

SLAM Handbook
从定位与建图到空间智能

中文翻译

2026 年 8 月 9 日

版权与翻译说明

本书原著由 Luca Carlone、Ayoung Kim、Timothy Barfoot、Daniel Cremers 和 Frank Dellaert 编辑，© Cambridge University Press。

本 PDF 收录已完成审查的 SLAM Handbook 中文译文。原作者、编辑、出版方及原项目的版权和署名信息均予以保留。中文翻译内容仅在获得相应授权的范围内公开使用。

目录

版权与翻译说明	i
第一章 SLAM 的因子图	1
历史注记	1
1.1 用因子图直观表示 SLAM	2
1.1.1 一个玩具示例	2
1.1.2 因子图视角	3
1.1.3 将因子图作为一种语言	6
1.2 从 MAP 推断到最小二乘	7
1.2.1 用因子图进行 MAP 推断	7
1.2.2 指定概率密度	8
1.2.3 非线性最小二乘	9
1.3 求解线性最小二乘	10
1.3.1 线性化	10
1.3.2 将 SLAM 写成线性最小二乘	11
1.3.3 用矩阵分解求解最小二乘	11
1.4 非线性优化	13
1.4.1 最速下降	13
1.4.2 高斯—牛顿	13
1.4.3 Levenberg—Marquardt	14
1.4.4 Dogleg 最小化	14
1.5 因子图与稀疏性	15
1.5.1 稀疏雅可比矩阵及其因子图	15
1.5.2 稀疏信息矩阵及其图	15
1.5.3 稀疏分解	17
1.6 消元	17
1.6.1 变量消元算法	17
1.6.2 线性高斯消元	20
1.6.3 作为贝叶斯网络的稀疏 Cholesky 因子	23
1.7 增量式 SLAM	23
1.7.1 贝叶斯树	24
1.7.2 更新贝叶斯树	24

1.7.3	增量平滑与建图	25
1.8	延伸阅读与近期趋势	26
第二章	高级状态变量表示	27
2.0.1	流形上的优化	27
2.0.2	旋转与位姿	27
2.0.3	矩阵 Lie 群	28
2.0.4	Lie 群优化	30
2.0.5	不确定性与 Lie 群	32
2.0.6	Lie 群的其他工具	32
2.0.6.1	$\oplus$ 与 $\ominus$ 算子	32
2.0.6.2	逆	33
2.0.6.3	伴随	33
2.0.6.4	雅可比	33
2.1	连续时间轨迹	34
2.1.1	样条	34
2.1.2	从参数化到非参数化	36
2.1.3	高斯过程	37
2.1.4	Lie 群上的样条与高斯过程	38
2.1.4.1	Lie 群上的样条	38
2.1.4.2	Lie 群上的高斯过程	40
2.2	延伸阅读与近期趋势	41
第三章	对错误数据关联和离群值的鲁棒性	43
3.1	离群值的成因及其危害	43
3.2	数据关联与离群值	43
3.3	存在离群值时的最小二乘	44
3.4	在 SLAM 前端检测和剔除离群值	45
3.5	随机采样一致性 (RANSAC)	49
3.6	基于图论的离群值剔除与成对一致性最大化	50
3.7	提高 SLAM 后端对离群值的鲁棒性	53
3.8	迭代重加权最小二乘	64
3.9	Black-Rangarajan 对偶性	65
3.10	交替最小化	73
3.11	渐进非凸性	74
3.12	延伸阅读与近期趋势	85
第四章	可微优化	89
4.1	非线性最小二乘回顾	89
4.2	穿过非线性最小二乘进行微分	90
4.3	展开微分	91
4.4	截断展开微分	92

4.5	隐式微分	93
4.6	流形上的微分	94
4.7	Lie 群导数	95
4.8	流形上的微分运算	97
4.9	自动微分的数值挑战与现代库	98
4.10	可微优化的实现示例	99
4.11	相关开源库	100
4.12	延伸阅读与近期趋势	101
第五章	稠密地图表示	102
5.1	范围感知基础	102
5.2	传感器测量模型	103
5.3	转换为点云	103
5.4	稠密建图基础	105
5.5	占据地图	106
5.6	隐式表面与距离场	106
5.7	占据地图还是距离场?	109
5.8	地图表示	109
5.9	目标空间结构的显式性	109
5.10	空间抽象类型	109
5.11	点	109
5.12	Surfel	112
5.13	网格	112
5.14	体素	113
5.15	连续函数	113
5.16	转换	113
5.17	数据结构与存储	114
5.18	朴素数据存储	114
5.19	哈希表	114
5.20	基于树的数据结构	115
5.21	混合数据结构	116
5.22	构建地图: 方法与实践	116
5.23	点	117
5.24	Surfel	117
5.25	网格	117
5.26	体素	118
5.27	按估计量分类的方法	118
5.28	按数据结构分类的方法	119
5.29	按测量融合算法分类的方法	119
5.30	按可扩展性分类的方法	120
5.31	高斯过程	120

5.32 高斯过程隐式表面	121
5.33 Hilbert map	122
5.34 建图中的深度学习	122
5.35 使用考量	126
5.36 环境因素	126
5.37 下游任务类型	126
5.38 建图方法总结	127
5.39 致谢	128

第一章 SLAM 的因子图

作者： Frank Dellaert、Michael Kaess、Timothy Barfoot

本章介绍因子图，并在高斯先验和高斯测量噪声的条件下，建立最大后验（maximum a posteriori, MAP）推断与最小二乘之间的联系。我们关注 SLAM 后端，即前端已经提取测量并完成数据关联之后的部分。首先，我们使用因子图对 SLAM 问题进行可视化（第 1.1 节）；随后说明 MAP 推断如何导出最小二乘优化（第 1.2 节）。接着讨论线性优化（第 1.3 节）和非线性优化（第 1.4 节）的求解方法，并进一步明确稀疏性、因子图和贝叶斯网络之间的联系（第 1.5 节）。然后介绍变量消元算法及其图模型解释（第 1.6 节），最后据此构建贝叶斯树以及增量平滑与建图（iSAM）算法（第 1.7 节）。本章最后讨论延伸阅读与近期趋势（第 1.8 节）。

历史注记

SLAM 的平滑方法不仅考虑机器人当前时刻的位置，还考虑截至当前时刻的完整机器人轨迹。许多作者只研究机器人轨迹的平滑问题^[172, 701, 700, 420, 594, 312]，这类方法如今称为**基于位姿的 SLAM**（pose-based SLAM）。它尤其适用于激光测距仪等传感器，因为这类传感器会在相邻机器人位姿之间产生两两约束。

更一般地，可以考虑完整 SLAM 问题^[1088]，即在最优意义下，同时估计全部传感器位姿以及环境中所有特征的参数。从计算角度看，这种基于优化的平滑方法具有明显优势：(a) 与基于滤波的协方差矩阵或信息矩阵不同，后两者都会随时间推移逐渐变成完全稠密的矩阵^[849, 1087]，而平滑对应的信息矩阵保持稀疏；(b) 在典型建图场景中（也就是机器人并非反复穿行于一个很小的环境），该矩阵能够更紧凑地表示地图协方差结构。由此，在 2000—2005 年间，大量工作开始将这些思想应用于 SLAM^[287, 340, 339, 1088]。

基于平方根的平滑与建图（square-root smoothing and mapping, SAM），也称为“因子图方法”，是在^[249, 254]中提出的，其依据是：信息矩阵或测量雅可比矩阵可以分别通过稀疏 Cholesky 分解或 QR 分解进行高效分解。这样便得到一个平方根信息矩阵，可立即用于求取最优机器人轨迹和地图。在序贯估计文献中，对信息矩阵进行分解被称为**平方根信息滤波**（square-root information filtering, SRIF）；该方法于 1969 年为 JPL 的水手 10 号（Mariner 10）金星探测任务而发展^[83]。使用平方根能够得到更加准确、稳定的算法。引用 Maybeck^[742]的话说：“许多实践者以充分的逻辑依据主张：平方根滤波器应始终优先于标准的卡尔曼滤波递推。”

下面我们将详细讨论：因子图为何是表示 SLAM 问题固有稀疏性的自然工具；将矩阵进行（稀疏）分解并得到矩阵平方根为何是求解这类问题的核心；以及这些内容最终如何与更一般的变量消元算法联系起来。本章的大部分内容是 Dellaert 等人^[255]长篇文章的压缩版。

1.1 用因子图直观表示 SLAM

本节首先通过一个玩具示例及其因子图表示，引入因子图，直观展示 SLAM 问题的稀疏性。随后说明多种不同形式的 SLAM 都可以采用这种表示，即使面对更大规模的问题，其稀疏结构也通常一目了然。

1.1.1 一个玩具示例

我们先考察一个简单的 SLAM 场景，以说明如何构造因子图。图 1.1 给出了该问题结构的示意图。机器人依次经过三个位姿 p_1 、 p_2 和 p_3 ，并对两个路标 ℓ_1 和 ℓ_2 进行方位观测。为了将解锚定在空间中，假设在第一个位姿 p_1 上还有一个绝对位置/方向测量。没有这一测量，就无法获得绝对位置的信息，因为方位测量都是相对的。¹

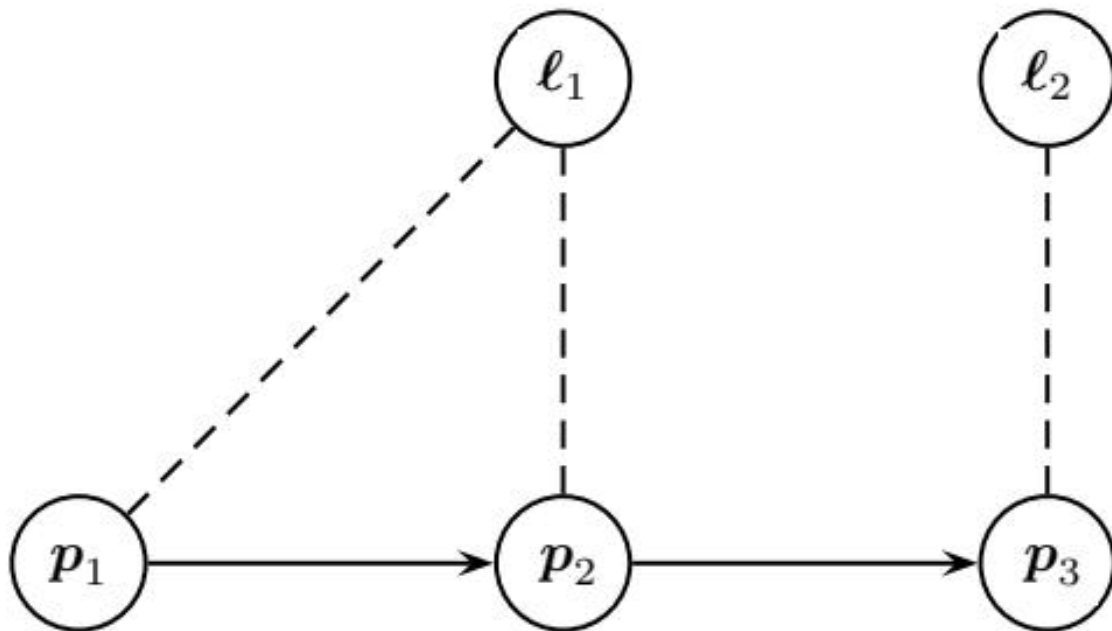


图 1.1: 图 1.1 三个位姿和两个路标构成的玩具同步定位与建图（SLAM）示例。图中用箭头示意机器人运动，用虚线表示方位测量。

由于测量存在不确定性，我们不可能恢复世界的真实状态，但可以得到一个关于测量能够推断出的内容的概率描述。在贝叶斯概率框架中，我们使用概率论的语言，为不确定事件赋予主观的可信程度。具体而言，对未知变量 $\mathbf{x}$ 定义概率密度函数（probability density function, PDF） $p(\mathbf{x})$ 。PDF 是满足下式的非负函数：

$$\int p(\mathbf{x}) d\mathbf{x} = 1, \quad (1.1)$$

这就是全概率公理。在图 1.1 的简单示例中，状态 $\mathbf{x}$ 为

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \\ \mathbf{p}_3 \\ \ell_1 \\ \ell_2 \end{bmatrix}, \quad (1.2)$$

也就是将各个未知量依次堆叠起来。

在 SLAM 中，给定一组观测测量 $\mathbf{z}$ 后，我们希望刻画关于未知量 $\mathbf{x}$ 的知识；在本例中，未知量包括机器人位姿和未知的路标位置。用贝叶斯概率的语言，这就是条件密度或后验密度：

$$p(\mathbf{x}|\mathbf{z}), \quad (1.3)$$

获得这样的描述称为**概率推断**。在此之前，必须先为感兴趣的变量以及它们如何产生（不确定的）测量指定概率模型。这正是概率图模型发挥作用的地方。

概率图模型利用概率分布中的结构，为复杂概率密度提供紧凑的描述机制^[592]。贝叶斯网络可能是最广为人知的图模型：它由变量节点组成，每个变量节点都关联一个先验概率密度或条件概率密度。图 1.2 展示了图 1.1 玩具示例对应的贝叶斯网络，其中明确给出了依赖于第一个位姿 $\mathbf{p}_1$ 的初始测量 z_1 ，以及 $z_2 \dots z_4$ 四个方位测量，每个测量都同时与一个位姿和一个路标相关。按照惯例，贝叶斯网络中已知量用方形节点表示，如图所示。不过，贝叶斯网络虽然非常适合建模，下面我们还要引入另一种更适合优化的图模型；它只关注问题中的未知变量。

1.1.2 因子图视角

由于局部性，高维概率密度通常可以分解为许多因子的乘积，每个因子只是定义在一个更小域上的概率密度。因子图是一类概率图模型，它允许我们将任意密度表示为因子的乘积。

为了说明因子图的动机，考虑如何对上述玩具 SLAM 示例执行推断。根据贝叶斯定律，后验 $p(\mathbf{x}|\mathbf{z})$ （式 1.3）可写为 $p(\mathbf{x}|\mathbf{z}) \propto p(\mathbf{z}|\mathbf{x})p(\mathbf{x})$ ，于是

$$p(\mathbf{x}|\mathbf{z}) \propto p(\mathbf{p}_1)p(\mathbf{p}_2|\mathbf{p}_1)p(\mathbf{p}_3|\mathbf{p}_2) \quad (1.4a)$$

$$\times p(\ell_1)p(\ell_2) \quad (1.4b)$$

$$\times p(z_1|\mathbf{p}_1) \quad (1.4c)$$

$$\times p(z_2|\mathbf{p}_1, \ell_1)p(z_3|\mathbf{p}_2, \ell_1)p(z_4|\mathbf{p}_3, \ell_2). \quad (1.4d)$$

这里假设位姿轨迹服从典型的马尔可夫链生成模型。每个因子都代表关于未知量 $\mathbf{x}$ 的一部分信息。

为了可视化这种分解，我们使用因子图。图 1.3 展示了该示例对应的因子图：所有未知状态 $\mathbf{x}$ （包括位姿和路标）都有一个对应节点。测量是已知量，因此不显式表示，也不是我们关

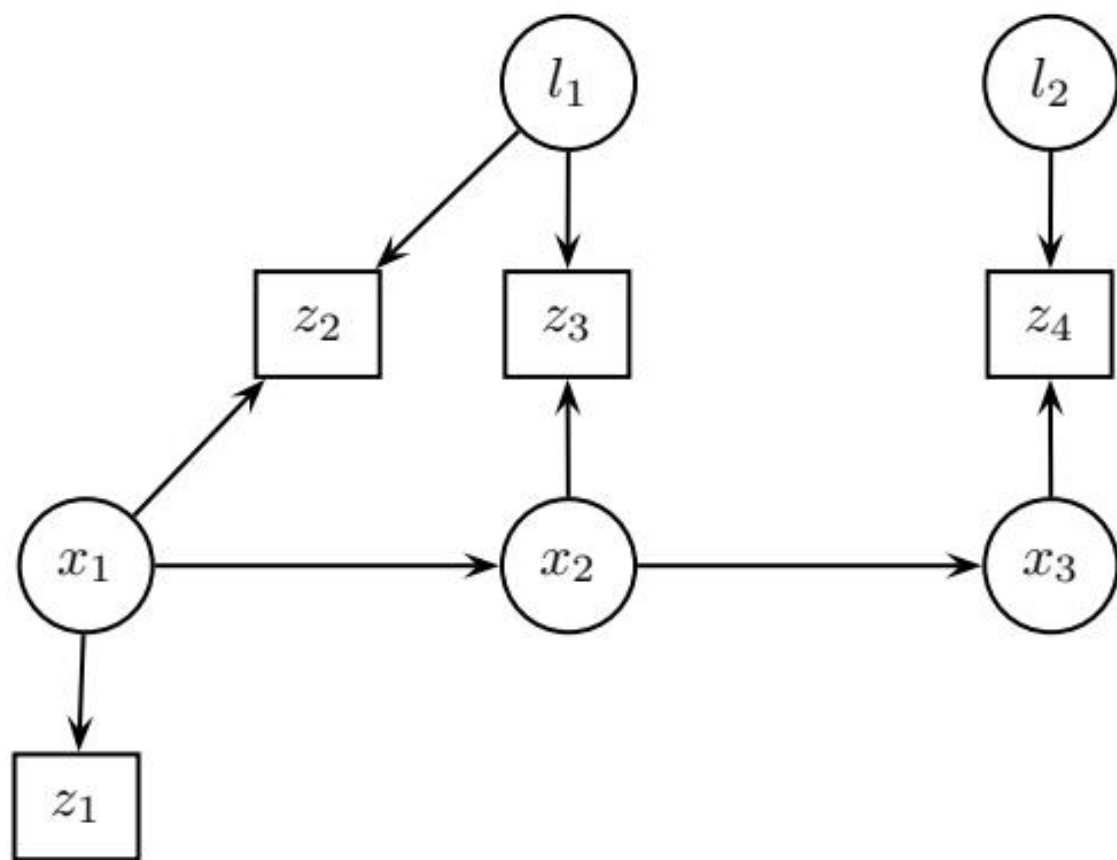


图 1.2: 图 1.2 图 1.1 示例对应的贝叶斯网络，其中将已知测量 $z_1 \dots z_4$ 明确表示为方形节点。

注的对象。在因子图中，我们显式引入另一类节点，用来表示后验 $p(\mathbf{x}|z)$ 中的每个因子。图中每个小黑节点表示一个因子；更重要的是，它只连接到该因子所依赖的状态变量。例如，因子 $\phi_9(p_3, \ell_2)$ 只连接到变量节点 p_3 和 ℓ_2 。具体地，有

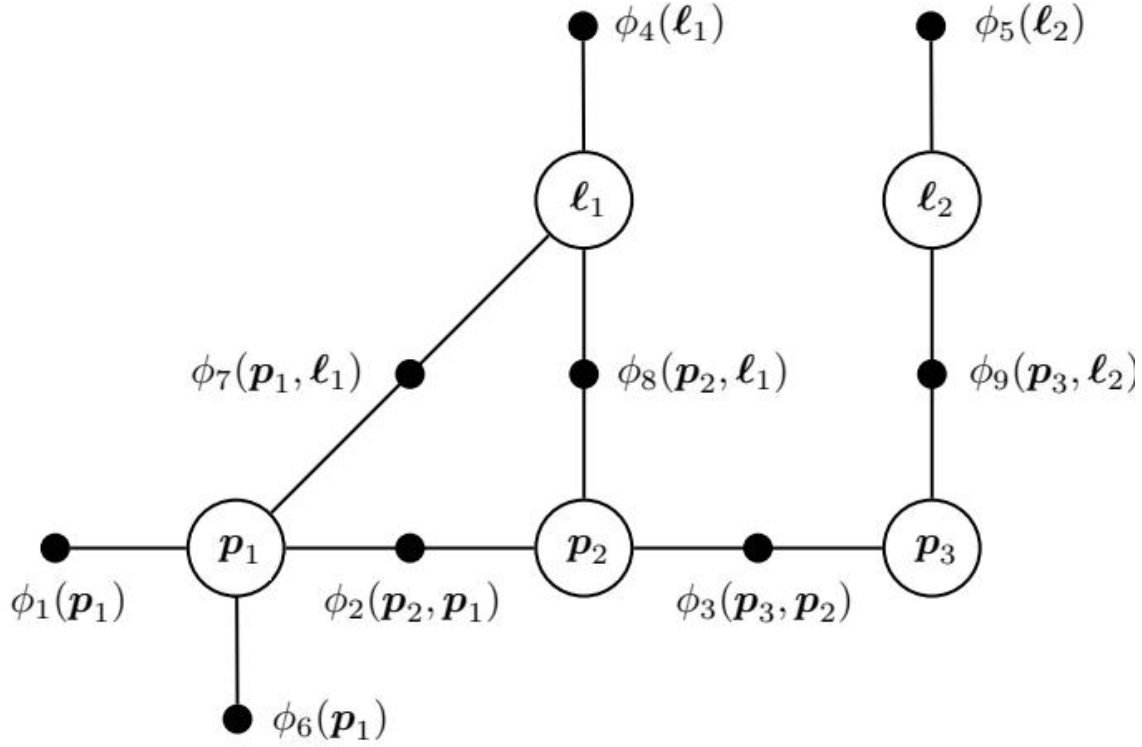


图 1.3: 图 1.3 图 1.1 示例得到的因子图。

$$\phi(p_1, p_2, p_3, \ell_1, \ell_2) = \phi_1(p_1)\phi_2(p_2, p_1)\phi_3(p_3, p_2) \quad (1.5a)$$

$$\times \phi_4(\ell_1)\phi_5(\ell_2) \quad (1.5b)$$

$$\times \phi_6(p_1) \quad (1.5c)$$

$$\times \phi_7(p_1, \ell_1)\phi_8(p_2, \ell_1)\phi_9(p_3, \ell_2), \quad (1.5d)$$

因子与式 (1.4a) — (1.4d) 中原始概率密度之间的对应关系应当是显而易见的。

因子值只需与相应概率密度成正比即可：任何与状态变量无关的归一化常数都可以省略，不会造成影响。在本例中，上述因子要么来自先验，例如 $\phi_1(p_1) \propto p(p_1)$ ；要么来自测量，例如 $\phi_9(p_3, \ell_2) \propto p(z_4|p_3, \ell_2)$ 。虽然测量变量 $z_1 \dots z_4$ 没有在因子图中显式显示，但这些因子都隐式地以它们为条件。有时为了使这一点更加明确，也可以将因子写成（例如） $\phi_9(p_3, \ell_2; z_4)$ ，甚至写成 $\phi_{z_4}(p_3, \ell_2)$ 。

1.1.3 将因子图作为一种语言

因子图除了为推断提供形式化基础之外，还能帮助我们可视化多种不同形式的 SLAM 问题、洞察问题结构，并充当一种通用语言，使不同团队的实践者能够在统一的概念下交流。图 1.3 中的每个因子都可以看作一个涉及其所连接变量的方程。通常，方程数量远多于未知量，因此必须量化先验信息和测量中的不确定性。这将导出最小二乘形式，从而适当地融合多个信息源的信息。

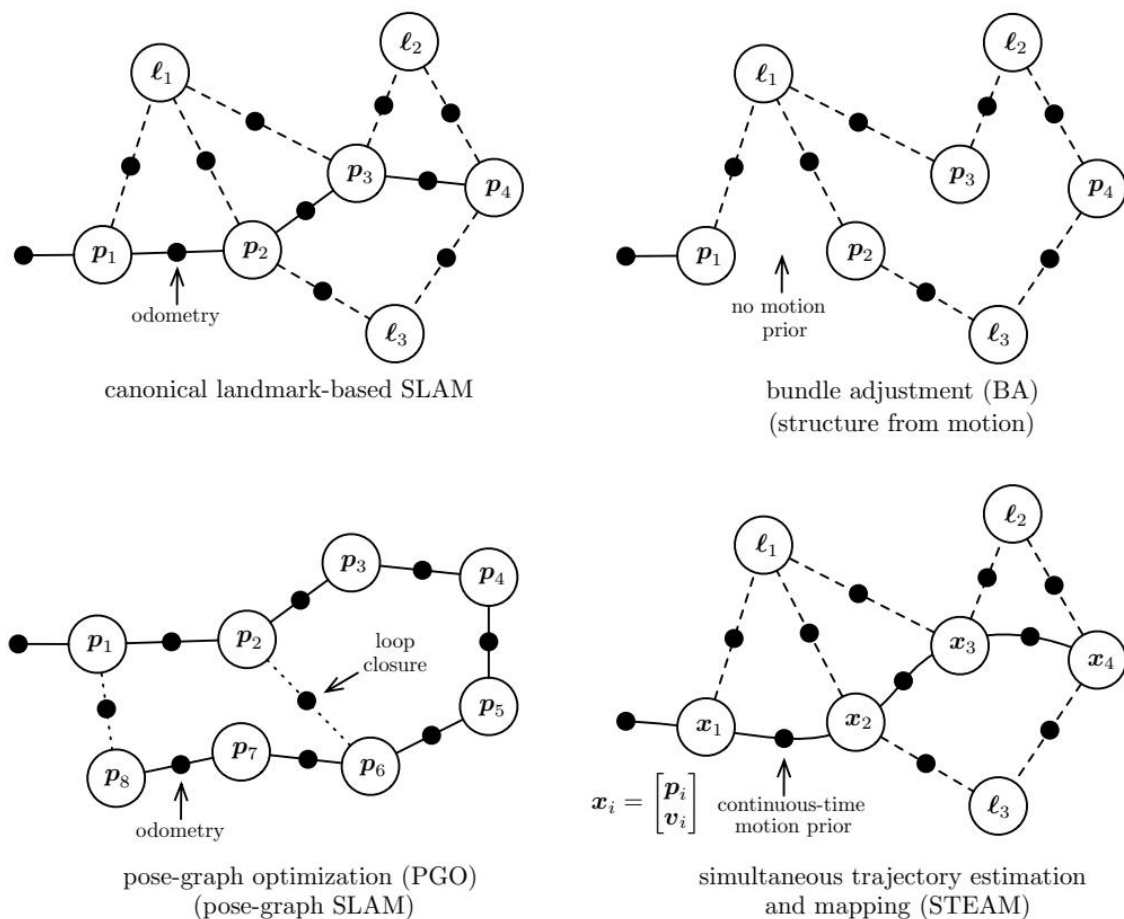


图 1.4: 图 1.4 几种都可以用因子图视角表示的 SLAM 问题变体。典型的基于路标的 SLAM 同时包含位姿变量和路标变量；从位姿观测路标，并且位姿之间通常存在基于里程计的运动先验。束调整 (BA) 与之相同，但不包含运动先验。位姿图优化 (PGO) 不含路标变量，但在位姿之间增加了回环测量。最后，同时轨迹估计与建图 (STEAM) 与基于路标的 SLAM 类似，只是将位姿替换为更高阶状态，并采用平滑的连续时间运动先验。

多种形式的 SLAM 问题都可以很容易地用因子图表示。图 1.1 是基于路标的 SLAM 示例，因为其中同时包含位姿变量和路标变量。图 1.4 给出了其他几种变体，包括束调整 (bundle adjustment, BA；与基于路标的 SLAM 相同，但没有运动模型)、位姿图优化 (pose-graph optimization, PGO；没有路标变量，但包含位姿之间的回环约束)，以及同时轨迹估计与建图 (simultaneous trajectory estimation and mapping, STEAM；在位姿中加入速度等导数项)。

比图 1.1 的玩具示例更接近实际的基于路标的 SLAM 问题，其因子图可能类似图 1.5。该图通过模拟一个在平面内运动约 100 个时间步、同时观测路标的二维机器人生成。为便于可

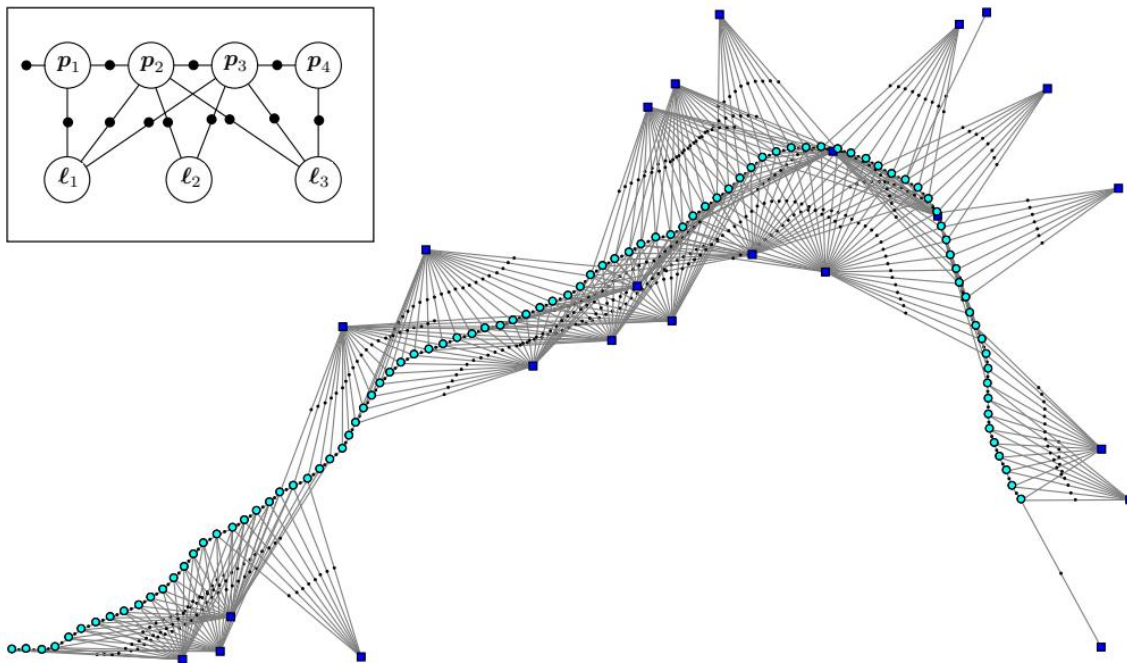


图 1.5: 图 1.5 一个规模更大的模拟 SLAM 示例因子图。

视化，每个位姿和路标都按其真实二维位置绘制。可以看到，里程计因子形成了显著的链状骨架，而在两侧则有二元²似然因子连接到约 20 个路标。在这类 SLAM 问题中，除先验之外，所有因子通常都是非线性的。

检查因子图可以揭示大量结构信息，帮助我们理解某个具体的 SLAM 问题实例。首先，有些路标拥有大量测量，我们预计它们会被非常准确地确定；另一些路标与图的联系很弱，因此估计结果也会更不确定。例如，右下方孤立的路标只有一个关联测量。

1.2 从 MAP 推断到最小二乘

最大后验 (MAP) 推断是确定未知量 $\mathbf{x}$ 取值的过程，使其与不确定测量和先验中包含的信息尽可能一致。在实际情况下，我们并不知道路标的真实位置，也不知道机器人随时间变化的位姿，尽管在许多应用中可能有一个较好的初始估计。下面将说明：在采用高斯测量噪声模型和高斯先验的条件下，MAP 推断对应的优化问题就是我们熟悉的非线性最小二乘问题。

1.2.1 用因子图进行 MAP 推断

给定测量 $\mathbf{z}$ ，我们关注位姿和/或路标等未知状态变量 $\mathbf{x}$ 。对于这些未知状态变量，最常用的估计量是最大后验 (MAP) 估计，其名称源于它最大化了给定测量 $\mathbf{z}$ 时状态 $\mathbf{x}$ 的后验密度 $p(\mathbf{x}|\mathbf{z})$ ：

$$\mathbf{x}^{\text{MAP}} = \arg \max_{\mathbf{x}} p(\mathbf{x}|\mathbf{z}) \quad (1.6a)$$

$$= \arg \max_{\mathbf{x}} \frac{p(\mathbf{z}|\mathbf{x})p(\mathbf{x})}{p(\mathbf{z})} \quad (1.6b)$$

$$= \arg \max_{\mathbf{x}} p(\mathbf{z}|\mathbf{x})p(\mathbf{x}) \quad (1.6c)$$

上式中的第二个等式是贝叶斯定律，它将后验表示为测量密度 $p(\mathbf{z}|\mathbf{x})$ 与状态先验 $p(\mathbf{x})$ 的乘积，再用 $p(\mathbf{z})$ 进行适当归一化。第三个等式省略了 $p(\mathbf{z})$ ，因为它与 $\mathbf{x}$ 无关，不会影响求 $\arg \max$ 的结果。换言之，式 (1.6c) 告诉我们：MAP 估计使似然 $p(\mathbf{z}|\mathbf{x})$ 与先验 $p(\mathbf{x})$ 的乘积最大。

我们使用因子图表示未归一化后验 $p(\mathbf{z}|\mathbf{x})p(\mathbf{x})$ 。形式上，因子图 $F = (\mathcal{U}, \mathcal{V}, \mathcal{E})$ 是一个二部图，包含两类节点：因子 $\phi_i \in \mathcal{U}$ 和变量 $\mathbf{x}_j \in \mathcal{V}$ 。边 $e_{ij} \in \mathcal{E}$ 始终连接因子节点和变量节点。与因子 ϕ_i 相邻的变量节点集合记作 $\mathcal{X}(\phi_i)$ ；我们用 $\Delta_{\mathbf{x}_i}$ 表示对该集合的一次赋值。按照这些定义，因子图 F 将全局函数 $\phi(\mathbf{x})$ 定义为

$$\phi(\mathbf{x}) = \prod_i \phi_i(\mathbf{x}_i). \quad (1.7)$$

换言之，独立关系由因子图中的边 e_{ij} 编码；每个因子 ϕ_i 只依赖于其邻接集合 $\mathcal{X}(\phi_i)$ 中的变量 $\Delta_{\mathbf{x}_i}$ 。

本章其余部分将说明，如何通过对因子图中的未知变量进行优化，找到最优赋值，即 MAP 估计。对于任意因子图，MAP 推断归结为最大化所有因子图势函数的乘积（式 1.7）：

$$\mathbf{x}^{\text{MAP}} = \arg \max_{\mathbf{x}} \phi(\mathbf{x}) \quad (1.8a)$$

$$= \arg \max_{\mathbf{x}} \prod_i \phi_i(\mathbf{x}_i). \quad (1.8b)$$

接下来需要推导因子 $\phi_i(\mathbf{x}_i)$ 的具体形式，而这在很大程度上取决于测量模型 $p(\mathbf{z}|\mathbf{x})$ 和先验密度 $p(\mathbf{x})$ 的建模方式。下面将详细讨论。

1.2.2 指定概率密度

上述密度 $p(\mathbf{z}|\mathbf{x})$ 和 $p(\mathbf{x})$ 的具体形式高度依赖于应用和所使用的传感器。最常用的密度是多元高斯密度，其概率密度为

$$\mathcal{N}(\boldsymbol{\theta}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{|2\pi\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}\|\boldsymbol{\theta} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}}^2\right), \quad (1.9)$$

其中， $\boldsymbol{\mu} \in \mathbb{R}^n$ 为均值， $\boldsymbol{\Sigma}$ 为 $n \times n$ 协方差矩阵，并且

$$\|\boldsymbol{\theta} - \boldsymbol{\mu}\|_{\boldsymbol{\Sigma}}^2 \triangleq (\boldsymbol{\theta} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{\theta} - \boldsymbol{\mu}) \quad (1.10)$$

表示平方马氏距离。归一化常数 $\sqrt{|2\pi\boldsymbol{\Sigma}|} = (2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}$ ，其中 $|\cdot|$ 表示矩阵行列式，它保证多元高斯密度在其定义域上的积分为 1。

未知量的先验通常用高斯密度指定；在许多情况下，将测量建模为受到零均值高斯噪声污染既合理又方便。例如，给定位姿 p 到给定路标 ℓ 的方位测量³ 可建模为

$$\mathbf{z} = \mathbf{h}(p, \ell) + \boldsymbol{\eta}, \quad (1.11)$$

其中 $h(\cdot)$ 是测量预测函数，噪声 $\boldsymbol{\eta}$ 服从协方差为 Σ_R 的零均值高斯密度。于是，关于测量 $\mathbf{z}$ 的条件密度 $p(\mathbf{z}|\mathbf{p}, \ell)$ 为

$$p(\mathbf{z}|\mathbf{p}, \ell) = \mathcal{N}(\mathbf{z}; \mathbf{h}(\mathbf{p}, \ell), \Sigma_R) = \frac{1}{\sqrt{|2\pi\Sigma_R|}} \exp\left(-\frac{1}{2}\|\mathbf{z} - \mathbf{h}(\mathbf{p}, \ell)\|_{\Sigma_R}^2\right). \quad (1.12)$$

在实际机器人应用中，测量函数 $h(\cdot)$ 往往是非线性的。不过，尽管它取决于所使用的传感器和 SLAM 前端，但通常并不难理解或写出。二维方位测量的测量函数就是

$$\mathbf{h}(\mathbf{p}, \ell) = \text{atan2}(\ell_y - p_y, \ell_x - p_x), \quad (1.13)$$

其中 ℓ_x, ℓ_y 是路标的 x, y 坐标， p_x, p_y 是位姿的 x, y 坐标， atan2 是众所周知的双参数反正切函数。因此，最终的概率测量模型 $p(\mathbf{z}|\mathbf{p}, \ell)$ 为

$$p(\mathbf{z}|\mathbf{p}, \ell) = \frac{1}{\sqrt{|2\pi\Sigma_R|}} \exp\left(-\frac{1}{2}\|\mathbf{z} - \text{atan2}(\ell_y - p_y, \ell_x - p_x)\|_{\Sigma_R}^2\right). \quad (1.14)$$

需要注意的是，我们并不总是假设测量噪声为高斯噪声。例如，为了应对偶发的数据关联错误，许多作者提出使用具有更重尾部的鲁棒测量密度；第 3 章将讨论这些方法。

并非所有涉及的概率密度都来自测量。例如，在玩具 SLAM 问题中，轨迹先验 $p(\mathbf{x})$ 由先验 $p(\mathbf{p}_1)$ 和条件密度 $p(\mathbf{p}_{t+1}|\mathbf{p}_t)$ 构成，用于描述在给定已知控制输入 $\mathbf{u}_t$ 的条件下机器人所遵循的概率运动模型。实践中，我们通常采用条件高斯假设：

$$p(\mathbf{p}_{t+1}|\mathbf{p}_t, \mathbf{u}_t) = \frac{1}{\sqrt{|2\pi\Sigma_Q|}} \exp\left(-\frac{1}{2}\|\mathbf{p}_{t+1} - \mathbf{g}(\mathbf{p}_t, \mathbf{u}_t)\|_{\Sigma_Q}^2\right), \quad (1.15)$$

其中 $\mathbf{g}(\cdot)$ 是运动模型， Σ_Q 是维度适当的协方差矩阵；例如，对于在平面内运动的机器人，它是 3×3 矩阵。

我们经常没有已知的控制输入 $\mathbf{u}_t$ ，而是测量机器人如何运动，例如通过里程计测量 $\mathbf{o}_t$ 。若假设里程计只是测量位姿差，并受到协方差为 Σ_S 的高斯噪声影响，则有

$$p(\mathbf{o}_t|\mathbf{p}_{t+1}, \mathbf{p}_t) = \frac{1}{\sqrt{|2\pi\Sigma_S|}} \exp\left(-\frac{1}{2}\|\mathbf{o}_t - (\mathbf{p}_{t+1} - \mathbf{p}_t)\|_{\Sigma_S}^2\right). \quad (1.16)$$

如果同时拥有已知控制输入 $\mathbf{u}_t$ 和里程计测量 $\mathbf{o}_t$ ，就可以将式 (1.15) 与式 (1.16) 结合起来。

对于在三维空间中运行的机器人，需要使用稍微复杂一些的工具，为 SE(3) 这类非线性流形指定概率密度；下一章将对此进行讨论。

1.2.3 非线性最小二乘

现在说明，对于采用上述高斯噪声模型的 SLAM 问题，MAP 推断等价于求解非线性最小二乘问题。假设所有因子都与多元高斯分布成正比，即

$$\phi_i(\mathbf{x}_i) \propto \exp\left(-\frac{1}{2}\|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2\right). \quad (1.17)$$

这既包括简单的高斯先验因子，也包括由零均值正态分布噪声污染的测量所导出的似然因子。将式 (1.17) 代入式 (1.8b)，取负对数并去掉因子 $1/2$ ，便可转而最小化一组非线性最小二乘项之和：

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2. \quad (1.18)$$

最小化这个目标函数，就是将多个由测量得到的因子以及可能存在的多个先验结合起来，执行传感器融合，从而确定未知量的 MAP 解。

一个重要而不太显然的观察是，式 (1.18) 中的因子通常只描述所涉及未知变量 $\Delta \mathbf{x}_i$ 的部分概率。除简单的先验因子外，测量 $\mathbf{z}_i$ 的维度通常低于未知量 $\Delta \mathbf{x}_i$ 的维度。在这种情况下，单个因子会为 $\mathbf{x}_i$ 定义域中的一个无限子集赋予相同的似然。例如，相机图像中的一个二维测量，与一整条能够投影到该图像位置的三维点射线相容。

尽管函数 $\mathbf{h}_i$ 是非线性的，但如果有一个足够好的初始猜测，本章讨论的非线性优化方法通常能够收敛到式 (1.18) 的全局最小值。不过需要提醒的是，式 (1.18) 的目标函数是非凸的，因此当初始猜测较差时，无法保证不会陷入局部最小值。这促成了所谓的**可证明最优求解器**，第 6 章将对此进行介绍。下面我们先关注局部方法，而不是全局求解器；首先考虑更容易的线性化问题。

1.3 求解线性最小二乘

在处理更困难的非线性最小二乘之前，本节首先说明如何将问题线性化，展示线性化如何导出线性最小二乘问题，并回顾如何通过矩阵分解高效求解相应的正规方程。关于这些方法的经典参考是 Golub 和 Van Loan 的著作^[389]。

1.3.1 线性化

可以使用简单的泰勒展开，将非线性最小二乘目标函数 (式 1.18) 中的所有测量函数 $\mathbf{h}_i(\cdot)$ 线性化：

$$\mathbf{h}_i(\mathbf{x}_i) = \mathbf{h}_i(\mathbf{x}_i^0 + \delta_i) \approx \mathbf{h}_i(\mathbf{x}_i^0) + \mathbf{H}_i \delta_i, \quad (1.19)$$

其中，测量雅可比 $\mathbf{H}_i$ 定义为在给定线性化点 $\mathbf{x}_i^0$ 处 $\mathbf{h}_i(\cdot)$ 的 (多元) 偏导数：

$$\mathbf{H}_i \triangleq \left. \frac{\partial \mathbf{h}_i(\mathbf{x}_i)}{\partial \mathbf{x}_i} \right|_{\mathbf{x}_i^0}, \quad (1.20)$$

而 $\delta_i \triangleq \mathbf{x}_i - \mathbf{x}_i^0$ 是状态更新向量。这里假设 $\Delta \mathbf{x}_i$ 位于向量空间中，或者等价地，可以用向量表示。当 $\mathbf{x}$ 中的一些未知状态表示三维旋转或其他更复杂的流形类型时，情况并不总是如此。第 2 章将重新讨论这个问题。

将泰勒展开 (式 1.19) 代入非线性最小二乘表达式 (式 1.18)，得到关于状态更新向量 δ 的线性最小二乘问题：

$$\delta^* = \arg \min_{\delta} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0) - \mathbf{H}_i \delta_i\|_{\Sigma_i}^2 \quad (1.21a)$$

$$= \arg \min_{\delta} \sum_i \|(\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0)) - \mathbf{H}_i \delta_i\|_{\Sigma_i}^2, \quad (1.21b)$$

其中, $z_i - h_i(x_i^0)$ 是线性化点处的预测误差, 即实际测量与预测测量之间的差。上式中的 δ^* 表示局部线性化问题的解。

通过一个简单的变量替换, 从这里开始可以去掉协方差矩阵 Σ_i : 将 $\Sigma^{1/2}$ 定义为 Σ 的矩阵平方根, 则平方马氏范数可以改写为:

其中

$$\|e\|_{\Sigma}^2 \triangleq e^{\top} \Sigma^{-1} e = \left(\Sigma^{-1/2} e \right)^{\top} \left(\Sigma^{-1/2} e \right) = \left\| \Sigma^{-1/2} e \right\|_2^2. \quad (1.22)$$

因此, 只需在式 (1.21b) 的每一项中, 用 $\Sigma_i^{-1/2}$ 左乘雅可比和预测误差, 就可以消除协方差矩阵 Σ_i :

$$A_i = \Sigma_i^{-1/2} H_i \quad (1.23a)$$

$$b_i = \Sigma_i^{-1/2} (z_i - h_i(x_i^0)). \quad (1.23b)$$

这个过程称为**白化**。例如, 对于标量测量, 它仅仅意味着将每一项除以测量标准差 σ_i 。注意, 这一步消除了测量的单位 (例如长度、角度), 因此不同的行可以组合到同一个代价函数中。

1.3.2 将 SLAM 写成线性最小二乘

线性化后, 最终得到如下标准最小二乘问题:

$$\delta^* = \arg \min_{\delta} \sum_i \|A_i \delta_i - b_i\|_2^2 \quad (1.24a)$$

$$= \arg \min_{\delta} \|A \delta - b\|_2^2. \quad (1.24b)$$

这里的 A 和 b 分别由收集所有白化雅可比矩阵 A_i 以及白化预测误差 b_i 得到, 并将它们组合成一个大的矩阵 A 和右端项向量 b 。

雅可比矩阵 A 是一个规模很大但稀疏的矩阵, 其分块结构与底层因子图的结构相对应。下面将详细考察这种稀疏结构。不过首先回顾经典线性代数方法, 说明如何求解这个最小二乘问题。

1.3.3 用矩阵分解求解最小二乘

对于一个满列秩的 $m \times n$ 矩阵 A ($m \geq n$), 式 (1.24b) 的唯一最小二乘解可以通过求解正规方程获得:

$$(A^{\top} A) \delta^* = A^{\top} b. \quad (1.25)$$

通常的做法是分解信息矩阵 (也称 Hessian 矩阵) Λ :

$$\Lambda \triangleq A^{\top} A = R^{\top} R. \quad (1.26)$$

这里的 Cholesky 因子 R 是一个 $n \times n$ 的上三角矩阵⁴，通过 Cholesky 分解计算得到；Cholesky 分解是针对对称正定矩阵的 LU（下三角—上三角）分解变体。随后，先求解

$$R^\top y = A^\top b \quad (1.27)$$

得到 y ，再通过前代和回代求解

$$R\delta^* = y \quad (1.28)$$

得到 δ^* 。对于稠密矩阵，Cholesky 分解需要 $n^3/3$ 次浮点运算；整个算法（包括计算对称矩阵 $A^\top A$ 的一半）需要 $(m+n/3)n^2$ 次浮点运算。还可以使用 LDL^{\{top\}} 分解，这是 Cholesky 分解的一个变体，可以避免开平方根。

另一种比 Cholesky 分解更准确、数值稳定性更好的方法是 QR 分解。该方法不必显式计算信息矩阵 $\mathbf{A}$ ，而是直接对 A 及其右端项进行 QR 分解：

$$A = Q \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix}, \quad \begin{bmatrix} d \\ e \end{bmatrix} = Q^\top b. \quad (1.29)$$

其中， Q 是 $m \times m$ 正交矩阵， $d \in \mathbb{R}^n$ ， $e \in \mathbb{R}^{m-n}$ ，而 R 是同一个上三角 Cholesky 三角因子。对于稠密矩阵 A ，通常按列从左到右计算 R 。对每一列 j ，通过用 Householder 反射矩阵 H_j 左乘 A ，将对角线下方的所有非零元素置零。经过 n 次迭代， A 完全分解为

$$H_n \dots H_2 H_1 A = Q^\top A = \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix}. \quad (1.30)$$

通常不会显式构造正交矩阵 Q ，而是将 b 作为额外一列附加到 A ，直接计算变换后的右端项 $Q^\top b$ 。由于 Q 是正交矩阵，有

$$\|A\delta - b\|_2^2 = \|Q^\top A\delta - Q^\top b\|_2^2 = \|R\delta - d\|_2^2 + \|e\|_2^2. \quad (1.31)$$

其中使用了式 (1.29) 的等式。显然， $\|e\|_2^2$ 就是最小二乘残差平方和，而最小二乘解 δ^* 可以通过回代求解三角系统

$$R\delta^* = d \quad (1.32)$$

得到。注意，QR 分解得到的上三角因子 R （对角线上的符号可能不同）与 Cholesky 分解得到的因子相同，因为

$$A^\top A = \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix}^\top Q^\top Q \begin{bmatrix} R \\ \mathbf{0} \end{bmatrix} = R^\top R. \quad (1.33)$$

这里再次使用了 Q 为正交矩阵这一事实。QR 的计算成本主要来自 Householder 反射，为 $2(m-n/3)n^2$ 次浮点运算。与 Cholesky 相比，当 $m \gg n$ 时，两种算法都需要 $O(mn^2)$ 次运算，但 QR 分解大约慢两倍。

总之，与 SLAM 线性化优化问题相关的线性代数可以简洁地概括为：将信息矩阵 $\mathbf{A}$ 或测量雅可比矩阵 A 分解成平方根形式。由于它们都基于 SAM 问题中的矩阵平方根，我们将这类方法称为平方根 SAM，简称 SAM^[249, 254]。

1.4 非线性优化

本节讨论求解式 (1.18) 所定义非线性最小二乘问题的一些经典优化方法。回顾一下，SLAM 中的非线性最小二乘目标函数为

$$J(\mathbf{x}) \triangleq \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2. \quad (1.34)$$

它对应于一个由测量以及部分或全部未知量的先验密度共同构成的非线性因子图。

一般来说，非线性最小二乘问题无法直接求解，而需要从合适的初始估计出发迭代求解。非线性优化方法通过依次求解式 (1.18) 的线性近似，逐渐逼近最小值^[264]。不同算法在局部近似代价函数的方式，以及根据这种局部近似寻找改进估计的方式上有所不同。关于非线性求解器的深入综述见^[814]；^[389] 则从线性代数角度进行讨论。

所有算法都具有如下基本结构：从初始估计 $\mathbf{x}^0$ 出发，在每次迭代中计算更新步 δ ，并将其应用到下一次估计 $\mathbf{x}^{t+1} = \mathbf{x}^t + \delta$ 。当达到某些收敛准则时停止，例如变化量 δ 的范数低于一个很小的阈值。

1.4.1 最速下降

最速下降 (steepest descent, SD) 使用当前估计处的最速下降方向来计算更新步：

$$\delta^{\text{sd}} = -\alpha \nabla J(\mathbf{x})|_{\mathbf{x}=\mathbf{x}^t}, \quad (1.35)$$

其中 $\mathbf{x}^t$ 是当前的 x 估计。这里使用负梯度确定最速下降方向。对于非线性最小二乘目标函数 (式 1.34)，在当前点附近将目标函数近似为二次函数 $J(\mathbf{x}) \approx \|\mathbf{A}(\mathbf{x} - \mathbf{x}^t) - \mathbf{b}\|_2^2$ ，在该线性化点可以得到精确梯度

$$\nabla J(\mathbf{x})|_{\mathbf{x}=\mathbf{x}^t} = -2\mathbf{A}^\top \mathbf{b}.$$

步长 α 必须谨慎选择，以平衡更新的安全性与合理的收敛速度。可以执行显式线搜索，在给定方向上寻找最小值。SD 算法简单，但在最小值附近收敛较慢。

1.4.2 高斯—牛顿

高斯—牛顿 (Gauss—Newton, GN) 通过使用二阶更新实现更快收敛。GN 利用非线性最小二乘问题的特殊结构，用 $\mathbf{A}^\top \mathbf{A}$ 近似 Hessian 矩阵。GN 更新步通过求解正规方程 (式 1.25) 得到：

$$\mathbf{A}^\top \mathbf{A} \delta^{\text{gn}} = \mathbf{A}^\top \mathbf{b}. \quad (1.36)$$

可以使用第 1.3.3 节中的任意方法求解。对于行为良好（即近似二次）的目标函数和较好的初始估计，高斯—牛顿表现出近似二次的收敛速度。如果二次近似很差，GN 步可能使新估计远离最小值，随后发生发散。

1.4.3 Levenberg—Marquardt

Levenberg—Marquardt (LM) 算法允许多次迭代直至收敛，同时控制我们愿意信任高斯—牛顿二次近似的区域。因此，这类方法通常称为**信赖域方法**。

为了结合 SD 和 GN 的优点，Levenberg^[646] 提议在正规方程（式 1.25）的对角线上加入非负常数 $\lambda \in \mathbb{R}^+ \cup \{0\}$ ：

$$(\mathbf{A}^\top \mathbf{A} + \lambda \mathbf{I}) \delta^{\text{lm}} = \mathbf{A}^\top \mathbf{b}. \quad (1.37)$$

当 $\lambda = 0$ 时得到 GN；当 λ 很大时，近似有 $\delta^* \approx \frac{1}{\lambda} \mathbf{A}^\top \mathbf{b}$ ，即沿代价函数 J （式 1.34）的负梯度方向更新。因此，LM 可以看作在 GN 和 SD 之间自然地进行折中。

随后 Marquardt^[731] 提议考虑对角元素的尺度，以获得更快的收敛：

$$(\mathbf{A}^\top \mathbf{A} + \lambda \text{diag}(\mathbf{A}^\top \mathbf{A})) \delta^{\text{lm}} = \mathbf{A}^\top \mathbf{b}. \quad (1.38)$$

这一修改会在梯度较小（目标函数近似平坦的方向）时沿最速下降方向采取更大的步长，因为此时对角元素的逆会较大；相反，在目标函数变化陡峭的方向上，算法会更加谨慎并采取更小的步长。对正规方程的这两种修改，都可以从贝叶斯角度解释为：给问题中的所有未知变量加入零均值先验。

GN 与 LM 的一个关键区别是，LM 会拒绝使残差平方和增大的更新。被拒绝意味着非线性函数在局部表现不佳，需要采取更小的步长。实现方式是启发式地增大 λ ，例如将当前值乘以 10，然后重新求解修改后的正规方程。另一方面，如果某一步使残差平方和降低，则接受该更新，并相应地更新状态估计；此时减小 λ （例如除以 10），然后以新的线性化点重复算法，直至收敛。

1.4.4 Dogleg 最小化

Powell 的 dogleg (PDL) 算法^[885] 可以作为 LM 的更高效替代方案^[697]。Levenberg—Marquardt 算法的一个主要缺点是：如果某一步被拒绝，就必须重新分解修改后的信息矩阵，而这正是算法中最昂贵的环节。因此，PDL 的核心思想是分别计算 GN 步和 SD 步，再将二者适当地组合起来。如果 LM 步被拒绝，GN 步和 SD 步的方向仍然有效，可以用另一种方式组合，直到代价下降。因此，状态估计的每次更新只需进行一次矩阵分解，而不是多次。

图 1.6 展示了 GN 步和 SD 步的组合方式。组合步从 SD 更新开始，随后急转弯（因此称为 dogleg，即“狗腿”）朝向 GN 更新，但在信赖域边界处停止。与 LM 不同，PDL 显式维护一个信赖域，在该区域内我们相信线性假设。线性近似是否合适由增益比决定：

$$\rho = \frac{J(\mathbf{x}^t) - J(\mathbf{x}^t + \delta)}{L(\mathbf{0}) - L(\delta)}, \quad (1.39)$$

其中 $L(\delta) = \mathbf{A}^\top \mathbf{A} \delta - \mathbf{A}^\top \mathbf{b}$ 是在当前估计 $\mathbf{x}^t$ 处对式（1.34）非线性二次代价函数 J 的线性化。如果 ρ 很小（即 $\rho < 0.25$ ），说明代价下降幅度没有达到线性化预测，因而缩小信赖域。相反，如果实际下降与预测相符或更好（即 $\rho > 0.75$ ），则根据更新向量的大小扩大信赖域，并接受该步。

1.5 因子图与稀疏性

到目前为止介绍的求解器假设相关矩阵可以是稠密的；但在 SLAM 中，稠密方法无法扩展到实际问题规模。对于图 1.1 的玩具问题，稠密方法能够正常工作。图 1.5 所示的较大模拟问题更接近真实世界，不过对于 SLAM 而言它仍然相对较小，因为实际问题常常包含数千乃至数百万个未知量。我们之所以仍能处理这类问题，正是由于其稀疏性。

直接观察因子图即可认识到这种稀疏性。由图 1.5 可见，该图是稀疏的（远非完全连接图）。连接 100 个未知位姿的里程计链，是由 100 个二元因子构成的线性结构，而不是可能存在的 100^2 个（二元）因子。此外，对于 20 个路标，最多可以有 2000 个因子将每个路标与每个位姿相连，但实际数量更接近 400。最后，路标之间完全没有因子，这反映了我们没有获得它们相对位置的任何信息。这种结构是大多数 SLAM 问题的典型特征。

1.5.1 稀疏雅可比矩阵及其因子图

现代 SLAM 算法的关键在于利用稀疏性。SLAM 中因子图的一个重要性质是：它准确表示了最终稀疏雅可比矩阵 A 的稀疏分块结构。为此，重新考察非线性 SLAM 问题内循环中的关键计算——最小二乘问题：

$$\delta^* = \arg \min_{\delta} \sum_i \|A_i \delta_i - b_i\|_2^2. \quad (1.40)$$

上式每一项都来自原始非线性 SLAM 问题中的一个因子，并在当前线性化点附近进行线性化（式 1.21b）。矩阵 A_i 可以按变量拆分成若干分块，并收集成一个大的分块稀疏雅可比矩阵，其稀疏结构恰好由因子图给出。

尽管这些线性问题通常作为非线性优化的内部迭代出现，下面省略更新量的上标，因为相关结论对其来源无关，适用于一般线性问题。

考虑图 1.1 玩具示例的因子图。线性化后得到具有图 1.7 所示分块结构的稀疏系统 $[A \ b]$ 。将其与因子图进行比较可以发现：每个因子对应一个分块行，每个变量对应 A 的一个分块列。总共有 9 个分块行，分别对应因子分解 $\phi(p_1, p_2, p_3, \ell_1, \ell_2)$ 中的 9 个因子。

1.5.2 稀疏信息矩阵及其图

如第 1.3.3 节所述，使用 Cholesky 分解求解正规方程时，首先要形成 Hessian 矩阵或信息矩阵 $\Lambda = A^\top A$ 。注意， $A^\top A$ 并不是真正的 Hessian，而是由残差的泰勒级数截断得到的“高斯—牛顿”近似。一般而言，由于雅可比 A 是分块稀疏的，Hessian Λ 也应当是稀疏的。按构造， Λ 是对称矩阵；当唯一 MAP 解存在时，它还是正定的。

信息矩阵 Λ 的稀疏模式自然定义了一个无向图 G ：如果两个变量曾经共同出现在同一个因子中，它们之间就存在一条边。在分块层面， $\Lambda = A^\top A$ 的稀疏模式正好是该图的邻接矩阵。这一结论不限于二元因子：一个 n 元因子会在其全部变量之间诱导一个团，因此对应所有变量对之间的非零分块。

图 1.8 展示了 (a) 玩具示例信息矩阵 Λ 的分块结构，以及 (b) 对应的无向图 G 。下面经常使用与 Λ 关联的图 G 来讨论稀疏性和填充。

$$\begin{bmatrix} \Lambda_{11} & & \Lambda_{13} & \Lambda_{14} & \\ & \Lambda_{22} & & & \Lambda_{25} \\ \Lambda_{31} & & \Lambda_{33} & \Lambda_{34} & \\ \Lambda_{41} & & \Lambda_{43} & \Lambda_{44} & \Lambda_{45} \\ & \Lambda_{52} & & \Lambda_{54} & \Lambda_{55} \end{bmatrix}$$

图 1.6: 图 1.7 图 1.1 玩具 SLAM 示例的稀疏雅可比矩阵 A 的分块结构，其中空白项为零。

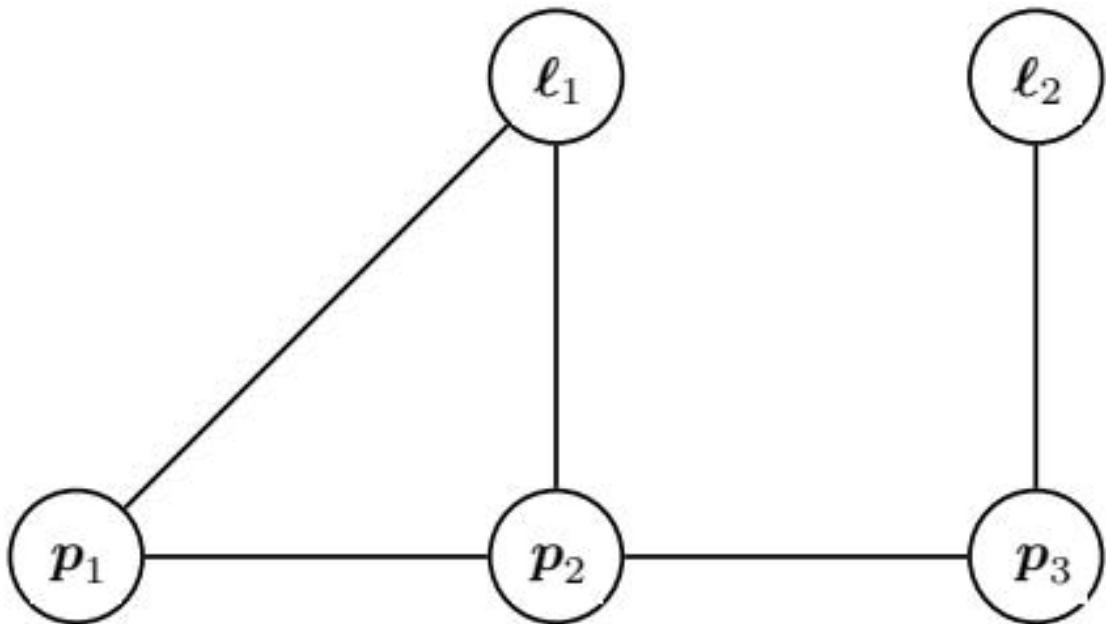


图 1.7: 图 1.8 玩具 SLAM 问题的信息矩阵 Λ 及其关联的无向图 G 。

1.5.3 稀疏分解

我们已经看到，MAP 估计等价于求解第 1.3.3 节所述的线性方程组；对于非线性最小二乘问题，则在迭代过程中反复求解这类系统。前两节说明， A 和 $A^\top A$ 都具有由因子图决定的稀疏性。无需展开细节，这种已知的稀疏模式可以显著加速 Cholesky 分解（处理 $A^\top A$ 时）或 QR 分解（处理 A 时）。例如，CHOLMOD^[189] 和 SuiteSparseQR^[240] 都提供了高效实现，许多软件包在底层使用它们。实践中，在稀疏问题上，稀疏 Cholesky 或 LDU 分解通常也优于 QR 分解，而且优势并非只是一个常数因子。

稀疏分解所需的浮点运算量远低于稠密矩阵。关键在于，稀疏矩阵采用的列排序会显著影响总计算量。任何排序最终都会产生相同的 MAP 估计，但变量排序决定矩阵因子中的填充量（即超出待分解矩阵原有稀疏模式的额外非零项）。已知寻找使矩阵分解填充最小的变量排序是 NP-hard 问题^[1239]，因此必须采用良好的启发式方法，这将进一步影响分解算法的计算复杂度。

仍以图 1.5 所示的较大模拟示例说明这一点。图 1.9 展示了对应稀疏雅可比矩阵 A 的模式，并在右上角给出信息矩阵 $\Lambda = A^\top A$ 的模式。图右侧还给出了两种不同排序下得到的上三角 Cholesky 因子 R 。两者都具有稀疏性，也都满足 $R^\top R = A^\top A$ （变量置换除外），但稀疏程度不同，而这正决定了分解 A 的代价。第一种排序最自然：先排列位姿，再排列路标，得到的稀疏 R 因子有 9399 个非零项。相比之下，右下角按列近似最小度（COLAMD）启发式方法^[29, 241] 重新排序后得到的 R 因子只有 4168 个非零项。然而，回代会为两种排序给出完全相同的解。

还可以使用预条件共轭梯度等工具迭代求解正规方程。在具有特殊稀疏模式的视觉 SLAM 中，幂迭代也已取得成功^[1171]。不过，对于大多数 SLAM 问题，稀疏分解仍然是首选方法，而且它还具有很好的图模型解释，下面将对此进行讨论。

1.6 消元

到目前为止，我们一直从线性代数角度解释 SLAM 推断。本节将视角扩展到更抽象的层次，直接利用图模型思考推断问题。这最终会引出当前最先进的 SLAM 求解器：下一节中用于增量平滑与建图的贝叶斯树。

1.6.1 变量消元算法

存在一种通用算法，可以针对任意（最好是稀疏的）因子图，计算未知变量 $\mathbf{x}$ 上相应的后验密度 $p(\mathbf{x}|\mathbf{z})$ ，并将其表示为易于恢复 MAP 解的形式。如前所述，因子图以因子乘积的形式表示未归一化后验 $\phi(\mathbf{x}) \propto p(\mathbf{x}|\mathbf{z})$ ；在 SLAM 问题中，该图通常直接由测量生成。变量消元算法提供了一套规则，可将因子图转换为另一个称为**贝叶斯网络**的图模型，该模型只依赖未知变量 $\mathbf{x}$ ，从而可以方便地进行 MAP 推断（以及采样、边缘化等其他操作）。

具体来说，变量消元算法可以将如下形式的任意因子图

$$\phi(\mathbf{x}) = \phi(\mathbf{x}_1, \dots, \mathbf{x}_n) \quad (1.41)$$

分解为具有如下形式的贝叶斯网络概率密度：

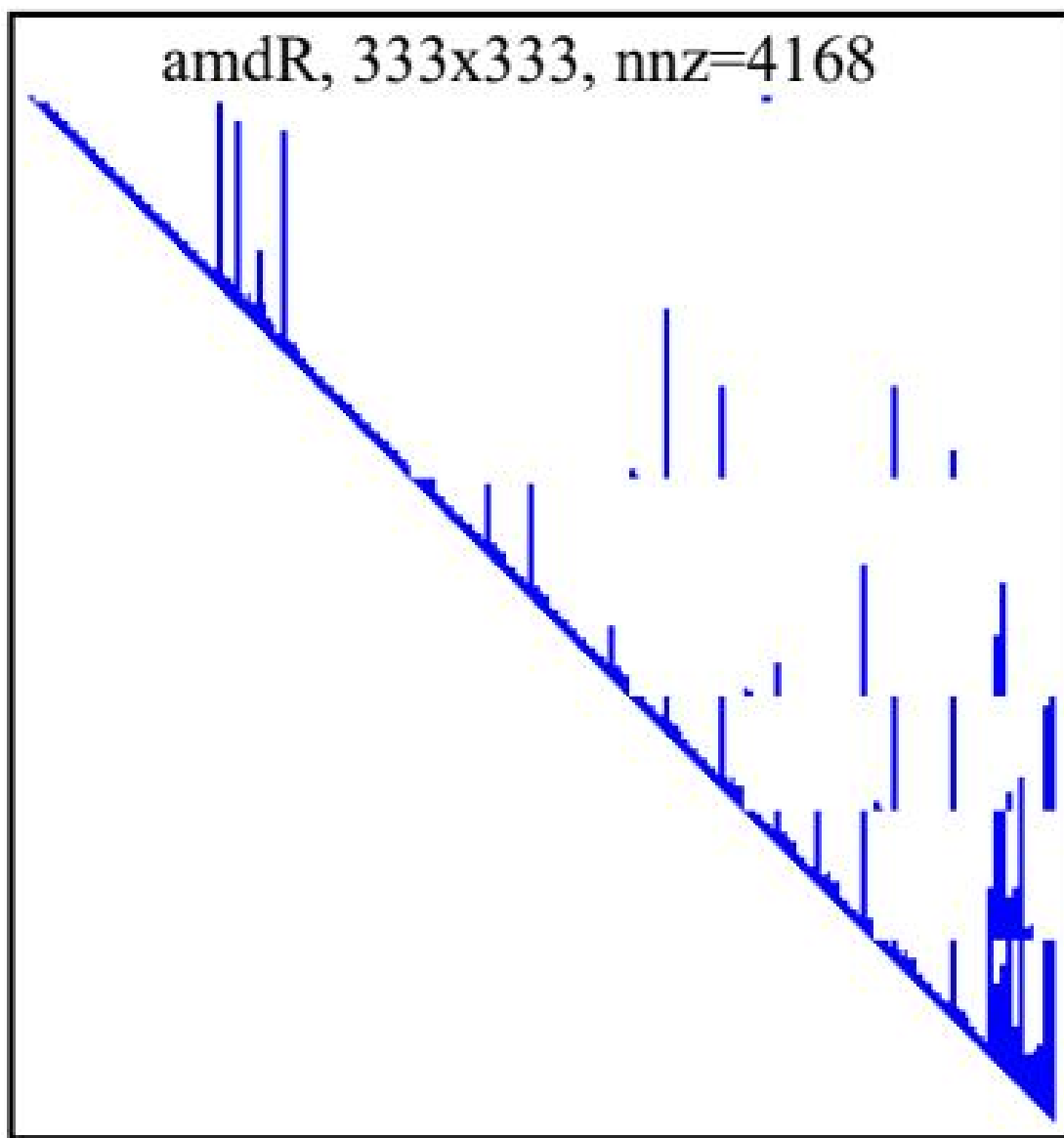


图 1.8: 图 1.9 左侧是图 1.5 问题对应的测量雅可比矩阵 A ；右侧从上到下依次为信息矩阵 $\Lambda = A^T A$ 、其上三角 Cholesky 因子 R ，以及采用更优变量排序（COLAMD）得到的替代因子 R 。

$$p(\mathbf{x}) = p(\mathbf{x}_1 | \mathbf{s}_1) p(\mathbf{x}_2 | \mathbf{s}_2) \dots p(\mathbf{x}_n) = \prod_j p(\mathbf{x}_j | \mathbf{s}_j), \quad (1.42)$$

其中, $\mathbf{s}_j$ 表示在给定变量排序 $\mathbf{x}_1, \dots, \mathbf{x}_n$ 下, 与变量 $\Delta \mathbf{x}_j$ 关联的分隔集 $\mathbf{s}(\mathbf{x}_j)$ 的一次赋值。分隔集定义为消元后 $\Delta \mathbf{x}_j$ 所依赖的变量集合。虽然这种分解类似链式法则, 但对稀疏因子图进行消元通常会得到很小的分隔集。

消元算法每次消去一个变量 $\Delta \mathbf{x}_j$, 从完整因子图 $\phi_{1:n}$ 开始。当消去每个变量时, 生成一个条件密度 $p(\mathbf{x}_j | \mathbf{s}_j)$, 以及只包含剩余变量的约简因子图 $\phi_{j+1:n}$ 。所有变量消元后, 算法返回具有所需分解形式的贝叶斯网络。

给定部分已消元的因子图 $\phi_{j:n}$, 要消除单个变量 $\Delta \mathbf{x}_j$, 首先移除所有与该变量相邻的因子 $\phi_i(\mathbf{x}_i)$, 并将它们相乘得到乘积因子 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 。随后把 ψ 分解为已消元变量上的条件分布 $p(\mathbf{x}_j | \mathbf{s}_j)$ 与分隔集上的新因子 $\tau(\mathbf{s}_j)$:

$$\psi(\mathbf{x}_j, \mathbf{s}_j) = p(\mathbf{x}_j | \mathbf{s}_j) \tau(\mathbf{s}_j). \quad (1.43)$$

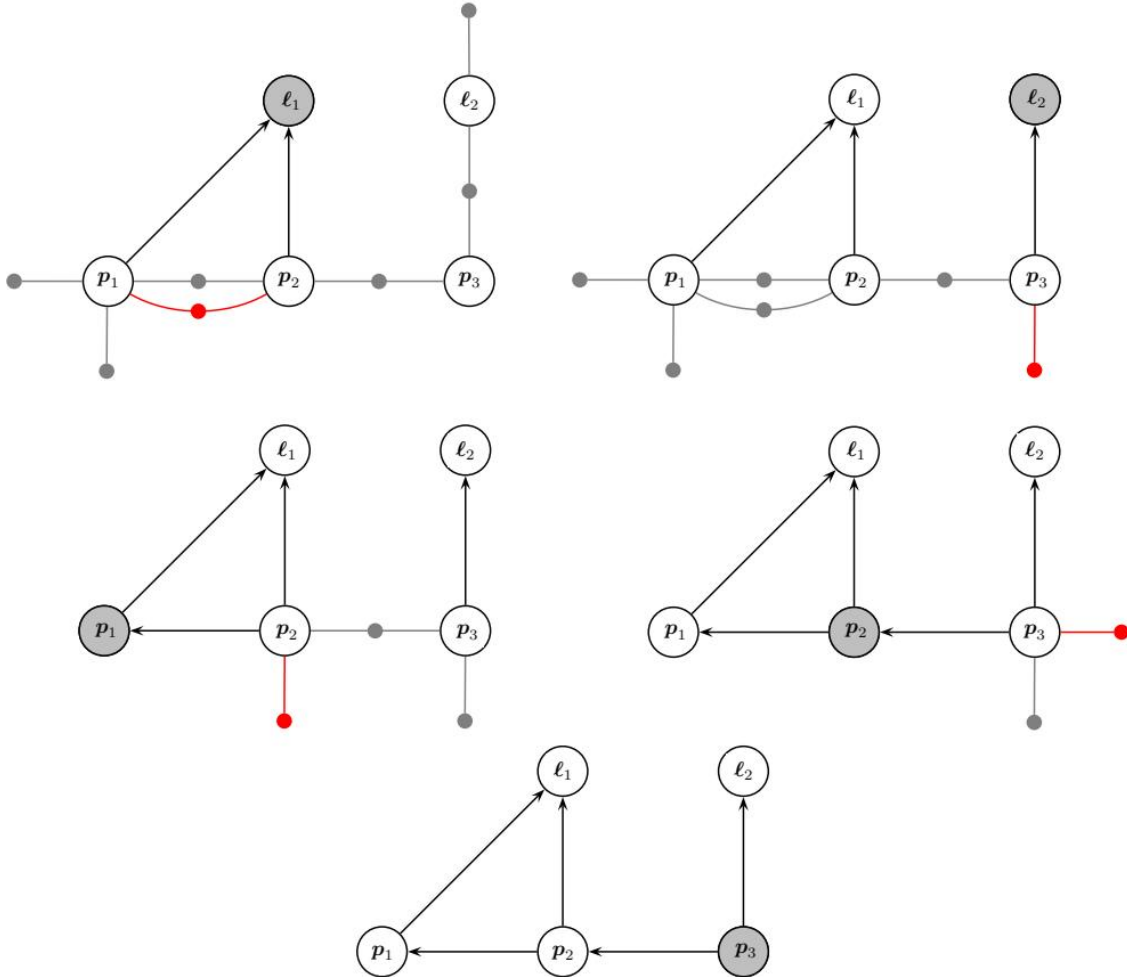


图 1.9: 图 1.10 玩具 SLAM 示例的变量消元: 按 $\ell_1, \ell_2, p_1, p_2, p_3$ 的顺序, 将图 1.3 的因子图转换为贝叶斯网络 (右下)。

因此，从 $\phi(\mathbf{x})$ 到 $p(\mathbf{x})$ 的完整分解，可以看作连续执行 n 个局部分解步骤。当消除最后一个变量 $\mathbf{x}_n$ 时，分隔集 s_n 为空，因此得到的条件密度只是对 $\mathbf{x}_n$ 的先验 $p(\mathbf{x}_n)$ 。

对于玩具示例，一种可能的消元序列如图 1.10 所示，排序为 $\ell_1, \ell_2, p_1, p_2, p_3$ 。每一步中，被消元的变量以灰色阴影表示，分隔集 s_j 上新生成的因子 $\tau(s_j)$ 以红色表示。整体而言，变量消元算法将因子图 $\phi(\ell_1, \ell_2, p_1, p_2, p_3)$ 分解为图 1.10（右下）的贝叶斯网络，对应的分解为

$$p(\ell_1, \ell_2, p_1, p_2, p_3) = p(\ell_1 | p_1, p_2) p(\ell_2 | p_3) p(p_1 | p_2) p(p_2 | p_3) p(p_3). \quad (1.44)$$

1.6.2 线性高斯消元

在线性测量函数和加性正态分布噪声的情况下，变量消元算法等价于稀疏矩阵分解；稀疏 Cholesky 分解和 QR 分解都是该通用算法的特例。

如前所述，消元算法逐个处理变量。对于每个变量 $\Delta \mathbf{x}_j$ ，移除与之相邻的所有因子 $\phi_i(\mathbf{x}_i)$ ，并形成中间乘积因子 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 。只需将所有矩阵 A_i 累积到一个更大的分块矩阵 $\bar{A}_j$ 中即可：

$$\psi(\mathbf{x}_j, \mathbf{s}_j) \leftarrow \prod_{i \in \mathcal{N}_j} \phi_i(\mathbf{x}_i) \quad (1.45a)$$

$$= \exp \left(-\frac{1}{2} \sum_i \|A_i \mathbf{x}_i - \mathbf{b}_i\|_2^2 \right) \quad (1.45b)$$

$$= \exp \left(-\frac{1}{2} \|\bar{A}_j [\mathbf{x}_j; \mathbf{s}_j] - \bar{\mathbf{b}}_j\|_2^2 \right), \quad (1.45c)$$

其中，新的右端项向量 $\bar{\mathbf{b}}_j$ 堆叠了所有 $\mathbf{b}_i$ ；符号 $[\cdot; \cdot]$ 表示按列（垂直）堆叠。

考虑在玩具示例中消去变量 ℓ_1 。与它相邻的因子为 ϕ_4, ϕ_7, ϕ_8 ，从而诱导出分隔集 $\mathbf{s}_1 = [p_1; p_2]$ 。乘积因子等于

$$\psi(\ell_1, p_1, p_2) = \exp \left(-\frac{1}{2} \|\bar{A}_1 [\ell_1; p_1; p_2] - \bar{\mathbf{b}}_1\|_2^2 \right), \quad (1.46)$$

其中

$$\bar{A}_1 \triangleq \begin{bmatrix} A_{41} & & \\ A_{71} & A_{73} & \\ A_{81} & & A_{84} \end{bmatrix}, \quad \bar{\mathbf{b}}_1 \triangleq \begin{bmatrix} \mathbf{b}_4 \\ \mathbf{b}_7 \\ \mathbf{b}_8 \end{bmatrix}. \quad (1.47)$$

观察图 1.7 的稀疏雅可比矩阵，这相当于取出第一列中存在非零分块的那些分块行，也就是与 ℓ_1 相邻的三个因子。

接下来可以用多种方式对乘积 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 进行分解。这里讨论 QR 变体，因为它与线性化因子的联系更加直接。具体而言，与乘积因子 $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 对应的增广矩阵 $[\bar{A}_j | \bar{\mathbf{b}}_j]$ 可使用部分 QR 分解^[389] 重写为

$$[\bar{A}_j | \bar{\mathbf{b}}_j] = Q \begin{bmatrix} R_j & T_j & d_j \\ & \tilde{A}_\tau & \tilde{\mathbf{b}}_\tau \end{bmatrix}, \quad (1.48)$$

其中 R_j 是上三角矩阵。由此， $\psi(\mathbf{x}_j, \mathbf{s}_j)$ 可以分解为

$$\begin{aligned}
 \psi(\mathbf{x}_j, \mathbf{s}_j) &= \exp \left\{ -\frac{1}{2} \left\| \bar{\mathbf{A}}_j[\mathbf{x}_j; \mathbf{s}_j] - \bar{\mathbf{b}}_j \right\|_2^2 \right\} \\
 &= \exp \left\{ -\frac{1}{2} \left\| \mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j \right\|_2^2 \right\} \exp \left\{ -\frac{1}{2} \left\| \tilde{\mathbf{A}}_\tau \mathbf{s}_j - \tilde{\mathbf{b}}_\tau \right\|_2^2 \right\} \\
 &= p(\mathbf{x}_j | \mathbf{s}_j) \tau(\mathbf{s}_j),
 \end{aligned} \tag{1.49a}$$

$$\left\| \bar{\mathbf{A}}_j[\mathbf{x}_j; \mathbf{s}_j] - \bar{\mathbf{b}}_j \right\|_2^2 = \left\| \mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j \right\|_2^2 + \left\| \tilde{\mathbf{A}}_\tau \mathbf{s}_j - \tilde{\mathbf{b}}_\tau \right\|_2^2. \tag{1.49b}$$

其中利用了旋转矩阵 Q 不会改变相关范数值这一事实。

在玩具示例中，图 1.11 展示了消去第一个变量 ℓ_1 （其分隔集为 $[p_1; p_2]$ ）的结果。图中同时展示了因子图上的操作，以及对图 1.7 稀疏雅可比矩阵的相应影响（省略右端项）。分界线以上的部分对应正在形成的稀疏上三角矩阵 R ；矩阵的新贡献用蓝色表示，新生成的因子用红色表示。为完整起见，图 1.12 展示了其余四个变量消元步骤，给出了 QR 分解如何在一个小型示例上端到端执行。最后一步展示了生成的贝叶斯网络与稀疏上三角因子 R 之间的等价性。

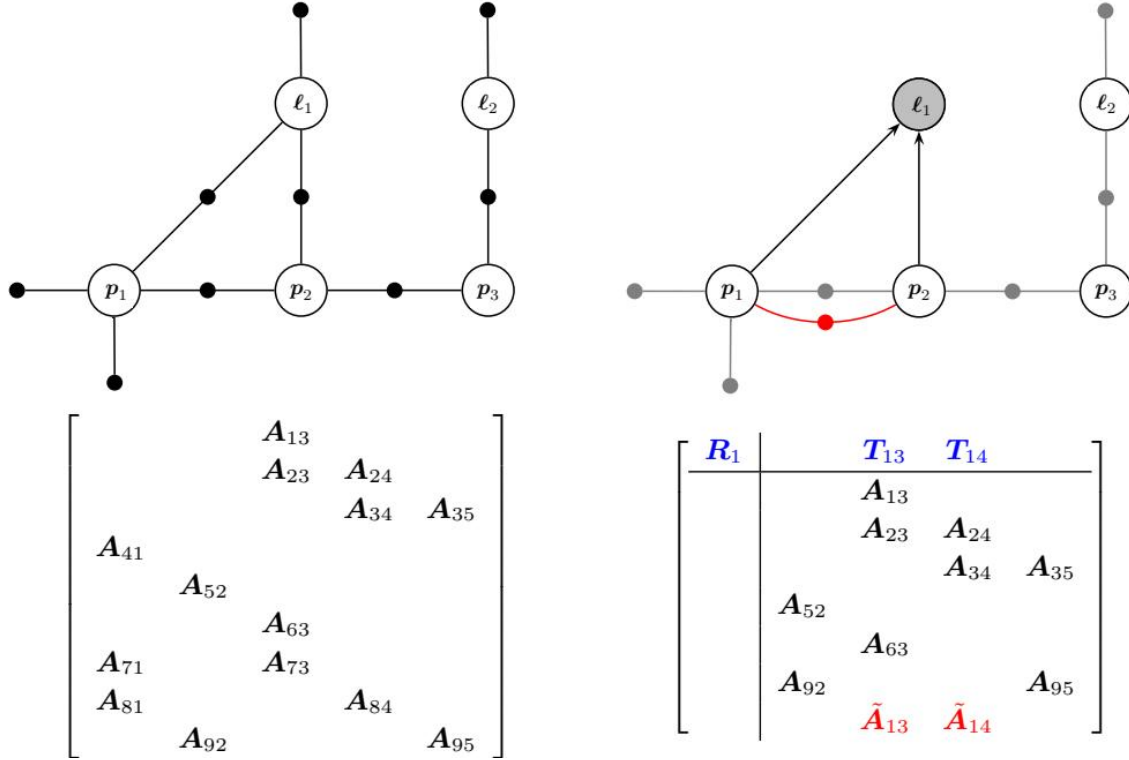


图 1.10: 图 1.11 将变量 ℓ_1 消元作为一次部分稀疏分解步骤。

使用部分 QR 分解消除单个变量的完整消元算法，等价于稀疏 QR 分解。由于上面的处理考虑了多维变量 $\mathbf{x}_j \in \mathbb{R}^{n_j}$ ，因此实际上这是多前沿 QR 分解^[288]的一个实例：每次同时消除多个标量变量，这有利于提高处理器利用率。在这里，标量变量按其在推断问题中的语义进行分组；而稀疏线性代数程序通常会按最大化计算效率的目标分析并分组问题。许多情况下，这两种策略高度一致。

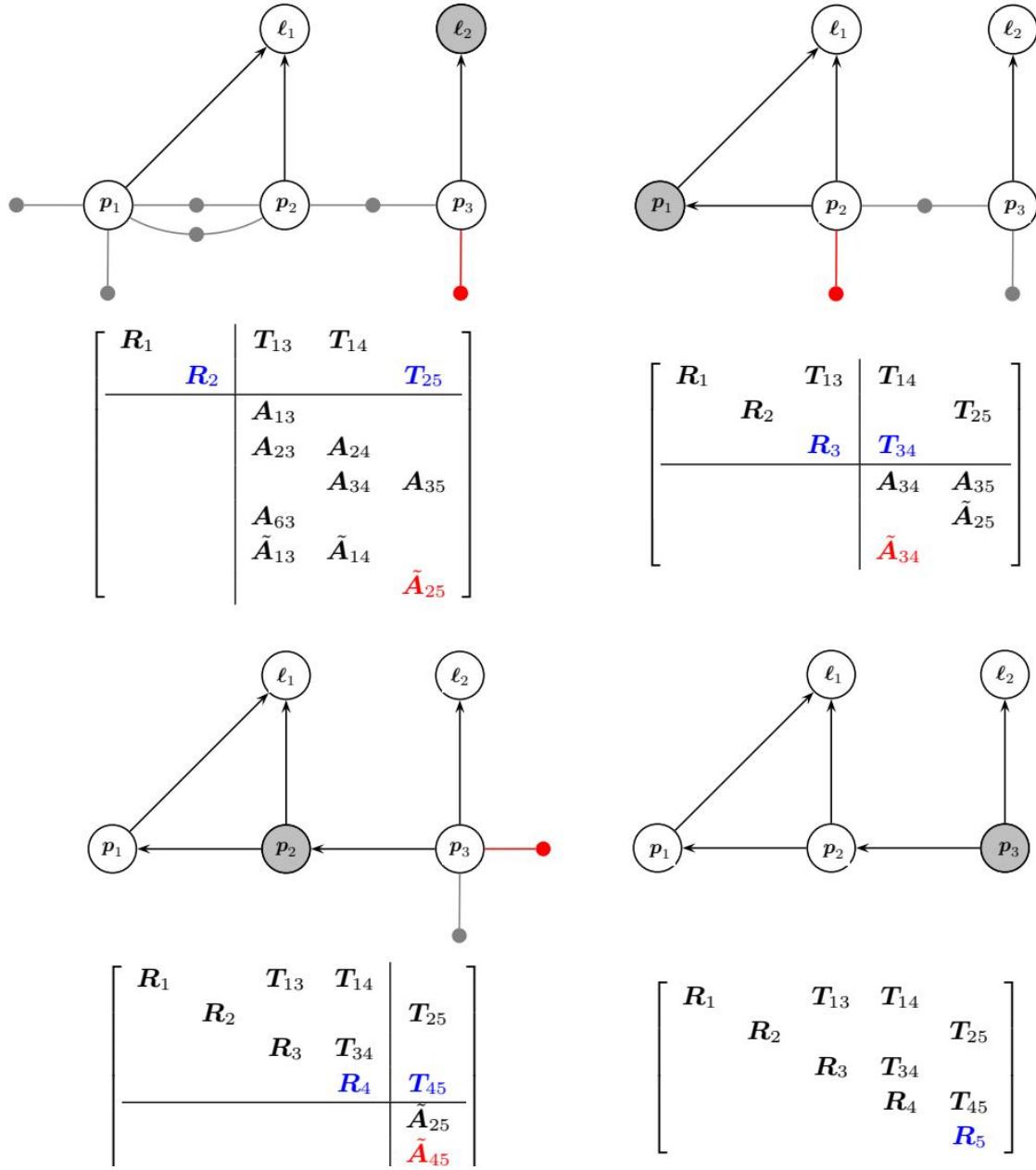


图 1.11: 图 1.12 玩具示例中剩余的消元步骤，完成一次完整 QR 分解。右下角的最后一步展示了最终贝叶斯网络与稀疏 Cholesky 因子 R 的等价性。

1.6.3 作为贝叶斯网络的稀疏 Cholesky 因子

变量消元与稀疏矩阵分解之间的等价性揭示了一个事实：与上三角矩阵关联的图模型就是贝叶斯网络！正如因子图是稀疏雅可比矩阵的图形化表示，贝叶斯网络揭示了 Cholesky 因子的稀疏结构。事后看来，这并不令人意外：贝叶斯网络是有向无环图 (DAG)，而这正对应矩阵的“上三角”性质。

此外，Cholesky 因子对应于高斯贝叶斯网络，即由线性高斯条件分布组成的贝叶斯网络。变量消元算法适用于一般密度；但当因子图只包含线性测量函数和高斯加性噪声时，得到的贝叶斯网络具有非常具体的形式。下面将讨论细节，以及在线性情形下如何求解 MAP 估计。

如前所述，与线性化非线性问题对应的高斯因子图，通过消元转化为密度 $p(\mathbf{x})$ ，其贝叶斯网络分解形式为

$$p(\mathbf{x}) = \prod_j p(\mathbf{x}_j | \mathbf{s}_j). \quad (1.50)$$

无论采用 QR 还是 Cholesky 变体，条件密度 $p(\mathbf{x}_j | \mathbf{s}_j)$ 都为

$$p(\mathbf{x}_j | \mathbf{s}_j) = k \exp \left(-\frac{1}{2} \|\mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j\|_2^2 \right), \quad (1.51)$$

这是关于已消元变量 $\mathbf{x}_j$ 的线性高斯密度。事实上，有

$$\|\mathbf{R}_j \mathbf{x}_j + \mathbf{T}_j \mathbf{s}_j - \mathbf{d}_j\|_2^2 = (\mathbf{x}_j - \boldsymbol{\mu}_j)^\top \mathbf{R}_j^\top \mathbf{R}_j (\mathbf{x}_j - \boldsymbol{\mu}_j) \triangleq \|\mathbf{x}_j - \boldsymbol{\mu}_j\|_{\Sigma_j}^2, \quad (1.52)$$

其中，均值 $\boldsymbol{\mu}_j = \mathbf{R}_j^{-1}(\mathbf{d}_j - \mathbf{T}_j \mathbf{s}_j)$ 线性依赖于分隔集 $\mathbf{s}_j$ ，协方差矩阵为 $\Sigma_j = (\mathbf{R}_j^\top \mathbf{R}_j)^{-1}$ 。因此，归一化常数为 $k = |2\pi \Sigma_j|^{-1/2}$ 。

消元步骤完成后，使用回代获得每个变量的 MAP 估计。如图 1.12 所示，最后被消元的变量不依赖于任何其他变量，因此可以直接从贝叶斯网络中提取该变量的 MAP 估计。按照消元顺序的逆序继续进行时，每个条件分布的所有分隔变量都将在前面的步骤中获得，从而可以计算当前前端变量的估计。

在每一步中，变量 $\Delta \mathbf{x}_j$ 的 MAP 估计就是条件均值：

$$\mathbf{x}_j^* = \mathbf{R}_j^{-1}(\mathbf{d}_j - \mathbf{T}_j \mathbf{s}_j^*), \quad (1.53)$$

因为到此时，分隔集 $\mathbf{s}_j$ 的 MAP 估计 $\mathbf{s}_j^*$ 已经完全确定。

1.7 增量式 SLAM

在增量式 SLAM 场景中，机器人穿行环境并不断接收新测量时，我们希望及时（或至少以固定间隔）计算最优轨迹和地图。一种方式是用新测量更新最近一次矩阵分解，复用已经纳入先前全部测量的计算结果。在线性情况下，可以通过增量分解方法实现这一点，其稠密版本在 Golub 和 Van Loan^[389] 中有详细讨论。然而，矩阵分解作用于线性系统，而实际关心的大多数 SLAM 问题都是非线性的。采用增量矩阵分解时，如何在不重新分解完整矩阵的情况下执行增量式重新线性化，并不显然。为解决这个问题，我们再次借助图模型，引入一种新的图模型——贝叶斯树。随后将说明，如何在系统加入新测量和新状态时增量更新贝叶斯树，从而得到增量平滑与建图 (iSAM) 算法。

1.7.1 贝叶斯树

众所周知，在树结构图中的推断是高效的。相比之下，典型机器人问题对应的因子图包含许多环。不过，我们可以分两步构造树结构图模型：第一步，对因子图执行变量消元，得到具有特殊性质的贝叶斯网络；第二步，利用该特殊性质，找到该贝叶斯网络中各个团之间的树结构。

通过因子图运行消元算法得到的贝叶斯网络具有一个特殊性质：它是弦图（chordal graph），即任意长度大于 3 的无向环都包含一条弦，也就是连接环上两个非相邻顶点的边。在人工智能和机器学习领域，弦图更常称为三角化图。由于它仍然是贝叶斯网络，其联合密度 $p(\mathbf{x})$ 可以在各个变量 $\Delta \mathbf{x}_j$ 上分解：

$$p(\mathbf{x}) = \prod_j p(\mathbf{x}_j | \pi_j), \quad (1.54)$$

其中， π_j 是 $\Delta \mathbf{x}_j$ 的父节点。不过，尽管贝叶斯网络是弦图，在变量层面它仍是一个非平凡图：既不是链式结构，也不是树结构。图 1.10 最后一步给出了当前玩具 SLAM 示例的弦贝叶斯网络；其中存在无向环 $\mathbf{p}_1 - \mathbf{p}_2 - \ell_1$ ，说明它不具备树结构。

通过识别这个弦图中的团，可以将贝叶斯网络改写为贝叶斯树。我们引入这种新的树结构图模型，用来捕捉贝叶斯网络中的团结构。贝叶斯网络中的团之所以能构成一棵树，并不显然；这是弦性质的结果，这里不再证明。将所有这些团列在一棵无向树上，就得到团树，在人工智能和机器学习中也称为连接树。贝叶斯树只是团树的有向版本，同时保留了消元顺序的信息。

更形式化地说，贝叶斯树是一棵有向树，其节点表示底层弦贝叶斯网络中的团 c_k 。具体而言，每个节点定义一个条件密度 $p(\mathbf{f}_k | \mathbf{s}_k)$ ，其中分隔集 $\mathbf{s}_k$ 是团 c_k 与其父团 ϖ_k 的交集 $c_k \cap \varpi_k$ 。前端变量 $\mathbf{f}_k$ 是剩余变量，即 $\mathbf{f}_k \triangleq c_k \setminus \mathbf{s}_k$ 。记号上，将一个团写成 $\mathbf{c}_k = \mathbf{f}_k : \mathbf{s}_k$ 。由贝叶斯树定义的变量联合密度为

$$p(\mathbf{x}) = \prod_k p(\mathbf{f}_k | \mathbf{s}_k). \quad (1.55)$$

对于根节点 f_r ，分隔集为空，即它只是根变量上的简单先验 $p(f_r)$ 。按照贝叶斯树的定义，团 c_k 的分隔集 $\mathbf{s}_k$ 始终是其父团 ϖ_k 的子集，因此图中的有向边与贝叶斯网络中的语义相同：表示条件依赖。

与当前玩具 SLAM 问题（图 1.3）关联的贝叶斯树如图 1.13 所示。根团 $\mathbf{c}_1 = \{\mathbf{p}_2, \mathbf{p}_3\}$ （蓝色）由 $\mathbf{p}_2$ 和 $\mathbf{p}_3$ 组成，并与另外两个团相交：绿色团 $\mathbf{c}_2 = \{\ell_1, \mathbf{p}_1\} : \mathbf{p}_2$ ，以及红色团 $\mathbf{c}_3 = \{\ell_2\} : \{\mathbf{p}_3\}$ 。颜色还表示平方根信息矩阵 R 的行如何映射到不同团，以及贝叶斯树如何捕捉它们之间的独立关系。例如，绿色和红色行只在属于根团的变量上相交，正如预期。

1.7.2 更新贝叶斯树

增量推断可以看作对贝叶斯树进行简单编辑。这种视角能够更好地解释和理解原本抽象的增量矩阵分解过程，也允许我们以贝叶斯树的形式存储和计算平方根信息矩阵，从而得到具有明确语义的稀疏存储结构。

为了增量更新贝叶斯树，我们有选择地将贝叶斯树的一部分转换回因子图形式。加入新测量时，相当于加入一个新因子；例如，涉及两个变量的测量会产生新的二元因子 $\phi(\mathbf{x}_j, \mathbf{x}_{j'})$ 。

此时，只有连接包含 $\mathbf{x}_j$ 和 $\mathbf{x}_{j'}$ 的团与根节点的路径会受到影响。这些团下方的子树不受影响，其他不包含这两个变量的子树也不受影响。因此，更新贝叶斯树时，只需将受影响部分转换回因子图，加入与新测量关联的新因子，然后使用任意方便的消元顺序重新消元这个临时因子图，构造新的贝叶斯树，最后重新接回未受影响的子树。

之所以只有树的上部受到影响，可以从贝叶斯树的两个重要性质理解。第一，贝叶斯树由弦贝叶斯网络按照消元顺序的逆序构成，因此每个团中的变量通过消去子团来收集来自子团的信息；任何团中的信息只会向上方根节点传播。第二，一个因子只有在与它相连的第一个变量被消去时才会进入消元过程。结合这两个性质可知，新因子不会影响那些不是其变量后继的变量。但是，如果一个因子涉及分别沿不同（即相互独立）路径连接到根的变量，那么为了表达它们之间新产生的依赖关系，就必须重新消去这些路径。

图 1.14 展示了如何在典型 SLAM 示例中应用这些增量分解/推断步骤。在该示例中，我们在 $\mathbf{p}_1$ 与 $\mathbf{p}_3$ 之间加入一个新因子，因此只影响树的左分支（左上图红色虚线标记部分）。随后构造右上图所示因子图：为每个团密度 $p(p_2, p_3)$ 和 $p(\ell_1, p_1 | p_2)$ 建立因子，并加入新因子 $f(p_1, p_3)$ 。右下图显示了按 ℓ_1, p_1, p_2, p_3 顺序消元后得到的图。最后，左下图给出重新组装的贝叶斯树，它由消元图得到的新树和原始贝叶斯树中未受影响的团（绿色）构成。

图 1.15 展示了一个小型 SLAM 序列的贝叶斯树示例，数据取自 Olson 等人^[824]著名的“曼哈顿世界”模拟序列第 400 步。机器人探索环境时，新测量通常只影响树的一小部分，因此只需重新计算这些部分。

1.7.3 增量平滑与建图

将上述内容结合起来，并考虑重新线性化等实际问题，就得到一种最先进的增量式非线性机器人 MAP 估计方法 iSAM。第一个版本 iSAM1^[531] 使用了 Golub 和 Van Loan^[389] 的增量矩阵分解方法，但它以次优方式处理线性化：定期在完整因子图上进行线性化，和/或在矩阵填充变得难以处理时进行线性化。第二个版本 iSAM2 使用贝叶斯树表示后验密度^[534]；随后如上所述，每当接收到新测量，就通过贝叶斯树增量更新来处理。

重新消除受影响团时，应该采用怎样的变量排序？只有贝叶斯树受影响部分中的变量需要更新。一种策略是在受影响变量上局部应用 COLAMD。但还可以做得更好：将最近访问的变量强制放到排序末端，即放入根团。对于这种增量变量排序策略，可以使用约束 COLAMD 算法^[241]。它既能将最近访问的变量置于末端，又能保持总体上良好的排序。通常，后续更新只会影响树的一小部分，因此除大型回环外，大多数情况下都能高效完成。

更新树之后，还需要更新解。贝叶斯树中的回代从不依赖其他变量的根节点开始，逐步向叶节点进行。不过，通常不必重新计算所有变量的解：树的局部更新往往不会影响远处的变量。相反，在每个团处检查向下传播的变量估计变化，当该变化低于一个很小的阈值时即可停止。

引入贝叶斯树的动机，是增量求解非线性优化问题。为此，我们有选择地对包含某些变量的因子重新线性化，前提是这些变量偏离线性化点超过一个很小的阈值。与上面的树修改不同，此时必须重新处理包含受影响变量的所有团，不仅包括它们作为前端变量的情形，也包括作为分隔变量的情形。这样会影响树的更大部分，但在大多数情况下仍显著低于重建整棵树的代价。此外，我们还需要回到原始因子，而不能直接把团转换成因子图，这要求在消元时缓存某些量。完整的增量非线性算法 iSAM2 可参见^[534]。

iSAM1 和 iSAM2 已成功应用于许多不同的机器人估计问题，其中变量之间存在非平凡约束，变量数量可达数百万；后续章节还会进一步讨论。两种算法都在 GTSAM 库中实现，代码见 <https://github.com/borglab/gtsam>。

1.8 延伸阅读与近期趋势

希望深入了解因子图的读者，可以参阅 Dellaert 等人^[255]的长篇文章。由于篇幅所限，本章相对简略，并弱化了一些高级概念，以尽可能降低入门门槛。不过，因子图和一般变量消元算法都非常强大，值得系统掌握。之后建议阅读 Kaess 等人^[534]，进一步理解贝叶斯树这一支撑 GTSAM 等现代求解器的核心概念。

如今，因子图范式可以处理流形上的状态变量，融合多种传感器，处理离群测量，甚至纳入深度学习方法的输出。本手册的其余部分将大量讨论这些趋势，读者可继续阅读以了解详情。

第二章 高级状态变量表示

作者： Timothy Barfoot、Frank Dellaert、Michael Kaess、Jose Luis Blanco-Claraco

上一章详细介绍了如何采用因子图范式建立并求解 SLAM 问题。为便于说明，我们有意避开了所估计状态变量的一些细节。本章重新审视状态变量的性质，并介绍现代 SLAM 形式化中普遍存在的两个重要主题。

首先，我们需要更好的工具来处理带有特定约束的状态变量；这些约束为变量定义了一个流形，因此在优化时必须特别处理。SLAM 中存在许多流形，最常见的是与机器人旋转相关的流形（尤其是三维旋转，但平面旋转也属于这一类）。

其次，状态变量的另一个方面来自时间本身的性质。上一章隐式地假设机器人以离散时间步在世界中运动。本章将引入轨迹的平滑连续时间表示，并讨论它们如何与因子图形式完全兼容。我们以 Barfoot^[47] 为主要参考，并采用 Sola 等人^[1022] 的简化记号。

2.0.1 流形上的优化

在某些机器人问题中，可以使用向量值未知量；但在大多数实际场景中，我们必须处理三维旋转以及其他非向量流形。粗略地说，流形是一个拓扑闭合的曲面（例如圆的周长或球面）；重要的是，在每个点附近，流形局部上都类似于欧氏空间。处理流形需要更精密的数学工具，以考虑其特殊结构。本节讨论如何在流形上执行优化，这将在上一章向量空间优化框架的基础上展开。作为示例，图 2.1 对球面流形 $\mathcal{M}$ 及其切空间 $T_x\mathcal{M}$ 进行了可视化；切空间可作为 $x \in \mathcal{M}$ 处用于优化的局部坐标系。

2.0.2 旋转与位姿

在 SLAM 语境中可以讨论许多流形，但最常见的两类用于表示旋转和位姿。旋转通常发生在二维（平面）或三维空间中，因此将旋转流形称为特殊正交群 $\text{SO}(d)$ ，其中 $d = 2$ 或 3 。平面旋转矩阵 $R_a^b \in \text{SO}(2)$ 的形式为

$$R_a^b = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad (2.1)$$

其中 $\theta \in \mathbb{R}$ 是旋转角，也是该情形下唯一的自由度。此外， R_a^b 可将参考坐标系 $\mathcal{F}^a$ 中表示的二维向量（例如路标）旋转到 $\mathcal{F}^b$ 中： $\ell^b = R_a^b \ell^a$ 。

三维旋转矩阵 $R_a^b \in \text{SO}(3)$ 同样用于在两个坐标系之间旋转向量，只是此时向量处于三维空间。三维旋转矩阵有 9 个元素，但只有 3 个自由度（例如滚转、俯仰和偏航）。二维和三维旋转矩阵都必须满足约束

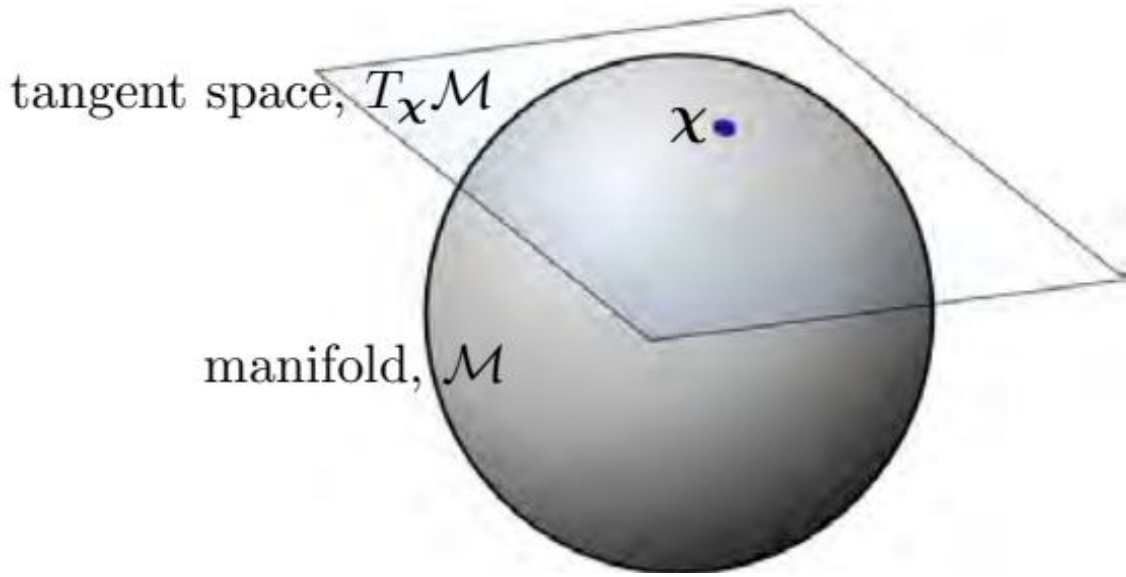


图 2.1: 图 2.1 对于球面流形 $\mathcal{M}$, $T_x \mathcal{M}$ 是局部切平面；配合局部基，可以定义局部坐标。

$$\mathbf{R}_a^{b\top} \mathbf{R}_a^b = \mathbf{I}, \quad \det(\mathbf{R}_a^b) = 1,$$

以适当地限制自由度。

机器人的位姿同时包含旋转变量 $\mathbf{R}_a^b \in \text{SO}(d)$ 和平移变量 $\mathbf{t}_a^b \in \mathbb{R}^d$ ，总共有 $3(d-1)$ 个自由度。有时我们分别跟踪这两类量，并使用 $\{\mathbf{R}_a^b, \mathbf{t}_a^b\} \in \text{SO}(d) \times \mathbb{R}^d$ 表示位姿。另一种方式是将它们组合成一个 $(d+1) \times (d+1)$ 的变换矩阵：

$$\mathbf{T}_a^b = \begin{bmatrix} \mathbf{R}_a^b & \mathbf{t}_a^b \\ \mathbf{0} & 1 \end{bmatrix}. \quad (2.2)$$

所有这类变换矩阵组成的流形称为特殊欧氏群 $\text{SE}(d)$ ，其中 $d=2$ 表示平面运动， $d=3$ 表示三维运动。使用 $\text{SE}(d)$ 的好处是，可以通过一次矩阵乘法同时对路标进行旋转和平移：

$$\underbrace{\begin{bmatrix} \ell^b \\ 1 \end{bmatrix}}_{\tilde{\ell}^b} = \underbrace{\begin{bmatrix} \mathbf{R}_a^b & \mathbf{t}_a^b \\ \mathbf{0} & 1 \end{bmatrix}}_{\mathbf{T}_a^b} \underbrace{\begin{bmatrix} \ell^a \\ 1 \end{bmatrix}}_{\tilde{\ell}^a}, \quad (2.3)$$

其中 $\tilde{\ell}$ 是路标的齐次表示。

由于旋转矩阵和变换矩阵的形式受到约束，它们并不是向量。例如，不能简单地将两个旋转矩阵相加，并期望结果仍是有效的旋转矩阵。不过， $\text{SO}(d)$ 和 $\text{SE}(d)$ 都属于具有额外有用结构的流形，即**矩阵 Lie 群**。我们可以利用这种结构，在因子图 SLAM 中继续执行无约束 MAP 优化（参见 Dellaert 等人^[255]、Boumal^[106] 或 Barfoot^[47]）。关于机器人中的 Lie 群，可参考 Chirikjian 和 Kyatkin^[207] 以及 Chirikjian^[205, 206] 的经典工作。

2.0.3 矩阵 Lie 群

在 $\text{SO}(d)$ 和 $\text{SE}(d)$ 上执行优化的关键，是利用它们的群结构。例如，矩阵 Lie 群在矩阵乘法下封闭：若 $\mathbf{R}_b^c, \mathbf{R}_a^b \in \text{SO}(d)$ ，则乘积也属于该群，且 $\mathbf{R}_a^c = \mathbf{R}_b^c \mathbf{R}_a^b \in \text{SO}(d)$ 。

矩阵 Lie 群还伴随一个非常有用的结构，称为 Lie 代数；它同时也是 Lie 群在单位元处的切空间。对我们而言，Lie 代数最重要的两个性质是：(i) 它是一个向量空间，其维度等于对应 Lie 群的自由度数量；(ii) 存在一个成熟的映射（矩阵指数）将 Lie 代数映射到 Lie 群。因此，可以从 Lie 代数元素相对容易地构造 Lie 群元素。例如，对于 $SO(2)$ ，暂时省略上下标，有

$$\mathbf{R} = \text{Exp}(\theta) = \exp(\theta^\wedge) = \sum_{n=0}^{\infty} \frac{1}{n!} (\theta^\wedge)^n = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \in SO(2), \quad (2.4)$$

其中

$$\theta^\wedge = \begin{bmatrix} 0 & -\theta \\ \theta & 0 \end{bmatrix}, \quad \theta \in \mathbb{R}. \quad (2.5)$$

量 $\theta^\wedge$ 属于 Lie 代数 $\mathfrak{so}(2)$ ，通过矩阵指数 $\exp(\cdot)$ 映射为 Lie 群元素 R 。反向也可以使用矩阵对数和 vee 算子完成： $\theta = \text{Log}(R) = (\log R)^\vee$ 。vee 算子 $(\cdot)^\vee$ 是 wedge 算子 $(\cdot)^\wedge$ 的逆。

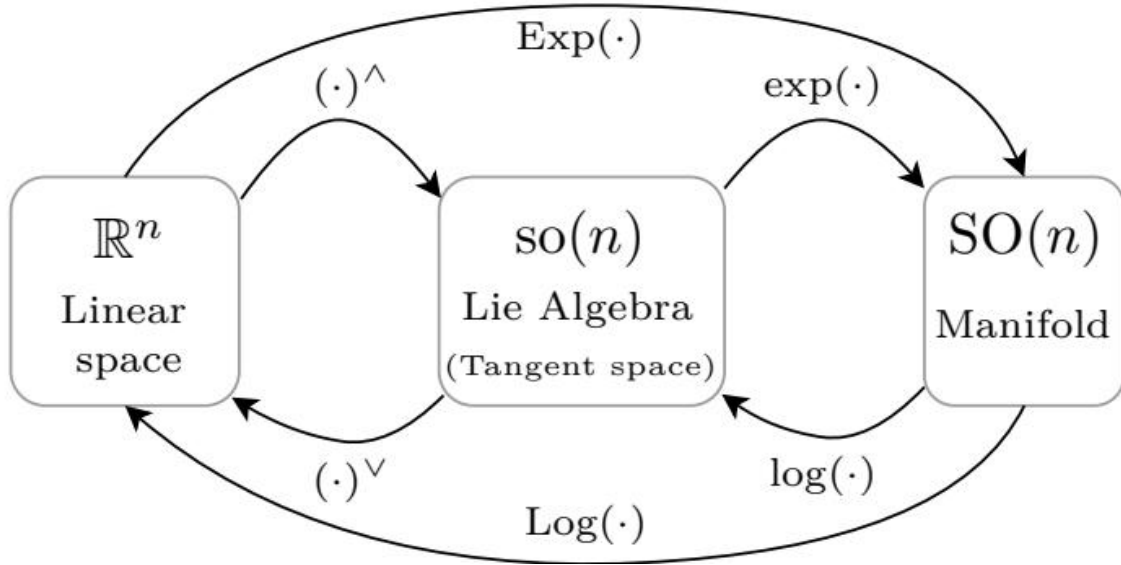


图 2.2: 本章记号概览：wedge 算子 $(\cdot)^\wedge$ 与 vee 算子 $(\cdot)^\vee$ ，指数映射 $\exp(\cdot)$ 与对数映射 $\log(\cdot)$ ，以及便捷记号 $\text{Exp}(\cdot) = \exp((\cdot)^\wedge)$ 和 $\text{Log}(\cdot) = (\log(\cdot))^\vee$ 。图中以 $SO(n)$ 为例，但这些算子对其他流形同样定义。

每个矩阵 Lie 群都有自己的线性 wedge 算子，用于从相应维度的标准向量空间构造 Lie 代数元素。对于 $SO(3)$ ，它是反对称算子：

$$\mathbf{R} = \text{Exp}(\boldsymbol{\theta}) = \exp(\boldsymbol{\theta}^\wedge) = \sum_{n=0}^{\infty} \frac{1}{n!} \boldsymbol{\theta}^{\wedge n} \in SO(3), \quad (2.6a)$$

$$\boldsymbol{\theta}^\wedge = \begin{bmatrix} 0 & -\theta_3 & \theta_2 \\ \theta_3 & 0 & -\theta_1 \\ -\theta_2 & \theta_1 & 0 \end{bmatrix} \in \mathfrak{so}(3), \quad \boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix} \in \mathbb{R}^3. \quad (2.6b)$$

对于 $SE(d)$ ，可写成

$$\mathbf{T} = \text{Exp}(\boldsymbol{\xi}) = \exp(\boldsymbol{\xi}^\wedge) \in \text{SE}(d), \quad (2.7a)$$

$$\boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\theta}^\wedge & \boldsymbol{\rho} \\ \mathbf{0} & 0 \end{bmatrix} \in \text{se}(d), \quad \boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{\rho} \\ \boldsymbol{\theta} \end{bmatrix} \in \mathbb{R}^{3(d-1)}, \quad \boldsymbol{\theta} \in \mathbb{R}^{2d-3}, \quad \boldsymbol{\rho} \in \mathbb{R}^d. \quad (2.7b)$$

这里 $d = 2$ (平面) 或 3 (三维)。注意, wedge 算子的具体版本可以由输入向量的大小确定。对于这里讨论的每个矩阵 Lie 群, Lie 代数与 Lie 群之间的映射还存在众所周知的闭式表达式, 可以替代矩阵指数的无穷级数形式 [47]。

2.0.4 Lie 群优化

建立矩阵 Lie 群之后, 就可以利用它们将非线性最小二乘项“线性化”, 从而执行 MAP 推断。回顾第 1.3.1 节, 我们仍然要线性化测量函数 $\mathbf{h}_i(\cdot)$, 只是其输入现在可能包含 Lie 群元素。

例如, 假设 $\mathbf{h}_i(\cdot)$ 是一个相机模型, 它接收在相机坐标系中表示的齐次路标 $\tilde{\ell}_i^c$, 并返回图像中的像素坐标 $\mathbf{z}_i \in \mathbb{R}^2$ 。传感器生成模型可写为

$$\mathbf{z}_i = \mathbf{h}_i(\mathbf{T}_w^c \tilde{\ell}_i^w) + \boldsymbol{\eta}_i. \quad (2.8)$$

其中, $\mathbf{T}_w^c \in \text{SE}(3)$ 是相对于相机坐标系的世界坐标系位姿, $\tilde{\ell}_i^w$ 是在世界坐标系中表示的齐次路标, $\boldsymbol{\eta}_i$ 是通常的传感器噪声。我们希望求解

$$\mathbf{T}_w^{c*} = \arg \min_{\mathbf{T}_w^c} \sum_i \left\| \mathbf{z}_i - \mathbf{h}_i(\mathbf{T}_w^c \tilde{\ell}_i^w) \right\|_{\Sigma_i}^2, \quad (2.9)$$

这就是透视 n 点 (perspective-n-point, PNP) 问题。本例假设世界坐标系中的路标位置已知; 当然, 在 SLAM 中也可能需要同时估计它们。

为了线性化传感器模型, 利用 Lie 代数产生位姿的扰动:

$$\mathbf{T}_w^c = \mathbf{T}_w^{c^0} \text{Exp}(\boldsymbol{\xi}). \quad (2.10)$$

这里 $\boldsymbol{\xi} \in \mathbb{R}^6$ 产生一个“小的”位姿变化, 用于扰动初始猜测 $\mathbf{T}_w^{c^0} \in \text{SE}(3)$ 。也常用简写记号

$$\mathbf{T}_w^c = \mathbf{T}_w^{c^0} \oplus \boldsymbol{\xi}, \quad (2.11)$$

它是式 (2.10) 的缩写。由于群的封闭性, 这两个量的乘积必然属于 $\text{SE}(3)$ 。使用 Lie 代数定义位姿扰动, 可以将其维度限制为三维位姿的实际自由度数量, 从而在优化过程中无需额外引入约束。

还可以用矩阵指数的级数形式近似扰动位姿:

$$\mathbf{T}_w^c \approx \mathbf{T}_w^{c^0} (\mathbf{I} + \boldsymbol{\xi}^\wedge). \quad (2.12)$$

其中只保留了关于 $\boldsymbol{\xi}$ 的一次项。将式 (2.12) 代入测量函数 (式 2.8), 得到

$$\mathbf{z}_i \approx \mathbf{h}_i \left(\mathbf{T}_w^{c^0} (\mathbf{I} + \boldsymbol{\xi}^\wedge) \tilde{\ell}_i^w \right) + \boldsymbol{\eta}_i. \quad (2.13)$$

利用齐次点的线性算子 $\odot$ ，还可写成

$$\mathbf{z}_i \approx \mathbf{h}_i \left(\mathbf{T}_w^{c^0} \tilde{\ell}_i^w + \mathbf{T}_w^{c^0} \tilde{\ell}_i^{w\odot} \boldsymbol{\xi} \right) + \boldsymbol{\eta}_i, \quad (2.14)$$

其中

$$\tilde{\ell}^\odot = \begin{bmatrix} \boldsymbol{\ell} \\ 1 \end{bmatrix}^\odot = \begin{bmatrix} \mathbf{I} & -\boldsymbol{\ell}^\wedge \\ \mathbf{0} & \mathbf{0} \end{bmatrix}. \quad (2.15)$$

现在还需对相机函数 $\mathbf{h}_i(\cdot)$ 进行线性化。利用一阶泰勒展开，有

$$\mathbf{z}_i \approx \mathbf{h}_i \left(\mathbf{T}_w^{c^0} \tilde{\ell}_i^w \right) + \underbrace{\frac{\partial \mathbf{h}_i}{\partial \boldsymbol{\ell}} \bigg|_{\mathbf{T}_w^{c^0} \tilde{\ell}_i^w}}_{\mathbf{H}_i \text{ (链式法则)}} \mathbf{T}_w^{c^0} \tilde{\ell}_i^{w\odot} \boldsymbol{\xi} + \boldsymbol{\eta}_i. \quad (2.16)$$

这里两部分相乘得到总体雅可比 $\mathbf{H}_i = \partial \mathbf{h}_i / \partial \boldsymbol{\xi}$ ，其链式法则结构十分清晰。也可以将传感器模型分解成若干基本操作，并使用中间向量状态逐步求导；对于本例，利用链式法则，关于相机位姿的雅可比可以拆成传感器模型、位姿作用、位姿复合和指数映射四项的乘积：

$$\mathbf{H}_i = \frac{\partial \mathbf{h}_i}{\partial \boldsymbol{\xi}}, \quad (2.17)$$

各个单独雅可比都有已知表达式^[87]。注意，每个雅可比的线性化点都考虑了扰动发生在 $\boldsymbol{\xi} = \mathbf{0}$ 附近这一事实。

回到式 (2.11b)，该测量对应的线性化最小二乘项（即负对数因子）为

$$\left\| \left(\mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}_w^{c^0} \tilde{\ell}_i^w \right) \right) - \mathbf{H}_i \boldsymbol{\xi} \right\|_{\Sigma_i}^2. \quad (2.18)$$

此时唯一未知量是位姿扰动 $\boldsymbol{\xi}$ 。将其与其他因子结合并求得状态变量的最优更新（包括 $\boldsymbol{\xi}^*$ ）后，需要按照式 (2.10) 的扰动方案更新初始猜测：

$$\mathbf{T}_w^{c^0} \leftarrow \mathbf{T}_w^{c^0} \oplus \boldsymbol{\xi}^*. \quad (2.19)$$

这样可以确保解始终位于 $\text{SE}(3)$ 中。优化照常迭代，直到所有状态更新（包括 $\boldsymbol{\xi}$ ）的变化足够小。式 (2.19) 适用于右侧位姿扰动；也可以选择左侧扰动，具体讨论见 Barfoot^[47]。

有时测量本身也位于流形上。例如，可能获得一个带噪位姿测量 $\mathbf{Z}_i$ ，其值属于 $\mathbf{T}_w^c \in \text{SE}(3)$ 。此时在 Lie 代数中构造误差：

$$\left\| \text{Log} \left(\mathbf{T}_w^c \mathbf{Z}_i^{-1} \right) \right\|_{\Sigma_i}^2 \approx \left\| \text{Log} \left(\mathbf{T}_w^{c^0} \mathbf{Z}_i^{-1} \right) - \mathbf{H}_i \boldsymbol{\xi} \right\|_{\Sigma_i}^2. \quad (2.20)$$

其中 $\mathbf{T}_w^c$ 按前述方式扰动， $\mathbf{H}_i$ 是该误差对应的雅可比^[47]。这样，任意类型的测量都可以在因子图范式中处理。

总之，对于属于 Lie 群的状态变量，可以通过上述方式执行无约束优化。关键是将测量函数相对于 Lie 代数中的扰动线性化，如式 (2.16) 所示。多数情况下可以解析地完成；也可以利用数值方法或自动微分计算所需雅可比。更一般地，这种针对 Lie 群元素函数的优化方

法属于黎曼优化^[106]：Lie 代数也是流形的切空间，因而优化被限制在位姿（或旋转）流形的切方向上；按式 (2.19) 更新则相当于将更新量重新收回流形。黎曼优化同样适用于并非矩阵 Lie 群的流形，且除了矩阵指数外还可以使用其他回缩映射。

2.0.5 不确定性与 Lie 群

我们通常将估计中的不确定性表示为服从某种分布的随机状态变量。第 1.2.2 节已经讨论过，高斯分布最为常用。对于向量变量 x ，可写为

$$x = \mu + \delta, \quad \delta \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad (2.21)$$

其中 μ 是均值， Σ 是协方差矩阵，即将状态拆分为均值与零均值噪声之和。

对于 Lie 群，不能简单地将噪声加到旋转矩阵 $R \in \text{SO}(d)$ 上，因为这样会破坏群的封闭性，结果不再是有效旋转矩阵。通常使用矩阵指数的满射映射，将高斯噪声 δ 与“均值”旋转 $\bar{R} \in \text{SO}(d)$ 组合：

$$R = \bar{R} \oplus \delta = \bar{R} \text{Exp}(\delta), \quad \delta \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad (2.22)$$

这样可保证 $R \in \text{SO}(d)$ 。对于 $\text{SE}(d)$ 同理：

$$T = \bar{T} \oplus \delta = \bar{T} \text{Exp}(\delta), \quad \delta \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad (2.23)$$

此时可保证 $T \in \text{SE}(d)$ ，并且 δ 与 Σ 的维度必须相应匹配。这种做法本质上是在切空间中定义高斯分布，再通过指数映射将其诱导为旋转上的噪声分布。也可以直接在旋转上定义概率分布，某些情况下这样具有计算优势；第 6 章将再次讨论。

2.0.6 Lie 群的其他工具

关于 Lie 群还可以讨论很多内容^[1033, 106]。为保持简洁，这里只汇集本章及后续章节会用到的几个有用事实。关于 Lie 群元素函数求导的更多细节将在第 4.3 节给出。

2.0.6.1 $\oplus$ 与 $\ominus$ 算子

前面已经使用 $\oplus$ 算子将 Lie 代数向量与 Lie 群元素组合。对于 $\text{SE}(d)$ ，有

$$T = T^0 \oplus \xi = T^0 \text{Exp}(\xi) = T^0 \exp(\xi^\wedge) \in \text{SE}(d). \quad (2.24)$$

还可以定义两个 Lie 群元素之间的“差”运算 $\ominus$ ：

$$T = T^0 \text{Exp}(\xi) \implies \xi = T \ominus T^0, \quad (2.25a)$$

$$\xi = T \ominus T^0 = \text{Log} \left(T^{0^{-1}} T \right) = \log \left(T^{0^{-1}} T \right)^\vee \in \text{se}(d). \quad (2.25b)$$

这些算子可以将相关运算的细节抽象出来。

2.0.6.2 逆

进行扰动时，有时需要扰动旋转或变换矩阵的逆。在 $SE(d)$ 中，直接有

$$(\mathbf{T}^0 \oplus \boldsymbol{\xi})^{-1} = \text{Exp}(\boldsymbol{\xi})^{-1} \mathbf{T}^{0^{-1}} = \text{Exp}(-\boldsymbol{\xi}) \mathbf{T}^{0^{-1}} = (-\boldsymbol{\xi}) \oplus \mathbf{T}^{0^{-1}}. \quad (2.26)$$

可以看到，扰动从右侧移到左侧，同时带有负号。

2.0.6.3 伴随

Lie 群的伴随表示，是将群元素视为其 Lie 代数线性变换的一种方式。对于 $SO(d)$ ，伴随表示与群本身相同，故略去细节。对于 $SE(d)$ ，伴随表示不同于群的基本表示，因此需要进一步说明。

给定位姿 T ， $SE(d)$ 的伴随映射将局部坐标系中的 Lie 代数元素 $\boldsymbol{\xi}^\wedge \in \mathfrak{se}(d)$ 变换为全局坐标系中的元素 $\boldsymbol{\epsilon}^\wedge \in \mathfrak{se}(d)$ ：

$$\boldsymbol{\epsilon}^\wedge \oplus T = T \oplus \boldsymbol{\xi}^\wedge, \quad (2.27)$$

其对应的内自同构（共轭）映射为

$$\boldsymbol{\epsilon}^\wedge = \text{Ad}_T \boldsymbol{\xi}^\wedge = T \boldsymbol{\xi}^\wedge T^{-1}. \quad (2.28)$$

等价地，也可以用伴随矩阵 $\text{Ad}(T)$ 将 $\boldsymbol{\xi} \in \mathbb{R}^6$ 线性变换到 $\mathbb{R}^6$ ：

$$\text{Ad}_T \boldsymbol{\xi}^\wedge = (\text{Ad}(T) \boldsymbol{\xi})^\wedge. \quad (2.29)$$

称 $\text{Ad}(T)$ 为 T 处 $SE(d)$ 的伴随表示。若

$$T = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix},$$

则

$$\text{Ad}(T) = \begin{bmatrix} \mathbf{R} & \mathbf{t}^\wedge \mathbf{R} \\ \mathbf{0} & \mathbf{R} \end{bmatrix}. \quad (2.30)$$

伴随矩阵的一个用途，是把已知变换两侧的扰动互换：

$$T \text{Exp}(\boldsymbol{\xi}) = \text{Exp}(\text{Ad}(T) \boldsymbol{\xi}) T. \quad (2.31)$$

这一等式不需要任何近似。

2.0.6.4 雅可比

每个 Lie 群都有相应的雅可比，用于联系群元素的变化与其 Lie 代数元素。以 $SO(d)$ 为例，旋转矩阵 R_a^b 与角速度 $\boldsymbol{\omega}^b \in \mathbb{R}^{2d-3}$ 之间的运动学方程（Poisson 方程）为

$$\dot{R}_a^b = \boldsymbol{\omega}^{b^\wedge} R_a^b. \quad (2.32)$$

这里 ω^b 表示坐标系 $\mathcal{F}^a$ 相对于 $\mathcal{F}^b$ 的角速度，并在 $\mathcal{F}^b$ 中表达。若将 R_a^b 参数化为 $\exp(\theta)$ ，则等价地有

$$\dot{\theta} = J^{-1}(\theta)\omega^b, \quad (2.33)$$

其中 $J(\theta)$ 是 $SO(d)$ 的（左）雅可比。它在组合矩阵指数乘积时非常有用。例如，当 θ_1 足够小时，

$$\text{Exp}(\theta_1)\text{Exp}(\theta_2) \approx \text{Exp}(\theta_2 + J(\theta_2)^{-1}\theta_1). \quad (2.34)$$

其级数形式为

$$J(\theta) = \sum_{n=0}^{\infty} \frac{1}{(n+1)!} (\theta^\wedge)^n. \quad (2.35)$$

闭式表达式可参阅 Barfoot^[47]。下文也用 $J(\xi)$ 表示 $SE(d)$ 的左雅可比，具体含义由上下文决定。Lie 群雅可比也是利用可微神经网络估计位姿的核心工具（见第 4.2 节）。

2.1 连续时间轨迹

连续时间轨迹可以表示平滑的机器人运动。到目前为止，我们假设需要估计轨迹上的一系列离散位姿；但机器人通常在世界中相当平滑地运动，因此有必要采用更平滑的轨迹表示。连续时间轨迹主要有两类：参数化方法将已知的时间基函数组合成平滑轨迹，通常选择具有局部支撑的基函数（例如分段多项式/样条），从而保持因子图稀疏；非参数化方法利用核函数获得更强的表示能力，具体可以使用以时间为自变量的一维高斯过程（GP）表示轨迹。选择合适且具有物理动机的核函数后，GP 关联的因子图同样可以保持稀疏。

连续时间轨迹不仅能保证平滑性，在处理高频传感器（如 IMU，见第 11.2 节）和/或异步传感器时尤其有用。若每收到一次测量就向因子图加入一个机器人位姿，那么高频传感器或不同传感器的异步采样会迅速产生难以处理的因子图。连续时间方法可以用远少于测量数量的变量表示轨迹，从而保持问题可处理。这对于旋转式激光雷达、雷达以及卷帘快门相机等存在运动畸变的传感器特别有价值，因为可以使用每个点或像素的精确时间戳，将其与该时刻的轨迹关联起来。

完成 MAP 推断后，连续时间轨迹还允许在任意感兴趣时刻查询轨迹，而不仅限于测量时刻；可以进行插值，也可以谨慎地外推。这对 SLAM 输出的下游使用者很有帮助。将测量时刻、估计变量时刻与查询时刻分离，是参数化和非参数化连续时间方法的主要优势。

2.1.1 样条

参数化连续时间轨迹方法的思想，是使用 K 个已知时间基函数 $\Psi_k(t)$ 的加权和表示位姿：

$$p(t) = \sum_{k=1}^K \Psi_k(t) c_k. \quad (2.36)$$

其中 c_k 是未知系数。暂时仍在向量空间中讨论，后面再介绍 Lie 群上的实现。基函数通常选用样条，即分段多项式（例如 B 样条和三次 Hermite 多项式）。样条具有局部支撑，在其

局部影响区域之外取零，因此非常有利。图 2.3 展示了这一设置：在任意时刻最多只有 4 个基函数非零，由此得到稀疏因子图。

与前面的离散时间方法相比，主要区别是使用系数变量而不是位姿变量；但这与一般因子图方法完全兼容。当在时刻 t_i 观测到路标 ℓ 时，传感器模型为

$$z_i = h_i(p(t_i), \ell) + \eta_i. \quad (2.37)$$

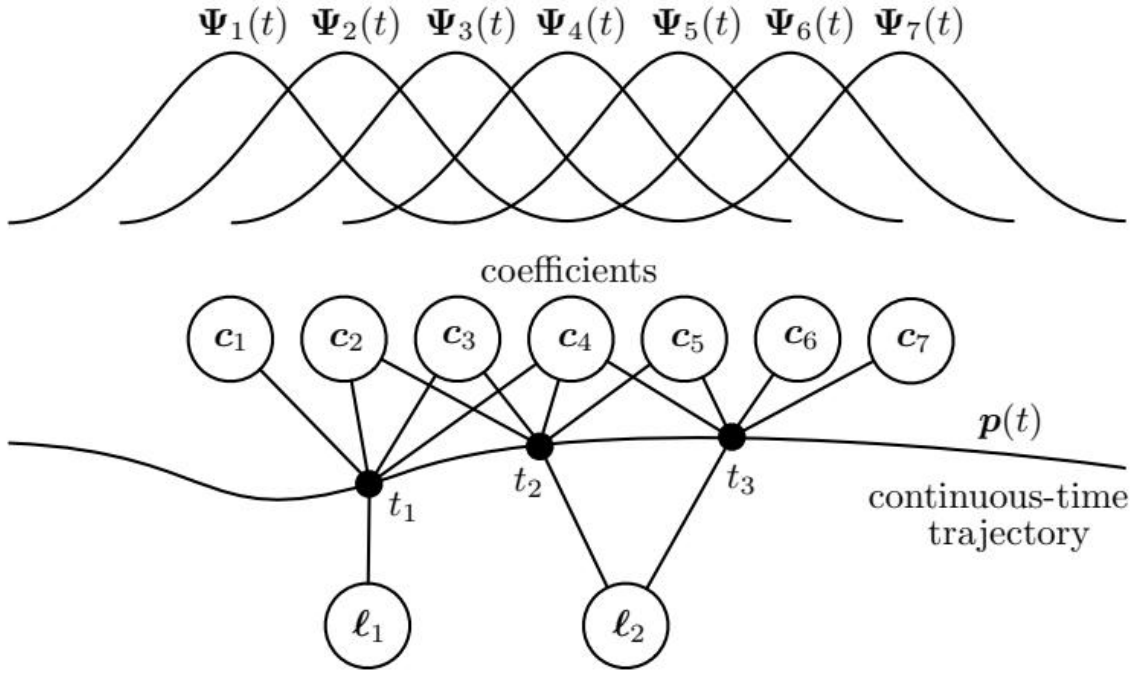


图 2.3: 图 2.3 用参数化样条表示连续时间轨迹。任意时刻的位姿由具有局部支撑的已知基函数加权求和得到；每个路标测量对应一个连接 4 个系数变量和 1 个路标变量的五元因子，整体因子图仍然非常稀疏。

代入式 (2.36)，有

$$z_i = h_i \left(\sum_{k=1}^K \Psi_k(t_i) c_k, \ell \right) + \eta_i. \quad (2.38)$$

如果基函数具有局部支撑，则在 t_i 处只有一小部分系数处于激活状态。令 $\mathbf{x}_i = [\mathbf{c}_i^\top \ell^\top]^\top$ 表示 t_i 处激活的系数变量以及路标变量，就重新得到

$$z_i = h_i(\mathbf{x}_i) + \eta_i. \quad (2.39)$$

于是可以使用一般方法构造非线性最小二乘问题并进行优化。

如果基函数足够光滑，还可以方便地求取轨迹导数：

$$\dot{\mathbf{p}}(t) = \sum_{k=1}^K \dot{\Psi}_k(t) c_k. \quad (2.40)$$

这样就能处理依赖速度甚至更高阶导数的传感器输出，而优化变量仍是同一组系数；只需计算基函数的相应导数即可。

通过 MAP 推断求得最优系数后，可以利用式 (2.36) 或式 (2.40) 在任意时刻查询轨迹及其导数。如果在推断过程中计算了系数估计的协方差（例如通过求信息矩阵的逆），还可以轻松将其映射为查询位姿或导数的协方差，因为式 (2.36) 和式 (2.40) 是线性关系；而基函数的局部支撑意味着只需使用相应的系数边缘协方差。

2.1.2 从参数化到非参数化

基本参数化连续时间方法的主要挑战，是必须决定基函数的类型和数量。基函数太多，容易对测量数据过拟合；太少，又可能没有足够能力表示真实轨迹形状，从而得到过度平滑的解。转向非参数化方法可以部分解决这一问题。

为简化说明，本节暂时假设没有路标变量，只关注位姿变量。采用上一节的参数化方法，线性化最小二乘项（负对数因子）为

$$\|(\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i^0)) - \mathbf{H}_i \Psi_i \delta_{c,i}\|_{\Sigma_i}^2. \quad (2.41)$$

其中， $\mathbf{x}_i^0$ 是当前解（激活的系数）， $\delta_{c,i}$ 是激活系数的更新量， Ψ_i 是在 t_i 处求值并堆叠后的激活基函数，雅可比为

$$\mathbf{H}_i = \left. \frac{\partial \mathbf{h}_i}{\partial \mathbf{x}} \right|_{\mathbf{x}_i^0}. \quad (2.42)$$

像以前一样收集各项，可以写成

$$\delta_c^* = \arg \min_{\delta_c} (\|\mathbf{b} - \mathbf{A} \Psi \delta_c\|^2 + \|\delta_c\|^2). \quad (2.43)$$

这里加入了正则项 $\|\delta_c\|^2$ ，倾向于使解的描述长度保持合理，即偏好接近零的样条系数，从而帮助避免过拟合。最优解满足

$$(\Psi^\top \mathbf{A}^\top \mathbf{A} \Psi + \mathbf{I}) \delta_c^* = \Psi^\top \mathbf{A}^\top \mathbf{b}. \quad (2.44)$$

这可以求得系数的最优更新 δ_c^* 。但我们真正关心的是位姿估计，而不是作为中间量的样条系数。测量时刻位姿变量的最优更新为 $\delta^* = \Psi \delta_c^*$ 。经过简单代数运算，可得

$$(\mathbf{A}^\top \mathbf{A} + \mathbf{K}^{-1}) \delta^* = \mathbf{A}^\top \mathbf{b}, \quad (2.45)$$

这是对式 (1.25) 正规方程的修改版本。核矩阵 $\mathbf{K} = \Psi \Psi^\top$ 起到正则化或平滑函数的作用。式 (2.45) 看起来比式 (2.44) 包含更大的线性系统，因为位姿数量多于样条系数。不过，非参数方法可以利用内置的插值能力，最终减少实际需要求解的系统规模。

为了摆脱显式基函数，可以使用所谓的核技巧，用所选核函数 $\mathcal{K}(t, t')$ （例如平方指数核）的计算结果替代基函数的显式内积。构造核矩阵只需要计算基函数内积，因此令 $\mathbf{K} = [\mathcal{K}(t_i, t_j)]_{ij}$ 即可。此时不再估计样条的系数（参数），因而称为非参数方法；但仍需调节所选核函数的超参数（例如平方指数核的长度尺度），以实现期望的轨迹平滑程度。由于式 (2.45) 需要核矩阵的逆，直接这样做似乎代价很高；下一节将说明如何选择核函数，使逆核矩阵非常稀疏。

2.1.3 高斯过程

下面构造一族经过专门设计的核函数，使逆核矩阵及其对应因子图保持稀疏。上一节说明，可以用核函数替换基函数，从而得到非参数连续时间方法；但若直接进行替换，逆核矩阵可能变得稠密。这里从另一条思路出发：图 2.4 给出了一个由本节思想得到的因子图示例，它既保持稀疏，又产生平滑轨迹。

首先选择一个由白噪声驱动的线性时不变随机微分方程 (SDE)：²

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{L}\mathbf{w}(t), \quad (2.46)$$

其中 $\mathbf{w}(t) = \mathcal{GP}(\mathbf{0}, \mathbf{Q}\delta(t-t'))$ 是零均值白噪声高斯过程， $\mathbf{Q}$ 是功率谱密度矩阵， $\delta(\cdot)$ 是 Dirac delta 函数。该 SDE 将充当运动先验。它可以写出闭式积分解：

$$\mathbf{x}(t) = \Phi(t, t_1)\mathbf{x}(t_1) + \int_{t_1}^t \Phi(t, s)\mathbf{L}\mathbf{w}(s)ds, \quad (2.47)$$

其中 $\Phi(t, s) = \exp(\mathbf{A}(t-s))$ 称为转移函数， t_1 是第一次测量的时间戳。若初始状态均值为零，则后续状态均值也保持为零；状态的协方差函数（即核函数）为

$$\mathcal{K}(t, t') = \Phi(t, t_1)\mathcal{K}(t_1, t_1)\Phi(t', t_1)^\top + \int_{t_1}^{\min(t, t')} \Phi(t, s)\mathbf{L}\mathbf{Q}\mathbf{L}^\top\Phi(t', s)^\top ds. \quad (2.48)$$

在全部测量时刻计算核函数并构造核矩阵时，可以利用简洁关系

$$\mathbf{K} = \Phi\mathbf{Q}\Phi^\top, \quad (2.49)$$

其中 $\mathbf{Q} = \text{diag}(\mathcal{K}(t_1, t_1), Q_1, \dots, Q_M)$ ，且

$$\Phi = \begin{bmatrix} \mathbf{I} & & & & \\ \Phi(t_2, t_1) & \mathbf{I} & & & \\ \Phi(t_3, t_1) & \Phi(t_3, t_2) & \mathbf{I} & & \\ \vdots & \vdots & \vdots & \ddots & \\ \Phi(t_M, t_1) & \Phi(t_M, t_2) & \cdots & \Phi(t_M, t_{M-1}) & \mathbf{I} \end{bmatrix}. \quad (2.50)$$

其中

$$Q_i = \int_{t_{i-1}}^{t_i} \Phi(t_i, s)\mathbf{L}\mathbf{Q}\mathbf{L}^\top\Phi(t_i, s)^\top ds.$$

矩阵 Φ 是由各时间点之间的转移函数构成的下三角块矩阵，其对角块为单位阵。由于我们需要的是逆核矩阵， $\mathbf{K}^{-1} = \Phi^{-\top}\mathbf{Q}^{-1}\Phi^{-1}$ ；而 $\mathbf{Q}$ 是块对角矩阵，逆可以逐个对角块计算。更重要的是， Φ^{-1} 只有主块对角线和紧邻其下的一个块对角线非零：

$$\Phi^{-1} = \begin{bmatrix} \mathbf{I} & & & & \\ -\Phi(t_2, t_1) & \mathbf{I} & & & \\ & -\Phi(t_3, t_2) & \mathbf{I} & & \\ & & -\Phi(t_4, t_3) & \ddots & \\ & & & \ddots & \mathbf{I} \\ & & & & -\Phi(t_M, t_{M-1}) & \mathbf{I} \end{bmatrix}. \quad (2.51)$$

因此，任意长度轨迹的逆核矩阵 K^{-1} 都是块三对角的。根据前面对因子图的讨论，式 (2.45) 左端的稀疏性与因子图结构密切相关；这里 K^{-1} 作为整条轨迹的运动先验，可以用非常稀疏的因子图表示。

K^{-1} 之所以具有这种稀疏因子图，是因为出发点 SDE 的状态具有马尔可夫性。实际含义是：根据式 (2.46) 想表达的运动先验，状态可能需要包含更高阶量，而不只是位姿，还包括其导数。例如，若希望采用“恒速”先验，可以选择

$$\underbrace{\begin{bmatrix} \dot{\mathbf{p}}(t) \\ \dot{\mathbf{v}}(t) \end{bmatrix}}_{\dot{\mathbf{x}}(t)} = \underbrace{\begin{bmatrix} \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} \mathbf{p}(t) \\ \mathbf{v}(t) \end{bmatrix}}_{\mathbf{x}(t)} + \underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{I} \end{bmatrix}}_{\mathbf{L}} \mathbf{w}(t). \quad (2.52)$$

这里的状态包含位姿及其导数。这种使用增广状态的形式有时称为同时轨迹估计与建图 (STEAM)，是 SLAM 的一种变体。

上述连续时间轨迹估计实际上是高斯过程回归^[909]的一个例子。建立这一联系后，在测量时刻求解状态，就可以利用 GP 插值在其他时刻查询轨迹的均值和协方差。采用稀疏核方法时，每次查询的代价相对于测量数量 M 为常数，因为只涉及夹住查询时刻的两个估计状态。

还可以利用 GP 插值减少所需控制点的数量，不必在每个测量时刻都设置控制点，这类似于 GP 的诱导点思想。例如，每次激光扫描设置一个控制点，但仍使用扫描过程中每个点的独立时间戳。这样，测量时刻、估计时刻和查询时刻可以彼此不同。GP 的核还提供了适当的正则化，因此无需担心设置过多估计时刻导致过拟合，但仍需有足够的估计时刻来捕捉轨迹细节。

2.1.4 Lie 群上的样条与高斯过程

当状态位于流形上时，同样可以使用样条和 GP 连续时间方法。若流形是 Lie 群，两种方法都利用 Lie 代数，但方式不同。下面先介绍样条，再介绍高斯过程。

2.1.4.1 Lie 群上的样条

使样条适用于 Lie 群的关键，是采用累积形式。对于向量空间，假设所有自由度使用相同基函数，式 (2.36) 可简化为

$$\mathbf{p}(t) = \sum_{k=1}^K \psi_k(t) \mathbf{p}_k, \quad (2.53)$$

其中 $\mathbf{p}_k$ 是样条控制点, $\psi_k(t)$ 是标量基函数。将其改写为累积形式:

$$\mathbf{p}(t) = \psi_1^c(t)\mathbf{p}_1 + \sum_{k=2}^K \psi_k^c(t)(\mathbf{p}_k - \mathbf{p}_{k-1}), \quad (2.54)$$

其中累积基函数为

$$\psi_k^c(t) = \sum_{\ell=k}^K \psi_\ell(t). \quad (2.55)$$

例如, 对时间间隔均匀为 T 的线性插值, 定义 $\alpha_k(t) = (t - (k-1)T)/T$, 普通基函数为

$$\psi_1(t) = \begin{cases} 1 - \alpha_1(t), & 0 \leq t < T \\ 0, & \text{其他情况,} \end{cases} \quad \psi_K(t) = \begin{cases} \alpha_K(t), & (K-1)T \leq t < KT \\ 0, & \text{其他情况,} \end{cases} \quad (2.56)$$

而对 $k = 2, \dots, K-1$,

$$\psi_k(t) = \begin{cases} \alpha_{k-1}(t), & (k-2)T \leq t < (k-1)T \\ 1 - \alpha_k(t), & (k-1)T \leq t < kT \\ 0, & \text{其他情况.} \end{cases} \quad (2.57)$$

对应的累积基函数为

$$\psi_1^c(t) = 1, \quad \psi_K^c(t) = \begin{cases} 0, & t < (K-1)T \\ \alpha_K(t), & (K-1)T \leq t, \end{cases} \quad (2.58)$$

以及对 $k = 2, \dots, K-1$,

$$\psi_k^c(t) = \begin{cases} 0, & t < (k-1)T \\ \alpha_k(t), & (k-1)T \leq t < kT \\ 1, & kT \leq t. \end{cases} \quad (2.59)$$

图 2.5 展示了这些基函数的形状。

累积基函数的主要优点是: 在给定时间戳处, 大多数基函数都不激活。对于线性样条, 当 $(k-1)T \leq t < kT$ 时,

$$\mathbf{p}(t) = \mathbf{p}_{k-1} + \psi_k^c(t)(\mathbf{p}_k - \mathbf{p}_{k-1}). \quad (2.60)$$

此时只需计算一个基函数; 更高阶样条在任意时刻也只会少量激活基函数。

在 Lie 群上使用样条时, 用 Lie 群运算 (矩阵乘法) 替代累积表达式中的加法。以线性样条为例, 在 $(k-1)T \leq t < kT$ 时, $\text{SE}(d)$ 中的元素可以写成

$$\mathbf{T}(t) = \text{Exp}(\psi_k^c(t)\text{Log}(\mathbf{T}_k\mathbf{T}_{k-1}^{-1}))\mathbf{T}_{k-1}. \quad (2.61)$$

这里选择了左侧扰动, 当然也可以采用右侧扰动。将 $\mathbf{T}(t_i)$ 代入任意测量表达式, 在控制点 $\mathbf{T}_k$ 的 Lie 代数扰动上进行线性化, 即可用于 MAP 框架。由于基函数具有紧支撑, 在一个测量时刻只有少量控制点激活。

对于线性样条，式 (2.61) 可写为

$$\mathbf{T}(t) = (\mathbf{T}_k \mathbf{T}_{k-1}^{-1})^{\alpha_k(t)} \mathbf{T}_{k-1}. \quad (2.62)$$

对 $\mathbf{T}(t)$ 中的表达式进行线性化时，分别扰动两个控制位姿，使

$$\text{Exp}(\boldsymbol{\xi}(t)) \mathbf{T}^0(t) = \left(\text{Exp}(\boldsymbol{\xi}_k) \mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \text{Exp}(-\boldsymbol{\xi}_{k-1}) \right)^{\alpha_k(t)} \text{Exp}(\boldsymbol{\xi}_{k-1}) \mathbf{T}_{k-1}^0. \quad (2.63)$$

插值位姿扰动 $\boldsymbol{\xi}(t)$ 与两个控制点扰动 $\boldsymbol{\xi}_k$ 、 $\boldsymbol{\xi}_{k-1}$ 的关系可近似为

$$\boldsymbol{\xi}(t) \approx (\mathbf{I} - \mathbf{A}(\alpha_k(t))) \boldsymbol{\xi}_{k-1} + \mathbf{A}(\alpha_k(t)) \boldsymbol{\xi}_k, \quad (2.64)$$

其中

$$\mathbf{A}(\alpha_k(t)) = \alpha_k(t) \mathbf{J} \left(\alpha_k(t) \mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \right) \mathbf{J} \left(\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \right)^{-1}. \quad (2.65)$$

这里 $\mathbf{J}(\cdot)$ 是 $\text{SE}(d)$ 的左雅可比。利用式 (2.64)，即可将测量时刻位姿的变化与两侧控制点位姿关联起来，形成用于 MAP 估计的线性化误差。例如，式 (2.16) 的线性化测量模型可写成

$$\mathbf{e}_i(t) \approx \mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}^0(t) \tilde{\ell}_i \right) - \mathbf{H}_i \boldsymbol{\xi}(t). \quad (2.66)$$

将式 (2.64) 代入即可得到关于两侧控制点的线性化误差：

$$\mathbf{e}_i(t) \approx \mathbf{z}_i - \mathbf{h}_i \left(\mathbf{T}^0(t) \tilde{\ell}_i \right) - \mathbf{H}_i (\mathbf{I} - \mathbf{A}(\alpha_k(t))) \boldsymbol{\xi}_{k-1} - \mathbf{H}_i \mathbf{A}(\alpha_k(t)) \boldsymbol{\xi}_k. \quad (2.67)$$

其中还需将 t 时刻的名义位姿 $\mathbf{T}^0(t) = (\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}})^{\alpha_k(t)} \mathbf{T}_{k-1}^0$ 代入 $\mathbf{h}_i$ 和 $\mathbf{H}_i$ 。这相当于沿线性样条链式传播导数；高阶样条也可以采用同样过程。

2.1.4.2 Lie 群上的高斯过程

在 Lie 群上使用高斯过程时，同样利用 Lie 代数。图 2.6 展示了相应的 GP 运动先验因子。根据所选运动先验，控制点状态也可以包含额外的轨迹导数。

我们在一组控制点状态之间使用局部 GP，这与样条类似。对 $\text{SE}(d)$ ，定义控制点之间的局部变量 $\boldsymbol{\xi}_k(t)$ 。例如，对 $\text{SE}(d)$ 的“随机游走”先验，可以选择

$$\dot{\boldsymbol{\xi}}_k(t) = \mathbf{w}(t), \quad \mathbf{w}(t) = \mathcal{GP}(\mathbf{0}, \mathbf{Q} \delta(t - t')), \quad (2.68)$$

这里的变量定义在控制点 $\mathbf{T}_k$ 和 $\mathbf{T}_{k+1}$ 之间。该 SDE 的转移函数为单位阵，随机积分得到

$$\boldsymbol{\xi}_k(t) = \underbrace{\boldsymbol{\xi}_k(t_k)}_{\mathbf{0}} + \int_{t_k}^t \mathbf{w}(s) ds, \quad (2.69)$$

于是运动先验为

$$\boldsymbol{\xi}_k(t) \sim \mathcal{GP}(\mathbf{0}, \min(t, t') \mathbf{Q}). \quad (2.70)$$

若控制点每隔 T 秒均匀分布, 则逆核矩阵为 $\kappa^{-1} = \Phi^{-\top} Q^{-1} \Phi^{-1}$, 其中

$$\Phi^{-1} = \begin{bmatrix} \mathbf{I} & & & & \\ -\mathbf{I} & \mathbf{I} & & & \\ & \ddots & \ddots & & \\ & & & -\mathbf{I} & \mathbf{I} \end{bmatrix}, \quad Q = \text{diag}(\mathcal{K}(t_1, t_1), T\mathcal{Q}, \dots, T\mathcal{Q}). \quad (2.71)$$

局部变量下的误差为

$$e_k = \begin{cases} \text{Log}(\mathbf{T}_1 \check{\mathbf{T}}_1^{-1}) & k = 1 \\ \boldsymbol{\xi}_{k-1}(t_k) - \boldsymbol{\xi}_{k-1}(t_{k-1}) & k > 1 \end{cases}, \quad (2.72)$$

其中 $\check{\mathbf{T}}_1$ 是初始位姿先验。用全局变量表示时, 同一误差为

$$e_k = \begin{cases} \text{Log}(\mathbf{T}_1 \check{\mathbf{T}}_1^{-1}) & k = 1 \\ \text{Log}(\mathbf{T}_k \mathbf{T}_{k-1}^{-1}) & k > 1 \end{cases}. \quad (2.73)$$

图 2.6 给出了这种随机游走 GP 运动先验的因子图。与线性样条类似, 若要在其他时刻查询轨迹, 可以使用 GP 插值; 对于随机游走先验, 结果再次是线性插值:

$$\mathbf{T}(t) = (\mathbf{T}_k \mathbf{T}_{k-1}^{-1})^{\alpha_k(t)} \mathbf{T}_{k-1}, \quad (2.74)$$

其中 $\alpha_k(t) = (t - (k-1)T)/T$, 且 $(k-1)T \leq t < kT$ 。与样条方法不同, 这里的线性插值是 SDE 选择的间接结果, 而不是显式指定的形式; 如果一开始选择高阶 SDE, 就会得到高阶插值样条。

最后, 需要将误差项线性化以用于 MAP 估计。再次利用 Lie 群扰动方法, 对于式 (2.73) 的第二种情形, 有

$$\begin{aligned} e_k &= \text{Log}(\text{Exp}(\boldsymbol{\xi}_k) \mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}} \text{Exp}(-\boldsymbol{\xi}_{k-1})) \\ &\approx \text{Log}(\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}}) + \boldsymbol{\xi}_k - \text{Ad}(\mathbf{T}_k^0 \mathbf{T}_{k-1}^{0^{-1}}) \boldsymbol{\xi}_{k-1}, \end{aligned} \quad (2.75)$$

其中 $\mathbf{T}_k^0$ 和 $\mathbf{T}_{k-1}^0$ 是当前猜测, $\boldsymbol{\xi}_k$ 和 $\boldsymbol{\xi}_{k-1}$ 是待求扰动, $\text{Ad}(\cdot)$ 是 $\text{SE}(d)$ 的伴随表示。式 (2.75) 的线性化形式可以在每次迭代中直接纳入标准 MAP 估计框架。

此外, 若想利用式 (2.74) 减少随机游走示例中的控制点数量, 可以采用式 (2.67) 中针对线性样条的同一方法, 因为两者最终都归结为 Lie 群控制点之间的线性插值。样条与 GP 的根本区别在于: GP 方法使用运动先验项 (见图 2.6) 对问题进行正则化, 而样条方法不使用这种先验。⁴

2.2 延伸阅读与近期趋势

关于流形, 尤其是 Lie 群上的估计, 已有大量研究。机器人领域的奠基性参考是 Chirikjian^[205] 及 Chirikjian 和 Kyatkin^[207]; 这些著作理论基础扎实, 并给出了处理流形不确定性的通用工具, 包括全局不确定性。关于流形优化, Boumal^[106] 是面向机器人应用的优秀参考。Barfoot^[47] 更详细地讨论了不确定性仍较为集中的 Lie 群估计, 这与本章内容相近。连续时

间估计方面, Talbot 等人^[1067]提供了参数化和非参数化方法的综合综述; Barfoot^[47]也对非参数方法作了较为详细的说明。

与上一章类似, 离群点处理、使算法可微, 以及利用本章工具支持不同类型的传感器等近期趋势, 将在后续章节中进一步讨论。

第三章 对错误数据关联和离群值的鲁棒性

作者： Heng Yang、Josh Mangelson、Yun Chang、Jingnan Shi、Niko Sunderhauf、Luca Carlone

在第 1 章中，我们已经看到，因子图是用于建模和可视化 SLAM 问题的强大表示；给定一组测量（例如里程计和路标测量）后，最大后验（maximum a posteriori, MAP）估计为推断感兴趣变量（例如机器人位姿和路标位置）提供了一个有理论依据且通用的框架。例如，我们看到，当测量 z_i 受协方差为 Σ_i 的加性零均值高斯噪声影响时，MAP 估计会导出一个非线性最小二乘优化问题：

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \|\mathbf{z}_i - \mathbf{h}_i(\mathbf{x}_i)\|_{\Sigma_i}^2, \quad (3.1)$$

其中， $\mathbf{x}_i$ 表示参与第 i 个测量的状态子集。¹ 本章指出，在实践中，许多测量 z_i 可能由于错误的数据关联而具有很大的误差，远不符合零均值高斯分布（第 3.1 节）；这些测量通常会使得式 (3.1) 中的估计 $\mathbf{x}^{\text{MAP}}$ 产生很大扰动。因此，我们将讨论如何在 SLAM 前端剔除严重离群值（第 3.2 节），然后重点讨论如何提高 SLAM 后端对剩余离群值的鲁棒性（第 3.3 节）。最后，我们将简要回顾近期趋势，并给出相关工作的进一步线索（第 3.4 节）。

3.1 离群值的成因及其危害

本节论证：离群值在大多数 SLAM 应用中不可避免；若未得到恰当处理，则会导致严重错误的估计。

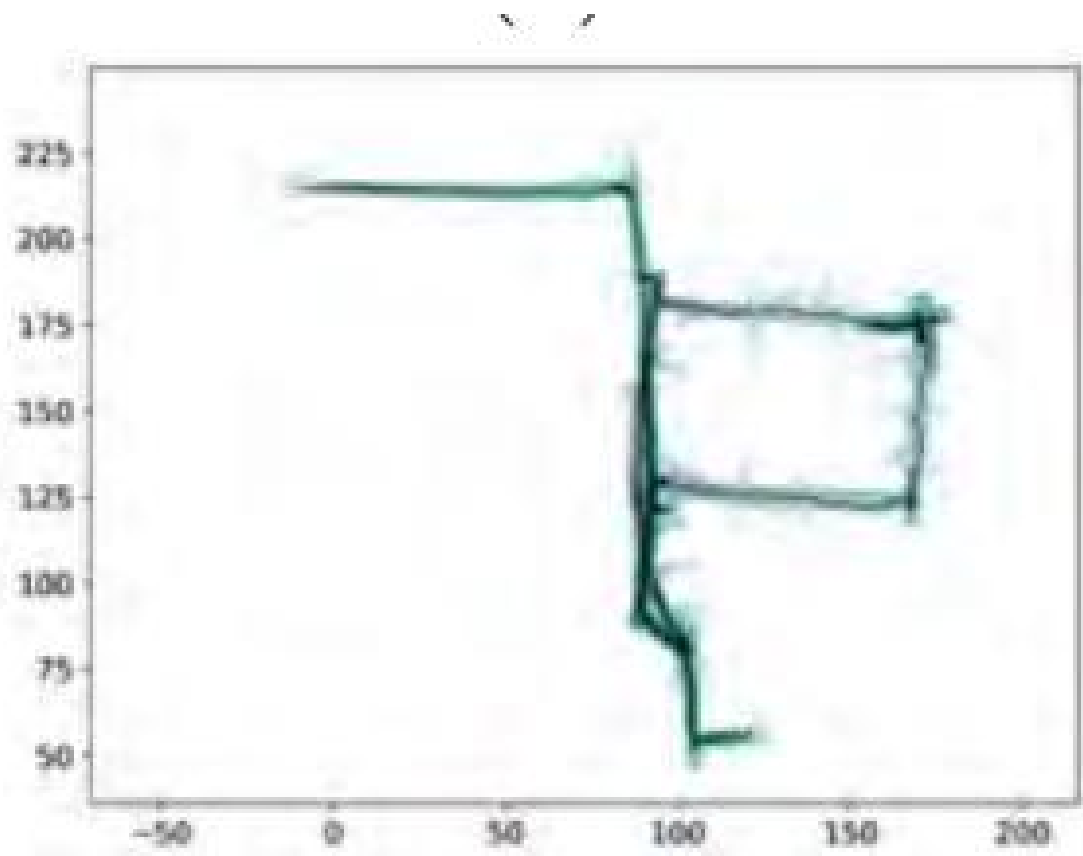
3.2 数据关联与离群值

为了解离群测量的成因，先考虑两个例子。首先，考虑一个基于路标的 SLAM 问题：我们需要根据里程计测量以及若干机器人位姿对路标的相对观测，重建机器人的轨迹和外部路标的位置。若如第 1 章那样假设路标测量具有零均值高斯噪声，则优化中会出现形如 $\|z_{ij} - h(p_i, \ell_j)\|_{\Sigma}^2$ 的项。这些项刻画了如下事实：给定测量 z_{ij} 是从位姿 p_i 对路标 ℓ_j 的观测，并带有高斯噪声，其中 $h(\cdot)$ 是描述相对测量类型（例如距离、方位等）的函数。实践中， z_{ij} 是由 SLAM 前端对原始传感器数据预处理得到的。例如，若机器人装有相机，且 z_{ij} 是对路标 ℓ_j 方位的视觉观测，则可先在图像中进行目标（或更一般地说，特征）检测与匹配，再计算与检测到的像素对应的方位，从而提取该测量。问题在于检测并不完美：图像中被检测为 ℓ_j 的路标，在现实中可能实际是另一个路标。这会使 z_{ij} 大幅偏离所假设的模型。将一个测量关联到

某个路标的问题通常称为**数据关联问题**，它也广泛存在于其他估计问题中（例如目标跟踪）。因此，错误的数据关联会在估计问题中产生离群值。

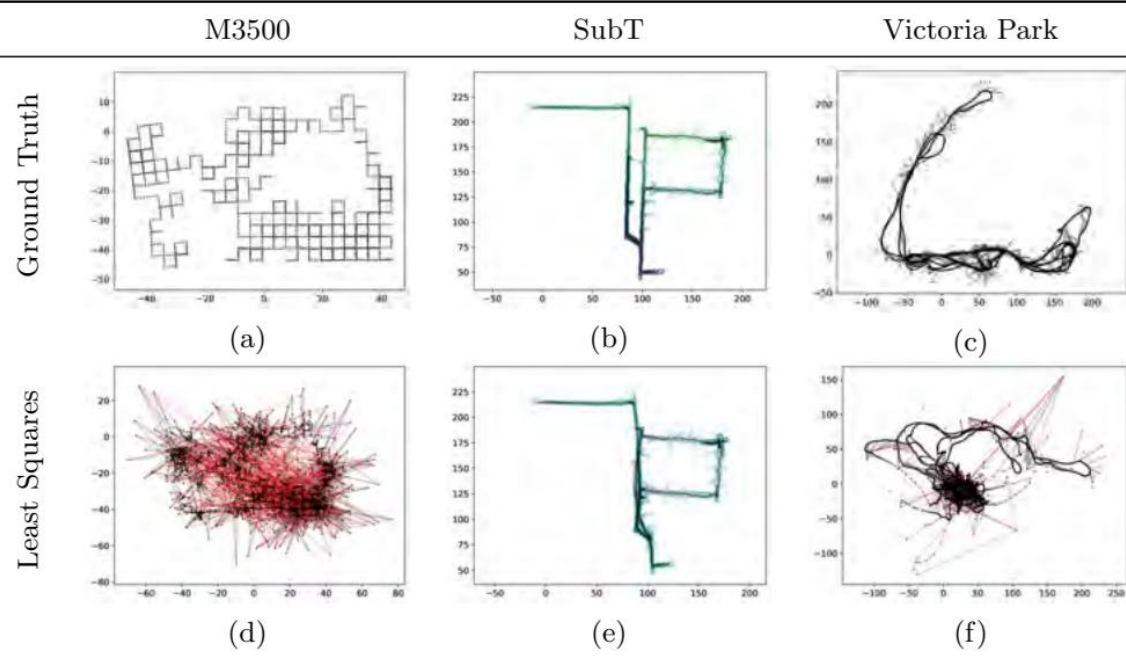
作为第二个例子，考虑位姿图优化问题。此时我们主要关心机器人的轨迹估计（由一组位姿表示），测量则是里程计测量（关联轨迹中相邻的位姿）或回环约束（关联非相邻、且可能在时间上相距很远的位姿）。实践中，回环由基于视觉或 LiDAR 的地点识别方法检测；这些方法负责判定一对位姿 p_i 和 p_j 是否观测到了环境中的同一部分。遗憾的是，当前的地点识别方法容易出错，会在没有观测到同一场景的位姿之间检测出回环。这部分源于现有方法的局限性，但更多时候源于**感知混叠**（perceptual aliasing）：两个外观相似的位置实际上对应不同地点，例如大学建筑中的两个教室，或办公环境中外观相似的隔间。这同样可理解为数据关联失败：我们错误地将回环测量关联到了两个选错的机器人位姿。

需要注意，离群值不仅由错误数据关联造成，也可能由 SLAM 方法中所作假设被违背而引起。例如，大多数 SLAM 方法假设路标是静态的；因此，对运动物体的检测，即使正确地关联到该物体，也可能产生具有大残差的离群测量。类似地，传感器故障和性能退化，例如故障的轮式编码器或相机镜头上的灰尘，也可能导致测量中出现离群值。



3.3 存在离群值时的最小二乘

存在离群值时，最小二乘形式 (3.1) 得到的估计可能严重错误。从理论角度看，我们为测量所假设的高斯噪声是“轻尾”（light-tailed）的，即它实质上排除了误差极大的测量。



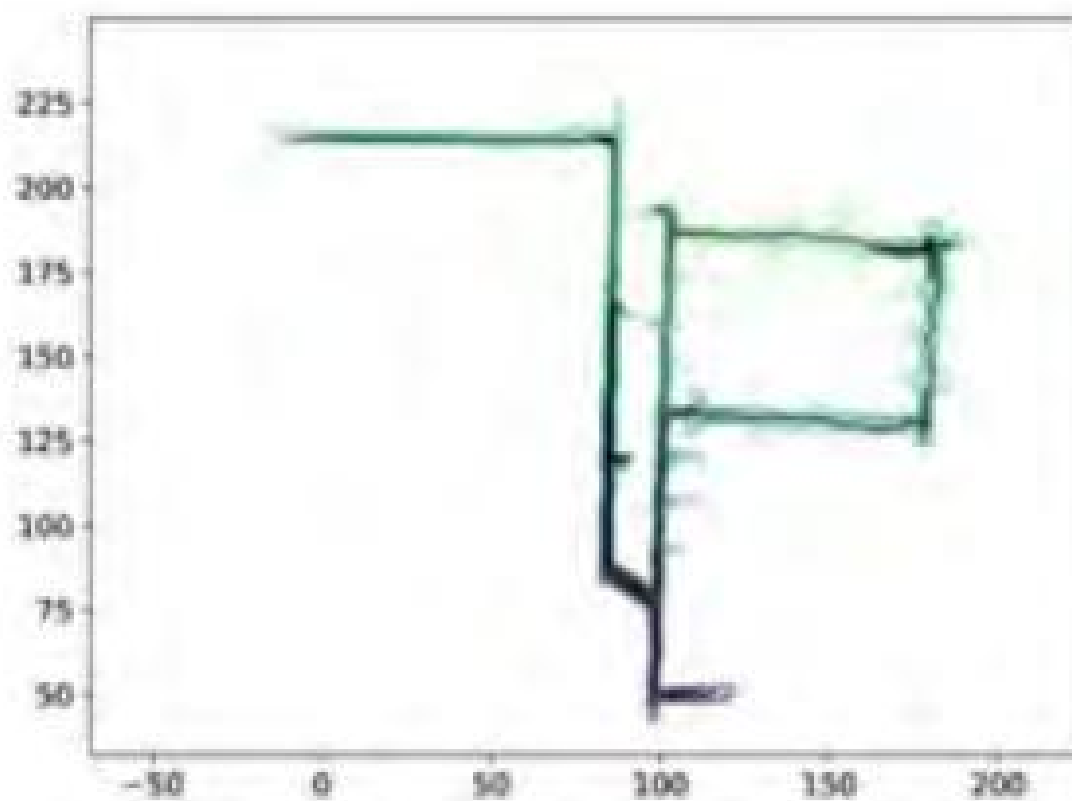
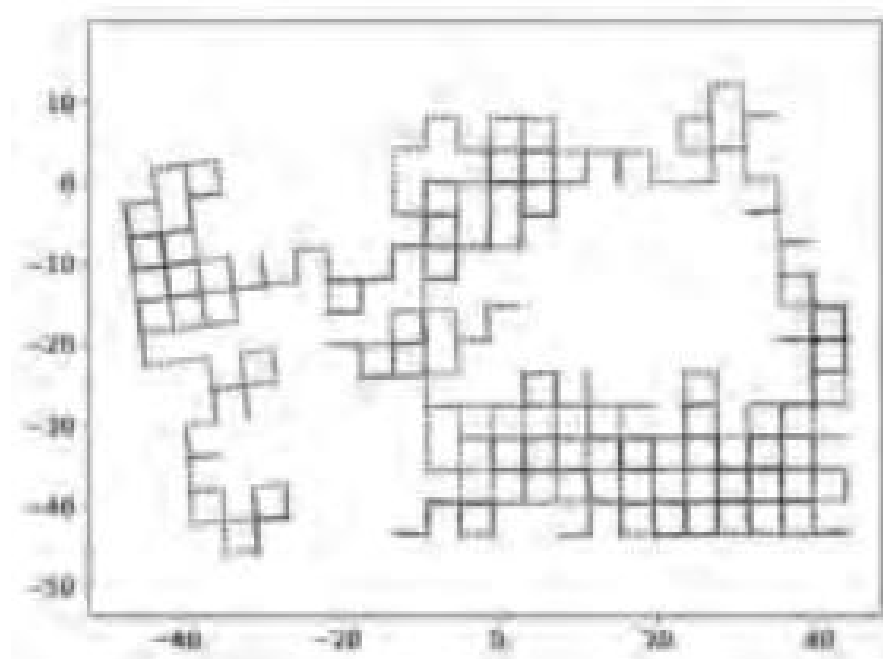
从更实际的角度看，离群值会在目标函数中产生一些项：当在真值附近求值时，其残差误差 $r_i(\mathbf{x}) := \|z_{ij} - h(\mathbf{p}_i, \ell_j)\|_\Sigma$ 非常大。由于优化目标将残差平方，即目标为 $\sum_i r_i(\mathbf{x})^2$ ，这些残差会对代价产生不成比例的巨大影响，优化过程会着重减小由离群值引起的大项，而非充分利用其余的内点 (inlier) 测量。

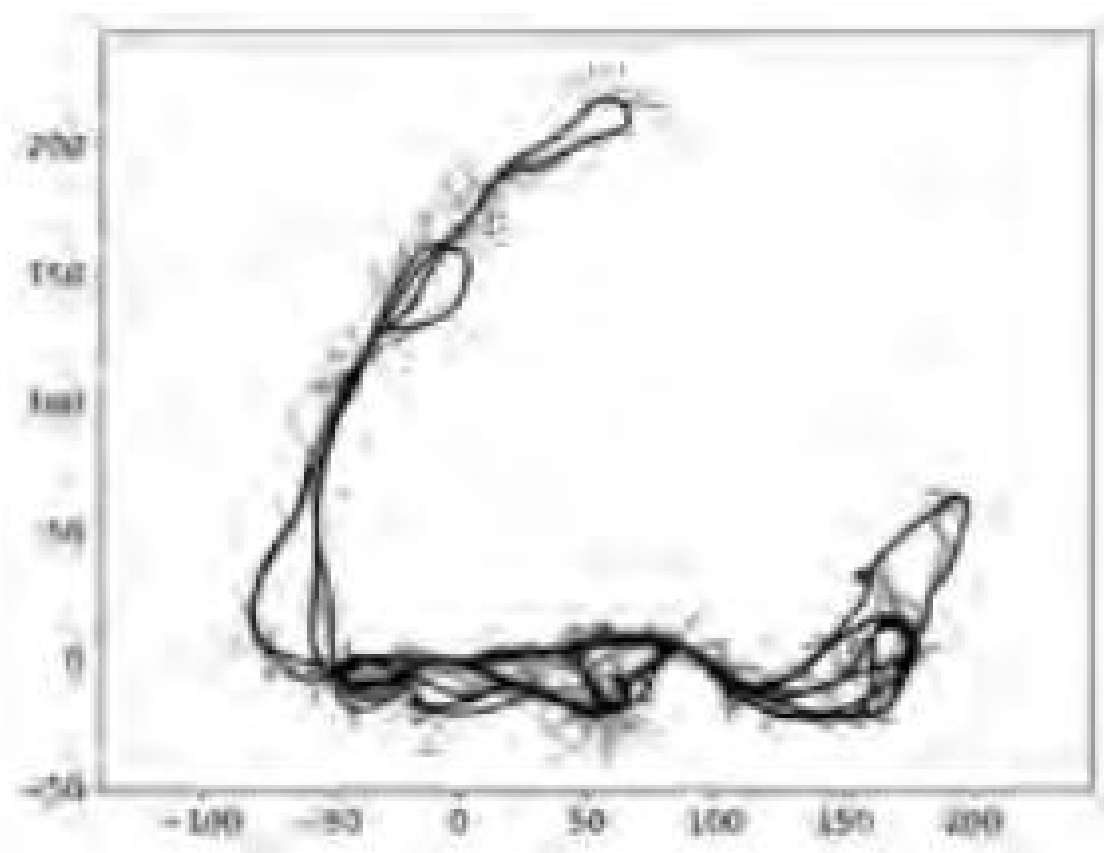
为说明这一点，图 3.1 给出了三个含离群值 SLAM 问题的结果。第一列是一个称为 M3500 的模拟位姿图优化基准，其中位姿按网格状构型排列；数据集包含 3500 个二维位姿和 8953 个测量。第二列是一个称为 SubT 的真实世界位姿图数据集，采集于 DARPA 地下挑战赛期间的一条隧道中^[299]；该数据集包含 682 个三维位姿和 3278 个测量。第三列是一个称为 Victoria Park 的真实世界基于路标的 SLAM 数据集^[809]；该数据集包含 7120 个二维位姿和路标，以及 17728 个测量。图 3.1(a)–(c) 展示这三个问题的真值轨迹。图 3.1(d)–(f) 展示存在离群值时由最小二乘形式产生的估计。具体地，对 M3500 和 Victoria Park，我们向（回环或路标）测量中加入 15% 的随机离群值；SubT 数据集本身已包含离群值。图中以红色显示离群测量。可以看到，离群值会导致完全错误的轨迹和地图估计。此外，离群值常揭示环境中的感知混叠：例如，SubT 数据集中间两条外观相似的竖直走廊会诱发许多伪回环，误导后端构建出只有一条竖直走廊的地图。

3.4 在 SLAM 前端检测和剔除离群值

SLAM 前端的主要作用，是从原始传感器数据中提取中间表示或（伪）测量，之后这些量将转换为后端的因子。典型的 SLAM 前端先计算一个初始测量集合（其中可能混入许多离群值），再对该初始集合进行后处理以去除离群值。本节讨论两种在 SLAM 前端剔除离群值的方法：RANSAC 和成对一致性最大化 (Pairwise Consistency Maximization)。

Ground Truth





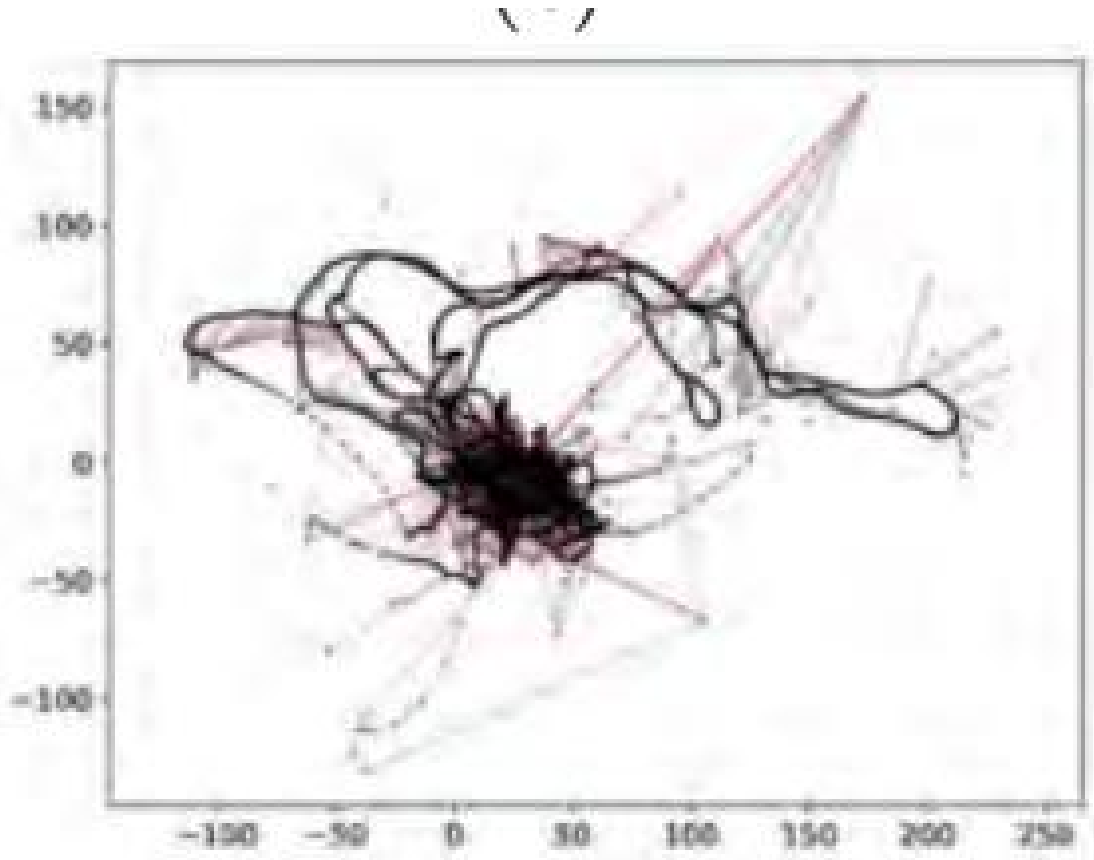


图 3.1: 图 3.1 含离群值的 SLAM 问题: (a)–(c) 分别为 M3500、SubT 和 Victoria Park 数据集的真值轨迹; (d)–(f) 为存在离群值时采用最小二乘形式得到的轨迹估计。内点测量显示为灰色边, 离群值显示为红色边。在 SubT 数据集中, 还展示了由 SLAM 位姿估计构建的稠密地图。

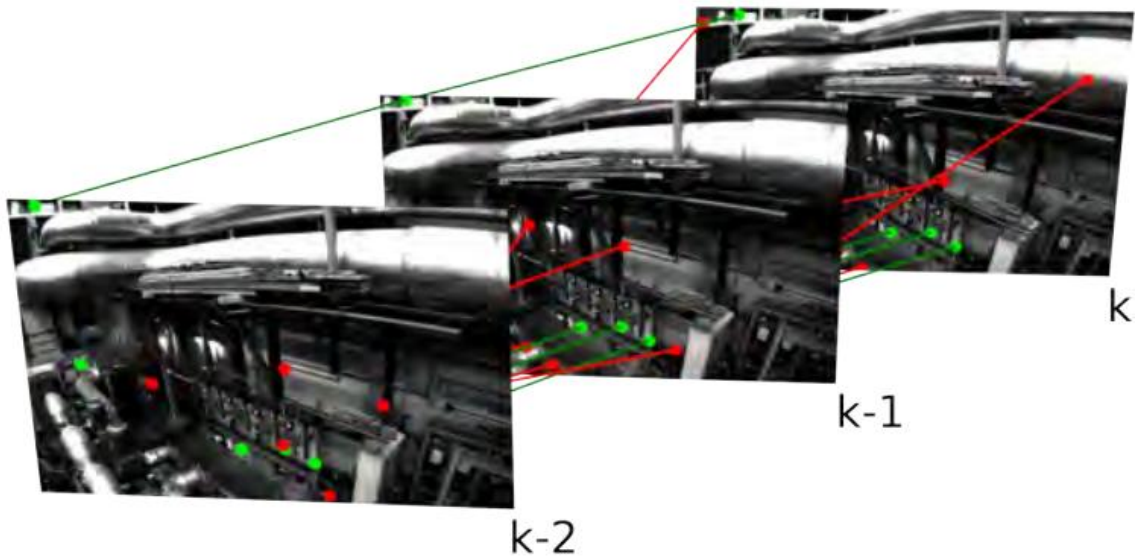


图 3.2: 图 3.2 视觉 SLAM 问题中跨三个帧 (分别采集于时刻 $k-2$ 、 $k-1$ 和 k) 的特征跟踪。内点以绿色显示: 它们是随时间描绘同一 (静态) 三维点的像素。离群值以红色显示。

3.5 随机采样一致性 (RANSAC)

RANSAC 是一种成熟的离群值剔除工具^[328],也是许多基于路标的 SLAM 系统的关键组成部分。为理解 RANSAC 是什么及其在 SLAM 中的作用,考虑一种基于路标的视觉 SLAM 方法。

例 3.1 (基于路标的视觉 SLAM 中的离群值)一种基于路标(或基于特征)的视觉 SLAM 方法会在每幅图像中提取二维特征点,然后通过基于光流的特征跟踪或基于描述子的特征匹配,在连续帧之间关联这些特征点(图 3.2)。具体而言,在时刻 k ,该方法检测二维特征点,并将其与前一帧(例如时刻 $k-1$)观测到的对应点匹配;这些匹配像素通常称为二维-二维对应(2D-2D correspondences)。由于光流或基于描述子的匹配并不精确,这一初始对应集合可能包含离群值。因此,在将其传递给估计机器人位姿和路标位置的后端之前,过滤严重离群值十分重要。

RANSAC 是一种在将对应关系传递给后端之前快速检测并去除其中离群值的工具。检测离群值依赖两个关键洞见。第一个洞见是,在 SLAM 问题中,内点对应必须满足几何约束。例如,在本例中,内点对应描述了相机运动时静态三维点的观测像素运动。由此产生的像素运动不可能任意,而必须遵循精确的几何约束,即**对极约束**(epipolar constraint);该约束规定了两个帧中对应像素如何随相机运动发生关联。具体地,对于已标定相机,对极约束要求对应像素 $z_i(k-1), z_i(k)$, 即分别在时刻 $k-1$ 和 k 描绘路标 i 的像素,满足

$$z_i(k-1)^\top ([t_k^{k-1}]_\times R_k^{k-1}) z_i(k) = 0, \quad (3.2)$$

其中, t_k^{k-1} 和 R_k^{k-1} 是描述相机在时刻 $k-1$ 与 k 之间(未知)运动的相对平移和旋转。²更一般地,若将第 i 个对应记为 z_i (在上述例子中, $z_i = \{z_i(k-1), z_i(k)\}$), 则这类几何约束具有如下形式:

$$C(z_i, x) \leq \gamma, \quad (3.3)$$

它表明对应关系必须满足某个不等式,而该不等式可能是未知状态 x 的函数。式 (3.3) 右侧的参数 γ 通常经过调节以计及噪声的存在。例如,虽然理想情况下式 (3.2) 中的对极约束应严格满足,但实践中像素检测带有噪声,因而可能存在小误差;因此,我们放宽该约束,只要求 $|z_i(k-1)^\top ([t_k^{k-1}]_\times R_k^{k-1}) z_i(k)| \leq \gamma$, 其中 γ 为某个较小的值。

第二个洞见是,假设离群值数量不太多,我们可以将内点找作:对某个 x , 满足几何约束 (3.3) 的最大对应集合:

$$\begin{aligned} S_{CM}^* &= \operatorname{argmax}_{x, S \subset M} |S| \\ \text{s.t. } C(z_i, x) &\leq \gamma, \quad \forall i \in S \end{aligned} \quad (3.4)$$

其中, M 是初始假定对应 (putative correspondences) 的集合, $|S|$ 表示子集 S 的基数(元素个数)^[728]。换言之,优化问题 (3.4) 在假定对应集合 M 中寻找最大的子集 S , 使得 S 中的测量对同一个 x 都满足几何约束。直观地说,问题 (3.4) 刻画了如下直觉:内点(由集合 S 估计)必须就同一个 x “达成一致”,例如它们必须全都与机器人的真实运动一致。问题 (3.4) 在计算机视觉中称为**一致性最大化**(consensus maximization)。注意,式 (3.4) 不要求求解完整的 SLAM 问题(后者可能包含许多位姿和路标),因为它只涉及 SLAM 状态的

一小部分；例如，对极约束 (3.2) 只涉及两个帧之间的相对位姿，而非整个 SLAM 轨迹。同时，问题 (3.4) 仍然是一个困难的组合问题，这与典型 SLAM 前端对快速运行时间的要求相冲突。因此，人们通常不寻求式 (3.4) 的精确解，而是采用快速启发式方法近似地求解它。

随机采样一致性 (RANDOM SAMPLE CONSENSUS, RANSAC) 可能是为一致性最大化问题 (3.4) 寻找近似解的最著名方法。RANSAC 建立在一个关键假设之上：式 (3.4) 中的 x 维数相对较低，且可利用快速估计器（即所谓的最小求解器，minimal solvers）从一小组测量（即所谓的最小集，minimal set）中估计得到。³ 例如，在我们的视觉 SLAM 示例中，可利用 Nister 的五点法^[810]，仅通过 5 个像素对应估计两个相机帧之间的相对运动。RANSAC 的核心思想是：无需穷尽检查每一个可能的子集 $S \subset M$ ，而可以对最小测量集进行采样以寻找内点。更具体地，RANSAC 迭代执行以下三步：

1. 采样 n 个对应关系的子集，其中 n 是当前问题最小集的大小；⁴
2. 使用最小求解器，根据采样的 n 个对应关系计算估计 $\hat{x}$ ；⁵
3. 对于上一步计算得到的 $\hat{x}$ ，选择满足几何约束 $C(z_i, \hat{x}) \leq \gamma$ 的对应关系 $S \subset M$ 。若该集合 S 大于先前迭代中计算的集合，则将其保存。

最后一步计算得到的集合 S 称为**一致集** (consensus set)。RANSAC 通常在计算到足够大的一致集（由用户参数指定）后，或达到最大迭代次数后停止。RANSAC 本质上试图从测量集合中采样 n 个内点，因为这些内点很可能会与其他所有内点“达成一致”，因而具有较大的一致集。

RANSAC 是众多离群值剔除问题的首选方案。特别是，在离群值数量较少且最小集较小时，它能迅速收敛到良好估计（即良好的对应集合）。若从测量集合中采样一个内点的概率为 ω ，⁶ 则可容易得出，RANSAC 找到一组内点所需的期望迭代次数为 $\frac{1}{\omega^n}$ 。例如，当 $n = 5$ 且 $\omega = 0.7$ （即 70% 的测量为内点）时，期望迭代次数小于 10。加之非最小求解器在实践中极快（常能在数毫秒内完成数百次迭代），RANSAC 极具吸引力。此外，RANSAC 还会提供估计 $\hat{x}$ （例如机器人里程计），可作为后端的初始猜测。

另一方面，RANSAC 未必适合所有问题。尤其是，当内点数量很少或最小集很大时，期望迭代次数会变得不切实际地大。例如，当 $n = 10$ 且 $\omega = 0.1$ 时，找到一组内点所需的期望迭代次数达到 10^{10} ；在更少的迭代次数后终止 RANSAC，很可能返回错误解（即错误的 $\hat{x}$ 和对应关系）。如下一节所述，在许多 SLAM 问题中，“内点很多且最小集很小”的假设并不总是成立。

3.6 基于图论的离群值剔除与成对一致性最大化

如前所述，当离群值比例适中（例如低于 70%）且最小集较小（例如小于 8）时，RANSAC 非常有效。然而，存在严重感知混叠的环境可能具有极高比例的离群值。此外，我们感兴趣的并非所有问题都存在最小集很小的快速最小求解器。例如，考虑一个具有 N 个节点的位姿图 SLAM 问题，最小集至少必须包括 $N - 1$ 个测量（构成位姿图的一棵生成树），而 N 通常有数千之多。

因此，本节介绍一种称为**成对一致性最大化** (Pairwise Consistency Maximization, PCM) 的替代方法。该方法不对最小集采样，而是借助图论寻找内部彼此“一致”的最大测量集合。该方法可处理离群值比例超过 90% 的测量集合，并在传递给后端之前剪除严重离群值。该方法最初由文献^[728]提出，并在^[1005, 332]中推广至超越成对一致性的情况。

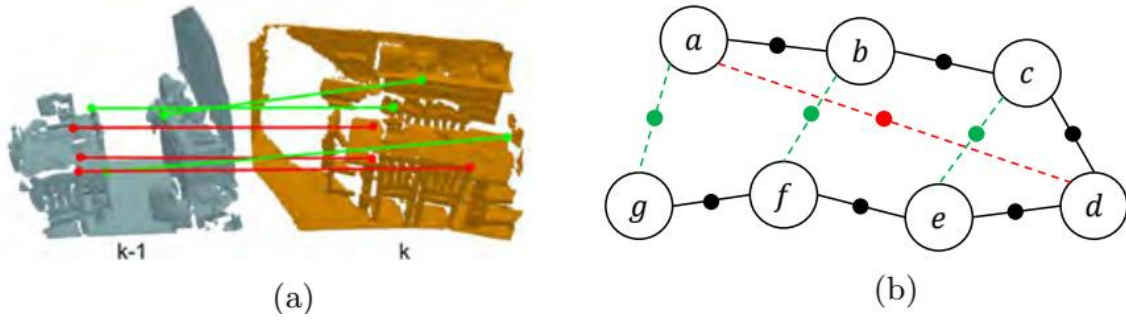


图 3.3: 图 3.3 (a) 来自两幅 RGB-D 扫描的三维-三维对应，它们表示场景的两个局部视图。绿色线条表示内点对应，红色线条表示离群值对应。(b) 回环中含离群值的位姿图。绿色虚线表示内点回环，红色点线表示一个离群回环。

PCM 的关键洞见是：对于许多问题，可以定义一致性函数来描述一对测量是否相互一致。下面通过两个例子说明这一点。

例 3.2 (带 RGB-D 相机的基于路标视觉 SLAM 中的一致性函数) 使用 RGB-D 相机的基于路标视觉 SLAM 方法会在每个 RGB-D 帧中提取三维特征点，再在连续帧之间将它们关联起来 (图 3.3(a))。具体地，在时刻 k ，该方法检测三维特征点，并将其与前一帧 (例如时刻 $k-1$) 观测到的对应点进行匹配；这些匹配的三维点通常称为三维-三维对应 (3D-3D correspondences)。时刻 k 和 $k-1$ 采集的三维点，理想情况下对应于从两个不同视点观测到的同一组三维静态点；因此，对应点对 $\{z_i(k-1), z_j(k-1)\}$ 与 $\{z_i(k), z_j(k)\}$ 之间的距离必须随时间保持不变 (至多存在噪声)：

$$\|z_i(k-1) - z_j(k-1)\| - \|z_i(k) - z_j(k)\| \leq \gamma \quad (3.5)$$

可以看到，与 RANSAC 中使用的几何约束不同，一致性函数 (3.5) (i) 不依赖于状态，因而无需最小求解器即可直接求值；(ii) 无论最小集大小如何，均涉及一对对应关系。尽管前一个例子也可使用 RANSAC 求解，⁷ 现在考虑一个维度更高的问题。

例 3.3 (位姿图 SLAM 中的一致性函数) 考虑一个位姿图 SLAM 问题，其中回环可能因地点识别失败和感知混叠而包含离群值；我们假设里程计可靠且不含离群值。为了判断两个回环是否彼此一致，需要注意：在无噪声情形下，图中沿环的位姿测量复合后必须为单位元 (图 3.3(b))。⁸ 因此，一对回环 T_b^a (位姿 a 与 b 之间) 和 T_d^c (位姿 c 与 d 之间) 必须满足：

$$\text{dist}(T_b^a \cdot \bar{T}_c^b \cdot T_d^c \cdot \bar{T}_a^d, \mathbf{I}) \leq \gamma \quad (3.6)$$

其中， $\bar{T}_c^b$ 和 $\bar{T}_a^d$ 分别是节点 b 到节点 c 、以及从节点 d 到节点 a 的里程计测量链； dist 是合适的距离函数，用于度量 $T_b^a \cdot \bar{T}_c^b \cdot T_d^c \cdot \bar{T}_a^d$ 与单位位姿之间的距离。与往常一样， γ 是为计及噪声而选择的参数：由于里程计和回环中存在噪声，沿一个环的测量未必复合为单位元。⁹

更一般地，一致性函数是关联两个测量并且必须满足给定约束的函数。对于一对测量 z_i 和 z_j ，所得的成对一致性约束形式为：

$$F(z_i, z_j) \leq \gamma, \quad (3.7)$$

其中, F 是一致性函数, γ 是计及测量噪声的用户指定参数。需要强调, 成对一致性约束与状态无关, 因此只需检查每一对测量, 无需求助于最小求解器即可高效验证。

利用式 (3.7), 我们可以给出另一种离群值剔除方法: 选择彼此成对一致的最大测量集合:

$$\begin{aligned} S_{\text{PCM}}^* = \operatorname{argmax}_{S \subset M} |S| \\ \text{s.t. } F(z_i, z_j) \leq \gamma, \quad \forall i, j \in S \end{aligned} \quad (3.8)$$

问题 (3.8) 寻找最大的测量子集 S , 使得 S 中每对测量均成对一致。我们将其称为成对一致性最大化 (PCM) 问题。该问题仍具有组合性质, 但看起来比式 (3.4) 稍易: 它不涉及 x , 且可对 M 中每一对 (i, j) 预先计算约束 $F(z_i, z_j) \leq \gamma$ 。此外, 该问题具有图论解释, 因此可使用图论中的成熟工具, 即最大团算法, 来求解。

为建立问题 (3.8) 与图论的联系, 将离群值剔除问题可视化为一个图 $\mathcal{G}$: 图的节点是假定测量 $i \in M$; 若

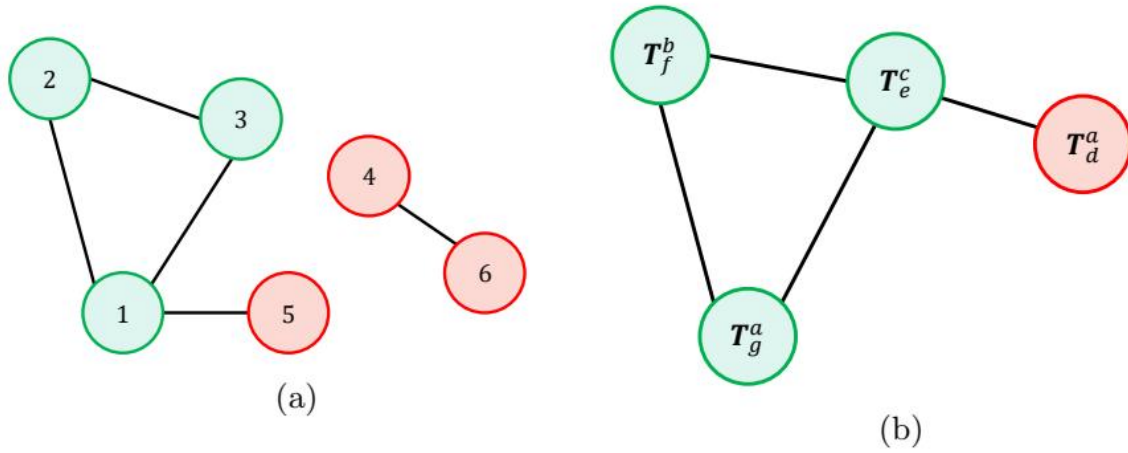


图 3.4: 图 3.4 (a) 图 3.3(a) 中三维-三维对应示例的一致性图。(b) 图 3.3(b) 中位姿图示例的回环一致性图。

$F(z_i, z_j) \leq \gamma$, 则节点 i 和 j 之间存在一条边。这通常称为**一致性图** (consistency graph)。问题 (3.8) 现在要求选择最大的节点子集 S , 使 S 中每对节点之间都由一条边相连; 这正是图中**最大团** (maximum clique) 的定义。更具体地, 在图论中, 团是一个节点子集, 其中每一对节点之间都有边; 最大团则是图中这样的最大节点子集。因此, 问题 (3.8) 的解就是一致性图的最大团。这一图论联系在实践中很有价值, 因为对给定图寻找最大团是图论中研究充分的问题, 称为最大团问题。最大团问题是 NP-hard 问题^[1197], 且难以近似^[1314, 322], 这意味着寻找任意接近真实解的解同样是 NP-hard 的。不过, 已有数十种最大团算法被提出, 其中一些可根据图的稠密度处理规模相当大的问题。所提出的方法大多可分为精确方法与基于启发式的方法。所有精确算法的复杂度均为指数级, 且通常基于分支定界; 启发式算法则常试图利用问题中的某种结构而更快, 但代价是不一定保证最优解^[1197]。相对较新的工作, 如^[853], 提出了可并行化的最大团算法, 能够在大型稀疏图中快速寻找最大团。

总而言之, 使用最大团算法求解 PCM 问题包括以下步骤:

1. 为当前问题选择一致性函数 F 。

2. 对每一对假定测量 $(i, j) \in M$ 求值一致性函数；当 $F(z_i, z_j) \leq \gamma$ 时，在 i 与 j 之间连边，从而构建一致性图。

3. 使用精确或近似最大团算法求解一致性图的最大团。

4. 返回（可能为近似的）最大团中的测量 S 。

需要指出，一致性函数的选择依赖于具体问题；选择好的函数会显著影响离群值剔除质量。例如，可以选择一个无意义函数 $F(z_i, z_j) = 0$ ，它无论输入为何都返回零；此类函数无法剔除任何离群值，从而使 PCM 无效。反之，若构造的函数使得只有内点能通过检验，则恰好剔除所有离群值。文献^[1005, 332]讨论了适用于广泛几何问题的一组潜在一致性函数。

在结束本节前，还需说明几点。尽管 PCM 相较 RANSAC 能处理更高比例的离群值，并且更适合某些问题（例如位姿图 SLAM），但 PCM 与 RANSAC 的权衡更为细致。RANSAC 使用最小求解器计算的估计来评估单个测量的一致性；PCM 则以成对方式评估一组测量彼此之间的一致性。在某些情形中，RANSAC 的单测量一致性不足以评估整个测量集合；位姿图优化常是如此，因为一对回环测量各自的一致性并不必然保证这两个回环之间成对一致。¹⁰另一方面，对于三维-三维位姿估计（例 3.2）等某些问题，PCM 使用的成对一致性函数 (3.5) 可能比 RANSAC 更宽松，从而将某些离群值归类为内点。

在 PCM 的语境下，还应注意，精确最大团求解器在稠密一致性图中（即许多测量对彼此一致时）往往较慢，因此对某些问题而言，基于启发式的最大团解可能是更好的选择。最后，对某些问题很难设计合适的一致性函数；例如，对于二维-二维对应，由于缺乏合适的不变量，难以严格地设计一个通用一致性函数（参见^[1005]的讨论）。

3.7 提高 SLAM 后端对离群值的鲁棒性

包括 RANSAC 和 PCM 在内的前端离群值剔除仍可能漏检离群值，并向后端传递受离群值污染的测量集合。¹¹如第 3.1.2 节所见，采用标准最小二乘估计时，少数离群值即可导致完全错误的结果。因此，提高后端对剩余离群值的鲁棒性十分重要。

第 3.1.2 节指出，使用残差平方会“放大”离群测量对代价函数的影响。本节遵循鲁棒统计中 M-估计 (M-Estimation) 的标准理论^[495]，对 SLAM 优化目标函数作轻微修改，以重新获得对离群值的鲁棒性。

M-估计（“maximum-likelihood-type estimation”）是鲁棒估计的一个框架。它建议将式 (3.1) 中的平方损失替换为恰当选择的鲁棒损失函数 ρ ：

$$\mathbf{x}^{\text{MAP}} = \arg \min_{\mathbf{x}} \sum_i \mathbf{r}_i(\mathbf{x})^2 \implies \mathbf{x}^{\text{MEST}} = \arg \min_{\mathbf{x}} \sum_i \rho(\mathbf{r}_i(\mathbf{x})). \quad (3.9)$$

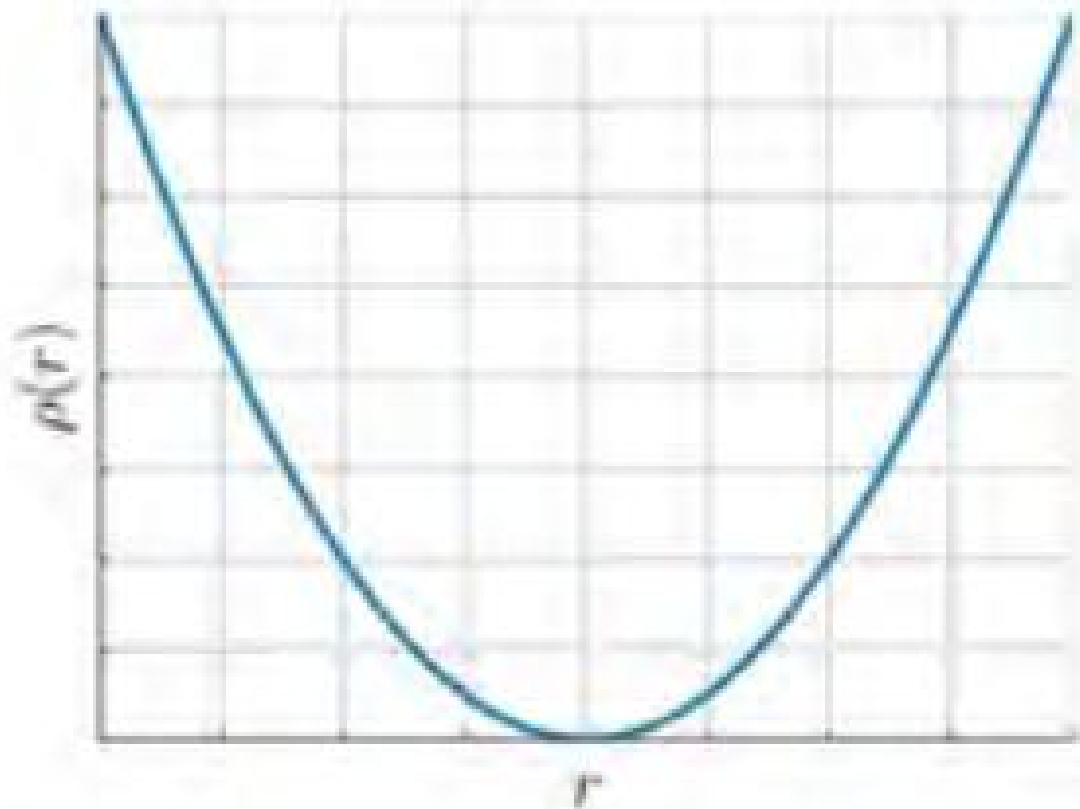
鲁棒损失 ρ 的关键要求是：它为非负函数，且对于大残差的增长速度低于二次；换言之，随着 r_i 变大，鲁棒损失函数需要具有导数 $\frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i} \ll \frac{\partial \|\mathbf{r}_i\|^2}{\partial \mathbf{r}_i} = 2\mathbf{r}_i$ ；在许多情况下，希望当 r_i 变大时， $\frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i}$ 趋于零。为阐明这一要求，考虑使用梯度下降求解 $\min_{\mathbf{x}} \sum_i \rho(\mathbf{r}_i(\mathbf{x}))$ 。由链式法则，目标 $f(\mathbf{x}) \doteq \sum_i \rho(\mathbf{r}_i(\mathbf{x}))$ 的梯度为：

$$\frac{\partial f}{\partial \mathbf{x}} = \sum_i \frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i} \cdot \frac{\partial \mathbf{r}_i(\mathbf{x})}{\partial \mathbf{x}} \quad (3.10)$$

由式 (3.10) 可知，若从一个良好的初始猜测开始（即相对接近真值），离群测量具有较大残差，因而 $\frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i}$ 很小，对总体下降方向的影响也很小。因此它们在估计中几乎没有影响。

实际上, 函数 $\psi(\mathbf{r}_i) := \frac{\partial \rho(\mathbf{r}_i)}{\partial \mathbf{r}_i}$ 通常称为**影响函数** (influence function) [85]。

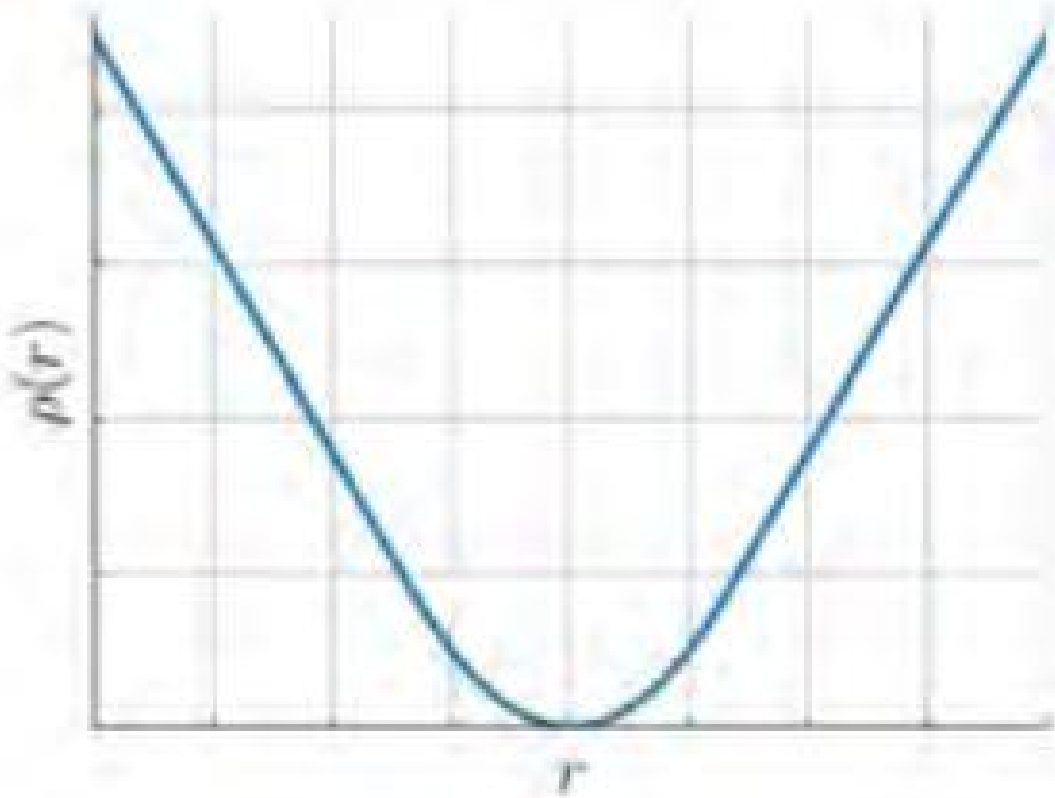
鲁棒估计文献并未给出唯一的鲁棒损失函数, 而是提供了多种可能选择。图 3.5 列出了常见损失函数, 其中包括 Huber、Geman-McClure、Tukey 双权 (Tukey's biweight) 和截断二次损失等常用鲁棒损失, 也包括一种更激进的选择, 即最大一致性损失。后一种通常不列在鲁棒估计文献的损失函数之中, 但这里提及它, 是因为它与我们在式 (3.4) 中讨论的一致性最大化问题相联系。¹² 鲁棒损失的选择高度依赖具体问题 [554, 1075]。例如, 当已知合理的截断阈值 (即内点的预期最大误差) 时, 具有硬截断的损失函数, 例如截断二次损失 (其二次区与函数“平坦”部分之间发生突变), 更为合适。还必须考虑计算因素。例如, Huber 常用于 BA 问题, 因为它是凸函数, 在优化过程中表现更稳定, 尽管它对离群值保留非零影响 (只有当大残差时损失保持常数, 影响才变为零)。另一方面, 截断二次和最大一致性损失对离群值尤不敏感, 但常需要专用求解器。¹³



- (a) 二次损失
- (b) Huber 损失
- (c) Geman-McClure
- (d) Tukey 双权
- (e) 截断二次 (f) 最大一致性损失

图 3.5 二次损失与鲁棒损失函数示例。鲁棒损失函数的形状由一个控制内点和离群值分界的参数确定。

图 3.6(g)–(l) 展示了对上述两种鲁棒损失, 即 Huber 损失和截断二次损失, 应用梯度下



降所得到的 SLAM 轨迹。这里采用与图 3.1 相同的数据集。我们使用启用 gradientDescent 标志的 GTSAM NonlinearConjugateGradientOptimizer^[253] 实现梯度下降求解器，并使用鲁棒噪声模型实例化鲁棒损失函数。最大迭代次数和停止条件阈值（相对与绝对容差）分别设为 10000 和 10^{-7} ，其余参数保留为 GTSAM 默认值。图中，若一条边被优化正确归类为内点或离群值，则显示为灰色（即落入 Huber 或截断二次损失二次区域的内点）；若离群值被错误归为内点，即“假阳性”（false positive），则显示为红色；若内点被错误归为离群值，即“假阴性”（false negative），则显示为蓝色。与非鲁棒最小二乘优化（图 3.1(d)–(f)）相比，鲁棒损失可使 M3500 和 SubT 数据集迅速恢复离群值鲁棒性；但简单的梯度下降仍可能无法正确优化截断二次代价等高度非凸函数，如 Victoria Park 数据集所示。下文将用基于渐进非凸性（graduated non-convexity）的更好求解器处理该问题。此外，尽管梯度下降在许多实例中已改善性能（图 3.6），它仍具有缓慢的收敛尾部；在我们的实验中，往往需要数千次迭代才收敛。因此，本节其余部分讨论同时提高收敛质量和速度的更高级求解器。

M3500

(a) SubT

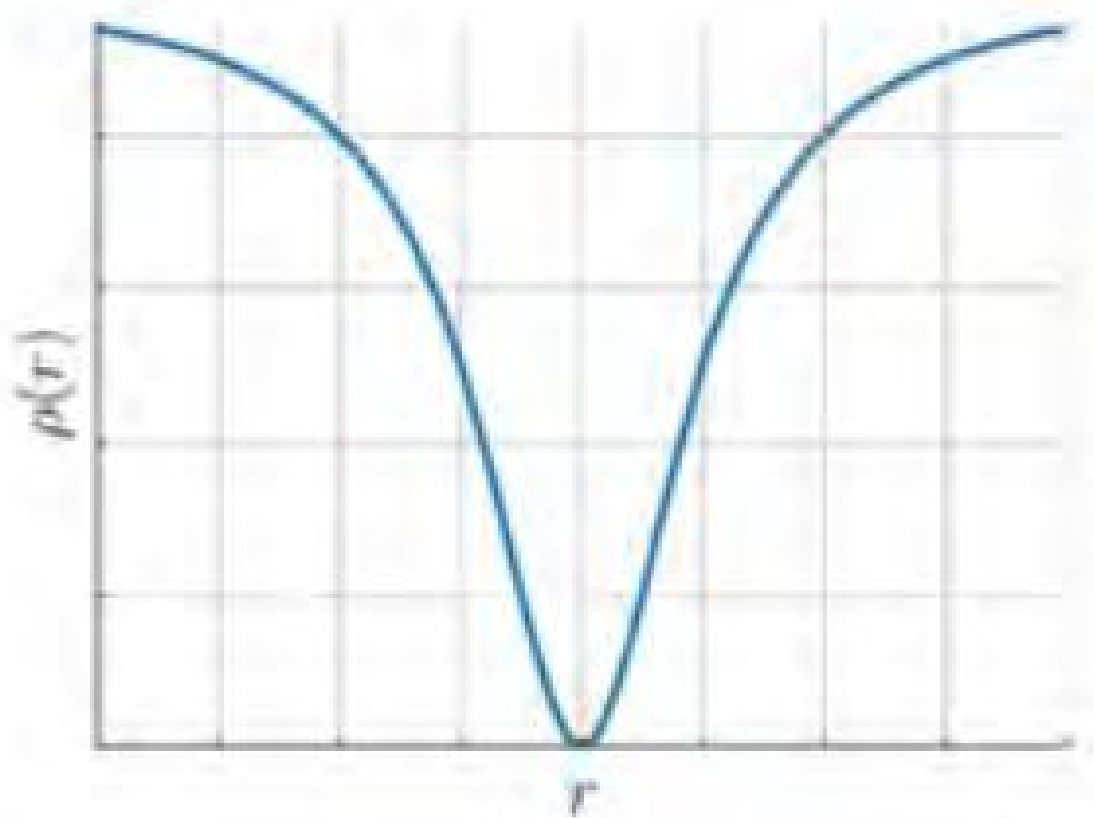
(b) Victoria Park

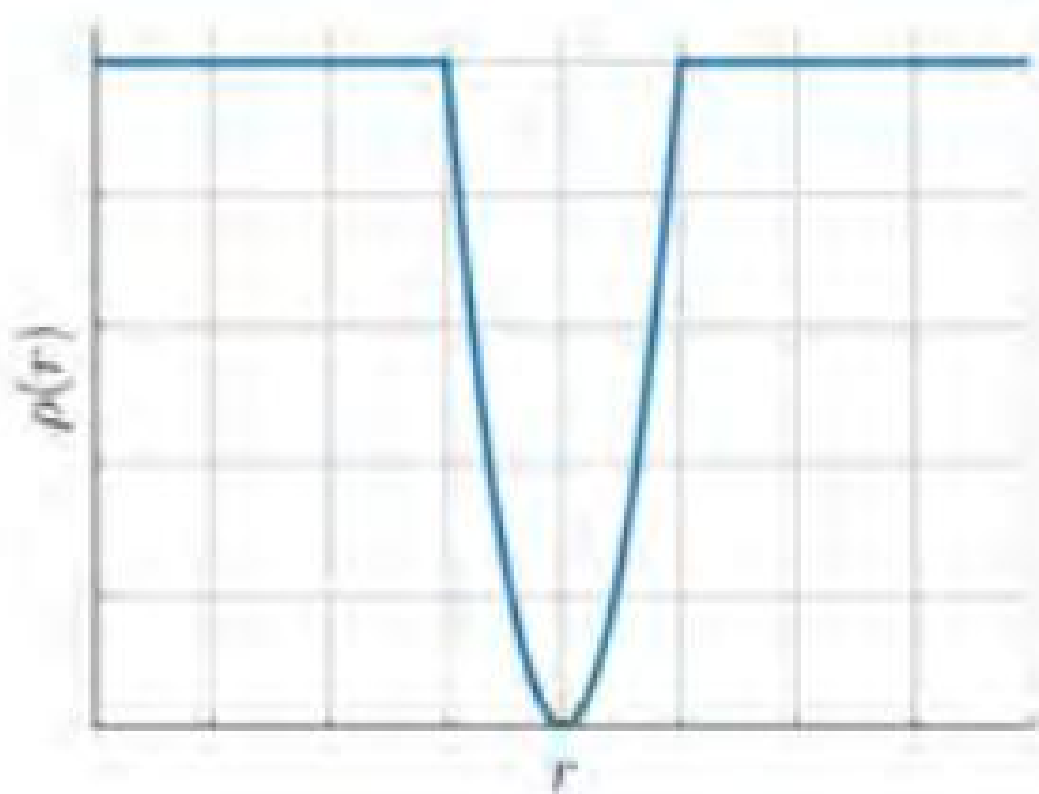
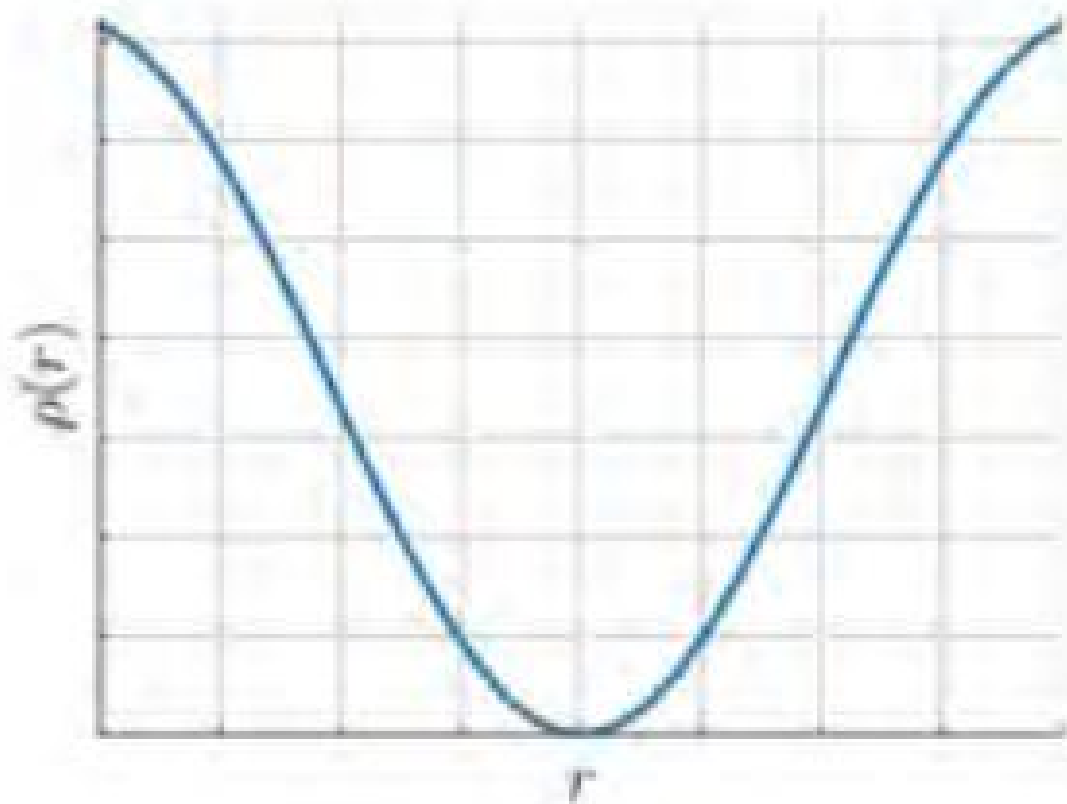
(g)

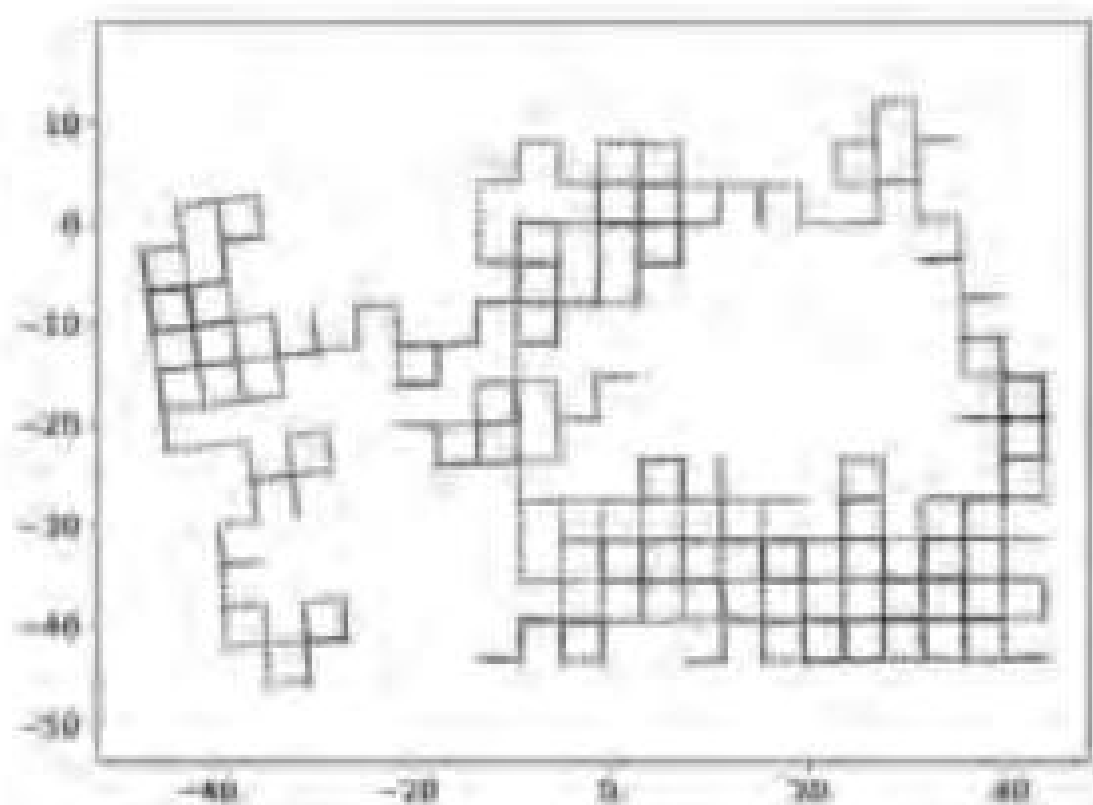
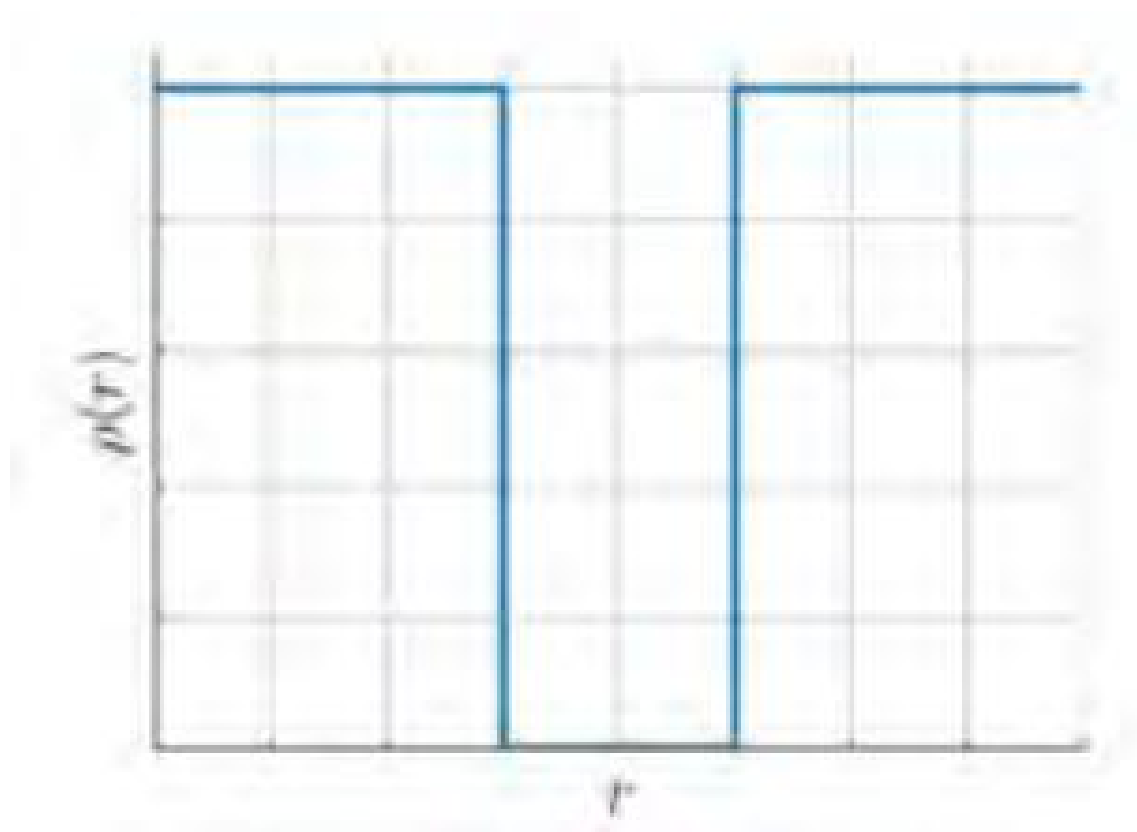
(c)

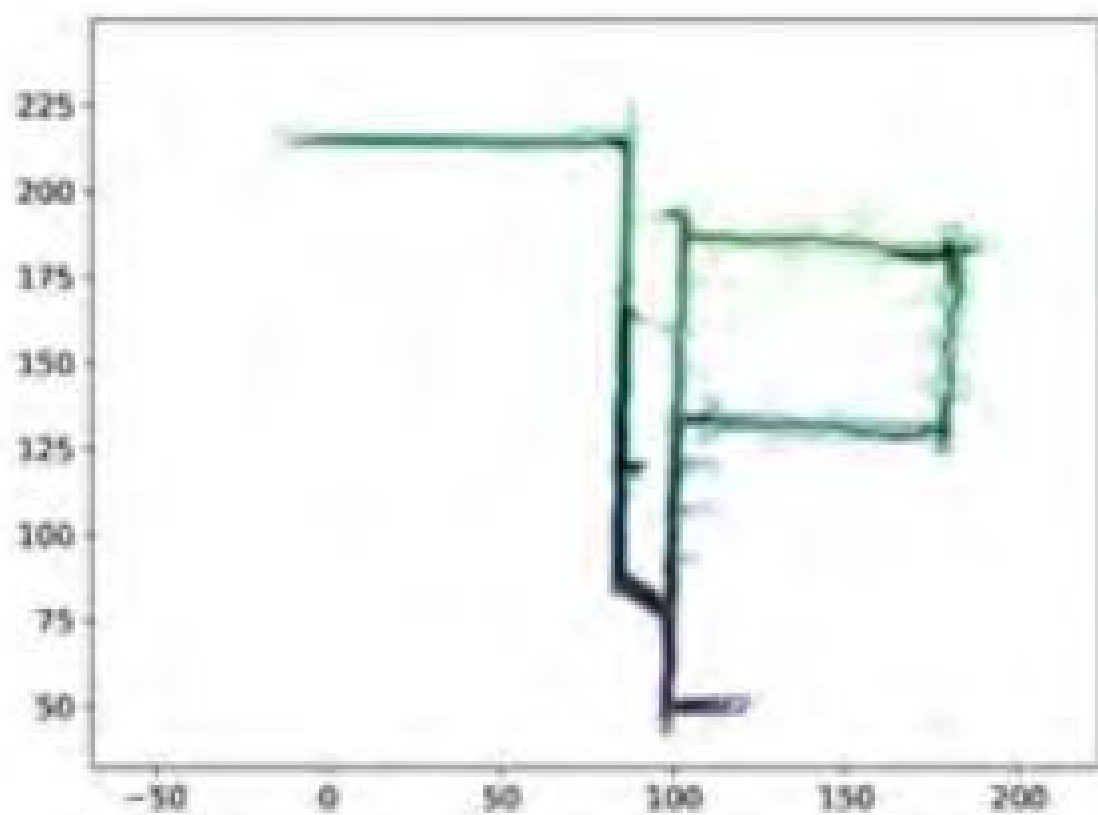
(j)

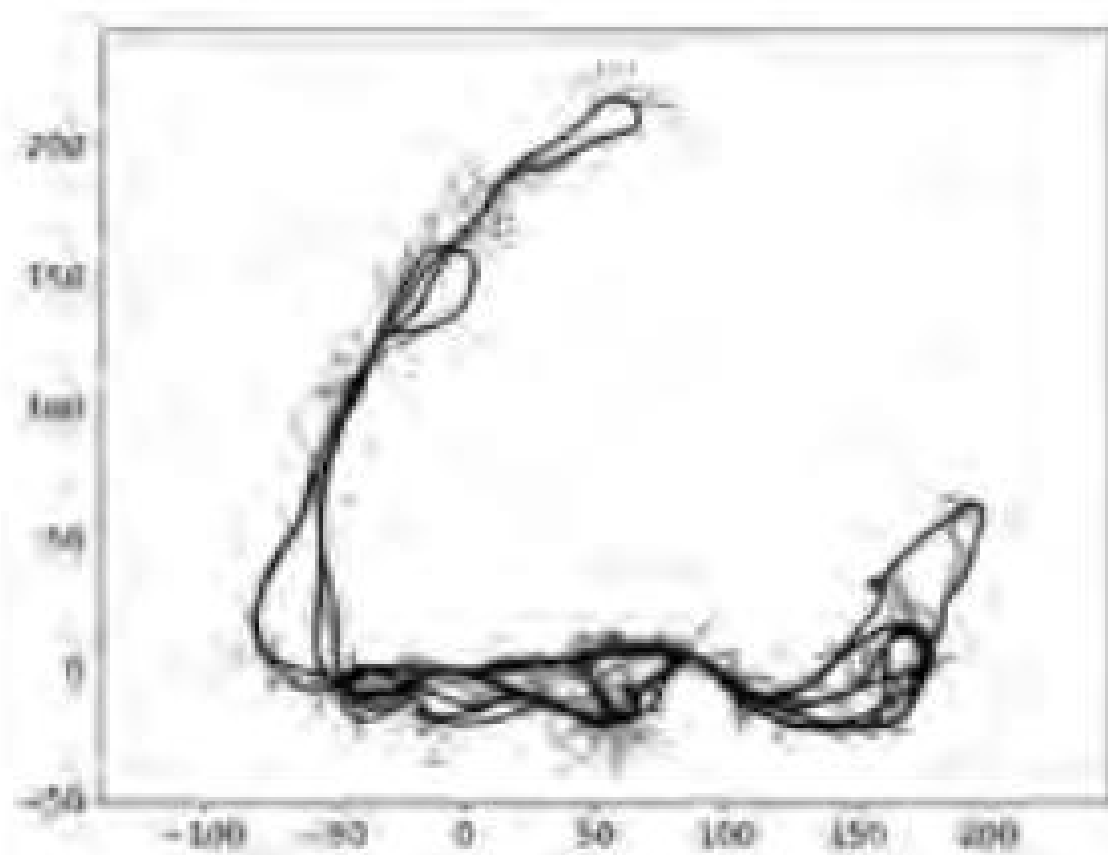
(h) (i)

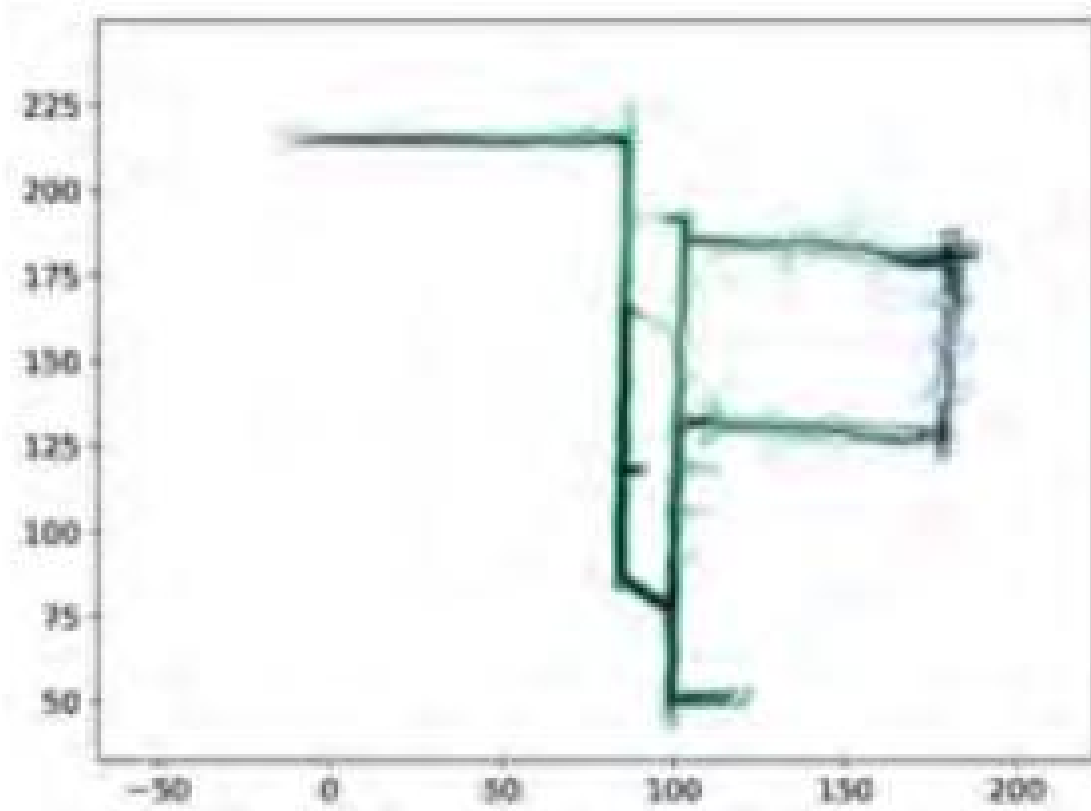
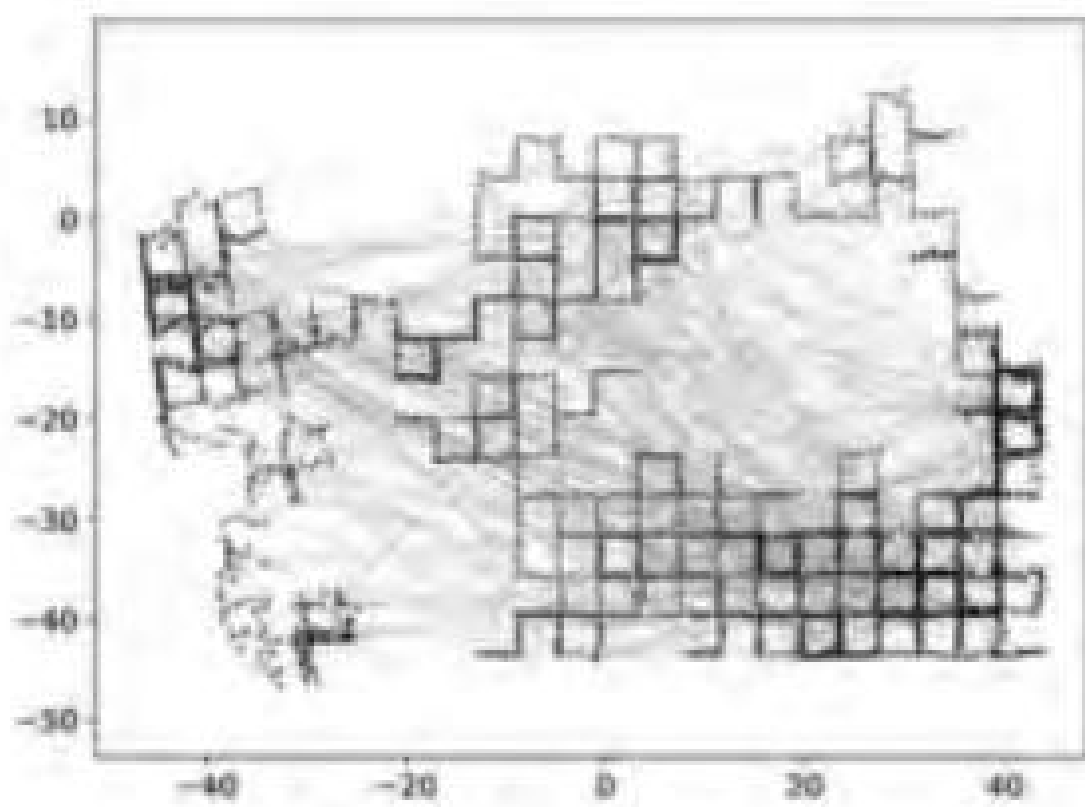


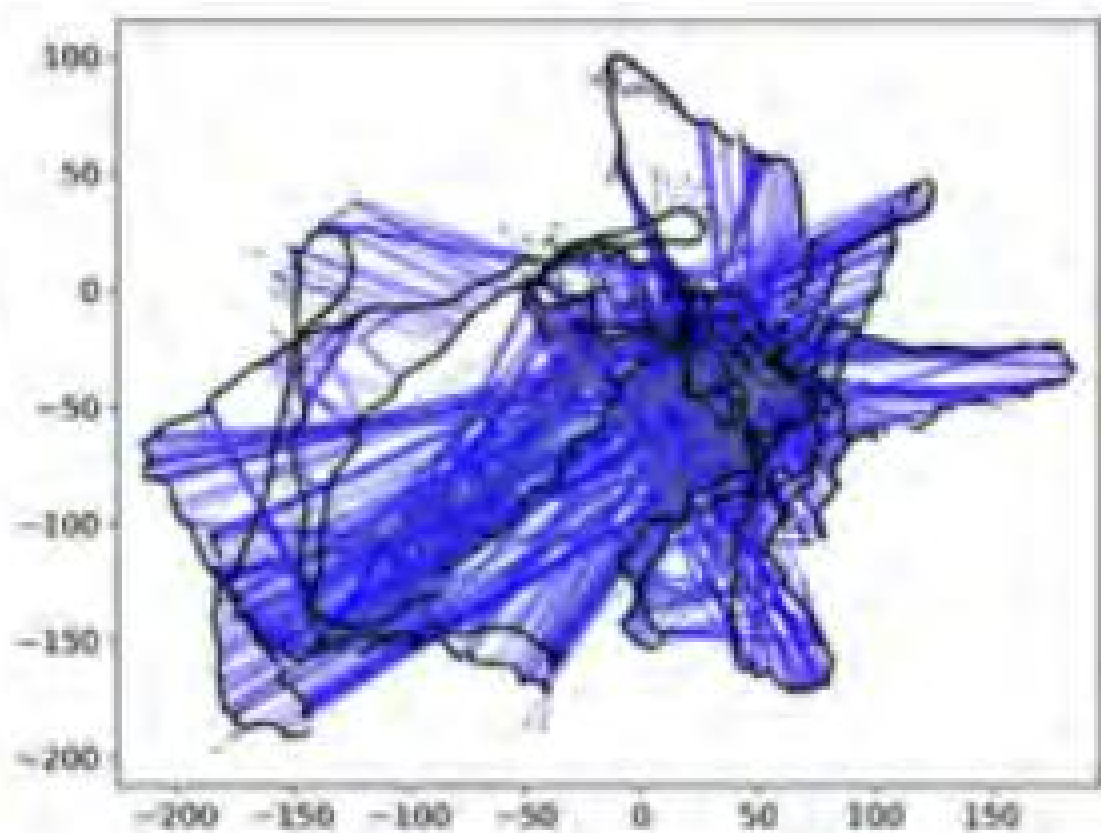


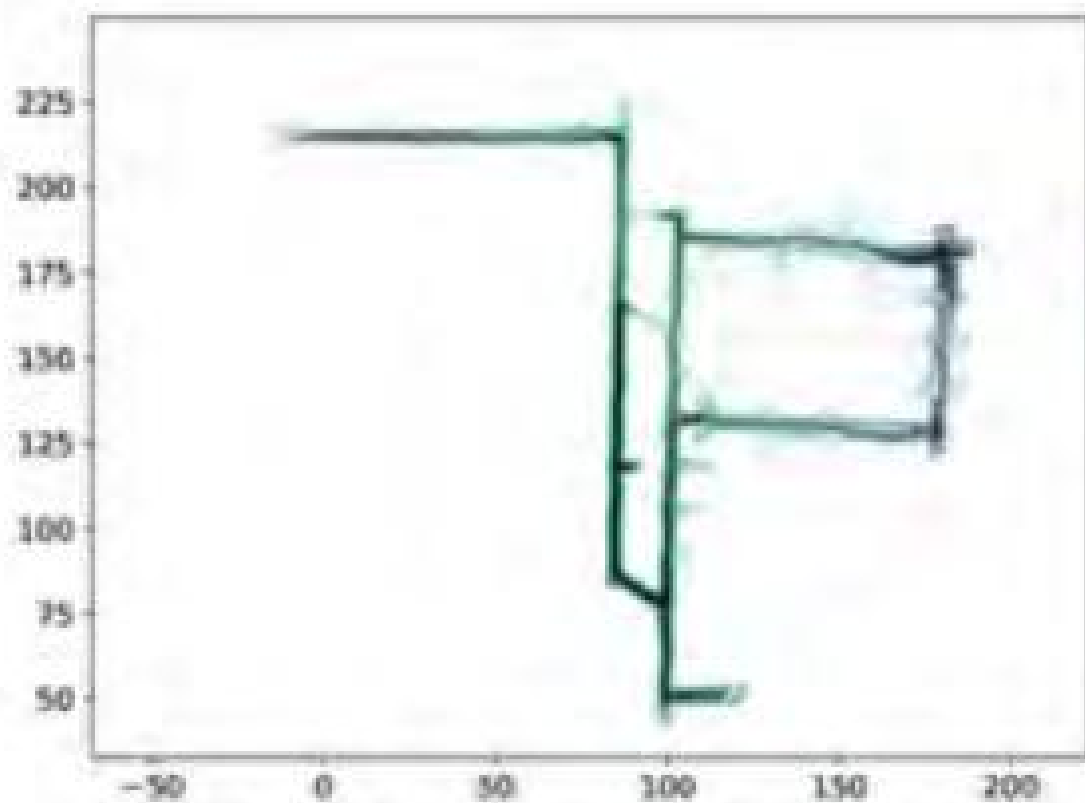
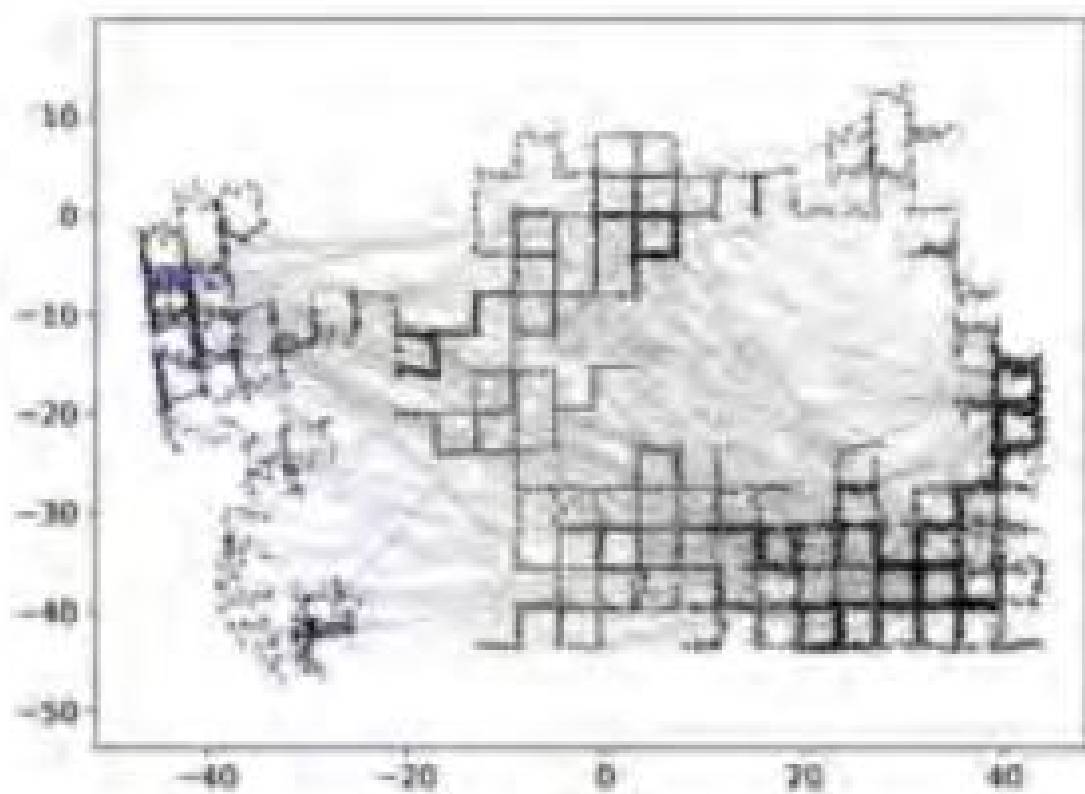




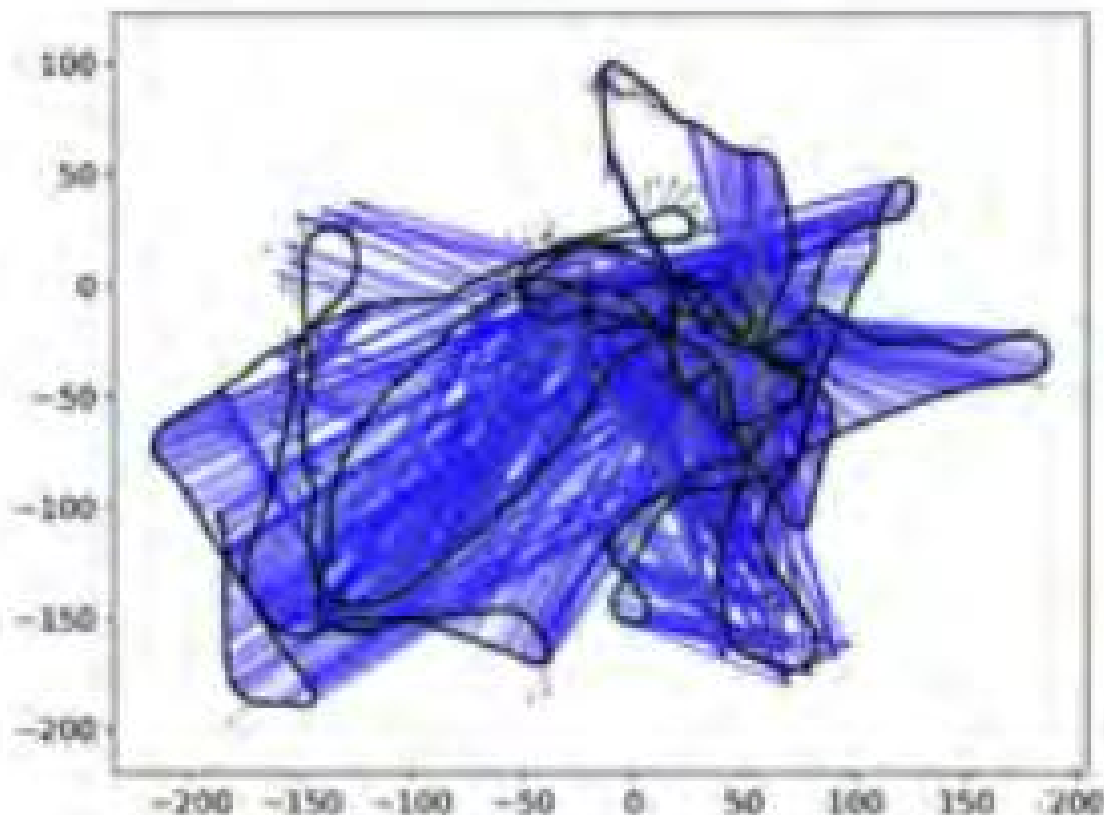








(k)



(l) 图 3.6 使用鲁棒损失函数和梯度下降求解含离群值 SLAM 问题：(a)–(c) 为 M3500、SubT 和 Victoria Park 数据集的真值轨迹；(d)–(f) 为使用梯度下降和 Huber 损失得到的轨迹估计；(g)–(l) 为使用梯度下降和截断二次损失得到的轨迹估计。测量以彩色边显示：灰色边为被优化正确归类为内点或离群值的边；红色边为被错误归类为内点的离群值（假阳性）；蓝色边为被错误归类为离群值的内点（假阴性）。

在深入讨论高级求解器前，最后指出：虽然切换到鲁棒损失函数看似放弃了概率框架，实际上可通过对重尾噪声分布应用 MAP 估计推导出若干鲁棒损失。例如，当假设噪声服从描述内点的高斯密度与描述离群值的均匀分布之间的 max-mixture 分布时，MAP 估计会导出截断二次损失^[34]。

3.8 迭代重加权最小二乘

M-估计在式 (3.9) 中用鲁棒损失 ρ 替换最小二乘损失，即用一个对大残差二次增长的函数替换它。这有两个代价。首先，我们失去了为最小二乘形式开发的高效解法；例如，高斯-牛顿和 Levenberg-Marquardt 方法是为最小二乘问题设计的。其次，由于 M-估计通常具有非凸的代价景观，迭代求解器（例如基于梯度下降的方法）对初始化质量敏感，常收敛至不期望的次优估计。本节介绍一种求解 M-估计的常用算法，即**迭代重加权最小二乘**（iteratively reweighted least squares, IRLS）；顾名思义，它允许复用高效的最小二乘求解器。下一节将

介绍渐进非凸性，以改善 IRLS 的收敛性。

IRLS 的基本思想是：在每次迭代中通过求解加权最小二乘问题来优化式 (3.9)：

$$\mathbf{x}^{(t+1)} = \arg \min_{\mathbf{x}} \sum_i w_i(\mathbf{x}^{(t)}) r_i^2(\mathbf{x}), \quad (3.11)$$

其中，权重 w_i 取决于上一次迭代的估计 $\mathbf{x}^{(t)}$ 。我们希望迭代解 $\mathbf{x}^{(t)}$ 收敛到 M-估计 (3.9) 的最优解。这意味着，式 (3.10) 所示鲁棒损失 ρ 的梯度，必须与式 (3.11) 中损失的梯度相匹配。将式 (3.11) 的梯度写为

$$\sum_i 2w_i(\mathbf{x}^{(t)}) r_i(\mathbf{x}) \frac{\partial r_i(\mathbf{x})}{\partial \mathbf{x}}$$

并与式 (3.10) 比较，可得 IRLS 权重更新规则：

$$w_i(\mathbf{x}^{(t)}) = \frac{1}{2r_i(\mathbf{x}^{(t)})} \frac{\partial \rho(r_i(\mathbf{x}^{(t)}))}{\partial r_i(\mathbf{x}^{(t)})} = \frac{\psi(r_i(\mathbf{x}^{(t)}))}{2r_i(\mathbf{x}^{(t)})}, \quad (3.12)$$

其中， $\psi(r_i) := \frac{\partial \rho(r_i)}{\partial r_i}$ 是影响函数。因而，IRLS 在为每个测量 i 计算权重 $w_i(\mathbf{x}^{(t)})$ 与对加权最小二乘问题 (3.11) 执行优化步骤（即一次 Gauss–Newton 或 Levenberg–Marquardt 迭代）之间交替进行。

图 3.7 给出了 IRLS 在 M3500、SubT 和 Victoria Park 数据集上的表现。IRLS 在数十次迭代内收敛，通常比梯度下降快得多；例如，在我们的 M3500 实验中，梯度下降优化 Huber 损失约需 5 秒，而 IRLS 少于 1.5 秒。另一方面，这种更快的收敛往往以精度略有下降为代价，比较图 3.7 与图 3.6 即可看出。文献 [10, 817] 已研究了更新规则 (3.12) 的收敛性质。

3.9 Black–Rangarajan 对偶性

权重更新规则 (3.12) 在实践中广泛使用，但其推导带有一定启发性；此外，在 ρ 的不可微点（例如截断二次损失的截断点）处，式 (3.12) 没有定义。下面引入一个更有原则的框架，即 Black–Rangarajan (B–R) 对偶性 [85]，用于通过 IRLS 求解 M-估计。

使用截断二次损失说明 B–R 对偶性的直觉：

$$\rho(r_i(\mathbf{x})) := \min\{r_i^2(\mathbf{x}), \beta_i^2\}, \quad (3.13)$$

其中， β_i^2 是第 i 个残差的界：若 $r_i^2(\mathbf{x}) \leq \beta_i^2$ ，第 i 个测量为内点，否则为离群值。通过引入新的权重变量 $w_i \in [0, 1]$ ，可将式 (3.13) 的代价等价地写作两项之和：

$$\rho(r_i(\mathbf{x})) := \min_{w_i \in [0, 1]} w_i r_i^2(\mathbf{x}) + (1 - w_i) \beta_i^2, \quad (3.14)$$

其中第一项恰为加权最小二乘，第二项是与 $\mathbf{x}$ 无关的 w_i 的函数。借助式 (3.14)，使用截断二次损失的 M-估计问题 (3.9) 可改写为

$$\min_{\mathbf{x}, w_i \in [0, 1], i=1, \dots, N} \sum_i [w_i r_i^2(\mathbf{x}) + (1 - w_i) \beta_i^2], \quad (3.15)$$

其中, 对每个测量残差 $r_i(\mathbf{x})$ 都引入一个 w_i 。问题 (3.15) 容易解释: $w_i = 1$ 意味着 $r_i^2(\mathbf{x}) \leq \beta_i^2$, 第 i 个测量为内点; $w_i = 0$ 意味着 $r_i^2(\mathbf{x}) > \beta_i^2$, 第 i 个测量为离群值。此外, 所有 $w_i = 0$ 的残差均被有效地从优化问题 (3.15) 中丢弃, 因而保证了鲁棒性。

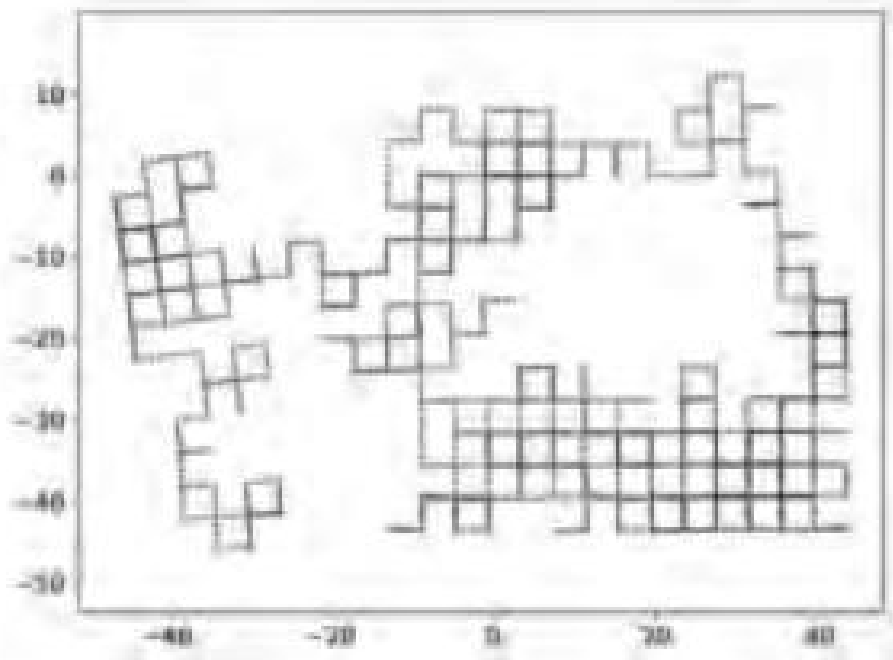
B-R 对偶性将上述推导推广到一族鲁棒损失。

定理 3.4 (Black-Rangarajan 对偶性^[85]) 给定鲁棒损失函数 $\rho(\cdot)$, 定义 $\phi(z) := \rho(\sqrt{z})$ 。若 $\phi(z)$ 满足 $\lim_{z \rightarrow 0} \phi'(z) = 1$ 、 $\lim_{z \rightarrow \infty} \phi'(z) = 0$ 且 $\phi''(z) < 0$, 则 M-估计问题 (3.9) 等价于

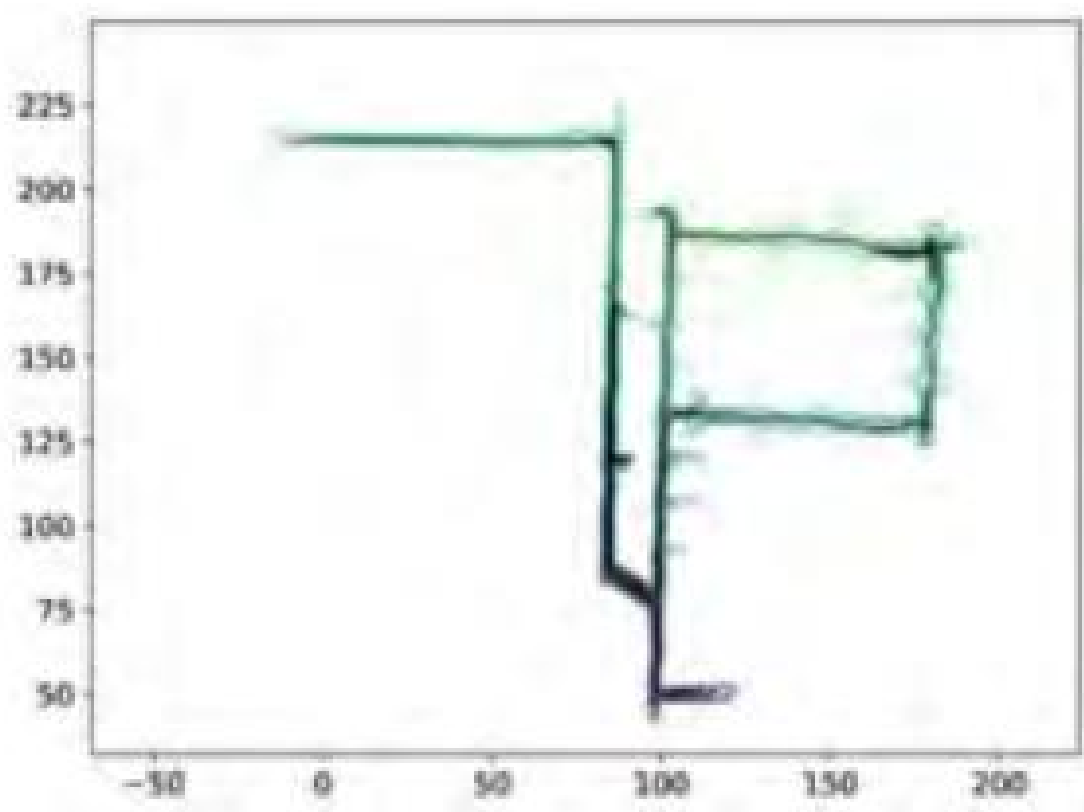
$$\min_{\mathbf{x}, w_i \in [0, 1], i=1, \dots, N} \sum_{i=1}^N [w_i r_i^2(\mathbf{x}) + \Phi_\rho(w_i)], \quad (3.16)$$

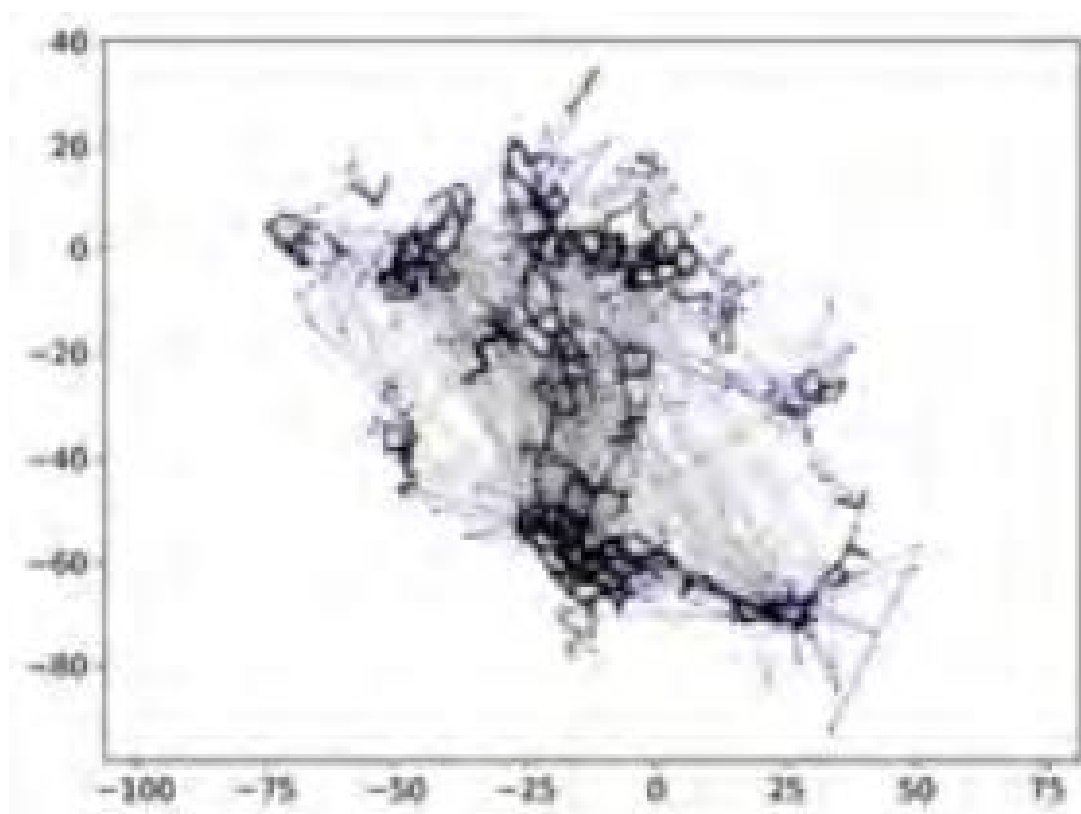
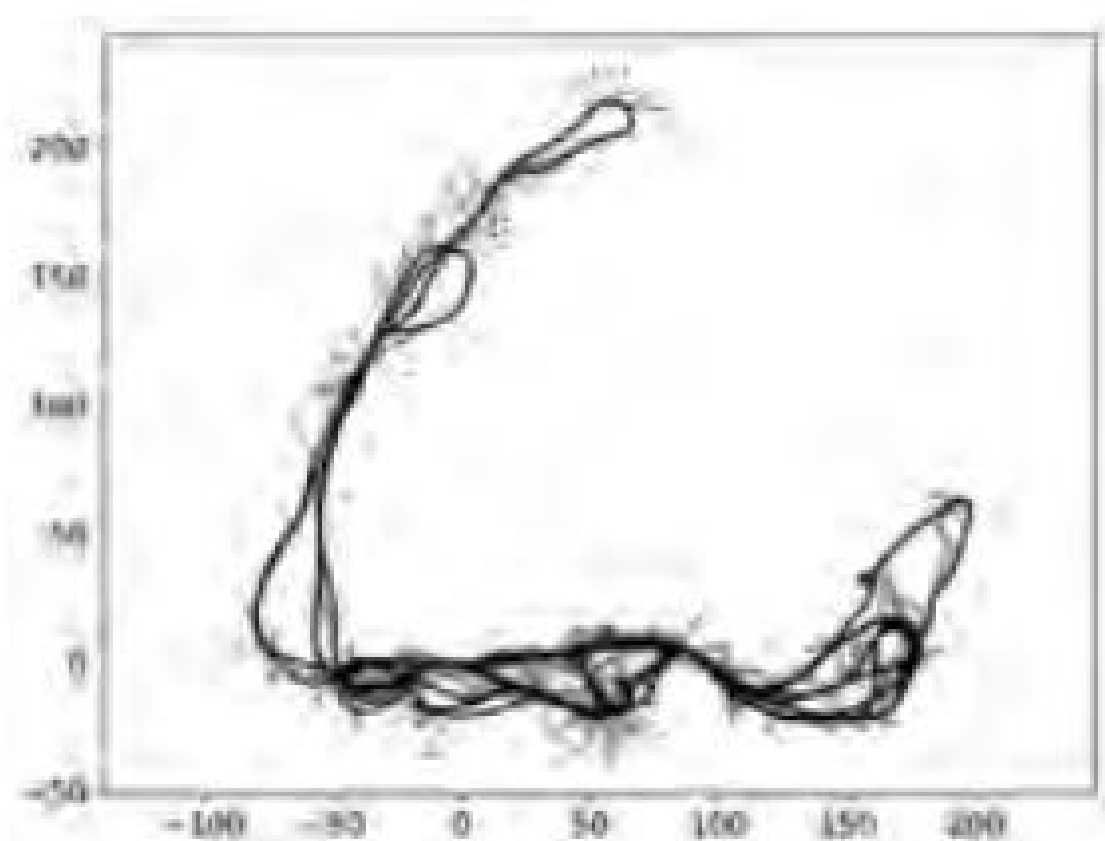
其中, $w_i \in [0, 1], i = 1, \dots, N$ 是与每个残差 r_i 关联的权重变量; 函数 $\Phi_\rho(w_i)$ 称为**离群值过程** (outlier process), 它定义对权重 w_i 的惩罚, 其形式取决于鲁棒损失 ρ 的选择。

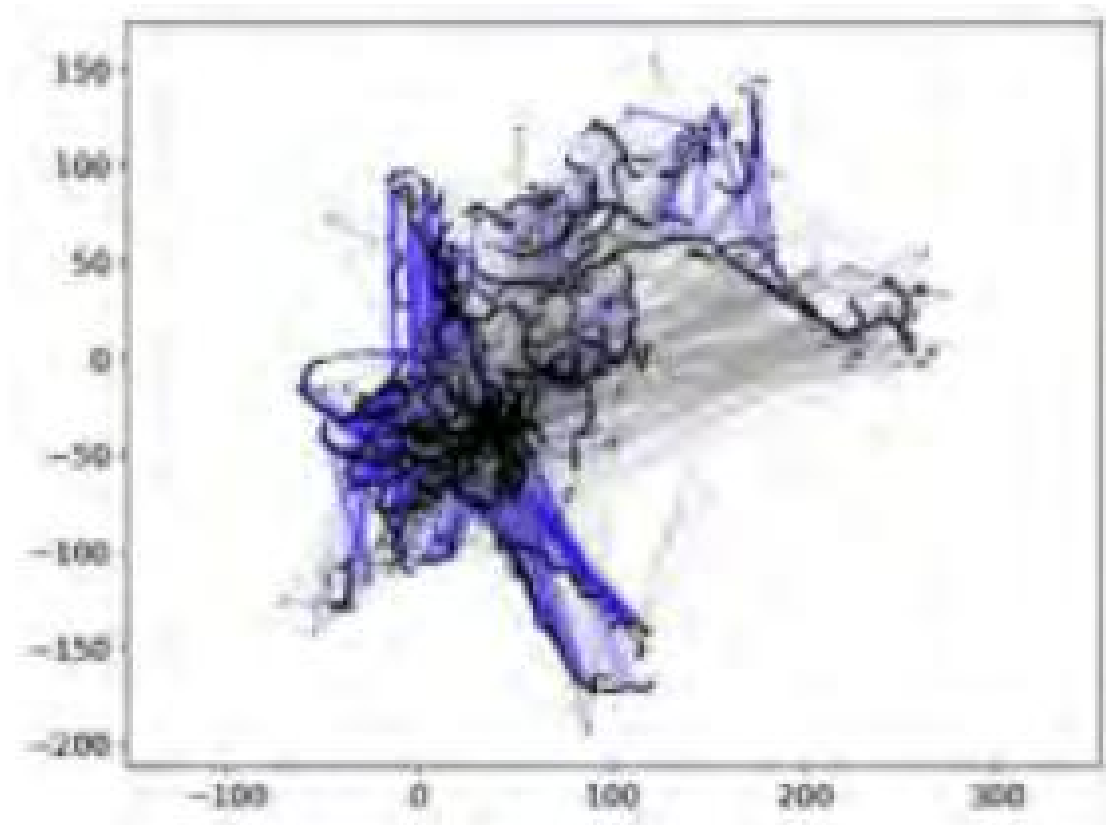
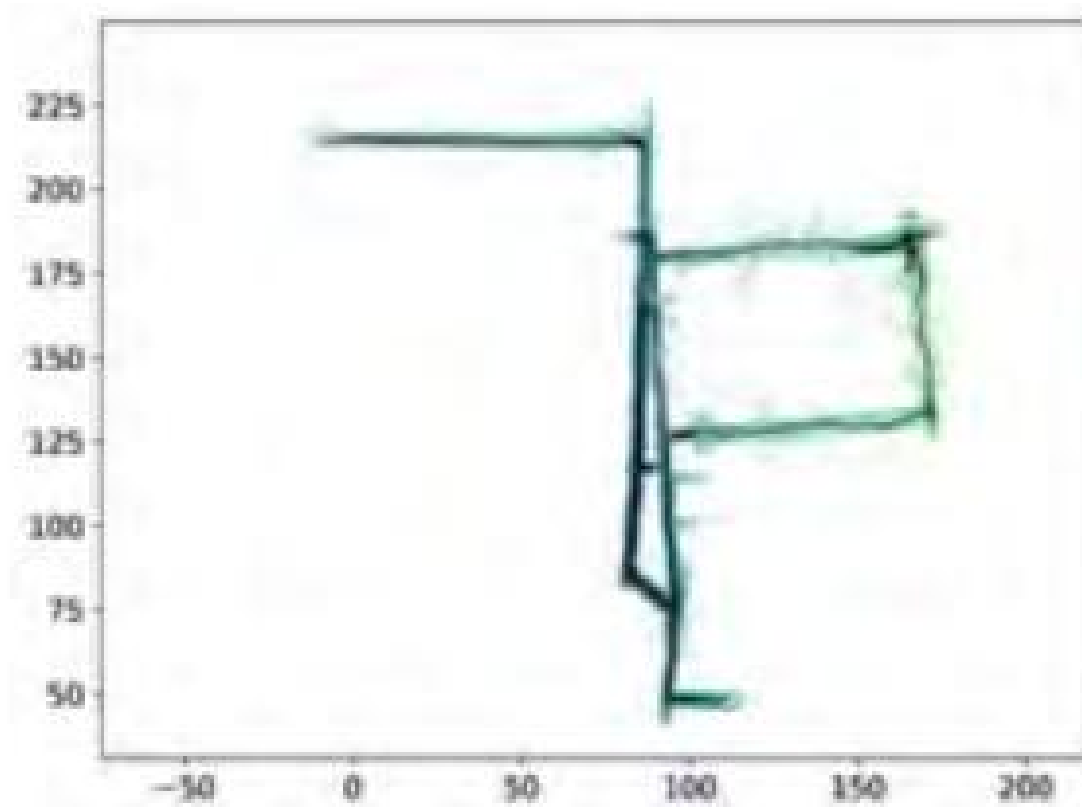
Ground Truth



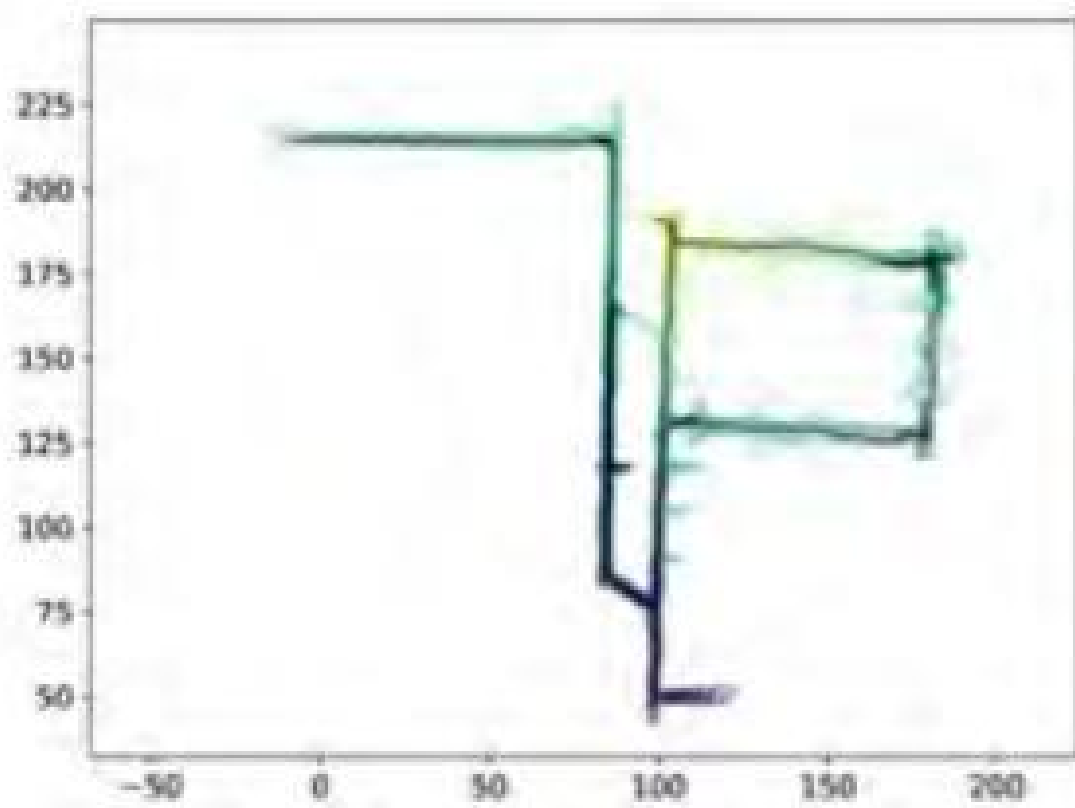
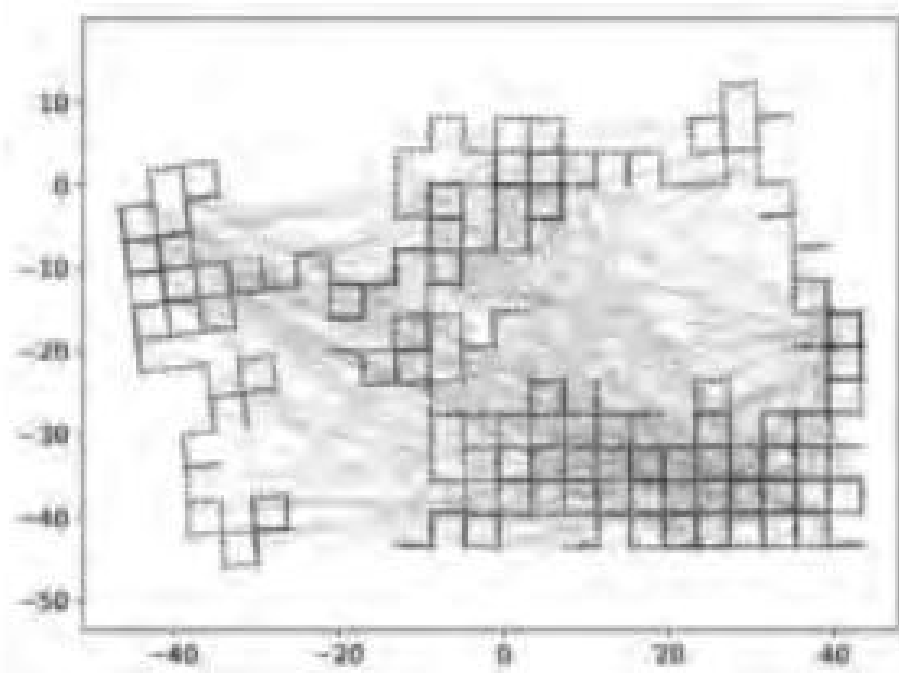
- (a)
- (b)
- (c)
- (d)
- (e)
- (f)
- (g)
- (j)
- (h)
- (k)
- (i)

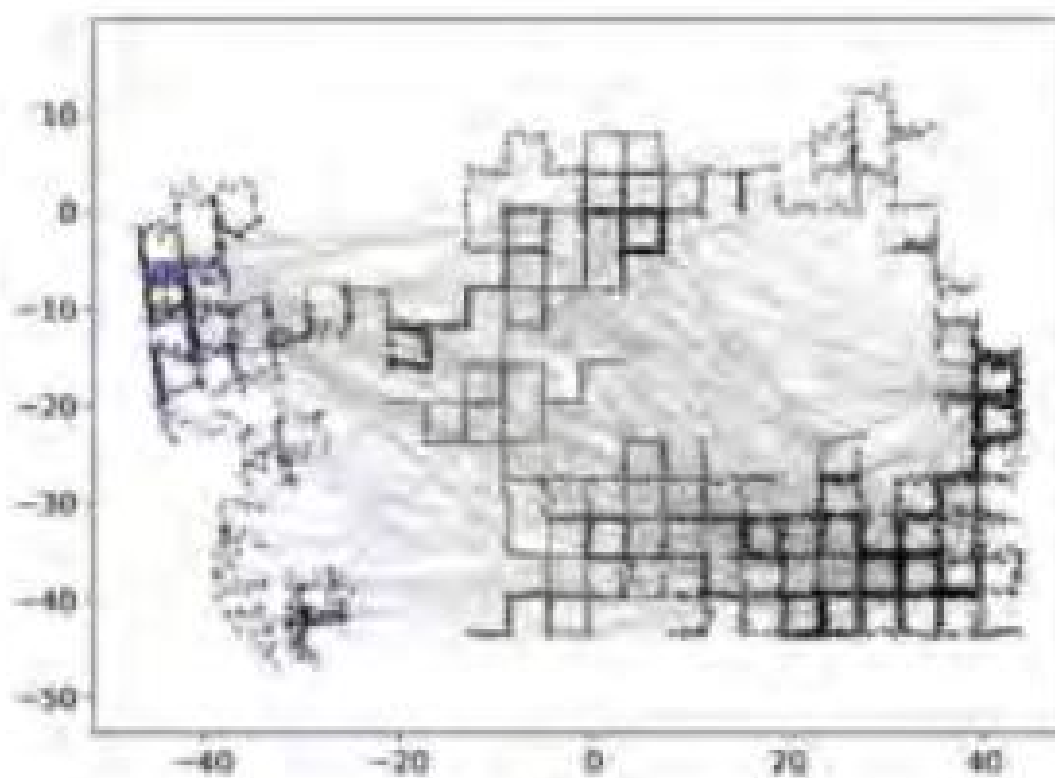
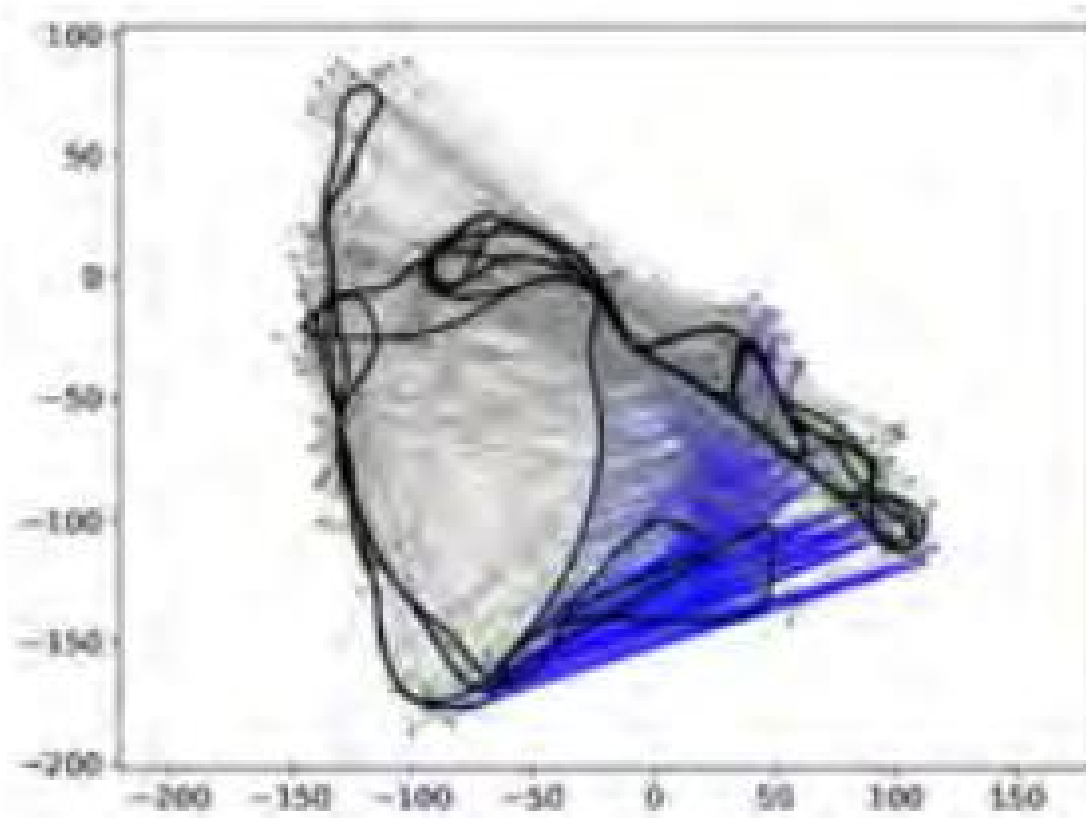


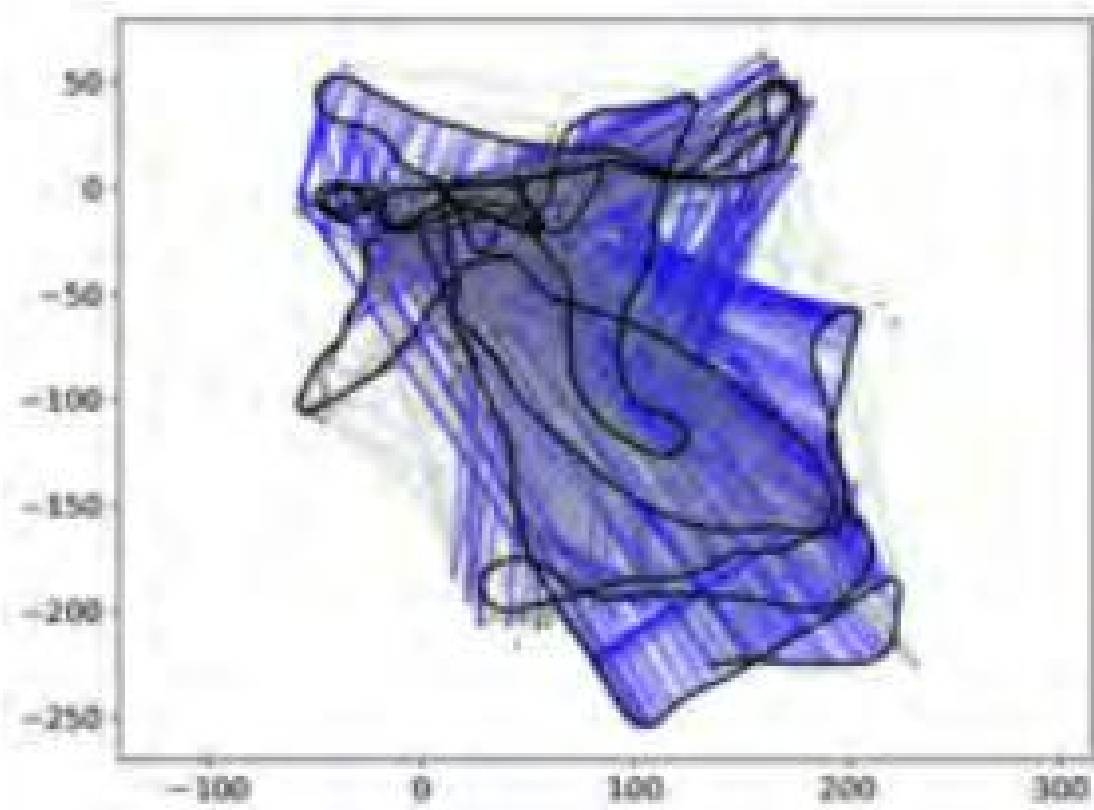
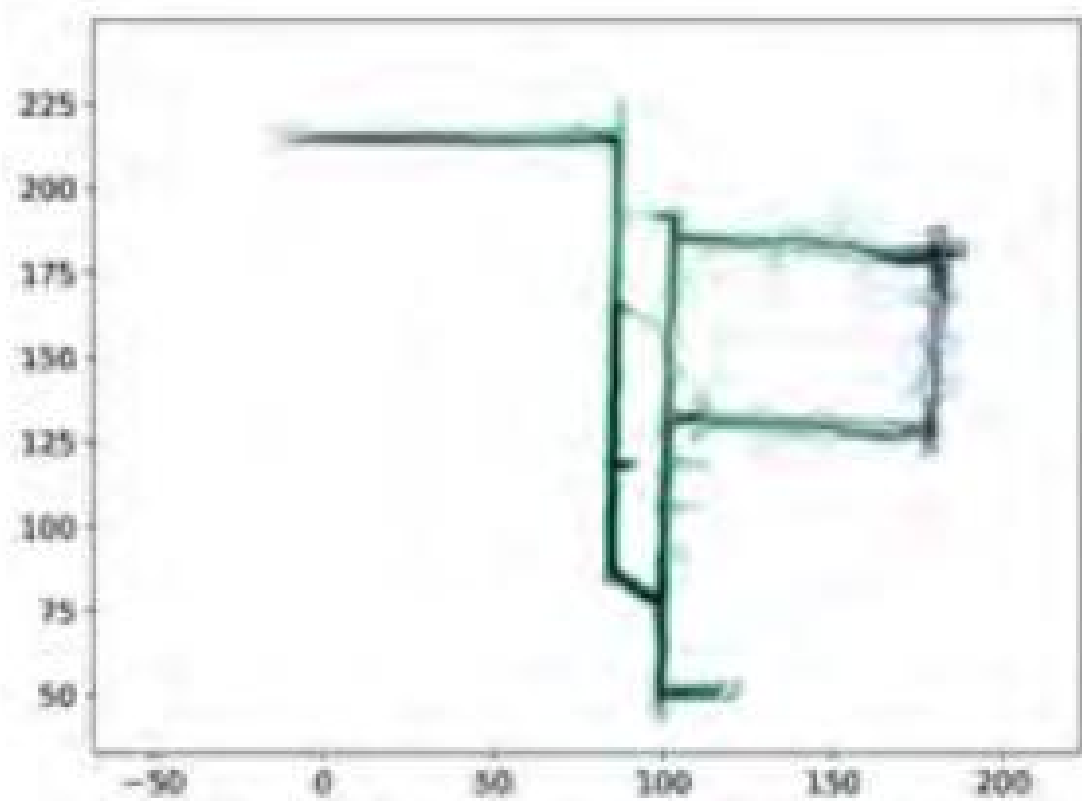




IRLS + GM







(l) 图 3.7 使用鲁棒损失函数和迭代重加权最小二乘 (IRLS) 求解含离群值的 SLAM 问题: (a)–(c) 为 M3500、SubT 和 Victoria Park 数据集的真值轨迹; (d)–(f) 为使用 Huber 损失得到的轨迹估计; (g)–(i) 为使用 Geman–McClure 损失得到的轨迹估计; (j)–(l) 为使用截断二次损失得到的轨迹估计。测量以彩色边显示: 灰色边为正确分类的边; 红色边为被错误归类为内点的离群值 (假阳性); 蓝色边为被错误归类为离群值的内点 (假阴性)。

当 ρ 为截断二次损失时, 我们已由式 (3.14) 轻易得到 $\Phi_\rho(w_i) = (1 - w_i)\beta_i^2$ 。当 ρ 取其他形式时, 文献 [85] 提供了推导 $\Phi_\rho(w_i)$ 的方法。下面给出 Geman–McClure (G–M) 鲁棒损失的例子。

例 3.5 (G–M 损失的 B–R 对偶性) 考虑 G–M 鲁棒损失函数

$$\rho(r_i(\mathbf{x})) = \frac{\beta_i^2 r_i^2(\mathbf{x})}{\beta_i^2 + r_i^2(\mathbf{x})}, \quad (3.17)$$

其中, β_i^2 是与式 (3.13) 类似的第 i 个残差的噪声界。与式 (3.17) 关联的离群值过程为

$$\Phi_\rho(w_i) = \beta_i^2 (\sqrt{w_i} - 1)^2. \quad (3.18)$$

为验证式 (3.18) 的正确性, 考虑

$$\min_{w_i \in [0,1]} w_i r_i^2(\mathbf{x}) + \Phi_\rho(w_i), \quad (3.19)$$

其最优解 (令式 (3.19) 的梯度为零) 是

$$w_i^* = \left(\frac{\beta_i^2}{r_i^2(\mathbf{x}) + \beta_i^2} \right)^2. \quad (3.20)$$

将式 (3.20) 代回式 (3.19) 的目标函数, 即可恢复 G–M 鲁棒损失 (3.17)。

3.10 交替最小化

引入 B–R 对偶性后, 使用称为**交替最小化** (alternating minimization) 的通用优化策略 [1112, 80], IRLS 算法便可自然导出。其思想是: 尽管在式 (3.16) 中同时优化 x 和 $w_i \in [0, 1]$, $i = 1, \dots, N$ 很困难, 但固定其中一者而优化另一者很容易。确实, 当 w_i 固定时, 问题 (3.16) 成为加权最小二乘; 类似地, 当 x 固定时, 问题 (3.16) 成为

$$\min_{w_i \in [0,1], i=1, \dots, N} \sum_{i=1}^N \Phi_\rho(w_i) + w_i r_i^2(\mathbf{x}),$$

它可拆为 N 个子问题, 每个子问题优化一个标量 w_i :

$$\min_{w_i \in [0,1]} \Phi_\rho(w_i) + w_i r_i^2(\mathbf{x}). \quad (3.21)$$

问题 (3.21) 易于求解, 且常有闭式解。事实上, 对 G–M 鲁棒损失, 式 (3.21) 的解正是式 (3.20)。对截断二次损失, 问题 (3.21) 为

$$\min_{w_i \in [0,1]} (1 - w_i)\beta_i^2 + w_i r_i^2(\mathbf{x})$$

并具有闭式解

$$w_i^* = \begin{cases} 1, & \text{if } r_i^2(\mathbf{x}) < \beta_i^2, \\ 0, & \text{if } r_i^2(\mathbf{x}) > \beta_i^2, \\ [0, 1], & \text{otherwise.} \end{cases}$$

总之，在 B-R 对偶性的语境下，第 t 次 IRLS 迭代在以下两步之间交替：

1. **变量更新**：使用当前权重 $w_i^{(t)}$ 求解加权最小二乘问题

$$\mathbf{x}^{(t)} \in \arg \min_{\mathbf{x}} \sum_i w_i^{(t)} r_i^2(\mathbf{x}). \quad (3.22)$$

2. **权重更新**：使用 $\mathbf{x}^{(t)}$ 更新权重

$$w_i^{(t+1)} \in \arg \min_{w_i \in [0,1]} \Phi_\rho(w_i) + w_i r_i^2(\mathbf{x}^{(t)}), i = 1, \dots, N. \quad (3.23)$$

通过 B-R 对偶性得到的权重更新规则实际上与广为使用的规则 (3.12) 一致。有趣的是，在 SLAM 中用 G-M 鲁棒损失实例化上述 IRLS 算法，会导出动态协方差缩放 (dynamic covariance scaling) 算法^[11]，该算法已在离群值鲁棒 SLAM 的语境下提出。

3.11 渐进非凸性

上一节利用 Black-Rangarajan 对偶性与交替最小化，导出了在式 (3.22) 和 (3.23) 之间交替求解的 IRLS 框架。然而，由于常见鲁棒损失的非凸性，IRLS 的收敛可能对初始化质量高度敏感，即 $\mathbf{x}^{(0)}$ 与式 (3.9) 最优解的接近程度，或 $w_i^{(0)}$ 对各测量内点/离群值归属的刻画程度。例如，文献^[1006]表明，即使测量中仅有 10% 离群值，采用截断二次损失或 Geman-McClure 损失的 IRLS 也可能失败（亦可参见图 3.7 的结果）。

本节介绍**渐进非凸性** (Graduated Non-Convexity, GNC) 算法，它可使 IRLS 对初始化质量显著不那么敏感。给定鲁棒代价函数 ρ ，GNC 的基本思想是利用标量平滑因子 μ 构造 ρ 的平滑版本 ρ_μ 。调节 μ 可控制 ρ_μ 的非凸程度：在参数范围的一端， ρ_μ 是凸的；在另一端，它恢复原始的 ρ 。下面以两个例子说明。

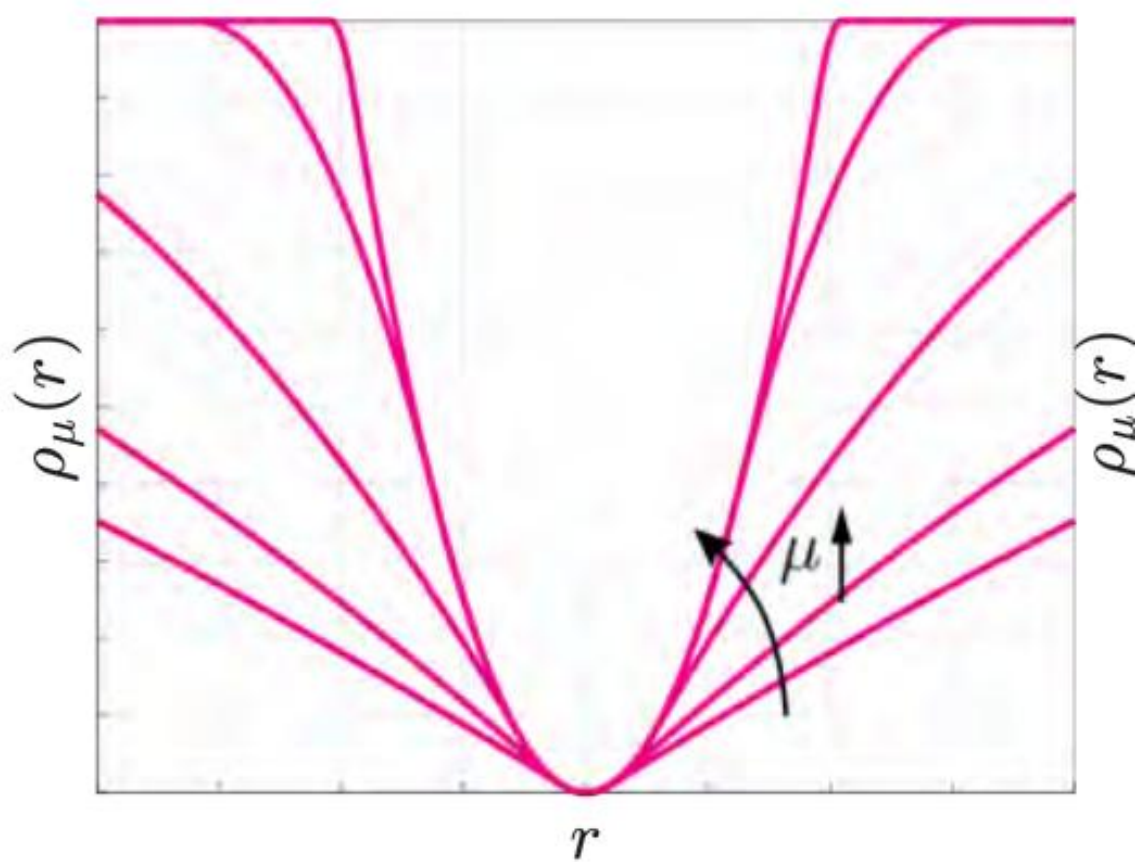
例 3.6 (GNC 截断二次损失) 考虑 GNC 截断二次损失函数

$$\rho_\mu(r_i(\mathbf{x})) = \begin{cases} r_i^2(\mathbf{x}) & \text{if } r_i^2(\mathbf{x}) \in \left[0, \frac{\mu}{\mu+1}\beta_i^2\right] \\ 2\beta_i|r_i(\mathbf{x})|\sqrt{\mu(\mu+1)} - \mu(\beta_i^2 + r_i^2(\mathbf{x})) & \text{if } r_i^2(\mathbf{x}) \in \left[\frac{\mu}{\mu+1}\beta_i^2, \frac{\mu+1}{\mu}\beta_i^2\right] \\ \beta_i^2 & r_i^2(\mathbf{x}) \in \left[\frac{\mu+1}{\mu}\beta_i^2, +\infty\right]. \end{cases} \quad (3.24)$$

当 μ 趋近于零时， ρ_μ 为凸函数；当 μ 趋近于无穷大时，它恢复式 (3.13) 的截断二次损失。

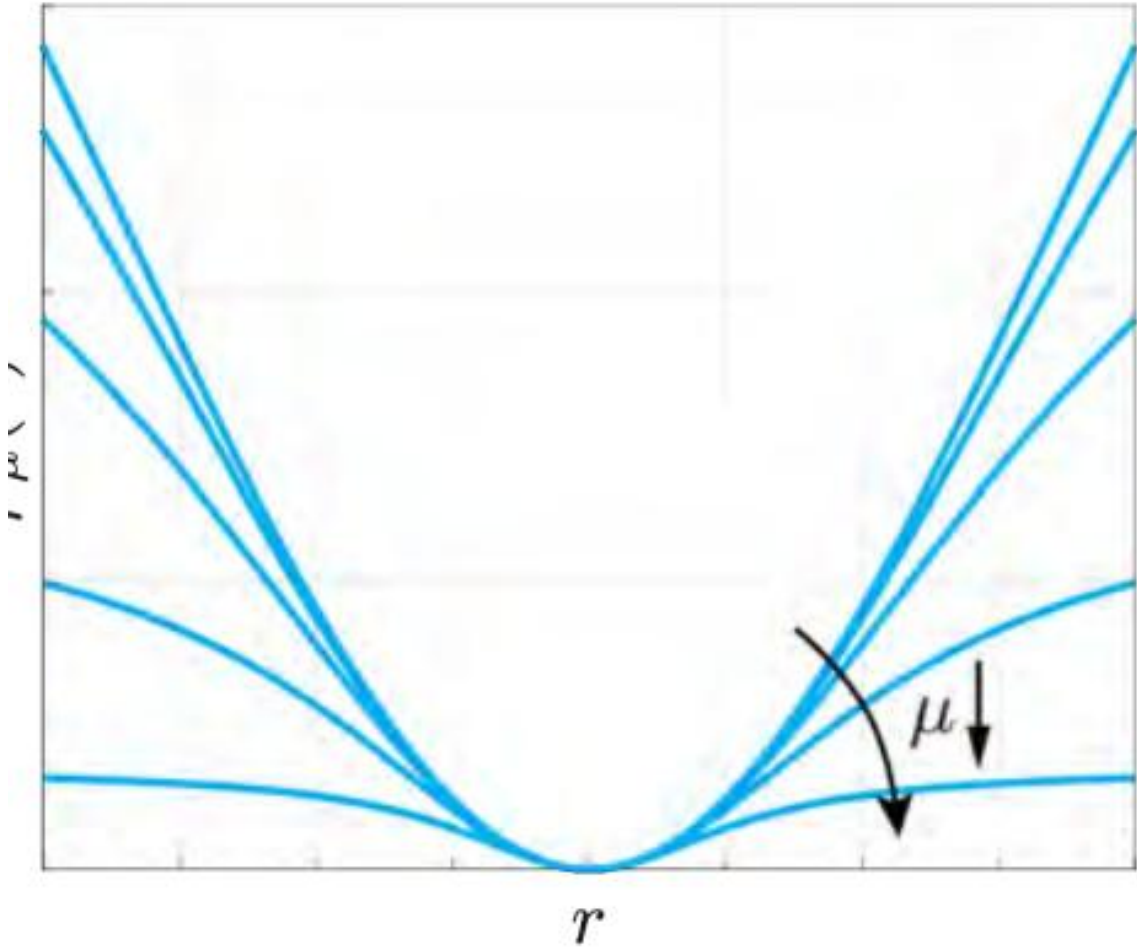
例 3.7 (GNC Geman-McClure 损失) 考虑 GNC Geman-McClure 损失函数

$$\rho_\mu(r_i(\mathbf{x})) = \frac{\mu\beta_i^2 r_i^2(\mathbf{x})}{\mu\beta_i^2 + r_i^2(\mathbf{x})}. \quad (3.25)$$



当 μ 趋近于无穷大时, ρ_μ 为凸函数; 当 $\mu = 1$ 时, 它恢复 G-M 损失 (3.17)。

(a)



(b) 图 3.8 使用控制参数 μ 的渐进非凸性: (a) 截断二次损失; (b) Geman-McClure 损失。^[1222] (©2020 IEEE)

图 3.8(a) 和 (b) 分别绘制了 GNC 截断二次损失和 GNC Geman-McClure 损失。可以看到, 增大或减小控制参数 μ 会为函数引入更多非凸性。

式 (3.24) 和 (3.25) 中平滑 GNC 函数的一个良好性质是, B-R 对偶性仍然适用。对 GNC 截断二次损失 (3.24), 应用 B-R 对偶性得到离群值过程

$$\Phi_{\rho_\mu}(w_i) = \frac{\mu(1-w_i)}{\mu+w_i}\beta_i^2.$$

对 GNC Geman-McClure 损失 (3.25), 应用 B-R 对偶性得到离群值过程

$$\Phi_{\rho_\mu}(w_i) = \mu\beta_i^2(\sqrt{w_i}-1)^2.$$

现在可以给出 GNC 算法。每次迭代执行以下三步:

1. **变量更新:** 使用当前权重 $w_i^{(t)}$ 求解加权最小二乘问题

$$\mathbf{x}^{(t)} \in \arg \min_{\mathbf{x}} \sum_i w_i^{(t)} r_i^2(\mathbf{x}). \quad (3.26)$$

2. **权重更新**: 使用 $\mathbf{x}^{(t)}$ 更新权重

$$w_i^{(t+1)} \in \arg \min_{w_i \in [0,1]} \Phi_{\rho_\mu}(w_i) + w_i r_i^2(\mathbf{x}^{(t)}), i = 1, \dots, N. \quad (3.27)$$

3. **控制参数更新**: 增大或减小 μ , 以向 ρ_μ 加入更多非凸性。

GNC 算法类似 IRLS, 区别在于它从凸的平滑函数 ρ_μ 开始, 然后迭代更新控制参数 μ , 逐渐向 ρ_μ 加入更多非凸性, 以逼近原始损失函数 ρ 。依据平滑损失 ρ_μ 的定义, 通过增大或减小 μ 可恢复原始 ρ 。例如, GNC 截断二次损失在 μ 较大时恢复原始截断二次损失, 因此每次 GNC 迭代将 μ 乘以常数因子 $\gamma > 1$ (例如 [1222] 中的 $\gamma = 1.4$)。反之, GNC Geman–McClure 损失在 μ 接近 1 时恢复原始 G–M 损失, 因此每次 GNC 迭代将 μ 除以 $\gamma > 1$ 。

图 3.9 展示在图 3.6 的相同三个数据集上应用 GNC 所得的 SLAM 轨迹, 采用两种不同的鲁棒损失: Geman–McClure 损失和截断二次损失。我们用 GTSAM 的 GNCOptimizer 实现 GNC。比较图 3.6 和图 3.7 可知, GNC 保证了更好的收敛性 (即较不易陷入局部极小值), 并在全部三个数据集中恢复出相当准确的轨迹。尽管 GNC 已表现出对离群值极强的耐受性 (例如, 在真实问题中可耐受约 80%–90% 的错误回环 [1222, 165]), 但该方法不提供收敛保证。此外, 实验表明其性能依赖具体问题; 它在位姿图优化问题中表现优异, 但在其他感知问题中性能会显著退化 [1005]。

下面展示一个 GNC 失败的问题, 并说明组合前端和后端离群值剔除可能是有益的。为此, 考虑一个比此前更具挑战性的 SLAM 设置。本章前面讨论位姿图优化问题 (例如图 3.1) 时, 假设里程计可靠, 而回环或路标测量中可能存在离群值。该设置本质上假定存在一条“里程计骨架” (odometry backbone): 它提供一组可信测量, 并可构建机器人位姿的初始猜测。虽然这一假设在许多问题中现实可行, ¹⁴ 但某些 SLAM 应用可能缺乏里程计骨架。例如, 由于轮式里程计中的车轮打滑或 LiDAR 里程计中的错误对齐, 某些里程计传感器可能产生错误测量。另一个例子是多机器人 SLAM: 每个机器人都有里程计骨架, 但包括全部机器人轨迹的整体 SLAM 问题却没有 [728]。

M3500

(a) SubT

Victoria Park

(b) (c)

(d)

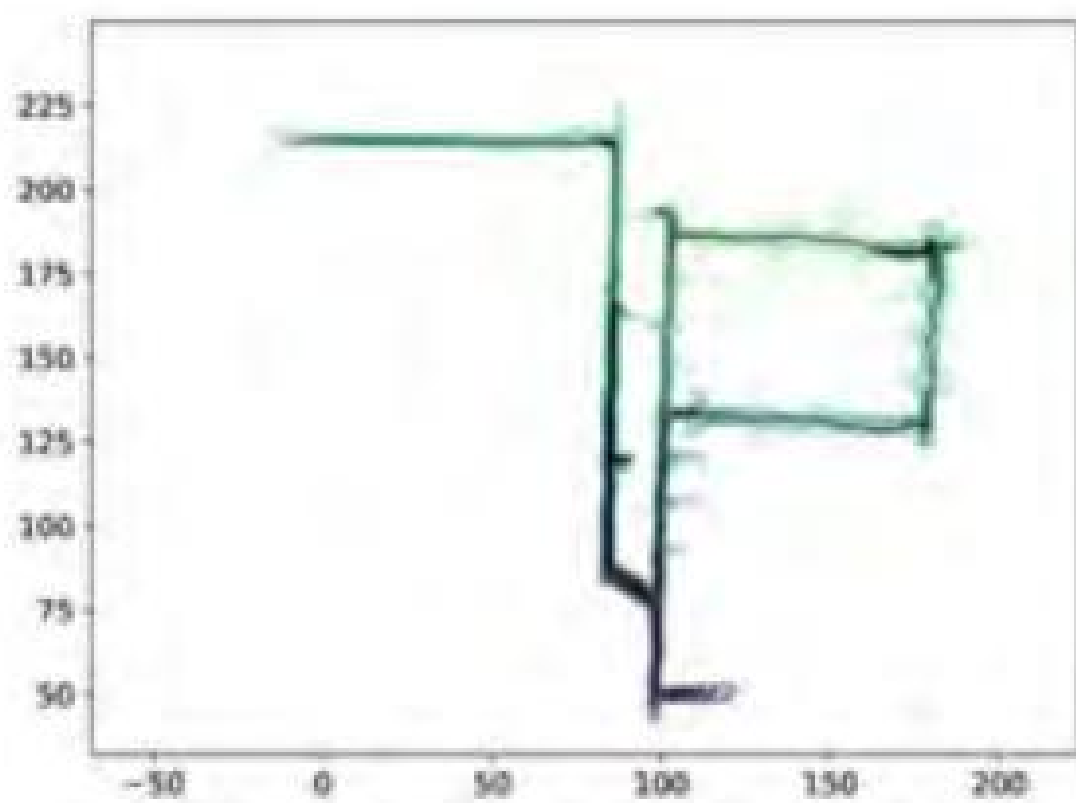
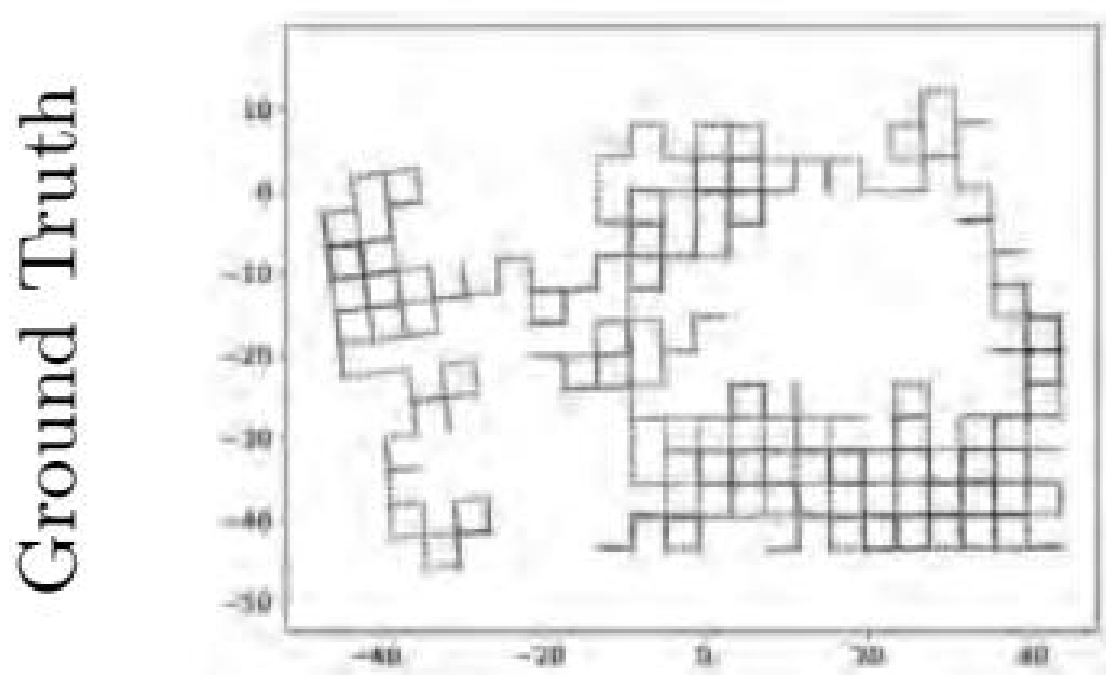
(g)

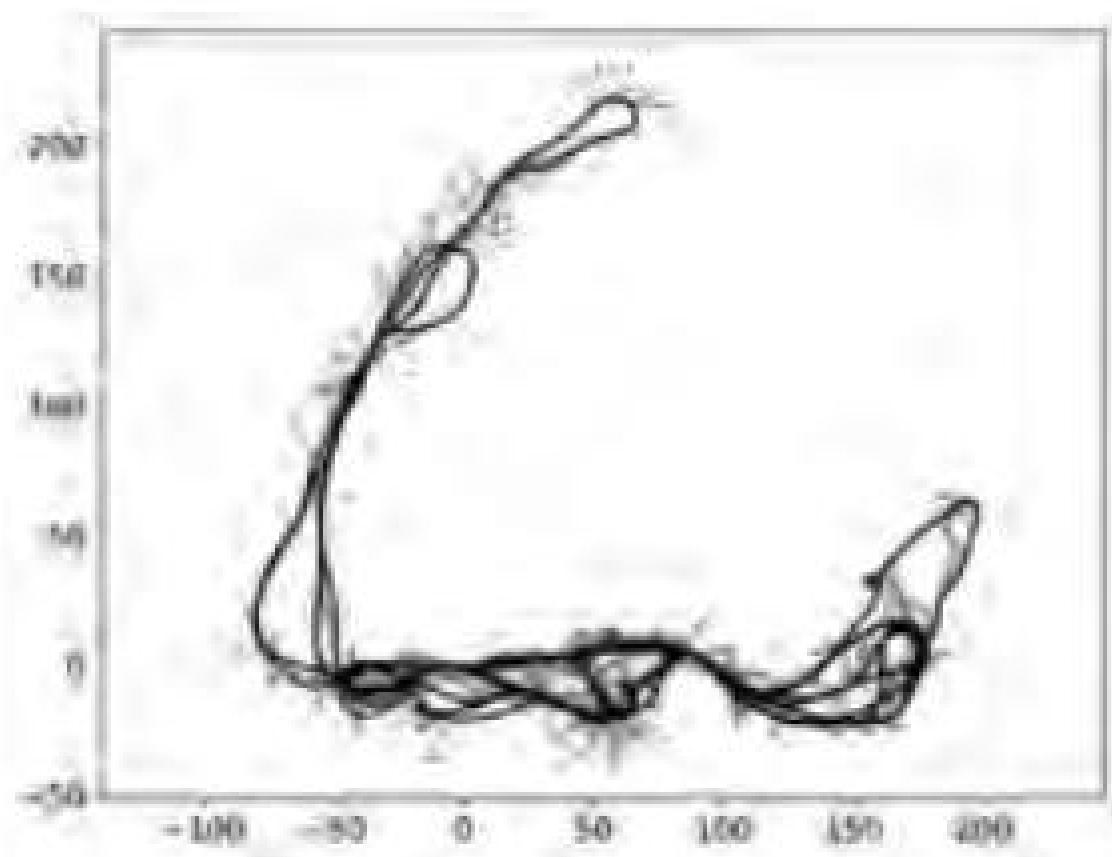
(f) (e)

(h)

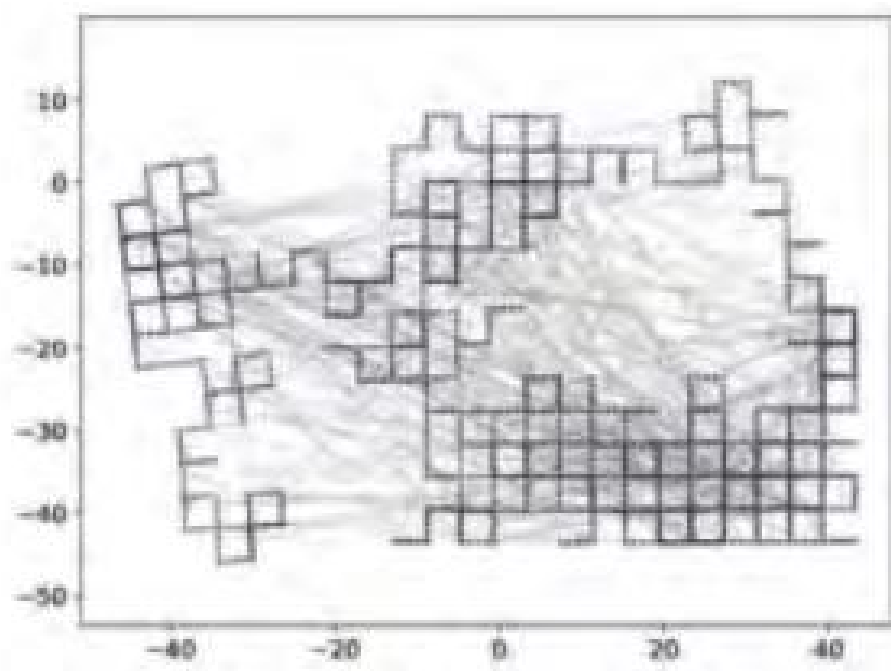
(i) 图 3.9 使用鲁棒损失函数和渐进非凸性 (GNC) 求解含离群值的 SLAM 问题: (a)–(c) 为 M3500、SubT 和 Victoria Park 数据集的真值轨迹; (d)–(f) 为使用 Geman–McClure 损失得到的轨迹估计; (g)–(i) 为使用截断二次损失得到的轨迹估计。测量以彩色边显示: 灰色边为正确分类的边; 红色边为被错误归类为内点的离群值 (假阳性); 蓝色边为被错误归类为离群值的内点 (假阴性)。

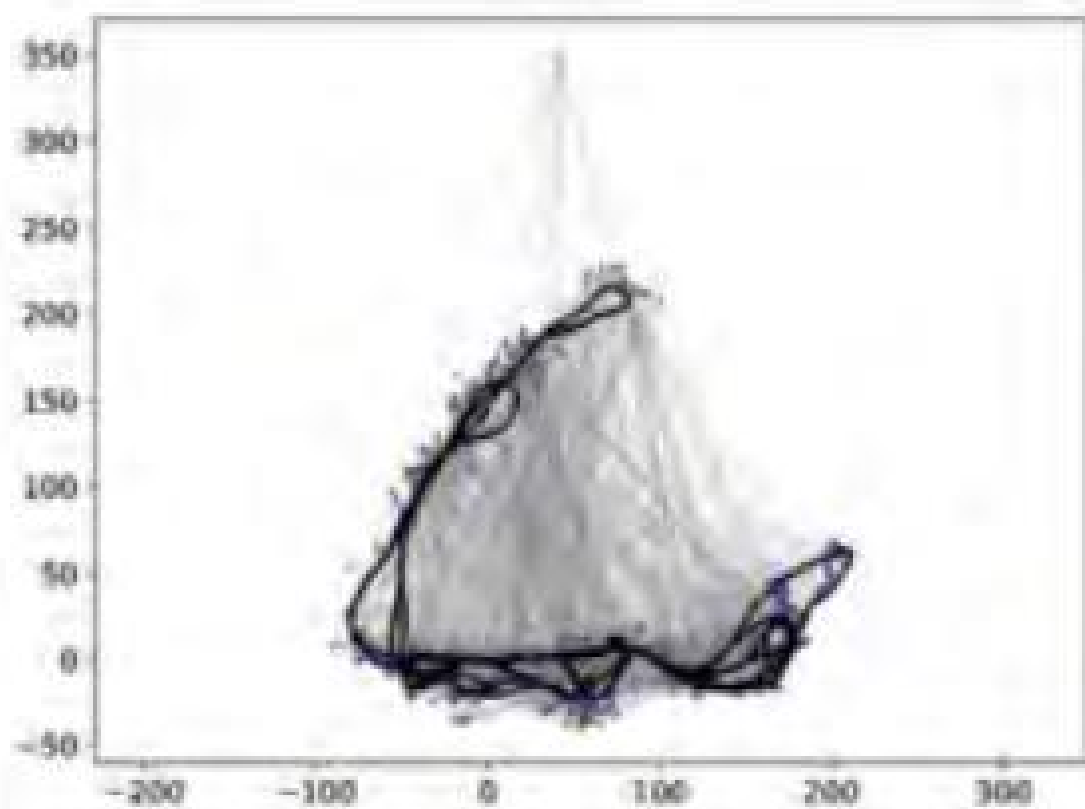
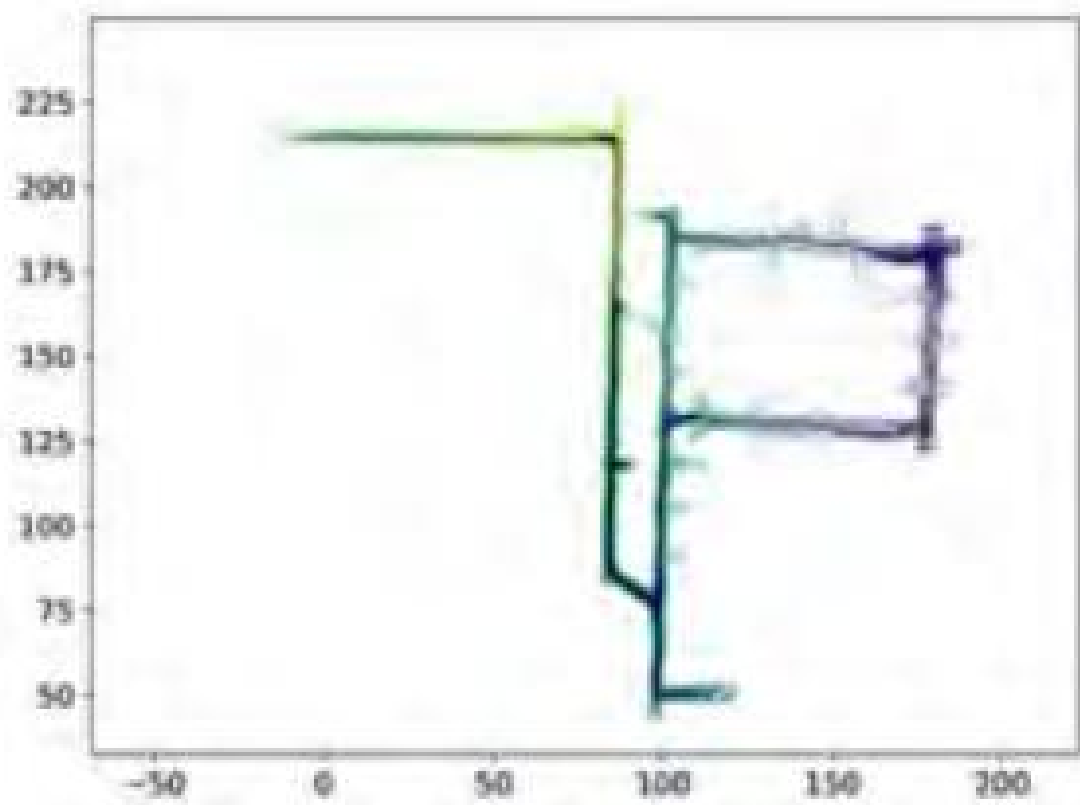
M3500

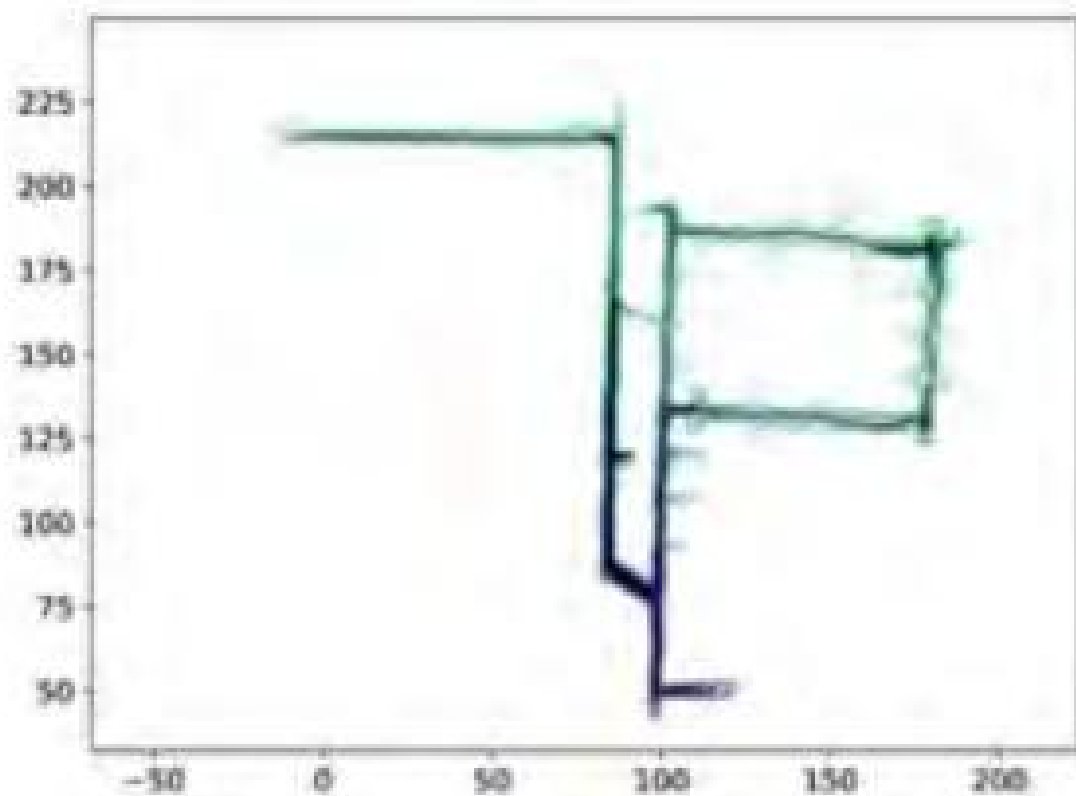
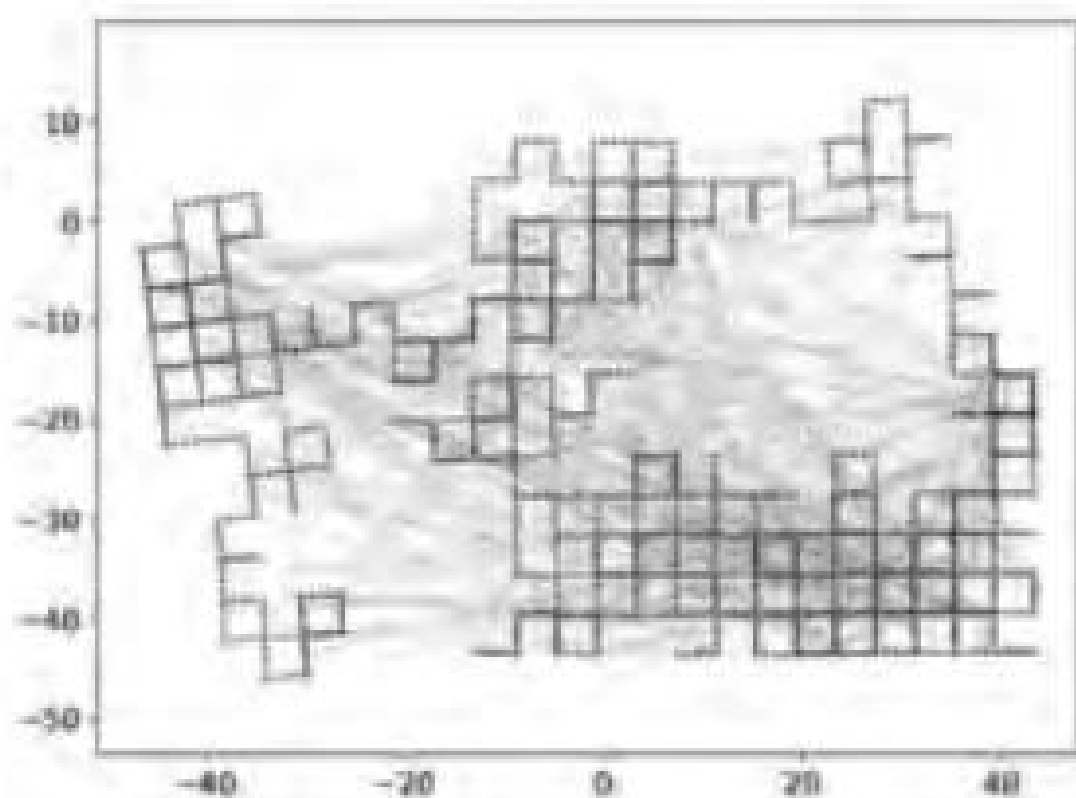


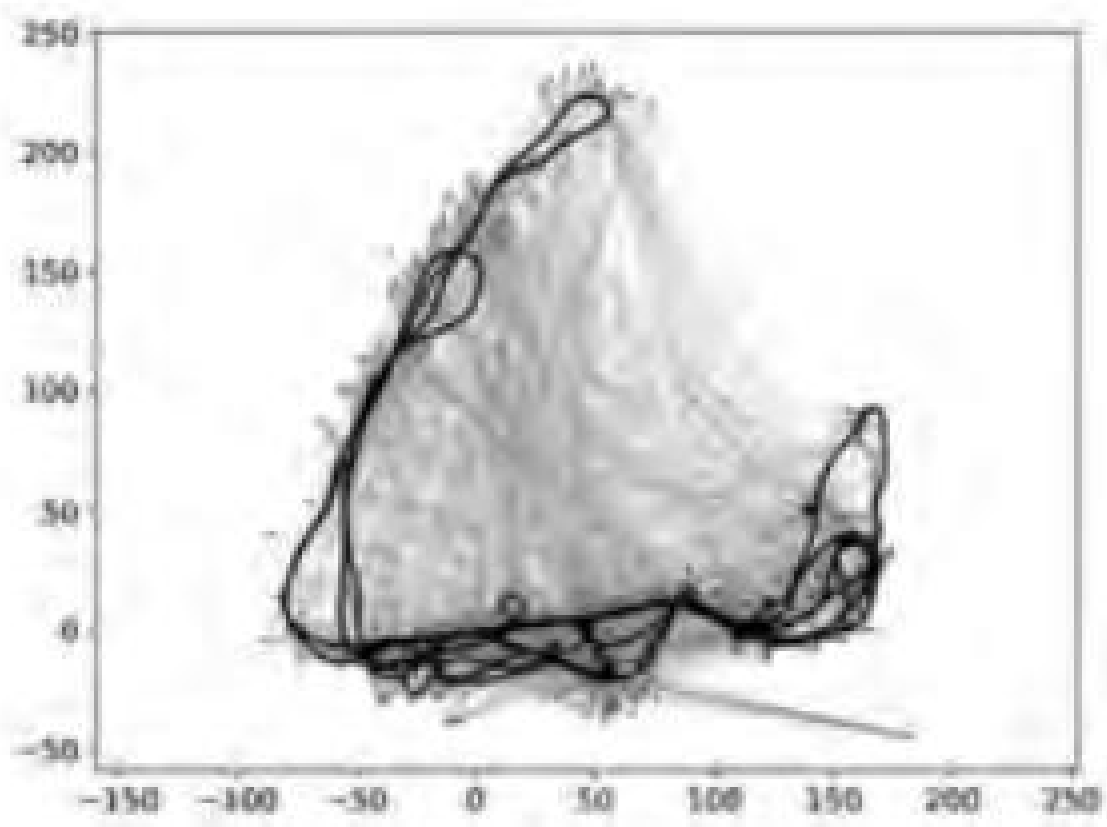


GNC + GM

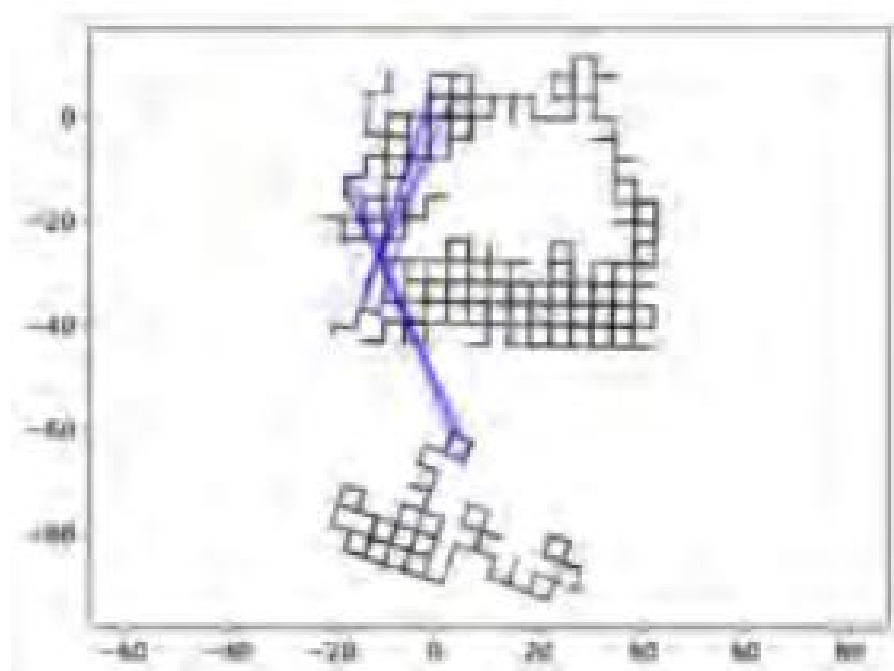




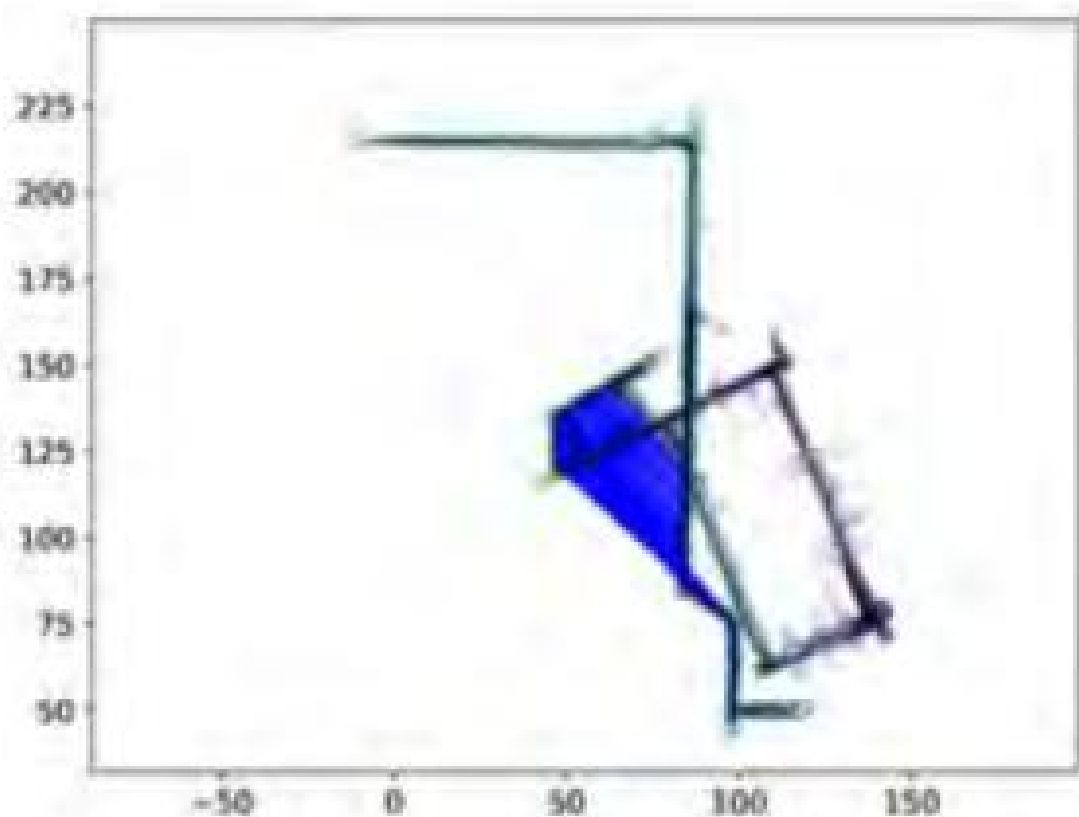




GNC



(a)
SubT

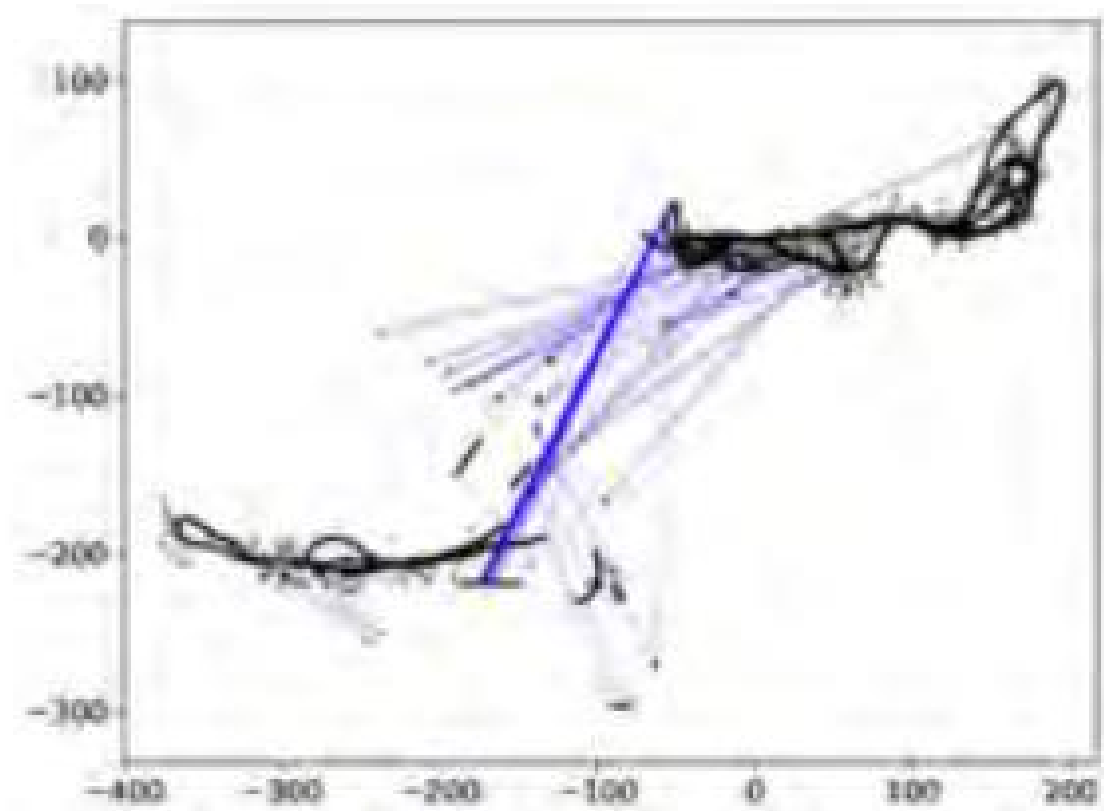


Victoria Park

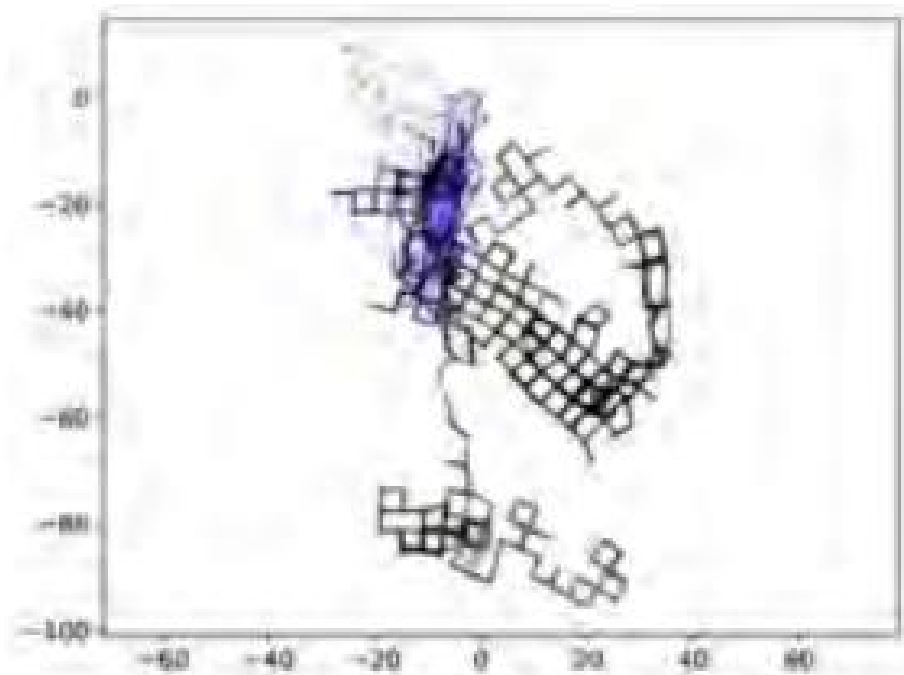
- (b)
- (c)
- (d)
- (g)
- (e)
- (h)
- (f)

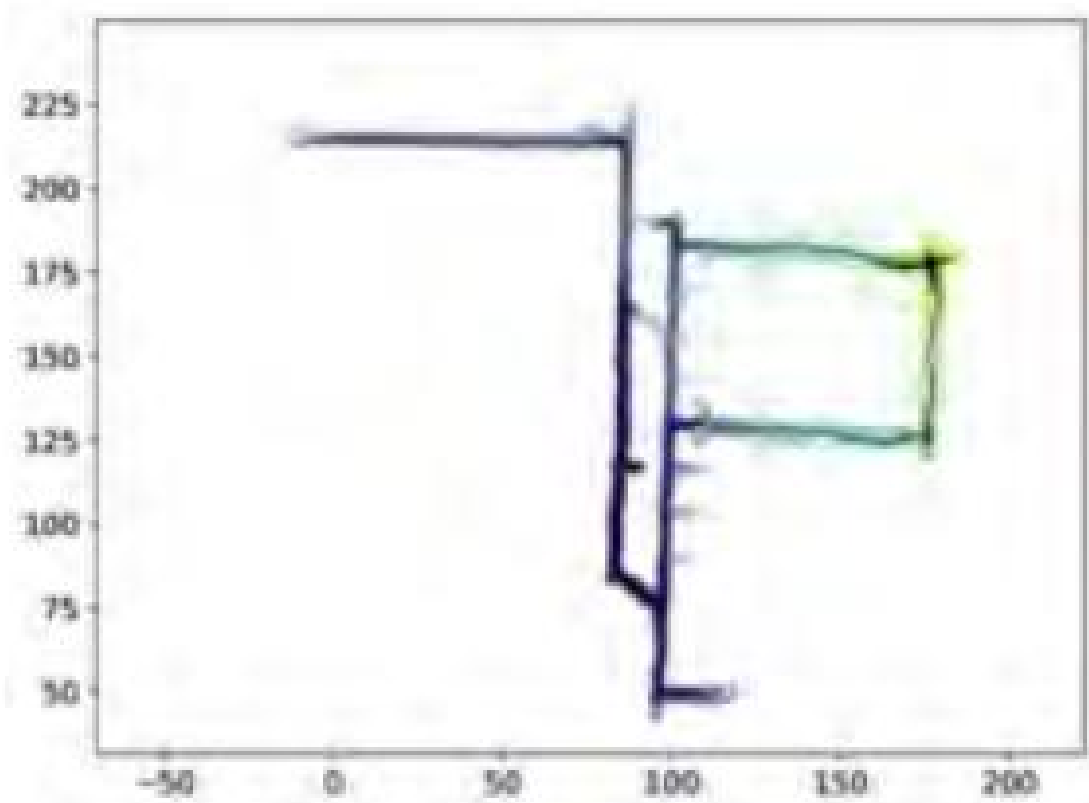
(i) 图 3.10 回环、路标测量和里程计中均含离群值的 SLAM 问题：(a)–(c) 为使用截断二次损失的 GNC 得到的轨迹估计；(d)–(f) 为使用 PCM 进行前端离群值剔除、随后采用最小二乘优化得到的轨迹估计；(g)–(i) 为先使用 PCM 进行前端离群值剔除、再使用带截断二次损失的 GNC 得到的轨迹估计。

在里程计与回环测量均可能为离群值时，鲁棒地求解 SLAM 问题极其困难。为展示所讨论方法在这个更复杂设置下的不足，我们先通过随机扰动两条里程计测量，修改图 3.1 中的位姿图问题，然后尝试用 GNC 求解。具体地，在 GNC 中，除了两条可能被扰动的测量之外，所有里程计测量均固定为内点。图 3.10 展示了 GNC 求解得到的轨迹。由于初始化受到扰动，GNC 失败并收敛到一个局部极小值，将所有测量（包括内点）都归类为离群值（蓝色边）。同



PCM





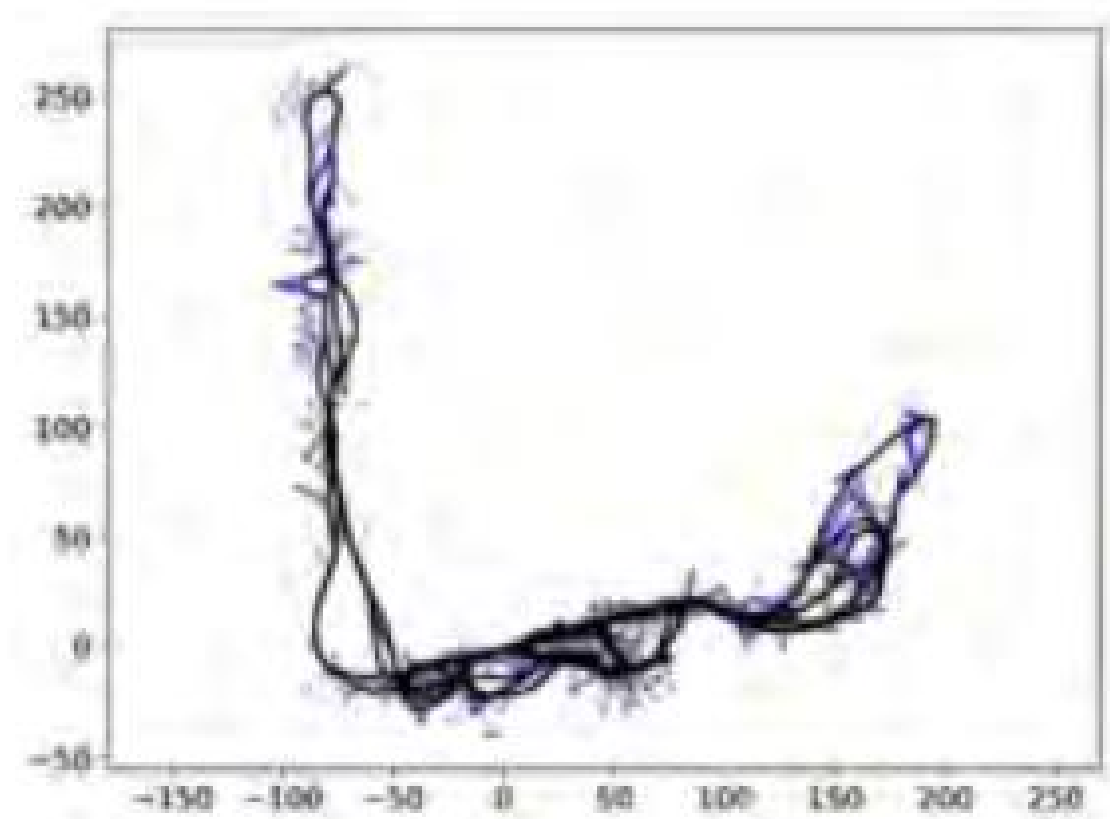
一图还展示了在采用最小二乘后端之前使用 PCM 过滤严重离群值的结果。PCM 与初始猜测无关，因此在 SubT 和 Victoria Park 数据集上能够收敛到更好的解（图 3.10(e) 和 (f)）。有趣的是，在 SubT 和 Victoria Park 数据集上，组合前端离群值剔除（PCM）与 GNC 获得了最佳结果。与此同时，三种方法（GNC、PCM 以及 PCM+GNC）在 M3500 数据集上均未能收敛到可接受的解，这证实了这一 SLAM 设置的困难性，以及现有 SLAM 算法在离群值剔除方面的局限。

3.12 延伸阅读与近期趋势

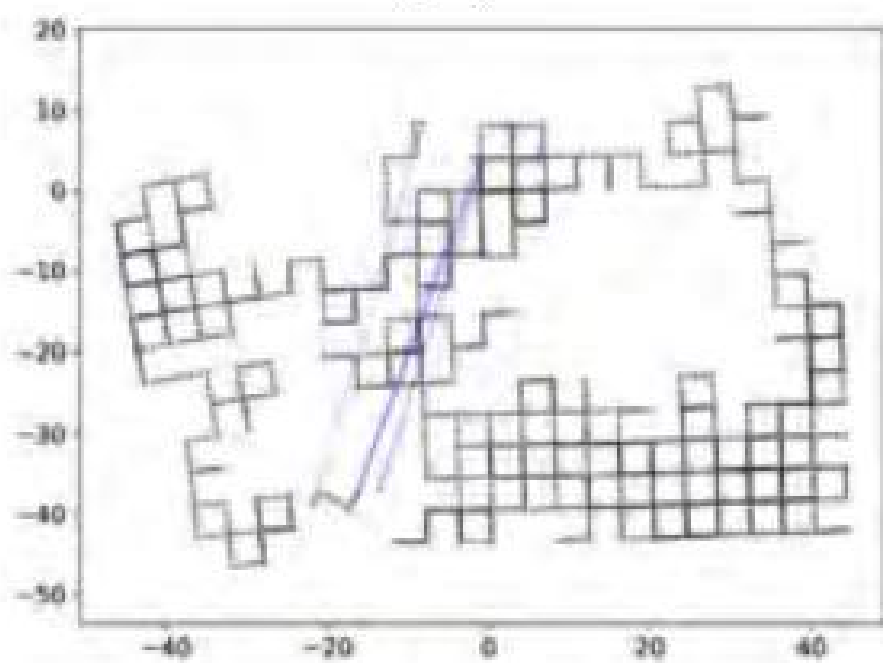
一致性最大化。尽管本章讨论的是 RANSAC 算法最基本的实例（依照 [328] 的最初提议），但值得一提的是，文献中给出了许多 RANSAC 变体，包括通过局部优化精化估计的变体 [216]、在 RANSAC 迭代中使用更好评分（而非一致集大小）的变体（例如 MLESAC [872]），以及在 RANSAC 迭代中对采样施加偏置的变体（例如 PROSAC [215]）。近期文献还包含可微 RANSAC 变体 [1174]，以及在式 (3.4) 中的参数 γ 未知时尝试寻找内点的变体（例如 [46]）。文献 [1109] 对 RANSAC 变体进行了近期综述与评估。

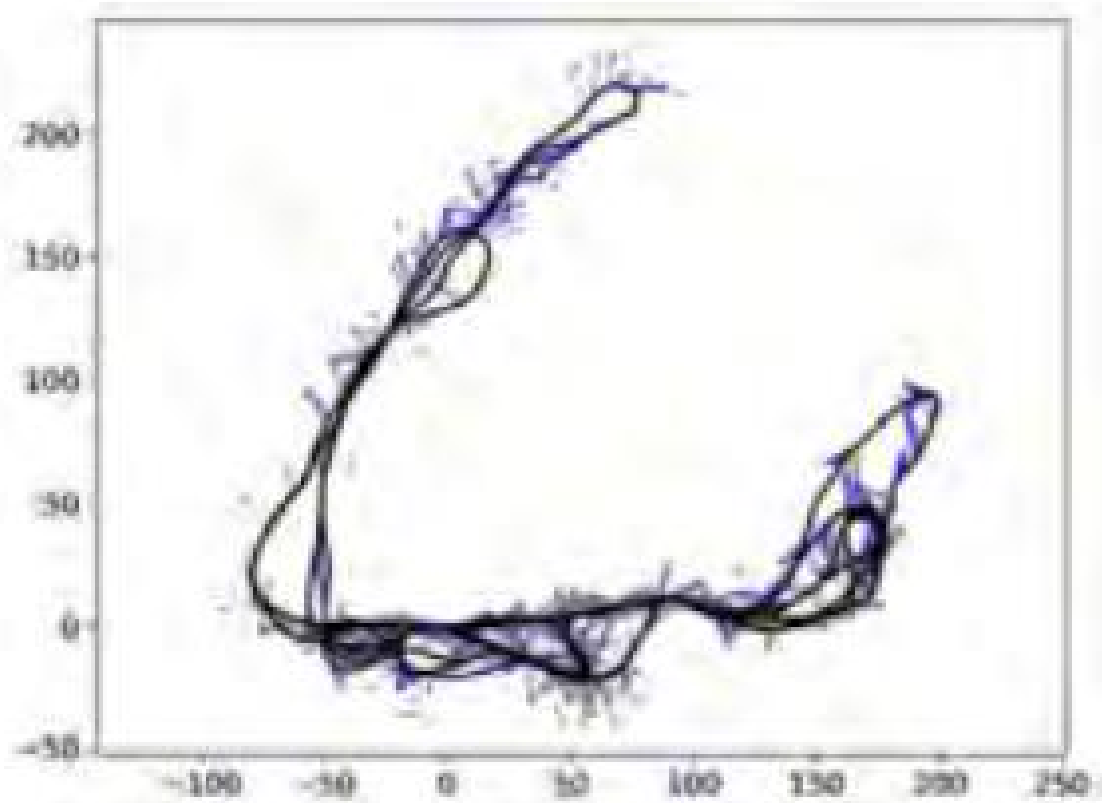
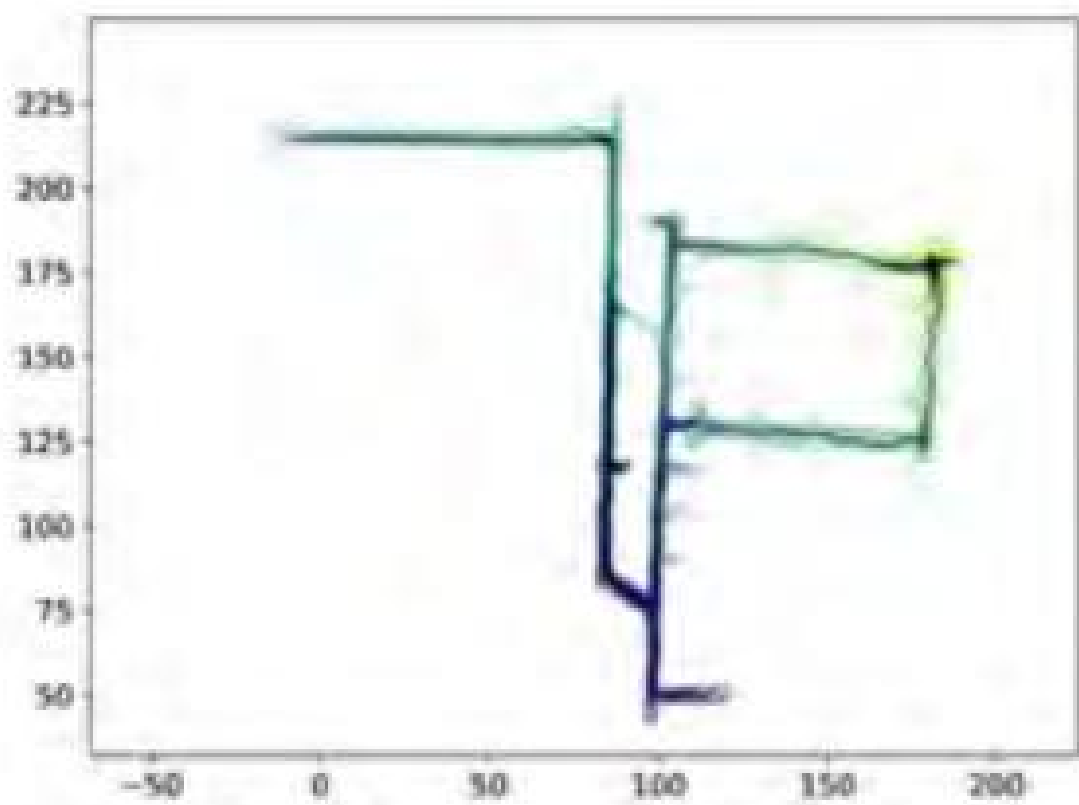
除 RANSAC 外，文献还包含精确一致性最大化方法，通常基于分支定界 [64, 436, 1290, 651, 1026, 203, 507, 132, 1226, 1]。尽管具有全局最优性保证，分支定界在最坏情形下具有指数运行时间，无法扩展到高维问题。

成对一致性最大化。第 3.2.2 节所述的 PCM 方法最初由 Mangelson 等人 [728] 在多机器人 SLAM 的语境下提出，并显示出特别的潜力。计算机视觉中也研究了基于图论的离群值剔



PCM + GNC





除。San Segundo 和 Artieda^[967] 构建关联图，并为二维图像特征匹配寻找最大团。Perera 和 Barnes^[869] 用团形式化方法分割刚体运动下的对象。Leordeanu 和 Hebert^[639] 通过近似谱方法，在对应图中寻找强连接簇来建立图像匹配。Enqvist 等人^[309] 开发了基于近似点覆盖的三维-三维和二维-三维配准离群值剔除算法。Yang 和 Carlone^[1217]、Yang 等人^[1223] 与 Bustos 等人^[133] 研究了基于最大团的三维-三维配准图论离群值剔除。检查一个测量子集内部一致性的思想也出现在 Latif 等人^[628] 中：该工作通过聚类测量，并用基于卡方的检验检查一致性，从而剔除回环离群值。PCM 论文^[728] 与本章讨论一样聚焦于成对一致性。近期，PCM 已在^[1005, 1006, 332] 中推广到群组 k 一致性，即一致性约束 (3.7) 涉及 k 个测量而非仅两个测量的情形。这些论文本质上将一致性图的概念推广为一致性超图，其中每条超边涉及 k 个节点。相关工作还考虑最大团问题的软变体：将二元条件 (3.7) 放宽，以在一致性图的边上产生连续权重^[712, 711]。这些方法已应用于包括地下探索^[298]、LiDAR 点云定位^[713]、多机器人度量-语义建图^[1095] 和非结构化环境全局定位^[33] 在内的实际应用。

交替最小化与渐进非凸性。 M-估计一直是机器人学^[99] 和视觉^[978, 173] 中常用的鲁棒估计方法。Tavish 和 Barfoot^[1075] 研究了不同损失函数的性能。若干论文研究了式 (3.16) 那样带辅助变量的形式化，却未认识到 Black-Rangarajan 对偶性 (定理 3.4) 所提供的与 M-估计的联系。例如，Sunderhauf 和 Protzel^[1059]、Sunderhauf 和 Protzel^[1058] 以及 Agarwal 等人^[11] 用负责禁用离群值的潜在二元变量扩展问题。Lee 等人^[637] 使用期望最大化。Olson 和 Agarwal^[827] 使用最大混合分布近似多模态测量噪声。近期，Barron^[57] 提出一个单参数函数，概括 M-估计中一族鲁棒损失函数。Chebrolu 等人^[174] 设计了一个期望最大化算法，以同时估计未知量 x 并选择合适的鲁棒损失函数 ρ 。渐进非凸性算法最初由^[86, 85] 为早期视觉应用中的离群值剔除提出；近期该算法已用于点云配准^[1297, 1223]、SLAM^[1222] 和其他应用^[34]。近期，Peng 等人^[860] 提出了一种与 GNC 和 IRLS 相似的算法，基于优化中的平滑主化 (smooth majorization) 思想，适用于广泛的鲁棒损失。此外，Peng 等人^[860] 还为 GNC 推导了全局和局部收敛保证。

可认证算法。至此所述算法大致可分为两类：(i) 快速启发式方法 (例如 RANSAC 或 M-估计的局部求解器)，它们高效但性能保证很少；(ii) 全局求解器 (例如分支定界)，它们提供最优性保证但难以随问题规模扩展。近年来出现了一类新方法，称为**可认证算法** (certifiable algorithms)，试图在可处理性与最优性之间取得平衡。可认证算法将非凸鲁棒估计问题松弛为凸半定规划 (semidefinite programs, SDP)，¹⁵ 其解可在多项式时间内获得，并提供可直接检验的事后全局最优性证书。鲁棒估计的可认证算法已在旋转估计^[1218]、三维-三维配准^[1223] 和位姿图优化^[618, 153] 的语境下提出。文献^[1219, 1221] 描述了一种为含离群值问题推导可认证算法的相当通用的方法，而^[147] 讨论了其与统计学并行工作的联系。除少数显著例外外，这些算法尽管在多项式时间内运行，计算代价仍高，且通常比启发式方法慢得多。在一些情形中，这些算法背后的洞见可用来认证快速启发式方法获得的解的最优性^[1219]，从而兼得二者之长。

第四章 可微优化

作者：Chen Wang、Krishna Murthy Jatavallabhula、Mustafa Mukadam

如第 I 章所述，当代 SLAM 系统的设计通常遵循前端与后端架构。在这一架构中，前端通常负责预处理传感器数据，并生成机器人轨迹和环境地图的初始估计；后端则细化这些初始估计，以提升整体精度。机器学习的近期进展带来了基于神经网络的新方法，有望增强 SLAM 前端的部分功能。例如，基于深度学习的方法在特征检测与匹配^[971, 267, 1267]以及前端运动估计^[1167, 1082]中表现出令人瞩目的性能。这些方法从大量样例构成的数据集训练神经网络，而无须针对任务显式编程。与此同时，基于几何的方法仍然是 SLAM 后端不可或缺的组成部分，主要原因在于它们具有普适性，并能通过求解优化问题产生全局一致的估计^[1208]。

原则上，可以将基于学习的 SLAM 前端直接接入 SLAM 架构，再将相应输出送入后端；但学习技术的使用也为不再单向的信息交换打开了空间。具体而言，后端现在可以向前端提供反馈，使前端能够直接从后端估计中学习，从而让两个模块以更协调的方式共同降低估计误差。调和几何方法与深度学习、利用两者的互补优势，是大量近期 SLAM 工作的一条主线。特别是，一个新兴趋势是对 SLAM 后端中出现的基于几何的优化问题进行微分。直观而言，对一个优化问题进行微分，可以理解该问题的最优解（例如 SLAM 估计）如何依赖于该问题的参数；在这里，这些参数就是学习型前端产生的测量。由此可以优化前端以最大化 SLAM 精度。也可以将其视为一个双层优化问题：上层优化过程受制于下层优化；具体来说，用于训练前端的基于神经网络的优化受制于根据给定前端输出计算 SLAM 解的基于几何的优化。

能够端到端地穿过优化过程计算梯度，是求解双层优化问题的核心；这使神经模型能够利用优化所编码的几何先验。这一方案的灵活性已在诸多应用中带来很有前景的先进结果，例如运动恢复（structure from motion）^[1080]、运动规划^[81, 1216]、SLAM^[511, 1082]、束调整（BA）^[1068, 1267]、状态估计^[1244, 182]和图像配准^[716]。

本章介绍如何对 SLAM 中出现的非线性最小二乘问题进行微分的基础知识。具体地，第 4.1 节重述非线性最小二乘（nonlinear least squares, NLS）问题；第 4.2 节说明如何穿过 NLS 问题进行微分；第 4.3 节讨论定义在流形上的问题如何微分；第 4.4 节讨论上述微分的数值挑战，并介绍相关机器学习库；最后，第 4.5 节给出当代 SLAM 系统中可微优化的示例。

4.1 非线性最小二乘回顾

非线性最小二乘（NLS）通过最小化观测值与模型预测值之差的平方和来估计模型参数。与线性最小二乘不同，NLS 中的模型关于参数是非线性的。除第 1 章中的因子图外，这一方法也广泛用于统计学、物理学和工程学等领域；当变量之间的关系并不直接时，它有助于将复杂模型拟合到数据，从而实现更准确且更鲁棒的预测。

具体地，NLS 通过求解下式来寻找变量 $\mathbf{x} \in \mathbb{R}^n$ ：

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}) = \arg \min_{\mathbf{x}} \frac{1}{2} \sum_i \underbrace{\|w_i \mathbf{c}_i(\mathbf{x}_i)\|}_{\mathbf{r}_i(\mathbf{x}_i)}^2, \quad (4.1)$$

其中，目标函数 $\mathcal{L}(\mathbf{x})$ 是向量值残差项 $\mathbf{r}_i$ 的平方和；每个残差项都是优化变量 $\mathbf{x} = \{\mathbf{x}_i\}$ 的（可能相互重叠的）子集 $\mathbf{x}_i \subset \mathbf{x}$ 的函数。当前假定 $\mathbf{x}_i$ 为向量；在本章后续内容中，将讨论推广至变量属于流形的情形。为保持灵活性，这里将残差 $\mathbf{r}_i(\mathbf{x}_i) = w_i \mathbf{c}_i(\mathbf{x}_i)$ 表示为权重 w_i 与向量代价 $\mathbf{c}_i$ 的乘积。

如第 1 章所述，NLS 通常通过迭代方式求解：先在当前变量附近线性化非线性目标，得到线性系统 $(\sum_i \mathbf{J}_i^\top \mathbf{J}_i) \delta \mathbf{x} = -\sum_i \mathbf{J}_i^\top \mathbf{r}_i$ ；再求解该线性系统以获得更新量 $\delta \mathbf{x}$ ；最后按 $\mathbf{x} \leftarrow \mathbf{x} + \delta \mathbf{x}$ 更新变量，直至收敛。第 2 章还指出，对于属于流形的变量，更新步骤中的加法更一般地是一个回缩映射（retraction mapping）。在线性系统中，

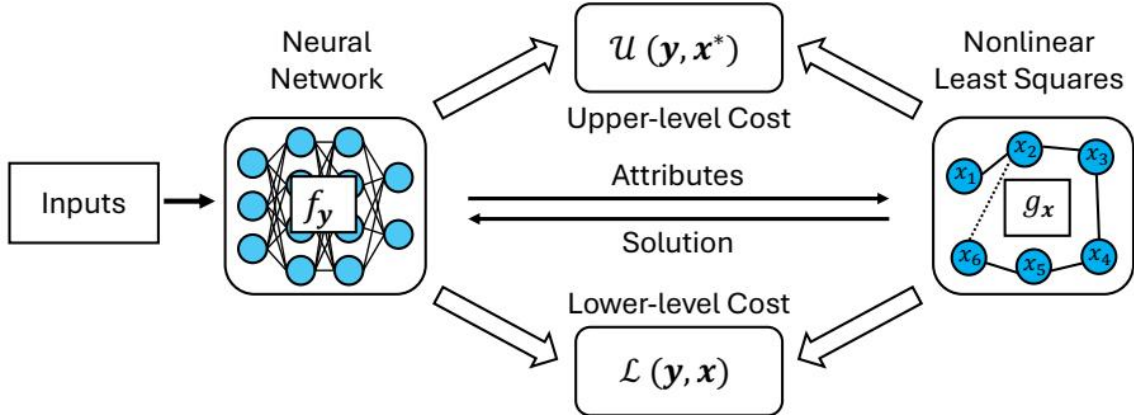


图 4.1: 图 4.1 现代 SLAM 系统通常同时包含神经网络和非线性最小二乘。为消除分别优化两个模块所引入的复合误差，可将整个系统表述为包含上层代价和下层代价的双层优化，从而进行端到端优化。

$\mathbf{J}_i = [\partial \mathbf{r}_i / \partial \mathbf{x}]$ 是残差关于变量的雅可比矩阵。上述迭代方法称为 Gauss-Newton (GN)，是一种（近似）二阶的非线性优化器，因为 $\sum_i \mathbf{J}_i^\top \mathbf{J}_i$ 是 Hessian 矩阵的近似。为改善鲁棒性和收敛性，Levenberg-Marquardt (LM) 等变体会对线性系统施加阻尼；另一些方法则利用线搜索调整更新步长，例如第 1 章介绍的 Dogleg 方法。

4.2 穿过非线性最小二乘进行微分

为将深度学习与非线性最小二乘无缝结合，通常需要可微非线性最小二乘（differentiable nonlinear least squares, DNLS）来求解图 4.1 所示的优化问题。这要求计算解 $\mathbf{x}^*$ 关于上层神经模型任意参数 $\mathbf{y}$ 的梯度；这些参数决定目标函数 $\mathcal{U}(\mathbf{x}; \mathbf{y})$ ，进而决定任意代价 $c_i(\mathbf{x}_i; \mathbf{y})$ 或变量初始化 $\mathbf{x}_{\text{init}}(\mathbf{y})$ 。目标是利用以 $\mathbf{x}$ 为函数定义的下层学习目标 $\mathcal{L}$ ，端到端地学习参数 $\mathbf{y}$ 。这构成一个双层优化（Bilevel Optimization, BLO）问题，可写为：

$$\mathbf{y}^* = \arg \min_{\mathbf{y} \in \Theta} \mathcal{U}(\mathbf{y}, \mathbf{x}^*), \quad (4.2a)$$

$$\text{s. t. } \mathbf{x}^* = \arg \min_{\mathbf{x} \in \Psi} \mathcal{L}(\mathbf{y}, \mathbf{x}), \quad (4.2b)$$

其中, $\mathcal{L} : \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}$ 是下层 (LL) 代价, $\mathcal{U} : \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}$ 是上层 (UL) 代价, $\mathbf{x} \in \Psi$ 和 $\mathbf{y} \in \Theta$ 为可行集。

在实践中, 变量 x 通常是具有明确物理含义的参数, 例如相机位姿; 而 y 通常是没有明确物理含义的参数, 例如神经网络的权重。下面用两个例子说明。

例 4.1 (具有学习特征的视觉 SLAM) 设想一个 SLAM 系统使用由 y 参数化的神经网络进行特征提取/匹配, 同时利用由 x 参数化的 BA 进行位姿估计, 并将特征匹配结果作为输入。在该示例中, 上层代价 (4.2a) 可以是用于优化网络的特征匹配误差, 而下层代价 $\mathcal{L}$ (4.2b) 可以是 BA 的重投影误差。直观地说, 相机位姿和路标位置的最优解 $\mathbf{x}^*$ 在神经网络训练中起监督信号的作用。因此, 优化 BLO (4.2) 可通过反向传播 BA 重投影误差, 进一步降低匹配误差 [1267]。

例 4.2 (PGO) 设想一个 SLAM 系统使用神经网络进行前端位姿估计, 并使用 PGO 作为后端消除里程计漂移。在此示例中, 上层和下层代价都可以是位姿图误差。区别在于, 上层代价优化由 y 参数化的网络, 而下层代价优化由 x 参数化的相机位姿。因此, 前端网络可通过从后端 PGO 反向传播位姿残差, 利用位姿图优化所获得的全局几何知识 [346]。

BLO 是一个历史悠久且研究充分的问题 [1148, 517, 679]。求解 BLO 通常依赖梯度下降技术。具体而言, 上层优化按 $\mathbf{y} \leftarrow \mathbf{y} + \delta \mathbf{y}$ 的形式更新, 其中 $\delta \mathbf{y}$ 是沿负梯度方向的一步。因此, 需要计算目标函数关于上层变量 $\mathbf{y}$ 的梯度, 可写为

$$\nabla_{\mathbf{y}} \mathcal{U} = \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}^*)}{\partial \mathbf{y}} + \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}^*)}{\partial \mathbf{x}^*} \frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}, \quad (4.3)$$

其中, 项 $\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}$ 涉及间接梯度计算。由于式 (4.3) 中其他直接梯度项易于获得, 求解 BLO (4.2) 的难点在于计算 $\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}$ 。为此, 人们从显式和隐式两个角度发展出一系列技术, 分别涉及穿过动力系统的递归微分与隐式微分理论, 通常称为展开微分 (unrolled differentiation) 和隐式微分 (implicit differentiation)。这些算法在 [679, 1148] 中已有总结; 这里的算法 1 给出包含两类方法的通用框架。下面分别介绍展开微分和隐式微分。

4.3 展开微分

展开微分需要穿过下层优化过程执行自动微分 (Automatic Differentiation, AutoDif), 以求解 BLO 问题。具体地, 给定 $t = 0$ 时的初始化 $\mathbf{x}_0 = \Phi_0(\mathbf{y})$, 展开的下层优化迭代过程为

$$\mathbf{x}_t = \Phi_t(\mathbf{x}_{t-1}; \mathbf{y}), \quad t = 1, \dots, T, \quad (4.4)$$

算法 1 通过展开微分或隐式微分求解 BLO。

- 1: 初始化: y_0, x_0 。
- 2: 当未收敛 ($\|y_{k+1} - y_k\|$ 足够大) 时:
- 3: 使用通用优化器 O 经 T 步求解 (4.2b), 获得 x_T 。
- 4: 按下列方式高效估计式 (4.3) 的上层梯度:
- 5: 展开微分: 通过式 (4.7) 中的 AutoDif, 计算 $\hat{\nabla}_{y_k} U = \frac{\partial U(y_k, x_T)}{\partial y_k}$ 。
- 6: 隐式微分 (算法 2): 计算

$$\hat{\nabla}_{y_k} \mathcal{U} = \left. \frac{\partial \mathcal{U}}{\partial y_k} \right|_{x_T} + \left. \frac{\partial \mathcal{U}}{\partial x^*} \frac{\partial x^*}{\partial y_k} \right|_{x_T},$$

其中隐式导数 $\frac{\partial x^*}{\partial y_k}$ 可通过求解由下层最优性条件导出的方程获得（后续各节将作概述）。

7:

8: 使用 $\hat{\nabla}_{y_k} U$ ，通过梯度计算 y_{k+1} 。

9: 结束循环

其中， Φ_t 表示第 t 步中基于下层问题的更新方案， T 为迭代次数。一种更新方案是梯度下降：

$$\Phi_t(\mathbf{x}_{t-1}; \mathbf{y}) = \mathbf{x}_{t-1} - \eta_t \cdot \frac{\partial \mathcal{L}(\mathbf{x}_{t-1}, \mathbf{y})}{\partial \mathbf{x}_{t-1}}, \quad (4.5)$$

其中， η_t 是学习率，项 $\frac{\partial \mathcal{L}(\mathbf{x}_{t-1}, \mathbf{y})}{\partial \mathbf{x}_{t-1}}$ 可通过 AutoDif 计算。¹ 因此，可用 $\mathbf{x}_T$ 近似替代 $\mathbf{x}^*$ ，进而计算 $\nabla_{\mathbf{y}} \mathcal{U}(\mathbf{y})$ ；完整的展开系统可定义为

$$\mathbf{x}^* \approx \mathbf{x}_T = \Phi(\mathbf{y}) = (\Phi_T \circ \cdots \circ \Phi_1 \circ \Phi_0)(\mathbf{y}), \quad (4.6)$$

其中符号 $\circ$ 表示函数复合。因此，只需考虑下列问题，而不必考虑式 (4.2) 中的双层优化：

$$\min_{\mathbf{y} \in \Theta} \mathcal{U}(\mathbf{y}, \Phi(\mathbf{y})), \quad (4.7)$$

该问题需要经由 AutoDif 计算 $\frac{\partial \Phi(\mathbf{y})}{\partial \mathbf{y}}$ ，而不是计算式 (4.3)。值得注意的是，计算递归梯度有两种途径：一种对应反向模式反向传播^[857]，另一种对应前向模式^[928]。这里省略 AutoDif 两种途径的细节，读者可参阅用于深度学习的 AutoDif 库（如 PyTorch^[850]），以及用于 SLAM 的 PyPose^[1145] 和 Theseus^[873]。Liu 等人^[679] 也对这些方法作了综述。

4.4 截断展开微分

反向模式和前向模式是精确的递归梯度计算方法，但完整迭代传播会耗费大量时间。这是因为上层问题对 $\mathbf{x}_t$ （其中 $t = 0, 1, \dots, T$ ）具有复杂的长期依赖；当 $\mathbf{y}$ 和 $\mathbf{x}$ 都是高维向量时，困难会进一步加剧。为克服这一挑战，研究者提出截断展开微分，以显著更少的计算时间和内存得到高质量的近似梯度。具体地，通过忽略长期依赖，并以部分历史近似式 (4.5) 的梯度，即仅保存最后 M 次迭代 ($t = T, T-1, \dots, T-M$)，可以显著降低时间和空间复杂度。Shaban 等人^[992] 已证明，用更少的反向步计算梯度，能够取得与精确梯度优化相当的性能，同时所需内存和计算量小得多。

在计算和内存约束更严格的情形下，截断展开微分仍常是现代机器人应用中的瓶颈。因此，研究者也尝试进一步简化截断微分：在式 (4.4) 中仅执行一次迭代，以移除递归结构^[676]，即

$$\nabla_{\mathbf{y}} \mathcal{U} = \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}_1(\mathbf{y}))}{\partial \mathbf{y}} + \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}_1(\mathbf{y}))}{\partial \mathbf{x}_1} \frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}}, \quad (4.8)$$

其中，项 $\frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}}$ 是由混合二阶导数给出的雅可比矩阵，可由式 (4.5) 计算为

$$\frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}} = -\frac{\partial^2 \mathcal{L}(\mathbf{x}_0, \mathbf{y})}{\partial \mathbf{x}_0 \partial \mathbf{y}}. \quad (4.9)$$

由于在某些应用中计算 Hessian 矩阵代价很高，可以采用数值解法：对变量 x 施加微小扰动，并将式 (4.8) 的第二项整体近似为

$$\frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}_1(\mathbf{y}))}{\partial \mathbf{x}_1} \frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}} \approx \frac{\frac{\partial \mathcal{L}(\mathbf{x}_0^+, \mathbf{y})}{\partial \mathbf{y}} - \frac{\partial \mathcal{L}(\mathbf{x}_0^-, \mathbf{y})}{\partial \mathbf{y}}}{2\epsilon}, \quad (4.10)$$

其中， ϵ 是一个小标量， $\mathbf{x}_0^\pm = \mathbf{x}_0 \pm \epsilon \frac{\partial \mathcal{U}(\mathbf{y}, \mathbf{x}_1(\mathbf{y}))}{\partial \mathbf{x}_1}$ 是微小扰动。该方法避免显式计算雅可比矩阵 $\frac{\partial \mathbf{x}_1(\mathbf{y})}{\partial \mathbf{y}}$ 。尽管如此，若包含非欧氏变量，例如属于 Lie 群的变量，就需关注扰动模型。幸运的是，现代库已支持针对 Hessian-向量积和 Jacobian-向量积的 Lie 群 AutoDif，例如将在第 4.4 节介绍的 PyPose^[1145]。

4.5 隐式微分

直观地说，式 (4.3) 中的项 $\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}$ 依赖于下层代价 (4.2b)，因此可使用隐式微分推导该梯度的解。

在微积分中，隐式微分是利用链式法则对隐函数求导的方法。对于由方程 $R(x, y) = 0$ 定义的隐函数 $y(x)$ ，通常无法先显式解出 y 再进行微分；取而代之的是，对 $R(x, y) = 0$ 关于 x 作全微分，再解所得线性方程，得到以 x 和 y 表示的导数 $\frac{dy}{dx}$ 。例如，考虑隐函数 $x + y + 5 = 0$ ，对等式两边关于 x 求导，可得 $\frac{dy}{dx} + \frac{dx}{dx} + \frac{d}{dx}(5) = 0 \Rightarrow \frac{dy}{dx} + 1 + 0 = 0$ 。解出 $\frac{dy}{dx}$ ，得到 $\frac{dy}{dx} = -1$ 。

假定下层代价关于 $\mathbf{x}$ 和 $\mathbf{y}$ 至少二次可微，则由最优性条件可知， $\mathbf{x}^*$ 为驻点，从而有 $\frac{\partial \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y})} = 0$ 。将该方程两边关于 $\mathbf{y}$ 求导，得到

$$\frac{\partial^2 \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y}) \partial \mathbf{y}} + \frac{\partial^2 \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y}) \partial \mathbf{x}^*(\mathbf{y})} \cdot \frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}} = 0. \quad (4.11)$$

这给出间接梯度 $\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}}$ ：

$$\frac{\partial \mathbf{x}^*(\mathbf{y})}{\partial \mathbf{y}} = -\left(\frac{\partial^2 \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y}) \partial \mathbf{x}^*(\mathbf{y})}\right)^{-1} \frac{\partial^2 \mathcal{L}(\mathbf{x}^*(\mathbf{y}), \mathbf{y})}{\partial \mathbf{x}^*(\mathbf{y}) \partial \mathbf{y}}, \quad (4.12)$$

式 (4.12) 的优点是，以求逆一个 Hessian 矩阵为代价，将变量 y 和 x 之间的间接梯度转化为 $\mathcal{L}$ 的直接梯度。其缺点是 Hessian 矩阵通常过大，难以计算；因此，常见做法是利用高效的 Hessian-向量积求解一个线性系统。

例 4.3 (存储和求逆 Hessian 矩阵的代价) 假定上层和下层代价均涉及一个仅有 10^6 万 (10^6) 个参数 (32 位浮点数) 的网络，则每个网络只需 $10^6 \times 4\text{Byte} = 4\text{MB}$ 的存储空间；而其 Hessian 矩阵需要 $(10^6)^2 \times 4\text{Byte} = 4\text{TByte}$ 的存储空间。这表明，低功耗计算机的内存甚至无法显式存储 Hessian 矩阵，因此直接计算其逆不可行。

回顾目标是计算式 (4.3) 的梯度，将式 (4.12) 代入式 (4.3) 可得：

$$\begin{aligned}\nabla_{\mathbf{y}}\mathcal{U} &= \frac{\partial\mathcal{U}(\mathbf{y},\mathbf{x}^*)}{\partial\mathbf{y}} - \underbrace{\frac{\partial\mathcal{U}(\mathbf{y},\mathbf{x}^*)}{\partial\mathbf{x}^*}}_{\mathbf{v}^\top} \underbrace{\left(\frac{\partial^2\mathcal{L}(\mathbf{x}^*(\mathbf{y}),\mathbf{y})}{\partial\mathbf{x}^*(\mathbf{y})\partial\mathbf{x}^*(\mathbf{y})}\right)^{-1}}_{(\mathbf{H}^\top)^{-1}} \frac{\partial^2\mathcal{L}(\mathbf{x}^*(\mathbf{y}),\mathbf{y})}{\partial\mathbf{x}^*(\mathbf{y})\partial\mathbf{y}} \\ &= \frac{\partial\mathcal{U}(\mathbf{y},\mathbf{x}^*)}{\partial\mathbf{y}} - \mathbf{q}^\top \cdot \frac{\partial^2\mathcal{L}(\mathbf{x}^*(\mathbf{y}),\mathbf{y})}{\partial\mathbf{x}^*(\mathbf{y})\partial\mathbf{y}}\end{aligned}\quad (4.13)$$

随后可通过优化下式，求解线性系统 $\mathbf{H}\mathbf{q} = \mathbf{v}$ 得到 $\mathbf{q}^*$ ：

$$\mathbf{q}^* = \arg\min_{\mathbf{q}} Q(\mathbf{q}) = \arg\min_{\mathbf{q}} \frac{1}{2}\mathbf{q}^\top \mathbf{H}\mathbf{q} - \mathbf{q}^\top \mathbf{v}, \quad (4.14)$$

可使用简单的梯度下降或共轭梯度法^[459]等高效线性求解器。对于梯度下降，需要计算 Q 的梯度 $\frac{\partial Q(\mathbf{q})}{\partial\mathbf{q}} = \mathbf{H}\mathbf{q} - \mathbf{v}$ ，其中 $\mathbf{H}\mathbf{q}$ 可通过高效 Hessian-向量积计算；即 Hessian-向量积是梯度-向量积的梯度：

$$\mathbf{H}\mathbf{q} = \frac{\partial^2\mathcal{L}}{\partial\mathbf{x}\partial\mathbf{x}} \cdot \mathbf{q} = \frac{\partial(\frac{\partial\mathcal{L}}{\partial\mathbf{x}} \cdot \mathbf{q})}{\partial\mathbf{x}}, \quad (4.15)$$

其中， $\frac{\partial\mathcal{L}}{\partial\mathbf{x}} \cdot \mathbf{q}$ 是一个标量。这意味着无需显式计算或存储 Hessian 矩阵 H 。算法 2 总结了通过线性系统计算隐式微分的过程；使用共轭梯度法的算法与之类似。

算法 2 通过线性系统计算隐式微分。

1: 输入：当前上层变量 $\mathbf{y}$ 和最优下层变量 $\mathbf{x}^*$ 。

2: 初始化： $k = 1$ ，学习率 η 。

3: 当未收敛（ $\|\mathbf{q}_k - \mathbf{q}_{k-1}\|$ 足够大）时：

4: 执行梯度下降：

$\mathbf{q}_k = \mathbf{q}_{k-1} - \eta(\mathbf{H}\mathbf{q}_{k-1} - \mathbf{v})$ ，式 (4.16)，

其中 $\mathbf{H}\mathbf{q}_{k-1}$ 通过高效 Hessian-向量积计算。

5: 结束循环

6: 令 $\mathbf{q} = \mathbf{q}_k$ 。

7: 按式 (4.3) 计算 $\nabla_{\mathbf{y}}\mathcal{U}$ ：

$$\nabla_{\mathbf{y}}\mathcal{U} = \frac{\partial\mathcal{U}(\mathbf{y},\mathbf{x}^*)}{\partial\mathbf{y}} - \underbrace{\left(\frac{\partial^2\mathcal{L}(\mathbf{x}^*(\mathbf{y}),\mathbf{y})}{\partial\mathbf{y}\partial\mathbf{x}^*(\mathbf{y})} \cdot \mathbf{q}\right)^\top}_{(\mathbf{H}_{\mathbf{y}\mathbf{x}} \cdot \mathbf{q})^\top}, \quad \text{式 (4.17),}$$

其中 $\mathbf{H}_{\mathbf{y}\mathbf{x}} \cdot \mathbf{q}$ 也可通过 Hessian-向量积高效计算。

近似。隐式微分实现复杂，但有一种近似方法是忽略隐式成分，仅使用直接部分 $\hat{\nabla}_{\mathbf{y}}\mathcal{U} \approx \frac{\partial\mathcal{U}}{\partial\mathbf{y}}\Big|_{\mathbf{x}_T}$ 。这等价于在上层问题中将下层优化得到的解 $\mathbf{x}_T$ 视为常数。该近似效率更高，但会引入误差项

$$\epsilon \sim \left| \frac{\partial\mathcal{U}}{\partial\mathbf{x}^*} \frac{\partial\mathbf{x}^*}{\partial\mathbf{y}} \right|. \quad (4.18)$$

不过，当隐式梯度包含较小二阶导数的乘积时，这一近似仍然有用，具体取决于所处理的 NLS 问题。

4.6 流形上的微分

典型 SLAM 系统的状态必然在流形上演化，因此，流形上的优化对于求解 SLAM 后端问题至关重要。下面推导对属于 Lie 群的变量求导时所需的雅可比矩阵，这是流形微分的关

键步骤。

4.7 Lie 群导数

第 2 章已经介绍 Lie 群、Lie 代数及其基本运算（例如指数映射和对数映射）的基本概念。本节将简要回顾这些概念，但主要关注其导数的定义，因为这是求解可微优化问题所必需的。

考虑一个 Lie 群的流形。该光滑流形上的每一点 χ 都有唯一的切空间，记为 $T_\chi \mathcal{M}$ ，微积分的基本原理在其中成立。Lie 代数 $\mathfrak{m}$ 是一个向量空间，并可在点 χ 的邻域局部定义为 $\mathfrak{m} = T_\chi \mathcal{M}$ 。指数映射 $\exp : \mathfrak{m} \rightarrow \mathcal{M}$ 将 Lie 代数中的元素映射至 Lie 群；对数映射 $\log : \mathcal{M} \rightarrow \mathfrak{m}$ 则是其逆，建立双向关系：

$$\chi = \exp(\tau^\wedge) \Leftrightarrow \tau^\wedge = \log(\chi), \quad (4.19)$$

其中 hat 算子 $\wedge$ 是线性可逆映射，且 $\tau^\wedge \in \mathfrak{m}$ 。若将 Lie 代数中的坐标表示为 $\mathbb{R}^n$ 中的向量 τ ，便可定义向量 τ 与 Lie 群 χ 之间的映射：

$$\chi = \text{Exp}(\tau) \Leftrightarrow \tau = \text{Log}(\chi), \quad (4.20)$$

这里重新定义指数映射和对数映射，使其分别直接以向量为输入和输出。

要计算 Lie 群上的导数，首先必须理解两个流形元素（例如 x_1 和 x_2 ）之间的相对变化。通过先定义 $\oplus$ 和 $\ominus$ 算子可对这些变化进行量化；它们各组合一个指数/对数映射和一个复合运算。由于复合通常不可交换，这些算子依操作数顺序分为右形式和左形式。右形式算子为：

$$\begin{aligned} \text{right-}\oplus : \quad \chi_2 &= \chi_1 \oplus \tau \triangleq \chi_1 \circ \text{Exp}(\tau), \\ \text{right-}\ominus : \quad \tau &= \chi_2 \ominus \chi_1 \triangleq \text{Log}(\chi_1^{-1} \circ \chi_2). \end{aligned} \quad (4.21)$$

式 (4.21) 中 τ 位于右侧，表示它在 x_1 处的局部坐标系中表示。相反，式 (4.22) 的左形式算子反映全局坐标系视角：

$$\begin{aligned} \text{left-}\oplus : \quad \chi_2 &= \varepsilon \oplus \chi_1 \triangleq \text{Exp}(\varepsilon) \circ \chi_1, \\ \text{left-}\ominus : \quad \varepsilon &= \chi_2 \ominus \chi_1 \triangleq \text{Log}(\chi_2 \circ \chi_1^{-1}), \end{aligned} \quad (4.22)$$

其中， ε 在全局坐标系中表示。 τ 和 ε 都可看作流形元素的增量扰动；借助相应的复合算子 $\oplus$ 和 $\ominus$ ，这些变化以切空间中的向量表示。

有了右形式的 $\oplus$ 和 $\ominus$ 算子后，使用雅可比矩阵 J 描述流形上的扰动。雅可比矩阵刻画了切空间 $\mathfrak{m}$ 中无穷小扰动 τ 的本质：

$$\begin{aligned} \frac{\partial f(\chi)}{\partial \chi} &\triangleq \lim_{\tau \rightarrow 0} \frac{f(\chi \oplus \tau) - f(\chi)}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{f(\chi \circ \text{Exp}(\tau)) - f(\chi)}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{\text{Log}(f(\chi)^{-1} \circ f(\chi \circ \text{Exp}(\tau)))}{\tau}. \end{aligned} \quad (4.23)$$

令 $g(\tau) = \text{Log}(f(\chi)^{-1} \circ f(\chi \circ \text{Exp}(\tau)))$ ，则右雅可比 J_R 可表示为 $g(\tau)$ 在 $\tau = 0$ 处的导数：

$$\frac{\partial f(\chi)}{\partial \chi} = J_R = \left. \frac{\partial g(\tau)}{\partial \tau} \right|_{\tau=0}. \quad (4.24)$$

这样，流形中 $f(\chi)$ 关于 χ 的导数由雅可比矩阵 $J_R \in \mathbb{R}^{m \times n}$ 表示，其中 m 和 n 分别是流形 $\mathcal{M}$ 和 $\mathcal{N}$ 的维数。右雅可比矩阵执行从切空间 $\mathfrak{m}$ 到切空间 $\mathfrak{n} = T_{f(\chi)}\mathcal{N}$ 的线性映射。

类似地，考虑施加于 Lie 群元素 χ 的无穷小扰动 $\epsilon \in T_\chi\mathcal{M}$ ，可利用左形式的加、减算子定义左雅可比 J_L ：

$$\begin{aligned} \frac{\partial f(\chi)}{\partial \chi} &\triangleq \lim_{\epsilon \rightarrow 0} \frac{f(\epsilon \oplus \chi) \ominus f(\chi)}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0} \frac{f(\text{Exp}(\epsilon) \circ \chi) \ominus f(\chi)}{\epsilon} \\ &= \lim_{\epsilon \rightarrow 0} \frac{\text{Log}(f(\text{Exp}(\epsilon) \circ \chi) \circ f(\chi)^{-1})}{\epsilon} \\ &= \left. \frac{\partial \text{Log}(f(\text{Exp}(\epsilon) \circ \chi) \circ f(\chi)^{-1})}{\partial \epsilon} \right|_{\epsilon=0}. \end{aligned} \quad (4.25)$$

所得左雅可比 $J_L \in \mathbb{R}^{n \times m}$ 同样是一个线性映射，但它在全局切空间中从 $T_\chi\mathcal{M}$ 映射至 $T_{f(\chi)}\mathcal{N}$ 。

为考察点 x_1 周围的局部扰动，将 τ 取作 $\tau = \chi \ominus \chi_1$ ，其中 x 是 x_1 的扰动版本。定义在切空间上的协方差矩阵 Σ_x 由期望算子 $\mathbb{E}$ 导出，从而表征不确定性及其传播：

$$\Sigma_\chi \triangleq \mathbb{E}[\tau \tau^\top] = \mathbb{E}[(\chi \ominus \chi_1)(\chi \ominus \chi_1)^\top]. \quad (4.26)$$

这些协方差矩阵可用于在流形上建立高斯分布，即 $\chi \sim \mathcal{N}(\chi_1, \Sigma_\chi)$ 。需要注意，协方差矩阵 Σ_χ 定义在切空间 $T_{\chi_1}\mathcal{M}$ 上，因此流形上的不确定性可以由向量表示，并以协方差矩阵形式传播。

例 4.4 (视觉惯性旋转估计) 考虑一个配备惯性测量单元 (IMU) 和相机的机器人。给定两个传感器的带噪观测 R_{IMU} 和 R_{Cam} ，可通过最小化测量之间的差异估计机器人的朝向；这可表述为流形 $\text{SO}(3)$ 上的非线性最小二乘问题：

$$\hat{R} = \arg \min_{R \in \text{SO}(3)} f(R, R_{\text{IMU}}, R_{\text{Cam}}), \quad (4.27)$$

其中， $f(\cdot)$ 是量化估计朝向 R 与传感器测量 R_{IMU} 和 R_{Cam} 之间差异的代价函数。有了雅可比矩阵后，便可以有效处理流形 $\text{SO}(3)$ 上用于位姿估计的优化。代价函数 $f(\cdot)$ 可写为：

$$f(R) = \|\text{Log}(R_{\text{IMU}}^{-1}R)\|^2 + \|\text{Log}(R_{\text{Cam}}^{-1}R)\|^2. \quad (4.28)$$

要最小化 $f(R)$ ，需要计算其关于流形 $\text{SO}(3)$ 上 R 的梯度。该梯度可由右雅可比 J_R 推导为：

$$\begin{aligned} \nabla f(R) &= 2 \left(\frac{\partial \text{Log}(R_{\text{IMU}}^{-1}R)}{\partial R} \right)^\top \text{Log}(R_{\text{IMU}}^{-1}R) \\ &\quad + 2 \left(\frac{\partial \text{Log}(R_{\text{Cam}}^{-1}R)}{\partial R} \right)^\top \text{Log}(R_{\text{Cam}}^{-1}R). \end{aligned} \quad (4.29)$$

梯度 $\nabla f(R)$ 可与梯度下降等优化算法结合使用：沿切空间移动并重新投影回流形，以迭代更新位姿 R ：

$$R_{k+1} = R_k \text{Exp}(-\alpha \nabla f(R)), \quad (4.30)$$

其中 α 为步长。该迭代过程持续到代价函数 $f(R)$ 收敛至极小值，从而提供利用两个传感器测量的位姿估计 $\hat{R}$ 。

4.8 流形上的微分运算

对于典型流形运算，可以推导与求逆、复合和群作用相关的雅可比矩阵闭式表达式。这些表达式能够通过链式法则计算流形上的函数导数，从而支持 SLAM 中完整的优化过程：

$$\frac{\partial \mathcal{Z}}{\partial \chi} = \frac{\partial \mathcal{Z}}{\partial \mathcal{Y}} \frac{\partial \mathcal{Y}}{\partial \chi}, \quad (4.31)$$

其中， $\mathcal{Z} = g(\mathcal{Y})$ ，且 $\mathcal{Y} = f(\chi)$ 。

求逆的雅可比可通过将函数 $f(\chi) = \chi^{-1}$ 代入式 (4.23) 的右雅可比 J_R 推导，得到：

$$\begin{aligned} \frac{\partial \chi^{-1}}{\partial \chi} &\triangleq \lim_{\tau \rightarrow 0} \frac{\text{Log}((\chi^{-1})^{-1}(\chi \text{Exp}(\tau))^{-1})}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{\text{Log}(\chi \text{Exp}(\tau)^{-1} \chi^{-1})}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{(\chi(-\tau)^\wedge \chi^{-1})^\vee}{\tau}. \end{aligned} \quad (4.32)$$

复合的雅可比可通过将函数 $f(\chi) = \chi \circ \chi_1$ 代入式 (4.23) 推导。复合算子 $x \circ x_1$ 关于 x 的导数为：

$$\begin{aligned} \frac{\partial(\chi \circ \chi_1)}{\partial \chi} &\triangleq \lim_{\tau \rightarrow 0} \frac{\text{Log}((\chi \chi_1)^{-1}(\chi \text{Exp}(\tau) \chi_1))}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{\text{Log}(\chi_1^{-1} \text{Exp}(\tau) \chi_1)}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{(\chi_1^{-1} \tau^\wedge \chi_1)^\vee}{\tau}. \end{aligned} \quad (4.33)$$

复合算子 $x \circ x_1$ 关于 x_1 的导数为：

$$\begin{aligned} \frac{\partial(\chi \circ \chi_1)}{\partial \chi_1} &\triangleq \lim_{\tau \rightarrow 0} \frac{\text{Log}((\chi \chi_1)^{-1}(\chi \chi_1 \text{Exp}(\tau)))}{\tau} \\ &= \lim_{\tau \rightarrow 0} \frac{\text{Log}(\text{Exp}(\tau))}{\tau} \\ &= \mathbf{I}. \end{aligned} \quad (4.34)$$

流形的雅可比可由 χ 的右雅可比表征，该雅可比从 $\tau \in \mathbb{R}^m$ 的指数映射导出，表示为：

$$\mathbf{J}_r(\tau) \triangleq \frac{\partial \text{Exp}(\tau)}{\partial \tau}. \quad (4.35)$$

右雅可比将 τ 的微小变化传递为 $\text{Exp}(\tau)$ 处局部切空间中的变化。类似地， x 的左雅可比将 τ 的变化映射为流形全局切空间中的变化，表示为：

$$\mathbf{J}_l(\tau) \triangleq \text{Ad}_{\text{Exp}(\tau)} \mathbf{J}_r(\tau). \quad (4.36)$$

群作用的雅可比取决于具体的群作用集合 $v \in \mathcal{V}$ 。群作用的雅可比定义为：

$$\begin{aligned} J_\chi^{X \cdot v} &\triangleq \frac{\partial(\chi \cdot v)}{\partial \chi}, \\ J_v^{X \cdot v} &\triangleq \frac{\partial(\chi \cdot v)}{\partial v}, \end{aligned} \quad (4.37)$$

其中， $\chi \in \mathcal{M}$ 且 $v \in \mathcal{V}$ 。

例 4.5 (机械臂) 考虑一个具有两个关节 R_1 和 R_2 的机械臂，每个关节均由 $\text{SO}(3)$ 中的一个元素表示。机器人末端执行器的最终朝向由关节旋转的复合决定：

$$\mathbf{R} = \mathbf{R}_1 \circ \mathbf{R}_2. \quad (4.38)$$

为了评估 $\mathbf{R}_1$ 和 $\mathbf{R}_2$ 中的微小扰动 τ 对末端执行器朝向 R 的影响, 可使用复合的雅可比对其量化:

$$\begin{aligned}\frac{\partial(\mathbf{R}_1 \circ \mathbf{R}_2)}{\partial \mathbf{R}_1} &= \lim_{\tau \rightarrow 0} \frac{(\mathbf{R}_2^{-1} \tau \wedge \mathbf{R}_2)^\vee}{\tau}, \\ \frac{\partial(\mathbf{R}_1 \circ \mathbf{R}_2)}{\partial \mathbf{R}_2} &= \mathbf{I}.\end{aligned}\quad (4.39)$$

该示例表明, 对第一个关节 $\mathbf{R}_1$ 的调整, 会经由受第二个关节 $\mathbf{R}_2$ 当前状态影响的变换作用于最终朝向 R 。而对第二个关节 $\mathbf{R}_2$ 的改变会直接影响 R , 不受第一个关节 $\mathbf{R}_1$ 的影响。

4.9 自动微分的数值挑战与现代库

AutoDif 是在包括流形微分在内的各种优化情境中准确、高效计算导数的核心技术。由于流形结构固有的复杂几何性质, 流形上的微分面临独特挑战, 这些性质可能影响 AutoDif 的性能和适用性。在可微优化中, 当 AutoDif 与流形的曲空间相互作用时, 这些挑战尤为突出, 可能引入数值不稳定性和不准确性的。

本节给出在基于流形的优化任务中使用自动微分时出现数值问题的例子, 尤其关注在流形约束下保持数值稳定性和精度的复杂性, 例如约束优化以及由流形上微分方程定义的系统。这里以 PyPose 库^[1145]为例; 它为 Lie 群和 Lie 代数定义了一种通用数据结构 LieTensor。具体而言, 将说明一些数值挑战, 以及 PyPose 如何处理这些挑战。

例 4.6 (指数映射和四元数) 指数映射是 Lie 群理论中的基本概念, 在以四元数表示的 Lie 代数和 Lie 群之间转换时尤为关键。该映射可将 $\mathbb{R}^3$ 代数结构中的角速度转换为单位四元数群 $\mathbb{S}^3$ 中的旋转朝向。从解析角度看, 四元数的指数映射由 Rodrigues 旋转公式导出; 该公式将 $\mathbb{R}^3$ 中的向量关联到相应旋转。给定 $\mathbb{R}^3$ 中向量 x , 它表示由旋转角缩放的旋转轴, 则旋转的四元数表示为:

$$\text{Exp}(\boldsymbol{\nu}) = \left[\sin\left(\frac{\|\boldsymbol{\nu}\|}{2}\right) \frac{\boldsymbol{\nu}^\top}{\|\boldsymbol{\nu}\|}, \cos\left(\frac{\|\boldsymbol{\nu}\|}{2}\right) \right]^\top \quad (4.40)$$

其中, $\|\boldsymbol{\nu}\|$ 表示 $\boldsymbol{\nu}$ 的模, 对应旋转角; $\frac{\boldsymbol{\nu}}{\|\boldsymbol{\nu}\|}$ 是沿 $\boldsymbol{\nu}$ 方向的单位向量。

实现可微 LieTensor 的一个挑战是, 通常必须为 Exp 和 Log 映射计算式 (4.40) 中的 $\frac{\sin(\|\boldsymbol{\nu}\|/2)}{\|\boldsymbol{\nu}\|}$ 等数值上存在问题的项^[1083]。当角度非常小时, 直接计算正弦和余弦函数会由于计算机中浮点数的有限表示而导致精度问题。为处理这些问题并保持数值稳定性, PyPose 采用 Taylor 展开, 以避免除以零。

$$\text{Exp}(\boldsymbol{\nu}) = \begin{cases} \left[\boldsymbol{\nu}^\top \gamma_e, \cos\left(\frac{\|\boldsymbol{\nu}\|}{2}\right) \right]^\top & \text{if } \|\boldsymbol{\nu}\| > \text{eps}, \\ \left[\boldsymbol{\nu}^\top \gamma_o, 1 - \frac{\|\boldsymbol{\nu}\|^2}{8} + \frac{\|\boldsymbol{\nu}\|^4}{384} \right]^\top & \text{otherwise.} \end{cases} \quad (4.41)$$

当 $\|\boldsymbol{\nu}\|$ 显著时, $\gamma_e = \frac{\sin(\|\boldsymbol{\nu}\|/2)}{\|\boldsymbol{\nu}\|}$; 当 $\|\boldsymbol{\nu}\|$ 很小时, $\gamma_o = \frac{1}{2} - \frac{\|\boldsymbol{\nu}\|^2}{48} + \frac{\|\boldsymbol{\nu}\|^4}{3840}$, 从而可在整个旋转幅值范围内确保精确计算。这里, eps 是使 $1 + \text{eps} \neq 1$ 成立的最小机器数。这一从解析到数值的过程说明: 在需要高保真旋转表示的应用中, 准确稳定地计算指数映射十分重要。

LieTensor 与现有库相比存在若干区别: (1) PyPose 支持任意阶梯度的 AutoDif, 且兼容 CPU、GPU、TPU 和 Apple silicon GPU 等主流设备; 而 LieTorch^[1083] 等库实现定制 CUDA 核函数, 只支持一阶梯度。(2) LieTensor 支持通过 vmap 算子并行计算梯度, 因此能更快地计算雅可比矩阵。(3) LieTorch、JaxLie^[1244] 和 Theseus 等库只支持 Lie 群, 而

PyPose 同时支持 Lie 群和 Lie 代数。因此，可直接从 LieTensor 实例调用 Exp 和 Log 映射，更加灵活易用；两类变量的梯度也可自动计算和反向传播。读者可在^[1144]找到受支持 LieTensor 运算的列表，在^[1147]找到 PyPose 教程。有关 LieTensor 及其自动微分的用法，参见 <https://github.com/pypose/slambook-snippets/blob/main/lietensor.ipynb>。

4.10 可微优化的实现示例

为使用双层优化实现端到端学习，需要集成除神经方法所需的 SGD^[930] 和 Adam^[579] 等基于梯度的方法之外的通用优化器，因为 SLAM 中的许多问题还需要约束优化或 2nd 阶优化^[51] 等其他优化算法。此外，实际问题中存在离群值，因此需要像第 3 章所述那样使损失函数鲁棒化。下面考虑第 3.3 节介绍的一种用于 SLAM 的 IRLS 方法，并说明 PyPose 面向优化的接口背后的思想，包括求解器 (solver)、核函数 (kernel)、校正器 (corrector) 和策略 (strategy)，以使用 2nd 阶 Levenberg-Marquardt (LM) 优化器。

首先考虑一个加权最小二乘问题：

$$\min_{\mathbf{x}} \sum_i (\mathbf{h}_i(\mathbf{x}_i) - \mathbf{z}_i)^T \boldsymbol{\Sigma}_i (\mathbf{h}_i(\mathbf{x}_i) - \mathbf{z}_i), \quad (4.42)$$

其中， $h(\cdot)$ 是回归模型（模块）， $\mathbf{x} \in \mathbb{R}^n$ 是待优化参数， $\mathbf{h}_i$ 表示第 i 个输入样本的预测， $\boldsymbol{\Sigma}_i \in \mathbb{R}^{d \times d}$ 是信息矩阵。LM 算法通过迭代更新估计 $\mathbf{x}_t$ 得到式 (4.42) 的解，即 $\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} + \boldsymbol{\delta}_t$ ，其中更新步 $\boldsymbol{\delta}_t$ 计算为：

$$\sum_i (\boldsymbol{\Lambda}_i + \lambda \cdot \text{diag}(\boldsymbol{\Lambda}_i)) \boldsymbol{\delta}_t = - \sum_i \mathbf{J}_i^T \boldsymbol{\Sigma}_i \mathbf{r}_i, \quad (4.43)$$

其中， $\mathbf{r}_i = \mathbf{h}_i(\mathbf{x}_i) - \mathbf{z}_i$ 是第 i 个残差； $\mathbf{J}_i$ 是在 $\mathbf{x}_{t-1}$ 处计算的 h 的雅可比矩阵； $\boldsymbol{\Lambda}_i = \mathbf{J}_i^T \boldsymbol{\Sigma}_i \mathbf{J}_i$ 是近似 Hessian 矩阵； λ 是阻尼因子。为得到步长 $\boldsymbol{\delta}_t$ ，需要一个线性求解器：

$$\mathbf{A} \cdot \boldsymbol{\delta}_t = \boldsymbol{\beta}, \quad (4.44)$$

其中， $\mathbf{A} = \sum_i (\boldsymbol{\Lambda}_i + \lambda \text{diag}(\boldsymbol{\Lambda}_i))$ ， $\boldsymbol{\beta} = - \sum_i \mathbf{J}_i^T \boldsymbol{\Sigma}_i \mathbf{r}_i$ 。实践中，方阵 $\mathbf{A}$ 通常是正定的，因此可使用 Cholesky 等标准线性求解器。若雅可比 $\mathbf{J}_i$ 很大且稀疏，也可使用稀疏 Cholesky^[161] 或预条件共轭梯度 (PCG)^[459] 等稀疏求解器。

实践中，常在式 (4.42) 中引入鲁棒核函数 $\rho: \mathbb{R} \mapsto \mathbb{R}$ ，以减小离群值的影响：

$$\min_{\mathbf{x}} \sum_i \rho(\mathbf{r}_i^T \boldsymbol{\Sigma}_i \mathbf{r}_i), \quad (4.45)$$

其中， ρ 用于降低具有较大残差 $\mathbf{r}_i$ 的测量的权重。此时需要调整式 (4.43) 以考虑鲁棒核的存在。一种常用方法是采用 IRLS 方法 Triggs 校正^[1107]，Ceres^[15] 库也采用该方法。然而，它需要核函数 ρ 的二阶导数，而该导数始终为负，可能导致包括 LM 在内的二阶优化器不稳定^[1107]。作为替代方案，PyPose 提出 IRLS 方法 FastTriggs；它仅涉及一阶导数，比 Triggs 更快也更稳定：

$$\mathbf{r}_i^\rho = \sqrt{\rho'(c_i)} \mathbf{r}_i, \quad \mathbf{J}_i^\rho = \sqrt{\rho'(c_i)} \mathbf{J}_i, \quad (4.46)$$

其中, $c_i = \mathbf{r}_i^T \Sigma_i \mathbf{r}_i$; $\mathbf{r}_i^\rho$ 和 J_i^ρ 分别是引入核函数后校正的模型残差和雅可比。FastTriggs 的更多细节及其证明参见 [1143]; IRLS 已在第 3.3 节介绍。

若初始猜测距离最优解过远, 简单的 LM 优化器可能无法收敛到全局最优解。因此, 常需采用自适应阻尼、Dogleg 和信赖域方法 [697] 等策略约束每一步, 避免步长“过大”。要采用这些策略, 只需将策略实例 (例如 TrustRegion) 传给优化器。总之, PyPose 只需在构造函数中传入优化器参数, 即可方便地扩展上述算法; 这些参数包括 solver、strategy、kernel 和 corrector。可用算法列表和示例见 [1146]。二阶优化的用法参见 <https://github.com/pypose/slambook-snippets/blob/main/optimization.ipynb>。

4.11 相关开源库

与可微优化相关的开源库可分为三类: (1) 线性代数库; (2) 机器学习库; (3) 专用优化库。

线性代数库是机器学习和机器人研究的基础。Python 线性代数库 NumPy [823] 提供全面的向量和矩阵操作, 并得益于底层经过良好优化的 C 代码而具有较高运行速度。高性能 C++ 线性代数库 Eigen [412] 已被 TensorFlow [2]、Ceres [15]、GTSAM [252] 和 g2o [402] 等众多项目采用。面向 C、C++、Fortran 和 Python 的 GPU 加速库 ArrayFire [727] 包含简洁 API, 并提供针对 GPU 优化的函数。

机器学习库更关注张量 (即高维矩阵) 上的运算和自动微分。Torch [228]、OpenNN [830] 和 MATLAB [738] 等早期机器学习框架为研究人员开发神经网络提供了基础工具, 但它们只支持 CPU 计算且缺少简洁 API, 限制了可用性。几年后, Chainer [1101]、Theano [24] 和 Caffe [518] 等深度学习框架出现, 以应对神经网络规模和复杂度的增长; 它们支持多 GPU 训练, 并为用户构建和训练神经网络提供了便捷 API。进一步地, TensorFlow [2]、PyTorch [850] 和 MXNet [184] 等近期框架提供了完整而灵活的生态系统, 例如多种编程语言 API、分布式数据并行训练, 以及基准测试和部署辅助工具。Gvnn [430] 将可微变换层引入基于 Torch 的框架, 从而实现端到端几何学习。JAX [110] 可自动微分原生 Python 和 NumPy 函数, 并是一个可组合函数变换的可扩展系统。这些框架的出现以多种方式促进了深度学习的发展。近来, 人们进一步努力将标准优化工具与深度学习结合。这些工作如 Theseus [873] 和 Cvxpy-Layer [18] 展示了如何将可微优化嵌入深度神经网络。PyPose [1145] 集成 Gauss-Newton 和 Levenberg-Marquardt 等二阶优化器, 并可计算 Lie 群和 Lie 代数的任意阶梯度, 这些能力对于机器人学至关重要。

其他专用优化库也已在机器人学中开发和应用。举例而言, Ceres [15] 是面向大规模非线性最小二乘优化问题的开源 C++ 库, 已广泛用于 SLAM。Pyomo [434] 和 JuMP [292] 是优化框架, 因其灵活支持构建、求解和分析优化模型的多种工具而被广泛使用。CasADi [32] 凭借对最优控制多种数值方法的快速有效实现, 已用于求解机器人学中的许多现实控制问题。位姿优化和因子图优化也在机器人学中发挥重要作用。例如, g2o [402] 和 GTSAM [252] 是用于基于图的非线性优化的开源 C++ 框架, 提供构建新问题的简洁 API, 并已用于解决 SLAM 中的多个优化问题。

优化库也已广泛用于机器人控制问题。举例而言, IPOPT [1139] 是基于内点法的非线性规划开源 C++ 求解器, 广泛用于机器人与控制。类似地, OpenOCL [590] 支持连续时间、离散时间、约束、无约束、多阶段和轨迹优化等多类优化问题, 可用于实时模型预测控制。另一种面向大规模最优控制和估计问题的库是 CT [383], 它为不同的最优控制求解器提供标准接口,

并可扩展到机器人应用中的广泛动力学系统。Drake^[1079] 为常见控制问题提供求解器，可直接与其仿真工具箱集成；其系统完整性使其受到研究人员青睐。

4.12 延伸阅读与近期趋势

近年来，深度学习方法取得显著发展^[1286]。作为数据驱动方法，相较于传统手工特征，深度学习被认为在视觉跟踪中表现更好。该领域的多数研究采用端到端结构，包括 DeepVO^[1163] 和 TartanVO^[1167] 等有监督方法，以及 UnDeepVO^[656] 和 Unsupervised VIO^[1173] 等无监督方法。一般认为，有监督方法能够从位姿、光流和深度等多样的真值中学习，因此性能优于无监督方法；但在真实世界中获取这类真值是一项劳动密集型工作^[1166]。

近来，混合方法日益受到关注，因为它们整合了基于几何方法与深度学习方法的优点。一些研究探索了将 BA 与深度学习结合，以在帧间施加拓扑一致性，例如为学习网络附加 BA 层的 BA-Net^[1068] 和 DROID-SLAM^[1082]。此外，另一些工作侧重于将图像特征压缩为编码（嵌入式特征），并在推理期间优化位姿-编码图，例如 DeepFactors^[236]。DiffPoseNet^[842] 则使用网络预测位姿和法向流，并通过 Cheirality 层微调粗略预测。然而，在这些工作中，基于学习的方法与基于几何的优化是解耦的，分别用于不同子模块。前端和后端缺少集成可能导致次优性能；此外，它们仅将位姿误差“穿过”BA 反向传播，因此监督信号来自真值位姿。在这种情形下，BA 只是网络的一个特殊层。近期，iSLAM^[346] 以双向方式连接前端和后端，并通过双层优化框架迫使学习模型从几何优化中学习，在不使用外部监督的条件下实现性能提升。其他任务也可表述为双层优化，例如强化学习^[1027]、局部规划^[1216]、全局规划^[186]、特征匹配^[1267] 和多机器人路径规划^[419]。

第五章 稠密地图表示

作者： Victor Reijgwart、Jens Behley、Teresa Vidal-Calleja、Helen Oleynikova、Lionel Ott、Cyrill Stachniss 和 Ayoun Kim

本章将关注 SLAM 的另一个方面，即建图组件。我们假设机器人位姿已经完成估计，目标是构建其周围环境的稠密地图。典型方法首先使用 SLAM 后端（前几章已讨论）求解机器人的运动轨迹，然后在给定轨迹的情况下重建稠密地图。本章将详细介绍稠密地图表示，重点讨论地图元素、数据结构和构建方法。

早期建图方法主要采用第 1 章所述的稀疏、基于路标的方案，仅从环境中提取少量显著特征。随着计算能力提升，以及机械式 3D LiDAR 或 RGB-D 相机等高精度 3D 范围传感器的出现（这些传感器能够以高频率提供详细的 3D 范围测量），稠密地图表示逐渐受到研究关注。对于规划、导航、操作和精确定位等需要详细理解环境的下游任务，稠密地图至关重要。本章解释这些稠密方法如何利用范围传感器数据的完整信息来细化和更新综合地图。

本章首先在第 5.1 节介绍支持稠密建图的主要传感器类型，重点是能够产生详细范围测量的范围传感器。第 5.2 节介绍基本表示元素和数据结构，并在第 5.4 节针对具体应用展开。与稀疏的基于路标建图相比，稠密地图表示的选择取决于所使用的传感器类型和预期的下游应用。影响表示类型选择的关键因素将在第 5.5 节总结。

5.1 范围感知基础

在深入讨论稠密地图表示之前，我们先简要总结一种 SLAM 中常用的关键感知模态，即范围传感器，为后续讨论提供必要背景。这类传感器测量环境中物体的距离，包括 LiDAR 传感器、飞行时间（time-of-flight, TOF）相机、RGB-D 相机和双目相机。这里我们重点介绍最常用的 LiDAR 传感器和 RGB-D 相机，它们主要用于室内外环境中的 SLAM 与稠密建图。

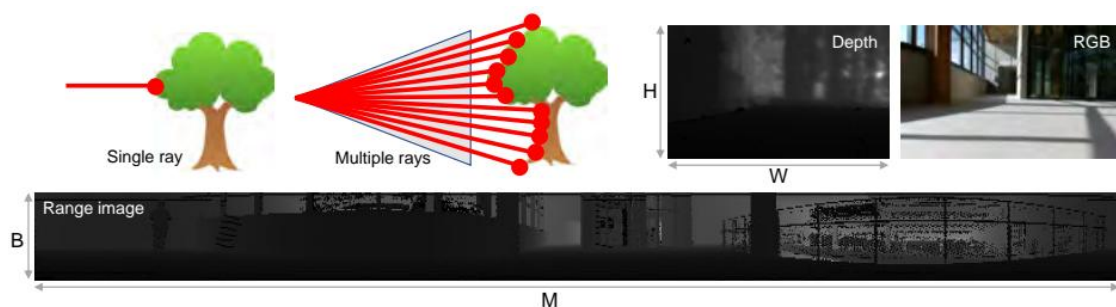


图 5.1: 图 5.1 LiDAR 传感器的单射线和多射线类型。图中给出了真实场景中的 RGB、深度和 LiDAR 范围图像示例。

5.2 传感器测量模型

先简要回顾感知机制及其测量模型。对于 LiDAR 传感器，¹ 激光束由传感器发射，经环境反射后被探测器接收^[935]。设激光束发射时刻为 t_{emit} ，探测时刻为 t_{detect} ，利用光速 c 可按下式得到范围 r ：

$$r = \frac{c(t_{\text{detect}} - t_{\text{emit}})}{2}. \quad (5.1)$$

单条射线测量可以通过按特定模式移动的射线阵列扩展为二维（2D）或三维（3D）点集合，例如进行 360 度旋转或沿特定形状扫描。传感器生成的点集合称为点云，它是构建地图的基本元素。LiDAR 测量也可以表示为范围图像 $\mathbf{R} \in \mathbb{R}^{B \times M}$ ：其中存储传感器完成一次完整旋转（即 360°）时 B 条射线各自的范围。因此，对于每条射线，在传感器的水平视场内有 M 个测量值；图 5.1 给出了示意和示例。

机器人应用中另一类常用范围传感器是 RGB-D 相机，例如 Microsoft Kinect、Azure Kinect DK 和 Intel RealSense。RGB-D 相机除了提供高为 H 、宽为 W 的 RGB 图像 $I_{\text{RGB}} \in \mathbb{R}^{3 \times H \times W}$ 外，还提供一个尺寸相同的深度图 $I_D \in \mathbb{R}^{H \times W}$ ，其中每个像素位置包含深度或范围。早期 RGB-D 相机使用具有已知图案的结构化红外（IR）光源，将该图案投射到环境中来生成深度图 I_D ；随后根据图案的畸变确定各像素的距离。由于阳光通常会干扰这种感知机制，采用投射光的 RGB-D 相机主要用于室内环境。新一代 RGB-D 传感器引入 IR 纹理投影器或飞行时间（TOF）技术，受干扰影响较小，因此可以支持室外应用。

5.3 转换为点云

利用范围传感器（例如 LiDAR 或 RGB-D 相机）的内参，可以将范围图像 R 或深度图 I_D 转换为点云 $\mathbf{P} = \{p_1, \dots, p_N\}$ 。其中点 $\mathbf{p}_i \in \mathbb{R}^3$ 在传感器的局部坐标系中表示。点是地图表示中最基本的单元，本章后续将进一步讨论。

对于 LiDAR，传感器为每条射线提供内参标定 $(\phi_{i,j}, \theta_{i,j})$ ，其中 $1 \leq i < B$ 且 $1 \leq j < M$ ；它们分别是方位角 $\phi_{i,j} \in [0, 2\pi]$ 与极角（俯仰角） $\theta_{i,j} \in [-\pi, \pi]$ ，如图 5.2 所示。利用每条射线的已知角度，可以将 $R_{i,j}$ 中的范围测量 $r_{i,j}$ 转换为三维点 $\mathbf{p} = (x, y, z)$ ：

$$x = r_{i,j} \cos(\theta_{i,j}) \cos(\phi_{i,j}) \quad (5.2)$$

$$y = r_{i,j} \cos(\theta_{i,j}) \sin(\phi_{i,j}) \quad (5.3)$$

$$z = r_{i,j} \sin(\theta_{i,j}). \quad (5.4)$$

对于 RGB-D 相机，通常采用针孔相机模型，将像素位置 (u, v) 处的范围 $r_{u,v} = R_{u,v}$ 转换为三维坐标。利用相机内参 $\mathbf{K} \in \mathbb{R}^{3 \times 4}$ ，将图像坐标中的齐次坐标 $\mathbf{x} = (u, v, r_{u,v})$ 通过伪逆转换到相机坐标中的点 $\mathbf{p}$ ：

$$\mathbf{p} = \mathbf{K}^\dagger \mathbf{x}. \quad (5.5)$$

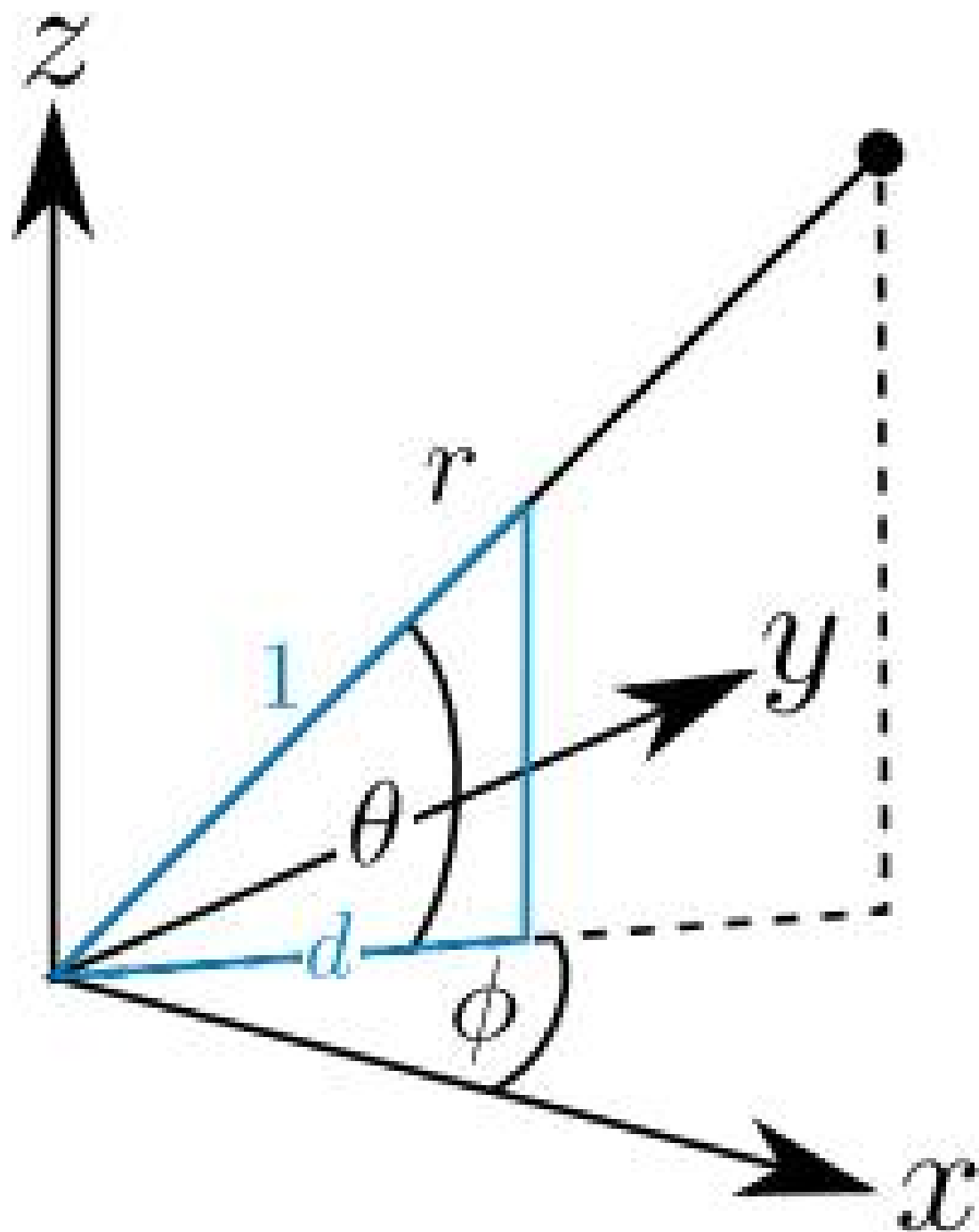


图 5.2: 图 5.2 球面坐标。

根据点的组织方式, 所得点云可分为有组织点云和无组织点云。从深度图转换得到的点云是有组织的, 每个点的位置与对应深度图的像素位置保持结构关系, 因此可通过简单索引计算相邻点。LiDAR 传感器生成的点云则更复杂。例如, 静止的 3D 机械式 LiDAR 生成的点云具有组织结构; 而对于非重复图案固态或 flash LiDAR, 以及运动中的机械旋转式 LiDAR, 这种组织结构不再成立。很多情况下, 无组织点云会因传感器运动而发生畸变, 范围图像中的测量邻域也不一定与空间邻域相关。因此, 需要估计传感器在完成一次扫描期间的运动 (例如利用惯性测量单元 (IMU) 测量或里程计信息), 结合每条射线的时间信息, 对 LiDAR 点云进行去畸变, 以补偿传感器运动^[1133, 256, 1231]。关于运动畸变及其补偿的更多细节, 请参阅第 8 章。

122

Dense Map Representations

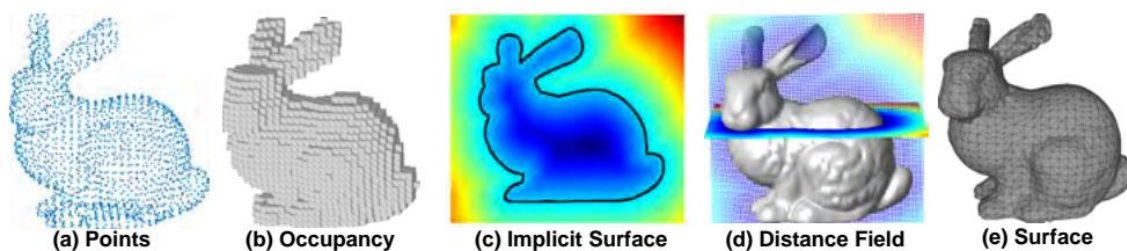


图 5.3: 图 5.3 常见稠密表示的示例。

5.4 稠密建图基础

地图是利用传感器信息和数据处理方法生成的、用于建模环境的符号结构^[1091, 136]。需要注意的是, 地图表示具有多样性, 同一空间信息可以有許多不同表示方式 (如图 5.3 所示)。场景表示的选择和精度会显著影响具体任务的性能, 因此表示方式应由使用场景决定。例如, 机器人中的运动估计和定位倾向于采用稀疏表示, 如 3D 点特征^[787, 896], 以利用这些特征实现一致的机器人位姿估计。另一方面, 场景重建的关键目标是获得准确、稠密且高分辨率的地图, 例如用于检查任务^[506, 822, 922]。类似地, 路径规划需要稠密信息, 例如障碍物占据状态或与障碍物的最近距离^[774, 304, 1110]。总体而言, 本章围绕以下三个问题组织内容。

问题 1: 稠密建图需要估计什么量? 表示环境时最常用的量是占据状态和距离。占据状态是建图中的关键属性, 用于区分自由空间与被占据空间。距离估计则通过测量到邻近表面或物体的范围, 为自由和占据空间提供更符合机器人视角的解释。

稠密建图基础。

问题 2: 应如何表示世界? 这是关于采用何种空间抽象进行表示的问题。概括而言, 表示可以是显式的, 也可以是隐式的。显式空间抽象还可根据所使用的几何类型分类, 而隐式表示可根据所选函数分类。图 5.4 展示了这些抽象类型。

问题 3: 应使用什么数据结构和存储方式? 选定的表示需要存储在内存中供后续使用。数据结构和存储方法应根据具体应用及预期用途进行选择。

文献中存在大量利用范围传感器生成场景稠密表示的方法, 它们在估计量、空间抽象、存储结构、连续性和应用领域等方面各不相同。

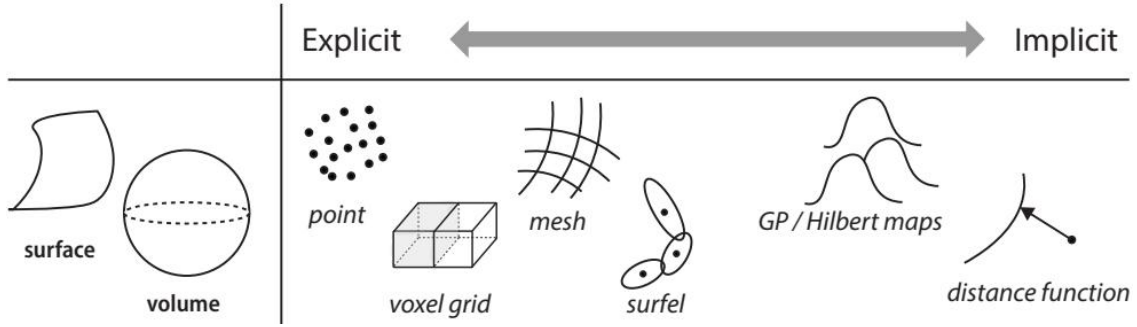


图 5.4: 图 5.4 表示可以是显式的，也可以是隐式的。为清晰起见，图示已作简化；在实际中，抽象并非严格可分，通常可以组合使用。

在讨论不同表示之前，我们先研究由范围测量估计的主要量，重点理解建图阶段最常估计的量。下面简要概述每个量的基本定义，并说明其重要性及发挥作用的具体场景。

5.5 占据地图

占据栅格由 Elfes 和 Moravec^[304, 774] 在三十多年前提出，此后得到广泛使用。占据栅格² 简单且计算高效，因此在室内（甚至室外）环境建图中很受欢迎。最简单的情形是估计某个栅格单元被占据的概率：将单元含有障碍物建模为概率，已占据单元的占据概率为 1，被判定为空的单元为 0。本质上，占据建图是一个二元分类问题，用于预测每个单元属于二元类别的概率。

给定传感器测量集合 $\mathbf{z}_{1:t}$ 和传感器位姿集合 $\mathbf{x}_{1:t}$ ，地图 m 中每个单元被占据的概率可表示为 $p(m|\mathbf{z}_{1:t}, \mathbf{x}_{1:t})$ 。图 5.5 给出了示例地图。假设每个单元 m_k 相互独立且测量条件独立，占据状态可用著名的对数几率形式高效更新^[774]：

$$l(m_k|\mathbf{z}_{1:t}) = l(m_k|\mathbf{z}_{1:t-1}) + l(m_k|z_t), \quad (5.6)$$

其中 $l(\cdot) = \log(o(\cdot))$ ， $o(\cdot)$ 为几率形式：

$$o(m_k|\mathbf{z}_{1:t}) = \frac{p(m_k|\mathbf{z}_{1:t})}{1 - p(m_k|\mathbf{z}_{1:t})}. \quad (5.7)$$

占据建图的主要优势体现在式 (5.6) 中：只需使用先前占据值和逆传感器模型 $l(m_k|z_t)$ ，通过简单加法即可更新概率。尽管如此，占据栅格要实现高效运行，依赖于对环境作出的很强假设。特别是，假设一个单元的占据似然与其他单元无关，会忽略空间相关性，而空间相关性对于推断附近未观测区域的占据状态可能十分重要。此外，传统占据栅格要求预先定义环境离散化方式，使地图各处空间分辨率保持不变。

5.6 隐式表面与距离场

表示机器人周围几何形状的另一方法不是使用占据概率，而是描述自由空间与占据空间之间的边界。在理想情况下，可以用关于三个变量 x, y, z 的解析函数描述该边界的形状和位置；当点 $\mathbf{p} = (x, y, z)$ 位于表面上时，函数值为 0。换言之，该函数的零交叉对应表面本身。

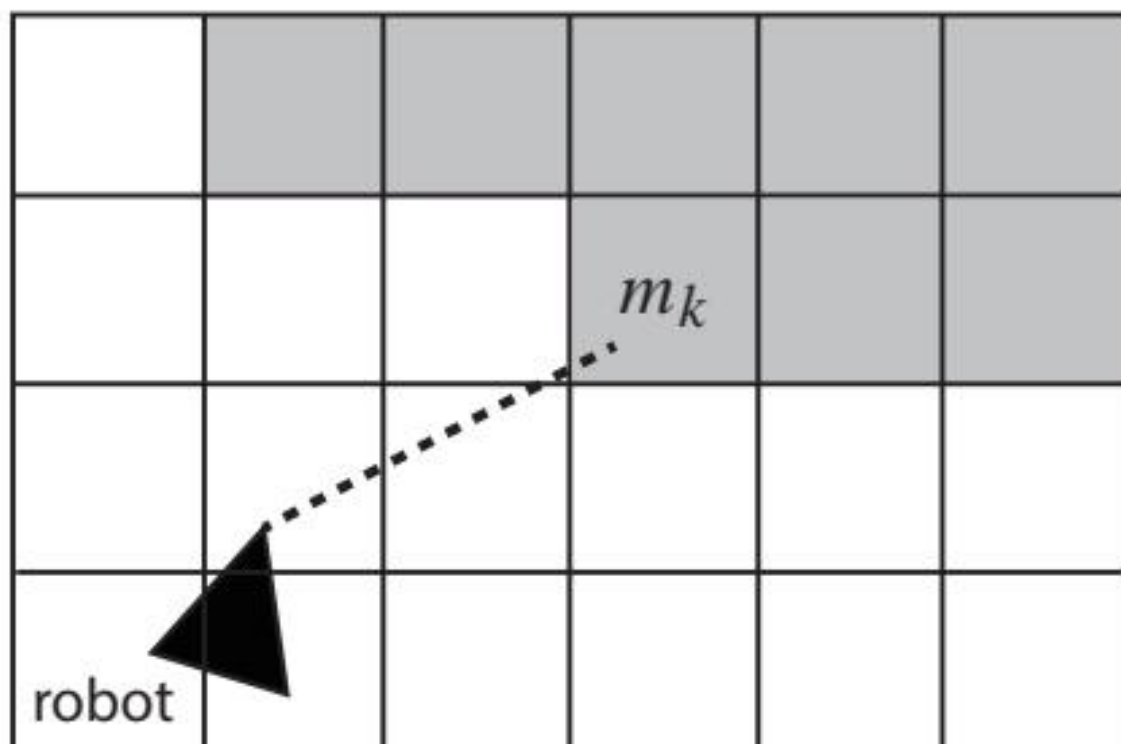


图 5.5: 图 5.5 二维简单射线投射示例。给定某个（计算或估计的）方位角处的范围测量，范围测量的回波表示存在障碍物。

这种函数称为隐式表面。按照约定，当 $\mathbf{p}$ 位于物体内部时函数符号为负，在外部时为正。图 5.6 给出了一个简单示例，说明如何确定隐式函数值。

连续隐式表面表示有许多优点。由于不存在固定分辨率，它可以在任意细节层次表示任意形状的物体。此外，只需计算函数符号，就能判断空间中的给定点位于障碍物内部还是外部。

可以使用不同类型的函数对隐式表面建模，其中欧氏有符号距离函数（Euclidean Signed Distance Function, ESDF）是一种常见选择。在任意查询点处，ESDF 的幅值表示到最近表面的距离，符号表示该点是否位于障碍物内部。ESDF 表示常用于加速几何算法，例如碰撞检查。此外，还可以从 ESDF 推导高质量的邻近梯度，用于基于优化的运动规划和形状配准。

ESDF 通过为地图中的每个点寻找（欧氏）最近表面点来计算。对于已知表面，可使用 Fast Marching 等技术高效完成。然而，在表面估计中，更常使用投影有符号距离，因为它可以从测量中高效计算，也更适合滤波。给定一条穿过查询点的测量射线，投影距离定义为沿射线从射线端点到查询点的距离，因此不必显式搜索最近点。尽管投影距离会高估欧氏距离，但其零交叉（估计表面）仍然正确。Curless 和 Levoy^[234] 提出的、后来由机器人领域 KinectFusion^[801] 推广的标准隐式表面估计方法，将所有测量的投影有符号距离进行加权平均。为减小高估的影响，投影有符号距离函数通常被截断到固定范围内，称为截断带，此时称为截断有符号距离函数（Truncated Signed Distance Function, TSDF）。注意，ESDF 可以从 TSDF 和占据地图高效计算，详见第 5.4.4 节。

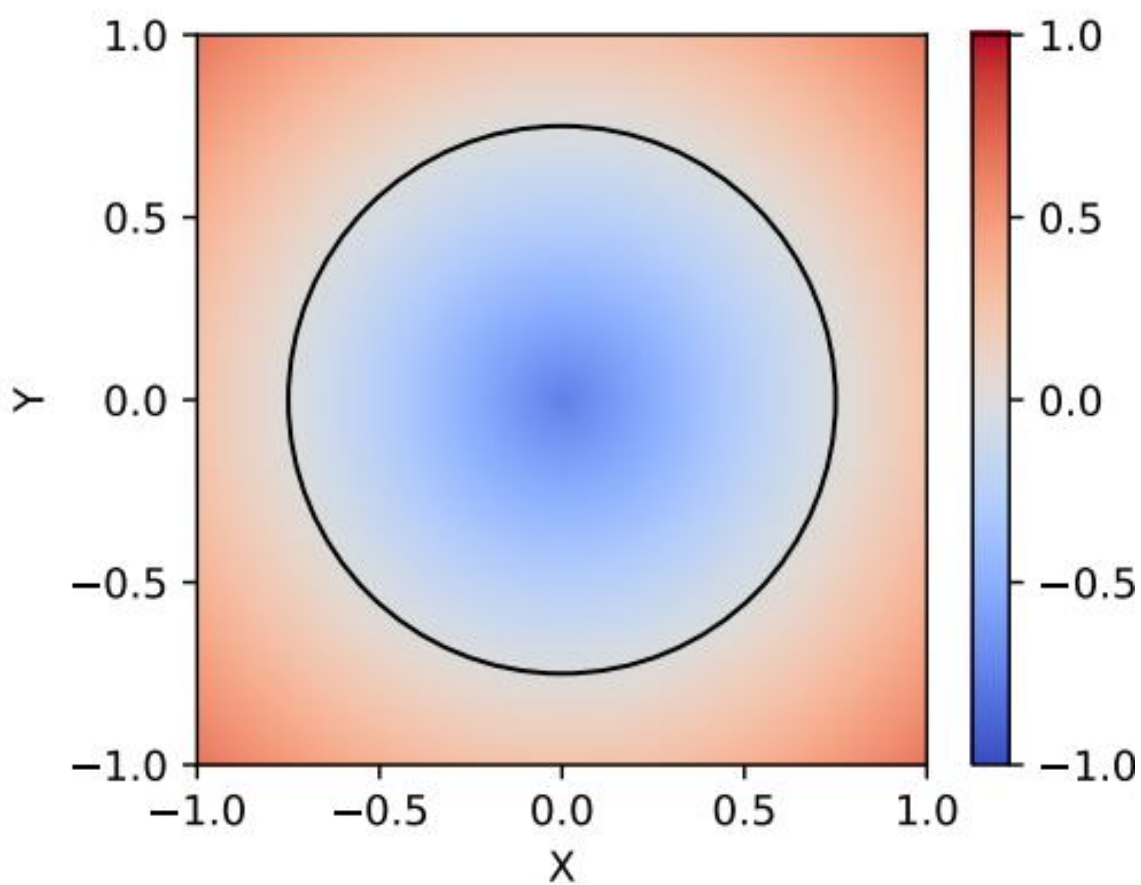


图 5.6: 图 5.6 用显式表面（黑色轮廓）和隐式表面（彩色场）表示的实心二维圆盘。隐式表面函数在物体内部（蓝色）为负，在外部（红色）为正。注意该函数的零交叉对应表面本身。

5.7 占据地图还是距离场？

占据表示和隐式表示都是体积方法，它们的共同点是：通过在观测体积内处处估计感兴趣的量来建模几何形状。然而，两种表示的优先目标不同，哪一种更好取决于具体应用。下面简要总结两个关键差异。

建模直接性：测量射线可以直接告诉我们空间的哪些部分是自由、被占据或未观测的，因此基于占据概率的地图更新所需的启发式方法和假设更少。相比之下，隐式表面通常对到表面的距离建模。对于已知表面，该距离可以精确计算；但对于部分测量，只能依赖前述 TSDF 等距离代理。

平滑性：隐式表面地图天然比占据地图平滑，后者建模的是二元属性。隐式表面的平滑性带来许多好处，最重要的是使其可微，从而产生对许多应用有价值的邻近梯度。平滑性还能降低离散化导致的近似误差，并通过插值获得良好的亚像素分辨率估计。然而，由于不连续性无法平滑表示，隐式表面往往会漏掉细薄障碍物。

5.8 地图表示

5.9 目标空间结构的显式性

如图 5.4 所示，可以根据表示的显式性及其目标空间进行分类。在 3D 建图中，表示体积很直接；但在机器人建图中，表面同样重要，因为它支持各种下游任务。我们可以考虑四类主要表示：显式表面、隐式表面、显式体积和隐式体积表示。

对于表面，可以采用显式或隐式方式表示。显式表示将表面视为二维流形，旨在用场景中物体的边界刻画空间。能够表示边界的最简单抽象就是范围传感器生成的点云。另一种通用表面表示是多边形网格（第 5.4.3 节），由顶点、边和面组成。网格通过形成相连的闭合多边形（更常见的是三角形）来编码体积的定向表面。Surfel（第 5.4.2 节）也是常用抽象，广泛用于建图。表面表示是可视化应用的基础，也用于渲染仿真环境、增强现实、计算机辅助设计和 3D 打印。

3D 体积建模采用类似策略。基于点的朴素表示常用于 LiDAR SLAM；此外，基于占据或距离的体素（第 5.4.4 节）也是显式表示的常见选择。存储体积地图时，需要仔细考虑数据存储，以降低计算成本。体积建图也使用隐式表示，通常通过函数实现。高斯过程（Gaussian process, GP，第 5.4.5 节）和 Hilbert map（第 5.4.6 节）是隐式表示的典型例子。

5.10 空间抽象类型

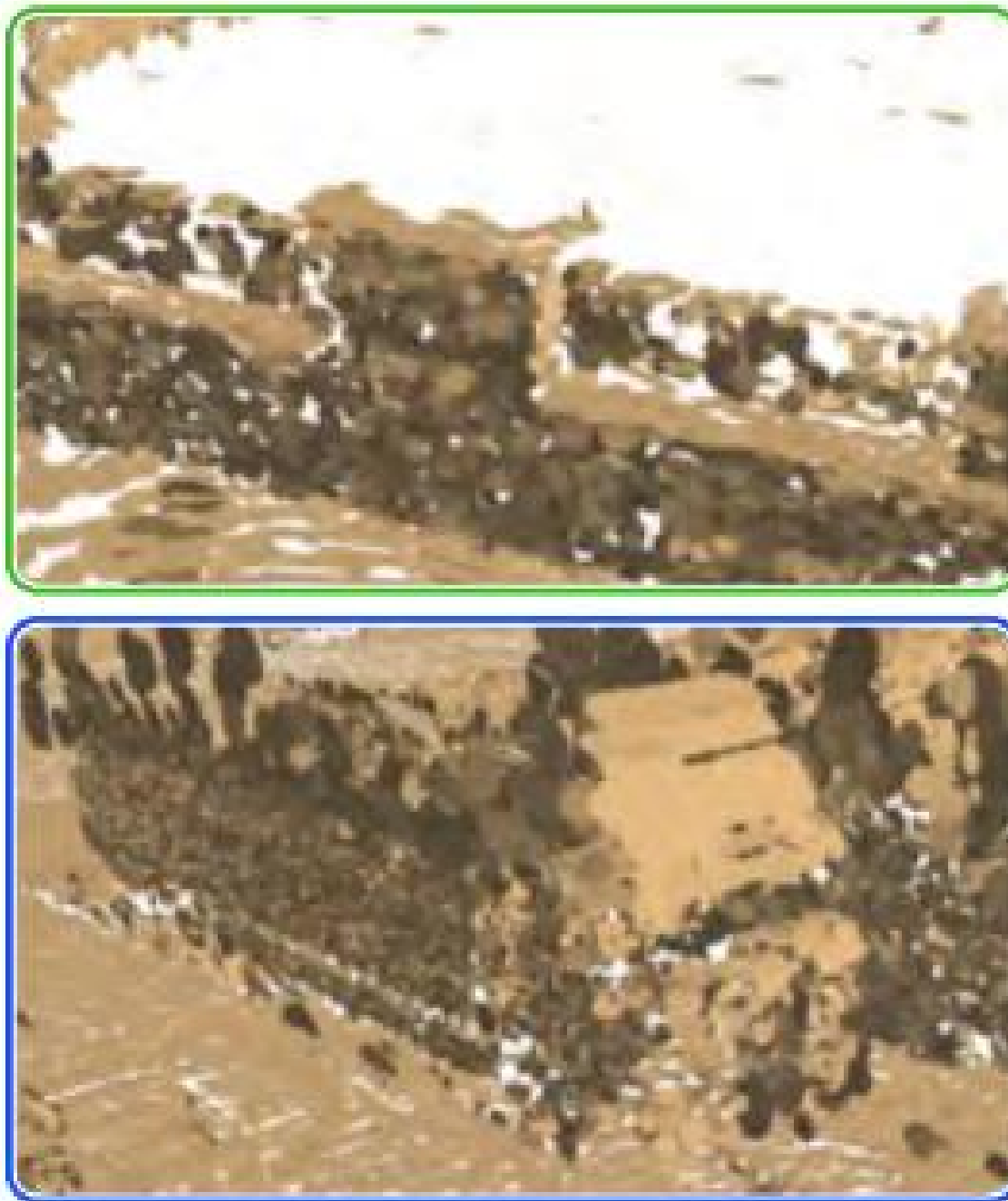
5.11 点

给定范围测量，一种直接的稠密地图表示是使用点云。构建时，我们利用估计的全局位姿 T_t^1 ，将时刻 t 在局部坐标系 $\mathcal{F}^t$ 中记录的点云 P_t 累积到全局地图点云 P_M 中。通常假设 P_M 的全局坐标系取为第一个点云的坐标系 $\mathcal{F}^1$ 。

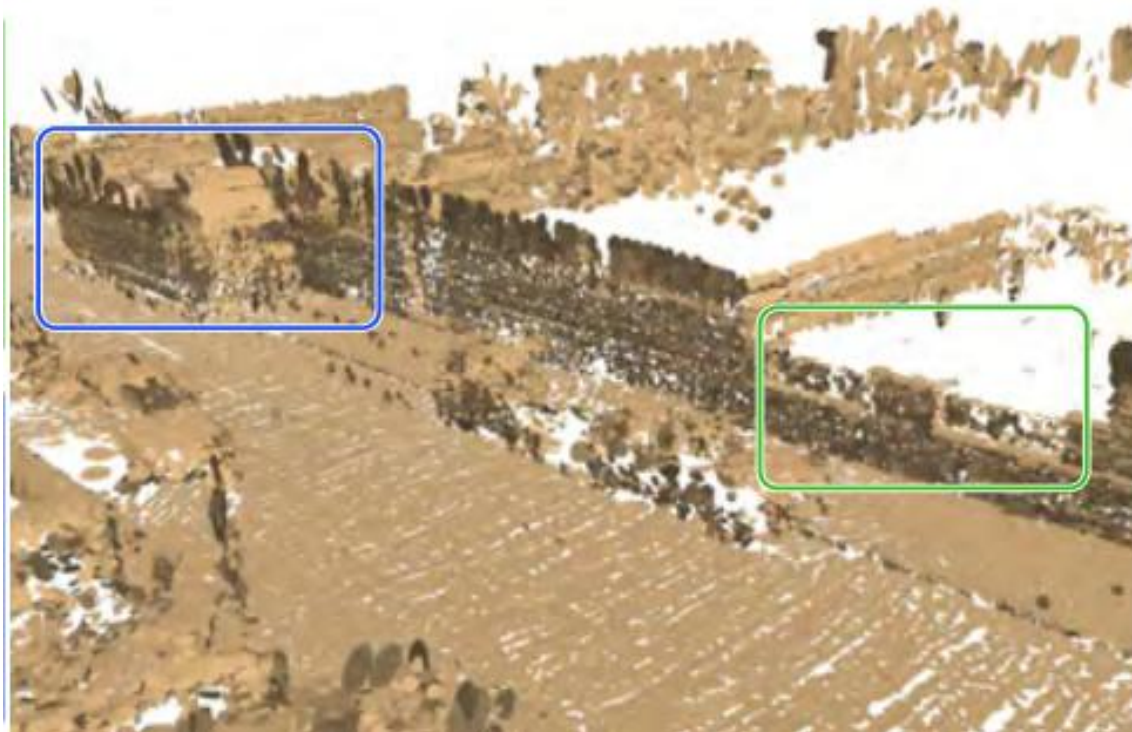
简单累积点云 $P_1, \dots, P_t$ 无法扩展到大型环境，因此常见策略是丢弃同一空间位置的冗余测量。为此，大多数方法^[1269, 256, 1133]使用高效的最近邻搜索（例如体素栅格或基于层次树的

表示, 见第 5.3.3 节) 对点云进行下采样和存储, 例如每个体素只存储有限数量的点^[256, 1133], 或仅保存满足特定准则的点^[1269]。也可以只保存关键帧, 即显式存储少量点云, 但这需要确定何时生成关键帧或子地图。

点云是最基本的表示形式, 可以转换为其他表示, 例如通过 Poisson 表面重建^[1131] 转换为网格, 或通过 Marching Cubes^[1132] 转换为有符号距离函数 (Signed Distance Function, SDF)。但这需要额外数据, 例如点的观测视点; 将多次测量合并为点云地图时, 该信息可能丢失。因此, 通常需要对表面方向作出假设, 以区分物体内部和外部区域。



(a)



(b) 图 5.7 对 KITTI 数据集^[373] 第 07 序列中一组 LiDAR 扫描, 通过累积点云和 surfel 生成的地图进行定性比较。(a) 点云地图, 点的亮度表示 LiDAR 测量的反射率。(b) 基于圆盘的对应 surfel 地图。累积所有点云的完整地图占用 2.95 GB, 而 SuMa^[66] 的对应 surfel 地图仅占用 160 MB。

5.12 Surfel

点云地图直接表示测量, 但存储的点不包含表面信息, 也无法表示点是从哪个方向测得的。通过 surfel^[870], 可以向点添加方向信息来编码这些内容。Surfel 通常表示为圆盘或椭圆^[549, 66, 1184, 104, 103], 更一般地也可以表示为用高斯模型描述的椭球^[1041, 1042, 282, 1318]。Splatting^[1318, 103] 不仅可以整合纹理信息, 还能将重叠 surfel 混合为特定视点下连贯的渲染结果; 更详细的讨论见第 14 章。

一个常用的圆形 surfel (表示圆盘) 由位置 $\mathbf{p} \in \mathbb{R}^3$ 、法向方向 $\mathbf{n} \in \mathbb{R}^3$ 和半径 $r \in \mathbb{R}$ 定义。利用现代图形处理器 (GPU) 的能力可以高效渲染这些几何基元, 从而加速点到 surfel 的关联并显著减少内存占用。

图 5.7 对比了基于点云和基于 surfel 的地图表示。稠密点云能够以很高的细节准确表示环境, 但代价是占用大量内存。相反, 将多次测量聚合为单个 surfel 会损失细节, 但能在保留大尺度表面主要结构的同时显著降低内存使用量。

与使用 surfel (即小型圆形表面片元) 显式表示表面几何密切相关的是正态分布变换 (normal distributions transform, NDT)^[82, 1035]。NDT 将空间划分为体素, 并用正态分布 $\mathcal{N}(\mu, \Sigma)$ 近似体素内的点, 其中均值 μ 和协方差 Σ 由所含点估计。协方差的特征值 $\lambda_1 < \lambda_2 < \lambda_3$ 及其对应特征向量 $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ 可用于估计体素内的表面属性。对于平面表面 ($\lambda_1 \ll \lambda_2$), 最小特征值 λ_1 对应的特征向量 $\mathbf{v}_1$ 即表面法向。因此, 对平面表面而言, NDT 表示一个 surfel, 同时也能更准确地表示无法用单个 surfel 近似的点分布。从这个意义上说, NDT 是一种混合表示: 空间被划分为体素栅格, 因此具有显式性; 而体素由正态分布表示, 能够连续表示体素内部空间, 因此又具有隐式性。

5.13 网格

点云和 surfel 地图虽然描述了局部表面属性, 但由于没有建模表面连通性, 仍然相对稀疏。为了获得更完整的表示, 可以使用网格, 将表面描述为一组相互连接形成多边形集合的点。这样就能表示水密表面、查询并插值新的表面点, 并沿相连表面高效遍历。

在网格术语中, 每个多边形称为一个面, 每个角点称为一个顶点。最常见的是三角形网格 (每个面由三个顶点围成) 和四边形网格 (每个面由四个顶点围成)。注意, 任意多边形 (面) 都可以拆分为等价的三角形集合; 因此, 三角形网格不仅最简单, 也是最通用的网格类型。

网格是一种非常灵活且节省内存的表示, 因为面和顶点的数量可以直接适应表面复杂度及所需细节层次。例如, 任意大小的平面只需两个三角面和四个顶点即可表示。此外, 网格适合并行处理和渲染。网格常用于与计算机图形学交叉的应用, 如渲染、表面分析、操作和形变, 也广泛用于数字模型、仿真或基于表面的算法, 例如地面机器人的路径规划。

5.14 体素

点云和网格适合表示沿表面定义的环境属性。然而，占据状态和有符号距离等估计量是在整个体积内定义的。一种直接存储和处理体积属性的方法，是在规则栅格上对其进行离散化。离散化的占据地图和有符号距离地图分别称为占据栅格和有符号距离场。

类比二维像素，3D 栅格中的单元称为体素。由于栅格结构规则，每个体素都可以分配唯一索引并存入数据结构。需要注意，体素只是一个容器，更形式地说是一个空间划分；其内容的含义因方法而异。在经典占据栅格中，体素值表示该体素内任意点被占据的可能性，因此地图任意点的占据值等于包含该点的体素值。但体素值不必表示一个常量小立方体。例如，有符号距离场中的体素值估计体素中心处的有符号距离；要获得任意点处的距离，需要查询该点邻近体素并通过插值得到其值。一些应用甚至使用（稀疏）体素栅格存储和高效查询非体积属性，例如点或表面颜色。

5.15 连续函数

函数是连续建图的关键抽象。在这种情况下，建图问题归结为拟合参数化或非参数化函数，即求解回归问题。上面提到的许多空间抽象都要求预先离散化环境，通常使地图各处空间分辨率固定；而连续函数（无论参数化还是非参数化）提供更大灵活性，可以重新计算分辨率，并通过插值填补数据缺口。

某些参数化函数，例如二维无穷直线^[1106]和三维平面^[533, 374, 1106]，需要对环境作出严格假设，从而限制场景表示。但它们在内存和计算复杂度方面高效。基于控制点的函数（如 B-spline^[932]）或非参数化函数（如基于 GP 的函数）能够在更少假设下连续建模环境。从占据状态^[816]、隐式表面^[1186, 636]、距离场^[1195]到表面本身^[1122]，尽管计算复杂度较高，基于 GP 的表示因其概率性质而成为表示环境的常用选择；概率性质可以对观测区域和未观测区域进行不确定性量化与推断^[378]。

连续函数用于建图的一个关键优势是：如果选取至少一次可微的函数，就能提供梯度。梯度信息对于定位（计算表面法向）^[1196]、闭环检测（计算地形特征）^[377]、数据融合^[636]和规划^[1196]都很重要。

5.16 转换

上述抽象并不总是单独使用；它们经常相互转换，或以多种形式同时使用。

例如，可以将点和网格等显式几何转换为隐式表面。一种灵活的距离计算方法是最近点查询：可以针对任意显式几何执行，并且只在需要的时间和位置按需计算。另一种方法是在规则栅格的所有点上使用波前传播计算有符号距离，这可以通过 Fast Marching 方法高效实现^[991]。

反过来，将隐式表面转换为网格表示也很常见。将距离场转换为网格的原始技术称为 Marching Cubes^[696]。该算法将隐式表面划分到固定大小的立方体栅格中，并独立处理每个立方体。每个立方体根据八个角点处的隐式表面值生成一组三角形元素（图 5.8），再通过线性插值细化顶点位置。包括 BIM（Building Information Modeling）和 CAD（Computer-aided Design）模型在内的网格，又可以采样生成点或 surfel。

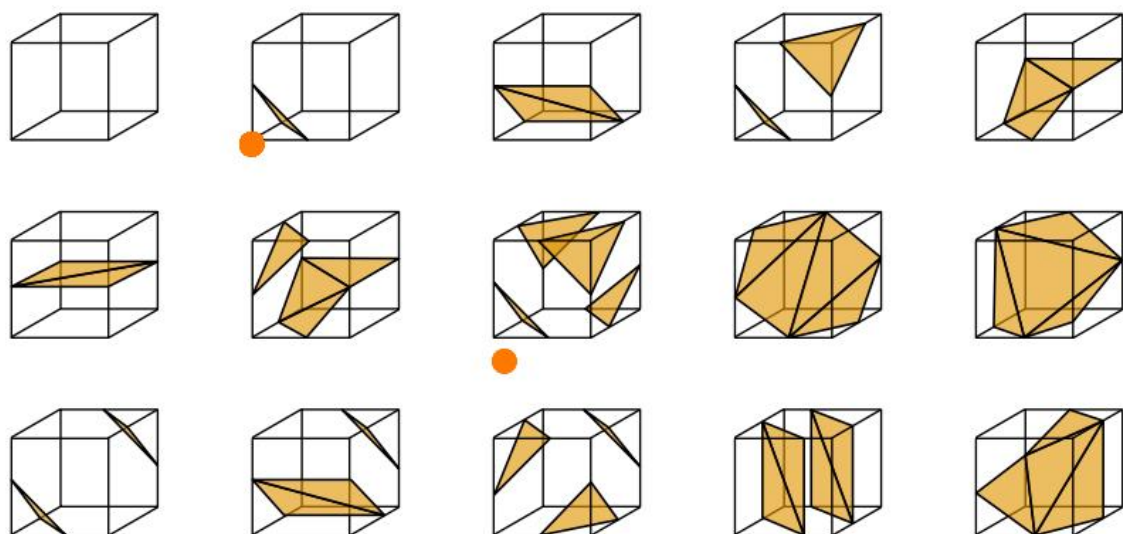


图 5.7: 图 5.8 Marching Cubes 算法将立方体投影到隐式表面上, 查询立方体顶点处函数值的符号, 并查表确定这些值对应的 15 种配置之一。(图片来源: Ryoshoru, “Marching cubes”, CC BY 4.0 许可, 来源: <https://commons.wikimedia.org/wiki/File:MarchingCubesEdit.svg>)

有时还需要将离散表示转换为连续表示。这通常通过对连续表示的参数求解优化问题来实现, 使相对于离散数据的拟合误差最小。

5.17 数据结构与存储

前几节介绍的抽象都需要存储在内存中。本节通过考察数据结构的选择及其优缺点, 讨论各种抽象如何存储。

5.18 朴素数据存储

对于许多表示, 简单的动态可调整大小数组是合理的起点。对于具有预定义空间划分的数据, 需要确定要存储的数据类型, 以及将空间坐标转换为空间索引坐标的函数。这常用于构建表示占据值或有符号距离值的地图。对于不规则数据, 只需确定存储的数据类型, 例如点云或 surfel。图 5.9 展示了使用映射函数存储有序数据以及存储无序点云的朴素方法。

这种朴素方法的优点是简单且支持快速随机访问。代价是表示可能需要大量内存。此外, 读写和修改操作虽然很快, 但改变表示的空间维度可能代价高昂, 因为需要复制整个数据结构的内容。

5.19 哈希表

解决上述朴素存储局限的一种自然扩展是使用哈希表。该方法将地图划分为多个分片, 对每个分片的坐标应用哈希函数得到单个值, 并将分片数据存入以该哈希值为索引的表中。分片通常表示具有明确坐标的地图子区域, 例如规则栅格中的立方体; 这些立方体可以对应单

Dense Map Representations

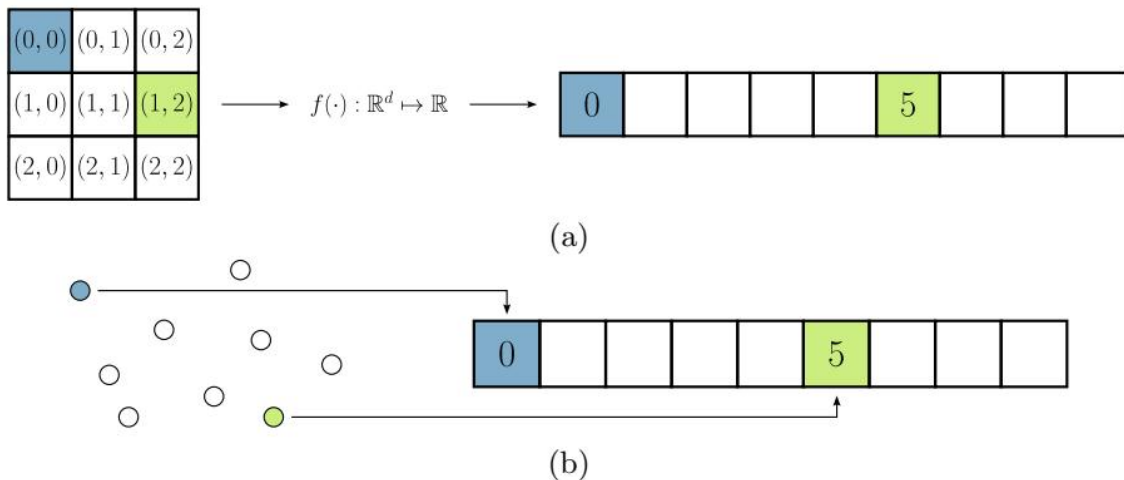


图 5.8: 图 5.9 (a) 通过函数 $f(\cdot)$ 将逻辑栅格坐标映射到朴素数组存储, 函数将坐标唯一映射为线性索引。(b) 将无序数据直接存入数组结构。

个体素或固定大小的体素组 (称为体素块), 也可以存储各自子区域中的点、surfel 或网格片段。

哈希表保留固定数组的 $O(1)$ 查找时间, 同时允许地图动态增长而无需重新分配。使用哈希表存储稠密地图时需要考虑三点:

1. **分片粒度**: 较小分片只在需要的位置分配数据, 可以提高稀疏性。但分片数量不应过多, 否则会降低哈希表的插入性能和内存效率。当哈希表存储仅沿表面存在的属性时, 这一权衡尤为重要。
2. **哈希函数**: 理想的哈希函数应在表中均匀分布键, 即使数据在空间上相邻 (建图中经常如此)。
3. **冲突处理**: 处理哈希冲突的方法 (例如链式法, 即每个条目包含链表, 或开放寻址) 会显著影响哈希表性能。

在大多数情况下, 哈希表在快速访问与高效插入、删除之间取得了良好平衡。但针对特定应用, 通常需要初始调参才能获得良好性能。

5.20 基于树的数据结构

另一种只为环境中相关部分分配内存、同时高效存储空间数据的方式, 是使用分层树结构表示。与哈希表类似, 树通常支持高效访问和插入; 其独特优势在于分层结构, 可高效存储多分辨率数据, 并加速最近邻搜索等空间操作。最常见的树变体包括 kD-tree^[72]、包围体层次结构 (bounding volume hierarchy, BVH)^[223] 和八叉树^[752]。

其中, 八叉树支持高效邻域搜索和增量整合新测量。八叉树中的每个节点由一个称为八分体的子空间表示。八分体由中心 $\mathbf{c} \in \mathbb{R}^3$ 和边长 $e \in \mathbb{R}$ 定义, 对应一个轴对齐包围盒。每个八分体最多有 8 个子八分体, 其边长为 $\frac{1}{2}e$, 如图 5.10 所示。

通常只在叶八分体 (即没有子节点的八分体) 中存储点, 并通过遍历树结构确定内部八分体中的点子集。

构建八叉树时, 在包含点云 P 的轴对齐包围盒内迭代地将空间划分为八分体。每次划分都将 P 分成子集 $P_1, \dots, P_8$, 对应范围为 $\frac{1}{2}e$ 的 8 个子八分体。非空子集 P_i 形成中心为 $\mathbf{c}$ 、

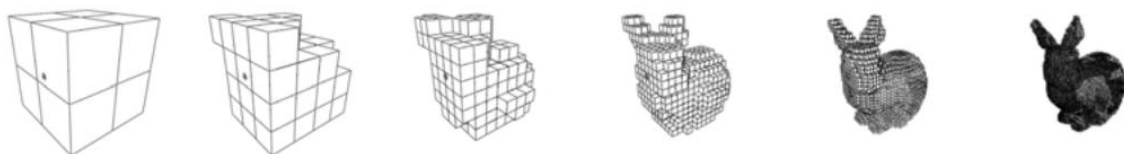


图 5.9: 图 5.10 不同树层级的八叉树及其八分体示例。八叉树的每一层都将空间细分为更细粒度的八分体。注意，八叉树的更深层级只表示被占据的空间。

范围为 $\frac{1}{2}e$ 的子八分体；当达到指定八分体大小或最小点数时停止。构建完成后，通过遍历树结构并按需添加内部节点，可以高效执行更新和插入。当新数据落在根八分体之外时，可以创建新的根节点，并将新数据和旧根节点分配给其子节点，从而扩展树。

与体素栅格相比，八叉树只表示包含数据的子空间，因此能够高效存储被占据空间。但这种内存效率需要通过树遍历访问特定叶八分体，可能增加定位一个点所需的时间；同时必须显式表示树结构，带来额外内存开销。近期一些方法针对这些内存开销进行了改进 [305, 820, 70]。

5.21 混合数据结构

为平衡内存需求与数据访问运行时间，一些数据结构以特定方式结合不同数据结构的优点，形成混合表示。在这种权衡中，我们接受较低的内存使用效率，以获得更高效的内存访问。

例如，哈希体素栅格 [808] 将环境划分为固定大小的稠密块（如 $8 \times 8 \times 8$ 个体素），再把这些块存储在哈希表中，从而结合稠密体素栅格和哈希表的优点。借助哈希表的灵活性，只需为包含有意义信息（如靠近表面的位置）分配块；同时，在每个块内使用普通 3D 数组存储体素，使操作简单、高效，并适合 GPU 加速。

另一种选择是将哈希表与树结合。类似于哈希体素栅格，VDB³ 数据结构 [792, 791] 将空间划分为哈希块，但在每个块内存储分层树。该结构提供层次树的全部优点，包括多分辨率表示和高效最近邻查询。然而，由于每个块大小固定，无论环境规模如何，树的最大高度都保持不变。因此，查找和插入可以在常数时间内完成，并显著快于纯树结构。

5.22 构建地图：方法与实践

节	空间抽象类型	表示的地图实体
5.4.1	点	表面
5.4.2	Surfel	表面
5.4.3	网格	表面（相连）
5.4.4	体素	占据状态或隐式表面
5.4.5–5.4.6	连续函数	占据状态或隐式表面

表 5.1 所介绍建图方法的总结。

到目前为止，我们讨论了需要估计的目标量以及建图可用的各种空间抽象。本节将详细考察用于构建这些地图元素的方法。方法按其主要空间抽象分类，如表 5.1 所示。注意，一些

方法会使用额外的空间抽象来提高性能，例如将点分组到体素中，以实现更高效的存储和更快的查询。

5.23 点

如第 5.3.2.1 节所述，简单累积测得点云来存储点无法扩展到大规模环境，还会造成测量冗余。因此，大多数方法^[1269, 256, 1133]将基于点的表示与体素栅格或八叉树结合，用于表示稠密地图。此外，数据结构的选择由高效最近邻搜索的需求驱动；最近邻搜索对于通过迭代最近点（iterative closest point, ICP）进行扫描配准至关重要，因为点对应关系需要迭代建立。

为了处理大规模环境，某些方法（例如著名的 LiDAR SLAM 方法 LOAM^[1269]）会滤波原始点云，提取角点和平面点，从而显著缩小点云规模。它们使用体素栅格只存储地图表示中的点子集，以剪枝冗余测量。基于 LOAM 中基于点的体素化，后续方法^[998, 1150, 839, 667, 894, 999]分别改进点提取^[998, 1150, 839]、优化流程^[839, 667]，或整合 IMU 信息^[894, 999]。

另一类方法采用不同方式处理点云数据量，以避免依赖能力很强的特征提取方法。通过体素栅格^[256]对点云进行规则采样，可以显著减少每次 LiDAR 扫描的点数并去除潜在冗余信息。关键在于，体素栅格中的点并不聚合和平均，而是保留原始测量。沿着这一思路，Dellenbach 等人^[256]和 Vizzo 等人^[1133]使用该策略对输入点云下采样，在体素栅格地图中仅存储每个体素内受限数量的点。

总的来说，如第 5.3.2.1 节所述，（哈希）体素栅格具有双重作用：通过每个体素存储有限数量的点测量来抽象空间，并通过直接索引相邻体素加速最近邻搜索。

5.24 Surfel

Surfel 可以采用与点云类似的策略。值得注意的是，Stückler 和 Behnke^[1041]以及 Droschel 和 Behnke^[282]的后续工作使用八叉树，在八叉树层级的多个层次表示 surfel，以进行数据关联。所谓多分辨率 surfel 地图通过累积的均值和协方差统计量间接表示 surfel，类似 NDT。

相比之下，Whelan 等人^[1184]将 surfel 存储为简单列表，并利用高效渲染技术生成投影，用于室内 RGB-D SLAM 的数据关联。在这种情况下，surfel 是显式几何基元，因此需要直接处理以相应更新 surfel 属性（即大小和方向）^[549]。Whelan 等人^[1184]的一项关键贡献是利用地图形变直接形变 surfel，而不是依赖 PGO；检测到闭环时，可以利用 surfel 表示的测量来形变地图。Park 等人^[846]也采用了类似的 surfel 地图形变策略。

同样，Behley 和 Stachniss^[66]面向室外环境，需要用多个空间范围为 $100\text{ m} \times 100\text{ m}$ 的子地图表示 surfel，并可将其从 GPU 内存换出。与 ElasticFusion^[1184]不同，该方法依赖 PGO，但利用 surfel 可以自由定位这一点，将 surfel 绑定到位姿，从而可以用位姿图优化后的位姿直接形变地图；其他方法^[1158]也采用了这一策略。

5.25 网格

如前文所述，网格以富有表现力且灵活的方式表示相连表面。网格生成方法可分为两类。第一类直接将测得的点转换为网格；第二类将问题分为两步：重建隐式表面，然后提取等值面

得到最终网格（见第 5.3.2.6 节）。

第一类方法通常直接计算输入点集的 Delaunay 三角剖分，并识别位于表面上的 Delaunay 三角形子集；^[157] 给出了详细综述。直接由点构建时，网格会隐式适应采样密度。这既可能是优点（提供自适应分辨率），也意味着方法对采样不规则和孔洞更加敏感。实践中，当整个表面可以由非常精确的深度传感器进行稠密采样时（例如使用测量设备），通常选择直接网格化方法。

第二类方法使用隐式表面作为中间步骤，在提取最终表面网格前简化数据融合和滤波。根据数据生成隐式表面的一种直观方法，是在规则栅格的每个点估计到表面的距离。如第 5.2.2 节所述，还必须根据每个点位于物体内部还是外部来设置隐式表面的符号。这一信息通常依据估计的表面法向确定，例如在小邻域上应用主成分分析（Principal Component Analysis, PCA）。然而，正如^[476]所指出的，这类方法可能产生不连续的隐式表面。为解决该问题，Carr 等人^[155]用一组径向基函数（Radial Basis Functions, RBF）对隐式表面建模，并通过求解全局优化问题拟合输入点。所得隐式表面在构造上是平滑的，并根据全局上下文可靠填补孔洞；但求解底层的大规模稠密优化问题计算代价很高。Shen 等人^[1002]使用移动最小二乘（moving least squares, MLS）对输入点进行局部近似，克服了这一限制。进一步地，Poisson 表面重建^[545]通过求解偏微分方程（partial differential equation, PDE）将隐式函数拟合到测量点的法向，得到稀疏且计算可处理的优化问题，对噪声尤其鲁棒。

在机器人应用中，通常希望从实时传感器流构建网格。通过增量更新，可以使表面重建达到实时效率。基于 TSDF 的表面重建因其天然的增量特性和总体简单性而广受实践采用。该方法属于第二类方法，通过平均投影距离估计隐式表面。由于更新 TSDF（或一般隐式表面）的代价远高于网格提取，实时方法主要集中于优化前者。

5.26 体素

基于体素的方法是 3D 重建和机器人学中最常用的体积表示之一。本节不按时间顺序罗列大量文献，而是聚焦实践中常见的概念，并依据三个基本决策标准组织内容：选择的估计量、数据结构和可扩展性。

5.27 按估计量分类的方法

基于体素的建图框架首先要选择估计量，最常见的选项是占据状态（见第 5.2.1 节）或距离度量（见第 5.2.2 节）；第 5.2.3 节的讨论可用于在两者之间作出选择。

自声呐的连续概率占据测量模型^[774]提出后，人们开发了简化的分段常数模型来降低计算成本^[478]。这一转变受到 LiDAR 技术出现以及从 2D 地图向 3D 地图发展的研究兴趣推动。最近，Loop 等人^[692]提出连续概率模型：该模型不膨胀物体，而是使物体表面处的占据概率收敛到 0.5。占据估计由于召回率高而常用于碰撞规避，但与基于距离的方法相比，其不连续性和缺乏信息的梯度限制了性能（见第 5.2.3 节）。

对于距离度量，不仅要估计距离场的正值部分，还要在表面后方外推负距离，因为表面由有符号距离场的零交叉表示。为限制融合不完美正、负距离估计对精度的影响（见第 5.2.2 节），更新通常被限制在表面边界附近的小截断带内。然而，基于距离的方法仍容易擦除几何形状。例如，从相反两侧观测细薄物体时，对观测到的正距离和虚构的负距离求平均，会使零

交叉翻转或消失。一些工作分析了截断带和权重衰减对最终重建质量的影响^[134]；从根本上说，该问题可以减轻但无法消除。总体而言，在平滑表面上，TSDF 估计的表面优于占据方法，但对细薄物体的召回率较低。

TSDF 提供的距离信息本身非常有价值。然而，TSDF 并不保守，而是严格高估欧氏距离。为解决这一安全问题，voxblox^[822] 推广了增量构建 ESDF 的方法：它将传感器数据融合到 TSDF 中，然后使用 brushfire 算法^[629] 更新 ESDF。随后，FIESTA^[426] 提出一种混合方法，直接从占据地图增量更新 ESDF。

5.28 按数据结构分类的方法

体积建图最简单的数据结构是静态 3D 数组。KinectFusion^[801] 表明，这种结构对于小型、固定大小的场景能够取得良好结果。但许多应用要求地图在运行时动态扩展，并且只在需要的位置分配体素以节省内存。

为此，Nießner 等人^[808] 提出体素块哈希方案，将体素分组为块（例如 $8 \times 8 \times 8$ 个体素），并将块存入哈希表。该结构很快被 TSDF 采用，提供常数时间 ($O(1)$) 查找和动态插入；当然也可用于存储占据概率，如 FIESTA^[426] 所示。与单独哈希每个体素相比，将体素分组为块可以在哈希表大小和体素分配粒度之间提供可调权衡。

体素也可以使用树结构存储。Octomap^[478] 首先推广了使用八叉树存储占据概率，多年来一直是体积建图的事实标准。树结构的主要优点是天然支持多分辨率；主要限制是编码树结构会产生显著内存开销，且单元查找时间与树高成正比。近期方法通过混合数据结构缓解这一限制。例如，Supereight^[1125] 在前几层使用标准（动态）八叉树，在最后几层使用静态八叉树。静态八叉树可视为使用固定大小数组存储的八叉树，这消除了用指针编码父子关系的内存开销，但由于静态八叉树按块分配，粒度有所降低。VDB^[792] 数据结构最初为视觉特效（VFX）行业提出，后来被多个体积建图框架采用^[721, 1132]。如第 5.3.3.4 节所述，它将块哈希与树结合，兼具两者优点：良好的内存效率、类似哈希表的常数时间查找和插入，以及类似树的多分辨率。

实际应用中，下游任务往往还要求在占据概率或 TSDF 旁存储颜色、语义^[398, 942] 或 ESDF^[822, 426] 等附加信息。尽管几乎所有数据结构都可以扩展以支持附加通道，但实现工作量会随底层数据结构的复杂度增加。这进一步说明应优先使用简单数据结构（如体素块哈希）或灵活的第三方库。

5.29 按测量融合算法分类的方法

根据深度测量更新地图的算法称为测量融合器。它通过应用测量模型，更新每个观测体素的估计量。融合测量主要有两类方法：射线追踪和基于投影的方法。

对于每个测得的点，射线追踪融合器从传感器向该点发射射线，并更新射线穿过的所有体素。这种方法非常通用，只需知道传感器原点的位置。然而，一个体素可能被多条射线击中，尤其是靠近传感器的体素，这会造成重复计算；在并行实现中处理由此产生的竞争条件还会带来实现和性能开销。

相反，基于投影的方法直接遍历观测体素，并将每个体素投影到传感器坐标系，查询用于计算更新的射线。遍历地图而不是射线天然避免了竞争条件，因此基于投影的方法广泛用于

多线程和 GPU 加速的体积建图框架。直接遍历地图产生的可预测访问模式还能减少内存瓶颈。但该方法的主要缺点是必须明确知道传感器的完整位姿和投影模型；对于无组织点云（包括 LiDAR 运动去畸变后的点云），该方法也更难使用。

5.30 按可扩展性分类的方法

内存和计算成本是体积建图的两大瓶颈。对于固定分辨率方法，内存和计算复杂度随地图总体积线性增长，并随所选分辨率的倒数立方增长。降低这些复杂度是重要研究方向，因为只有这样才能构建超越小型受限体积的高细节地图。

体积建图早期工作主要关注降低内存使用。例如，Octomap 使用八叉树内部节点存储子节点占据值的最大值或平均值。通过递归剪枝估计量接近父节点的叶节点，地图中的均匀区域会自动用更少、更低分辨率的节点表示。这种适应环境几何的策略在实践中非常有效，因为环境通常主要由自由空间组成。此外，在八叉树内部节点存储最小、最大或平均值对下游任务也有价值，可支持低分辨率地图查询，并使用层次算法执行快速碰撞检查或探索规划。但 Octomap 的核心限制是所有测量都以最高分辨率融合，因此其计算复杂度仍呈立方增长。

多分辨率还可用于降低测量更新的计算成本。由于测量射线以固定角度发射，击中远处几何体的射线更少，因此随着距离增加降低更新分辨率是合理的。可以通过多分辨率射线追踪^[286]或多分辨率投影融合^[1125]实现。Supereight2^[347]进一步根据测量更新的熵调节更新分辨率，从而降低计算复杂度。这些方法显著提高了更新性能，但不同分辨率层仍需显式同步。消除同步需求的一种方法是不在每个八叉树节点存储绝对值，而只编码各分辨率层之间的差异；形式上可通过小波分解实现。小波编码地图可以在任意时刻以任意分辨率高效查询。利用这一性质，wavemap^[923]以从粗到细的方式更新地图，进一步降低计算复杂度。除根据测量熵调整更新分辨率外，它还跳过无信息更新，例如地图某区域的占据状态已经收敛为空闲且所有测量一致时。

5.31 高斯过程

如第 5.3.2.5 节所述，将建图问题表述为回归问题有利于获得连续表示。此外，若希望尽量减少对环境作出的假设，采用非参数方法求解非线性回归问题是理想选择。高斯过程(Gaussian process, GP)^[909]是一种随机、非参数、非线性回归方法。给定其他位置处噪声且稀疏的测量，它可以估计任意查询点处未知函数的值。我们已在第 2.2 节介绍如何使用 GP 表示连续时间轨迹。显然，GP 也是连续量建图的有吸引力方案，并已广泛用于机器人学文献中，以利用深度传感器建模连续空间现象^[1122, 816, 377, 574]。

GP 模型中的信息由均值 $\mathbf{m}(\mathbf{x})$ 和核函数 $\kappa(\mathbf{x}, \mathbf{x}')$ 给出，对所估计的连续量建模为：

$$f(\mathbf{x}) \sim \mathcal{GP}(\mathbf{m}(\mathbf{x}), \mathcal{K}(\mathbf{x}, \mathbf{x}')). \quad (5.8)$$

设 $\mathbf{X} = \{\mathbf{x}_j \in \mathbb{R}^D\}$ 是具有测量值 $\mathbf{y}$ 的位置集合，在位置 $\mathbf{x}_j$ 处测得的估计量为 $y_j = f(\mathbf{x}_j) + \epsilon_j$ 。对于 J 个训练对 $(\mathbf{x}_j, y_j)$ ，假设噪声 ϵ_j 独立同分布，且满足高斯分布 $\epsilon_j \sim \mathcal{N}(\mathbf{0}, \sigma_j^2)$ 。给定测试位置集合 $\mathbf{X}^* = \{\mathbf{x}_n^* \in \mathbb{R}^D \mid n = 0, \dots, N\}$ ，函数值与观测目标值的联合分布可写为：

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}_* \end{bmatrix} = \mathcal{N}\left(\mathbf{1}, \begin{bmatrix} \mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_j^2 \mathbf{I} & \mathbf{K}(\mathbf{X}, \mathbf{X}^*) \\ \mathbf{K}(\mathbf{X}^*, \mathbf{X}) & \mathbf{K}(\mathbf{X}^*, \mathbf{X}^*) \end{bmatrix}\right), \quad (5.9)$$

其中, $\mathbf{K} = [\mathbf{K}(\mathbf{x}_i, \mathbf{x}_j)]_{ij}$ 。因此, 条件分布 $(f_* | \mathbf{X}, \mathbf{y}, \mathbf{X}^*) \sim \mathcal{N}(\bar{\mathbf{f}}_*, \text{cov}(\mathbf{f}_*))$, 其均值为:

$$\bar{\mathbf{f}}_* = \mathbf{K}(\mathbf{X}^*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_j^2 \mathbf{I}]^{-1} \mathbf{y}, \quad (5.10)$$

协方差为:

$$\text{cov}(\mathbf{f}_*) = \mathbf{K}(\mathbf{X}^*, \mathbf{X}^*) - \mathbf{K}(\mathbf{X}^*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_j^2 \mathbf{I}]^{-1} \mathbf{K}(\mathbf{X}, \mathbf{X}^*). \quad (5.11)$$

式 (5.10) 和式 (5.11) 是所估计量的预测方程。GP 对表示具有空间相关性的数据尤其有效, 因此能够克服环境建图中占据栅格方法所特有的单元独立性假设。高斯过程占据地图 (Gaussian Process Occupancy Map, GPOM) [816] 将传感器观测及其对应标签 (自由或被占据) 收集为训练数据; 地图单元构成测试位置, 并如式 (5.9) 所示与训练数据相关。利用式 (5.10) 和式 (5.11) 完成回归后, 通过二元分类函数将回归输出“压缩”为单元的占据概率。

在原始形式中, GPOM 是一种批处理建图技术, 计算复杂度为 $\mathcal{O}(J^3 + J^2 N)$ 。为应对这一计算复杂度, 尤其是实现增量 GP 建图, 后续提出了多种方法, 例如 [574, 575, 1157, 378]。

基于 GP 函数建图的一个关键优点是, 可以对估计量进行线性运算 [969], 结果仍为 GP。由于求导及梯度都是线性运算, 估计量的微分输出具有概率性。环境不确定性的连续表示可用于突出未探索区域, 并优化机器人的搜索规划 [378, 677]。GP 地图的连续性也能提高规划器的灵活性, 使其可以直接基于已收集的传感器数据推断, 而不受栅格或体素单元分辨率限制。

5.32 高斯过程隐式表面

隐式表面也可以由 GP 表示。高斯过程隐式表面 (Gaussian process implicit surface, GPIS) 技术 [1186, 736, 677, 505] 利用 GP 方法, 根据噪声测量估计隐式表面的概率连续表示。此外, GPIS 也可连续地估计表面和距离场 [576, 1034, 636, 1195]。

在 GPIS 表述中, 设要估计的距离场为 d 。给定表面上的点、这些点到最近表面的距离 d_i , 以及通过线性算子计算得到的相应梯度 ∇d [969], 由于表面上的距离为零, d 与 ∇d 可以建模为零均值的联合 GP:

$$\begin{bmatrix} d \\ \nabla d \end{bmatrix} \sim \mathcal{GP}(\mathbf{0}, \mathbf{K}(\mathbf{X}, \mathbf{X}')). \quad (5.12)$$

GPIS 方法能够估计连续隐式表面及其由梯度得到的法向, 并同时给出不确定性。一些工作考虑使用参数函数先验以更准确地捕捉给定形状 [736, 505]; 另一些方法不仅估计隐式表面, 还估计完整距离场。就标准 GPIS 表述而言, 距离在测量附近 (即表面上) 拟合良好, 但在远离表面处会回归到均值, 此处均值为零。为了在远离表面的位置以连续且概率的形式估计完整距离场, 一些工作将非线性操作应用于类似 GPIS 的表述 [1195, 1196, 634]。

所有这些方法都必须处理基于 GP 表述的计算复杂度; 作为交换, 仅凭点云就能获得环境的连续、生成式和概率表示。

5.33 Hilbert map

Hilbert map (HM) [904] 在许多方面与 GPOM [816] 相似。二者都是连续概率模型，与基于体素的方法不同，不对空间离散化；与基于点的方法相比，能够插值缺失数据。如前所述，GPOM 的主要挑战是计算开销高。因此，Hilbert map 的设计目标是：(i) 以在线方式连续处理数据；(ii) 建模观测之间的依赖；(iii) 纳入测量不确定性。

为实现这些目标，通常在投影特征空间中使用随机梯度下降训练逻辑回归器。分类器与优化器的组合支持利用大量数据在线更新模型；特征投影则使简单分类器也能表示复杂的空间细节。

特征投影与 GP 中核函数的思想相同，但这里使用近似而不是完整协方差。有多种选择，包括 Nystroem [1185]、随机 Fourier 特征 (Random Fourier Features) [901] 和稀疏核 (Sparse Kernel) [755]，这里采用稀疏核。稀疏核的目标是限制观测产生影响的范围，从而提高收敛速度和计算效率；得到的核函数会在特定距离处严格降为 0。

该核函数通过在待建图空间中按规则间隔放置诱导核，可以将二维或三维空间中的点投影到显著更高的维度。这样就能计算输入数据的高维特征空间表示，并使用小批量随机梯度下降训练逻辑回归分类器。最后，训练通过沿范围测量采样自由空间点，同时将回波加入为障碍物点来完成。

HM 面临的一个挑战是所用核的表达能力。图 5.11 所用的径向基函数 (radial basis function, RBF) 在二维中是圆、在三维中是球，需要组合多个函数值才能重建环境的复杂细节。因此，核的数量和长度尺度之间存在权衡，二者会影响计算成本、重建细节和插值能力。

(a)

(b)

(c) 图 5.11 (a) 机器人使用二维 LiDAR 传感器获得的观测，随后转换为 (b) 自由 (绿色) 和被占据 (红色) 点的数据集，再用来训练 (c) 所示的 Hilbert map。

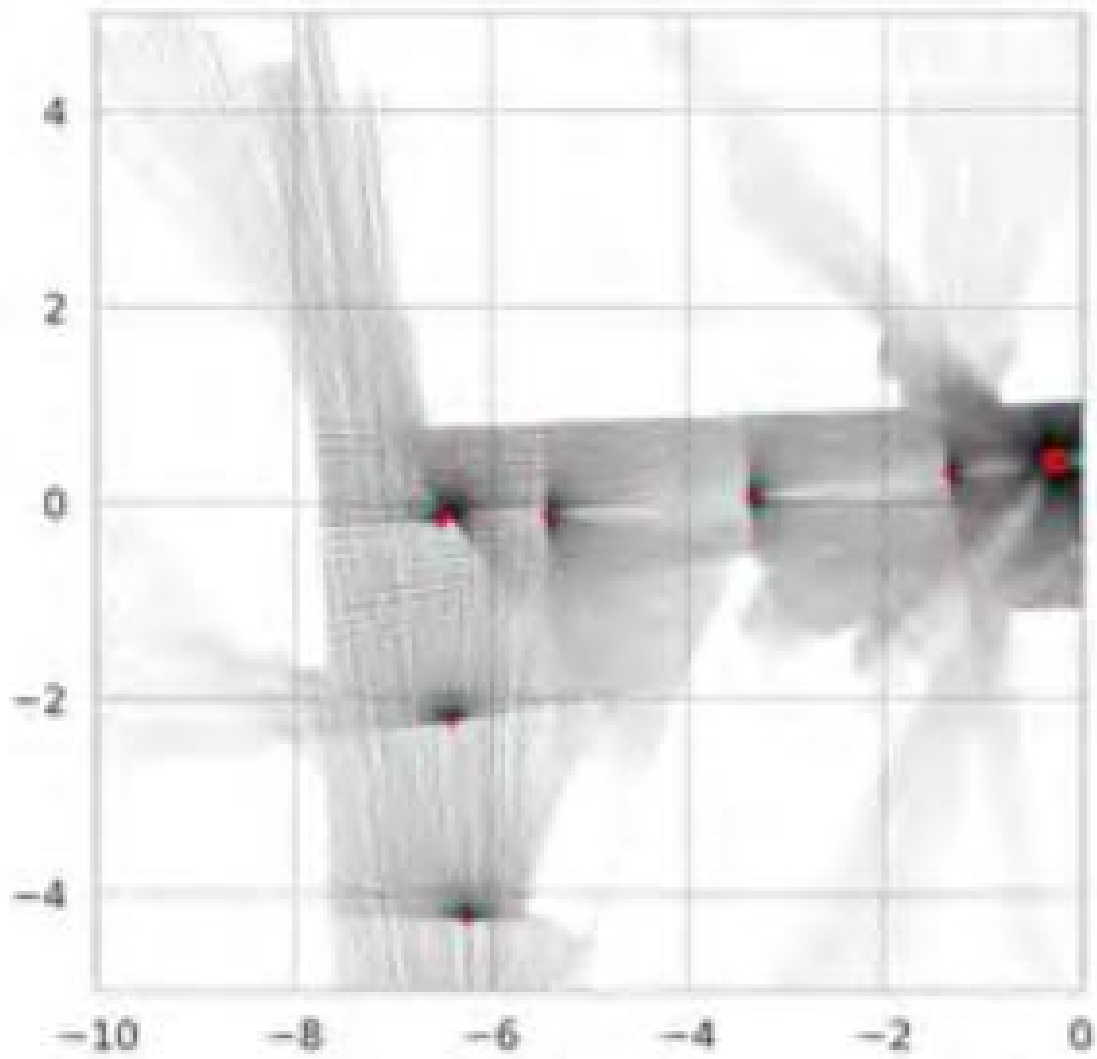
5.34 建图中的深度学习

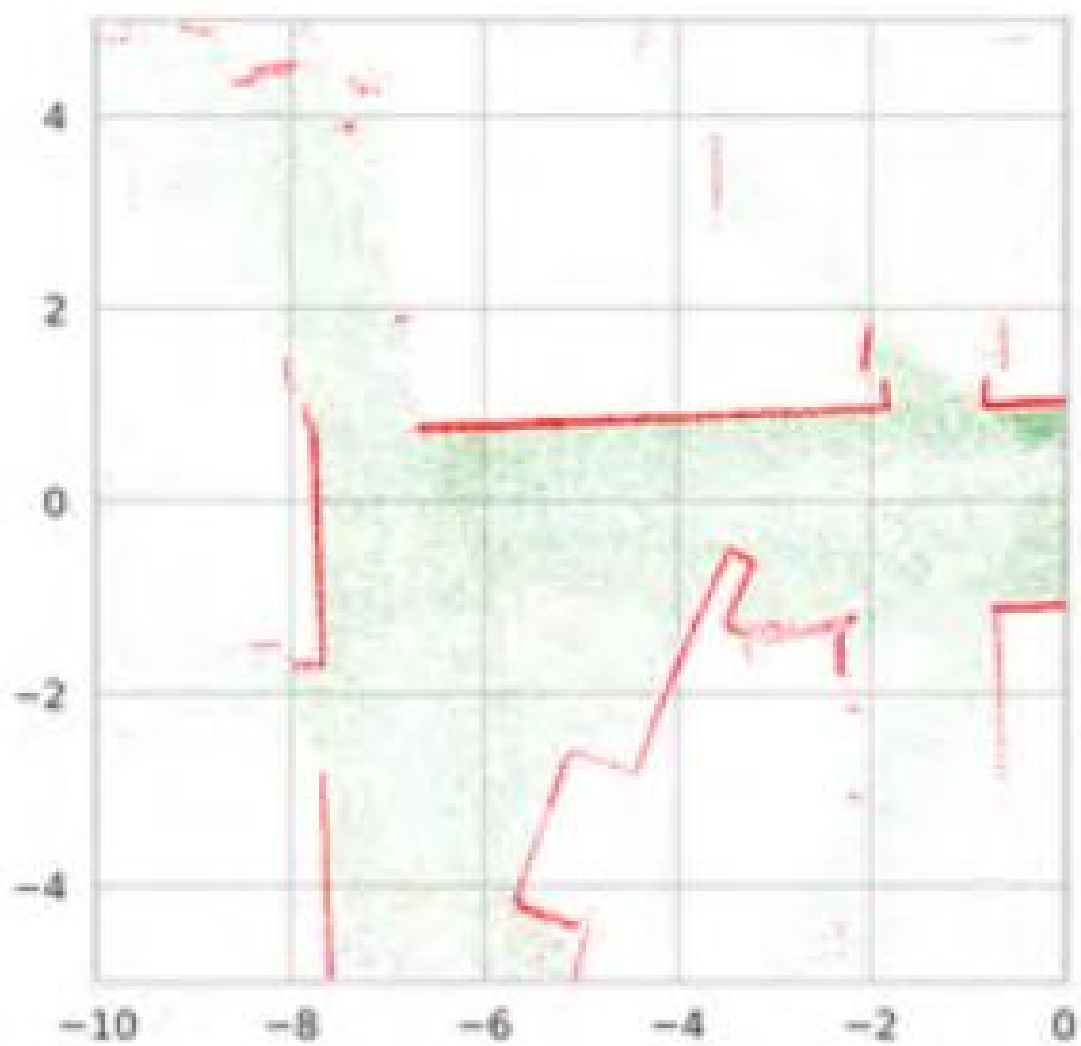
近年来，利用神经辐射场 (neural radiance fields, NeRF, 见第 14 章及 [764]) 进行新视图合成受到关注；它通过简单的多层感知机 (multi-layer perceptron, MLP) 实现出色的图像生成结果。一些方法因此研究使用神经表示来估计 SDF。在任意空间位置学习预测 SDF，可以得到连续表示，既能以任意分辨率转换为网格，也能凭借学习函数的插值能力生成更完整的表示。

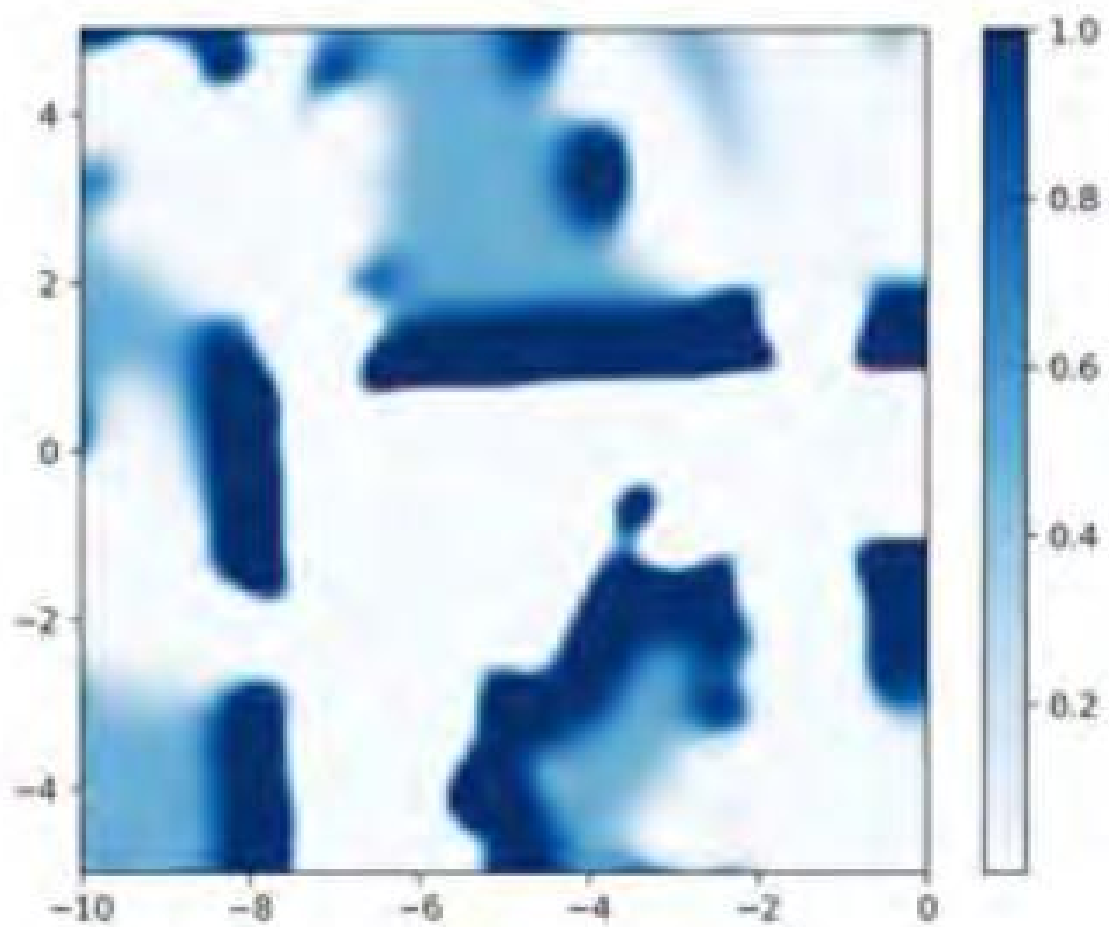
与隐式表示相似，这种表示从输入数据学习并近似为可在任意位置查询的连续函数。神经表示通常是在给定位姿下离线学习的；近来，人们也开始关注增量方法 [1047, 1293]，以及使用神经表示在线估计位姿的方法 [1047, 968]。

特别地，Sucar 等人 [1047] 的方法基于 RGB-D 帧，利用神经网络预测场景中任意点的 SDF 值。后续方法将特征的空间表示 (如体素栅格 [862, 1309]、八叉树 [1293]、点 [968, 261] 等) 与神经表示分离。这些方法在空间表示中存储小而具有描述力的特征，再利用小型神经网络确定场景中任意点的 SDF 值。这样就将学习函数与空间表示解耦，既可以依靠小型神经解码器将特征转换为有符号距离值，也能够有效应用于室外等大尺度场景。

基于深度学习的建图、重建和 SLAM 领域发展迅速，并结合了经典表示中的思想，例如







surfel splatting^[548, 740]，在重建质量和效率方面获得显著成果。第 13 章将介绍 Dust3r 等前馈网络如何从单目视频生成稠密表示。特别是，渲染新视图并在任意位置生成新数据的能力，可能在不依赖仿真环境的情况下用于机器人学习。例如，NeRF 和 Gaussian splatting^[1318, 103]已广受关注，并在多项 SLAM 相关工作中展示出显著潜力；第 14 章将进一步讨论这些方法。

5.35 使用考量

所有地图表示都需要在各自鲜明且往往互补的优点与缺点之间权衡。因此，为给定应用或机器人系统选择地图表示时，必须仔细考虑该地图在所有下游任务中的使用方式。其他需要考虑的因素是运行环境和可用传感器。下面先讨论环境因素，它们会促成一些明确的选择；然后讨论更细致、依赖任务的考量；最后就第 5.4 节所述现有方法给出简要使用建议。

5.36 环境因素

运行环境可以分为结构化和非结构化两类。在自动化工厂等受严格控制的空间中，针对机器人任务及其将遇到的特定对象量身定制的地图表示，在效率和精度方面通常优于通用稠密表示。相反，本章讨论的稠密表示能够对任意形状的物体建模，并可在任何环境中工作。在变化或部分未知的环境中，机器人通常必须能够区分已观测自由空间和未观测空间。该信息使路径规划器能够避免穿越未观测空间的不安全运动，因为这些空间可能被占据；也可用于探索规划。包括点、surfel 和网格在内的显式表面表示，通常无法区分自由空间和未观测空间，而所有基于占据的方法都能做到。基于隐式表面的方法也能提供这种区分，但在许多更专注于重建的应用中，为节省计算和内存成本，远离表面的此类信息经常被丢弃。

另一个考虑因素是可扩展性。显式表示通常比隐式表示更节省内存，因为它们只描述表面本身，并且可以轻易地使保真度适应场景各部分所需的细节。需要自由空间信息时，与固定分辨率体素表示相比，多分辨率方法可以在精度、内存以及捕捉极细物体的能力方面带来显著改进。

最后需要考虑环境中是否包含大量动态物体，以及应在多大程度上对其建模。从地图表示角度看，现有方法大多可归为三类。第一类不考虑动态性，直接将所有测量融合进本章介绍的一种表示。实践中，使用隐式表示时这可能已经足够，因为其自由空间更新通常能很好地消除物体移动后留下的残余。第二类方法试图只将环境静态元素融合到地图中，显式检测并丢弃所有对应动态物体的测量。这类方法在使用显式地图时尤其流行，因为显式地图中的残余更难移除；此类方法通常即使在高度动态的空间中也能生成干净的地图。最后一类方案不仅表示背景，还表示场景中的运动元素。注意，这通常通过混合表示完成，将本章介绍的基本几何表示与对象层级的专用表示结合。关于动态环境中的 SLAM（包括实现上述方法和更先进方法的具体技术）的详细讨论，请参阅第 15 章。

5.37 下游任务类型

除环境外，同样重要的是考虑机器人所需任务需要何种地图信息。任意给定操作通常都能在各种表示上执行，但效率和实现复杂度往往差异很大。最大的差别在于操作沿表面执行，还是在笛卡尔空间中执行。如表 5.2 所示，隐式表示通常可以简单高效地滤波以笛卡尔坐标表

达的属性，例如占据状态；相比之下，显式表示适合滤波沿表面表达的属性，例如视觉纹理。这解释了显式表示通常对深度测量质量更敏感，但能够创建非常细致、视觉效果出色的 3D 重建；而隐式方法适合融合 RGB-D 相机数据等噪声深度测量。

就查询而言，显式表示可以直接沿表面遍历，这解释了它们在渲染、图形应用和覆盖路径规划等任务中的流行。不过，要回答笛卡尔坐标中的查询，需要进行最近邻查找等额外步骤。隐式表示恰好相反，因此常用于碰撞检查任务。

操作	显式表示中高效	隐式表示中高效
滤波测量	沿表面（纹理等）	在笛卡尔空间中（占据状态等）
查询和遍历	在表面坐标中（覆盖规划等）	在笛卡尔坐标中（碰撞检查等）
修改表面	几何（形变等）	拓扑（合并、切割、简化等）

表 5.2 显式与隐式表面表示的互补优点和缺点。

最后，显式表示可以高效修改表面的几何形状，包括形变。实践中，地图通常根据不完美且存在漂移的里程计系统提供的位姿估计，通过融合深度测量构建。随着时间推移，累积位姿误差也会导致稠密地图不一致。与 SLAM 系统一样，检测到闭环时可以通过形变稠密地图来消除这些误差。显式和隐式表面都可以形变，但使用显式表示时该操作在本质上更简单、更高效。相反，使用隐式表示可以简化并提高影响表面拓扑或连通性的操作效率。因此，隐式表示常用于合并表面估计、组合或相减物体形状以及简化表面。

还应记住，不同表示可以配合使用，以发挥各自优势。一个很好的混合方法示例是基于 TSDF 的网格化（第 5.4.3 节）：先使用隐式表面表示（TSDF）方便地滤波噪声深度测量，再用 Marching Cubes 将其转换为显式表示（网格）。在判断混合表示的优势是否抵消其带来的开销时，值得考虑如何将转换限制为仅在局部、低频发生。

5.38 建图方法总结

最后总结本章介绍的现有方法的关键差异。先看显式表示：用点集合描述表面简单，所需假设最少，但信息量也最少。除无穷小点外，surfel 在小邻域或片元上表示表面属性。最后，网格显式表示表面连通性，并允许平滑插值其属性。然而，估计表面连通性需要最多假设，有时还伴随显著计算成本。

对隐式表示而言，隐式表面相对于占据地图的一项特殊优势是能提供快速、高质量的距离信息和梯度，有利于基于优化的规划。然而，滤波占据估计所需的假设更少；对于基于体素的方法，占据地图也更擅长捕捉细薄障碍物。在深度测量特别噪声大或稀疏时，基于 GP 和 Hilbert map 的非体素隐式表示能够提供特别好的不确定性估计。由于它们显式考虑几何形状的空间相关性，通常也更擅长插值部分观测到的表面。

学习型方法是一个快速发展的研究领域。对于基于学习的隐式表示，NeRF 已被证明能够支持很有前景的新能力，尤其是语义建模和空间推理。最近，Gaussian splatting^[552]（一种与 surfel 相似的显式表示）推动了人们对使用 splatting 的方法^[548, 740, 1261]的兴趣。研究人员正积极改善这些方法的计算和内存占用，测试其可支持的新能力，并探索如何将其整合到完整机器人系统中。展望未来，我们预计基于学习的方法将通过学习先验，提高稠密表示的通用性和表达能力，并改善其处理噪声测量、不完整观测和动态物体的能力。

5.39 致谢

作者感谢 Lan Wu 在本章准备过程中提供的支持。